个结点一次且仅访问一次。线性结构中元素之间的先后关系是确定且一对一的，从线性表的表头开始，沿着特定的描述关系，很容易找到它唯一的后继，进而能够访问线性表中的全部元素。所以，对线性表的遍历是轻而易举的事情。而二叉树的前驱、后继关系不是一对一的，二叉树中每个结点都可能有0个、1个或2个子结点，对这种非线性结构进行访问需要按照某种约定的方法进行，从而保证不会漏掉对某个结点的访问，当然也不会让结点被多次访问。

由二叉树的定义可知，一棵二叉树由三部分组成：根、左子树和右子树。因此，对非空的二叉树的遍历可以相应地分解为三项“子任务”：

① 访问根结点。

② 遍历左子树（即依相应的规律访问左子树中的全部结点）。

③ 遍历右子树（即依相应的规律访问右子树中的全部结点）。

根结点是一个单一的结点，“访问根结点”这个子任务很容易完成。当左子树或右子树中含有多个结点时，相应的“遍历子树”的子任务又类似于对原二叉树的遍历。根据定义，左子树和右子树仍是二叉树（可以是空二叉树），对它们的遍历任务可以按上述方法继续分解，直到子树为空时，它的遍历任务亦为空，任务完成。可以看出，这个遍历过程具有递归性，自然可想到通过编写递归程序来实现二叉树的遍历。

上述三项任务之间的次序可以组合出6种不同的排列。按照先左（子树）后右（子树）的惯例，保留3种排列次序。另外3种排列次序是先右（子树）后左（子树），分别与前面的3种排列一一对应，可以看作它们各自的镜像。

对二叉树进行遍历的3种常规算法分别是先序遍历、中序遍历和后序遍历。这3种遍历也分别称为先根遍历、中根遍历和后根遍历。这3种遍历的过程描述如下。

1）先序遍历算法

若二叉树为空，则返回；否则依次执行以下操作：

① 访问根结点。

② 先序遍历左子树。

③ 先序遍历右子树。

2）中序遍历算法

若二叉树为空，则返回；否则依次执行以下操作：

① 中序遍历左子树。

② 访问根结点。

③ 中序遍历右子树。

3）后序遍历算法

若二叉树为空，则返回；否则依次执行以下操作：

① 后序遍历左子树。

② 后序遍历右子树。

③ 访问根结点。

以L表示根的左子树中全部结点的输出序列，以R表示根的右子树中全部结点的输出序列，以V表示根的输出，则可以简洁地表示这3种遍历序列。先序遍历序列是V,L,R，中序遍历序列是L,V,R，后序遍历序列是L,R,V。

概括地说，在这 3 种遍历序列中，根的输出分别位于两棵子树的遍历序列之前、之间及之后。当然，两棵子树的遍历序列仍要遵守所依从的遍历规则。具体来说，在对整棵树进行先序遍历时，其左、右子树的遍历也要按先序遍历规则进行；在对整棵树进行中序遍历时，其左、右子树的遍历也要按中序遍历规则进行；在对整棵树进行后序遍历时，其左、右子树的遍历也要按后序遍历规则进行。

【例 5-5】 以图 5-10 所示的树 T 为例，分析它的遍历过程，给出遍历结果。

为方便起见，以输出一个结点的信息代表对这个结点的访问。

先看二叉树 T 的先序遍历。根据先序遍历的算法描述，首先访问树 T 的根。此时输出 0。接下来应该按先序遍历规则输出 0 的左子树中的全部结点，然后再按先序遍历规则输出 0 的右子树中的全部结点。

对于 0 的左子树的部分，按先根（1）、再左子树（3）、最后右子树（4）的次序进行遍历，遍历的结果是 1,3,4。对于 0 的右子树的部分，遍历根结点 2，因为 2 的左子树为空，故只需要遍历 2 的右子树部分。而对这棵子树的遍历，仍是按先根（5）、再左子树（6）、最后右子树（空）的次序，最后得到的结果是 2,5,6。

由此得到 T 的先序遍历序列为 0,1,3,4,2,5,6。可以看出，根在最前面，后面是左子树中的全部结点，再后面是右子树中的全部结点。

图 5-11 展示了 T 的先序遍历过程。从根结点开始，使用带数字标号的箭头表示对结点的遍历，箭头旁边的标号是遍历次序。使用带空心箭头的弧线或不带数字标号的箭头来示意结点间的回退转移。

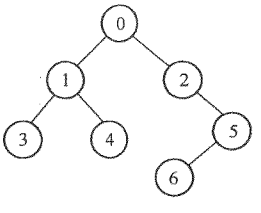


图 5-10　二叉树 T

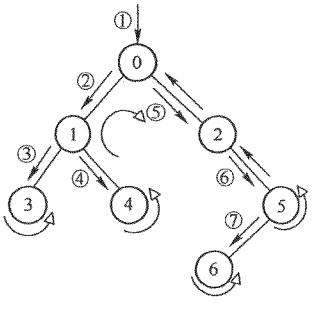


图 5-11　T 的先序遍历过程

再看 T 的中序遍历。按照规则，首先中序遍历根的左子树，然后访问根，最后中序遍历根的右子树。对于左子树的遍历，也是依先左子树（3）、再根（1）、最后右子树（4）的次序进行遍历，得到 3,1,4 的结果。后面输出根 0。对于右子树，因为 2 的左子树为空，所以第一个输出的是根 2，然后是 2 的右子树，输出 6,5。由此，得到的中序遍历结果是3,1,4,0,2,6,5。

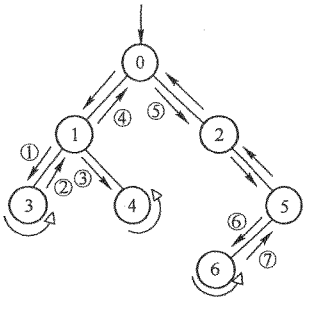


图 5-12　T 的中序遍历过程

图 5-12 展示了 T 的中序遍历过程。依然是从根开始进行遍历，但并不输出根的内容，而是沿左孩子指针一直找到最“左下角”的一个结点。这个“左下角”的意思是指，找到的结点的左孩子指针为空，即再也不能沿左孩子指针向前

“走”了。通常，在画二叉链表时，左孩子指针是向左下方向延伸的，所以这个结点看起来很像树的“左下角”。这个结点是第一个被遍历到的结点。在从根寻找“左下角”的过程中，除“左下角”结点以外，经过的结点均不输出，只是“路过”它们。

可以看出，根结点 0 的前面是左子树中的全部结点，0 的后面是右子树中的全部结点。根在两个子树的“中间”。

T 的后序遍历以先遍历根的左子树、再遍历根的右子树、最后输出根的次序进行。仍是从根开始，在输出根之前要先遍历左子树，与中序遍历类似，要去寻找树的“左下角”并输出。对于结点 1，遍历了它的左子树之后，还需要遍历它的右子树，然后才能输出 1。在从根到结点 3 的“路途”中，已经“路过”一次结点 1 了，遍历结点 3 之后，还要再回到这个结点，这是第二次“路过”它。这两次都不输出它的值。直到它的右子树遍历完毕，第三次到达这个结点时，才输出它。如图 5-13 标出的前 3 个步骤所示。

结点 1 输出完毕，意味着 0 的左子树遍历完毕，应该转到 0 的右子树了。类似地，第二次“路过”结点 0 但不输出。0 的右子树的根是 2，它的左子树为空，所以 2 的前面仅有其右子树中的全部结点。先输出 6，再输出 5，最后输出 2。得到的 T 的后序遍历结果是 3,4,1,6,5,2,0。

【例 5-6】给出如图 5-14 所示二叉树，写出其先序、中序及后序遍历序列。

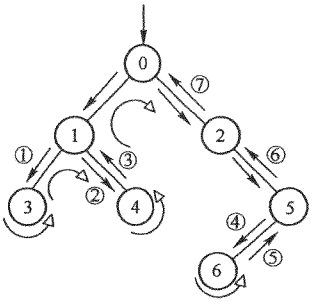


图 5-13　T 的后序遍历过程

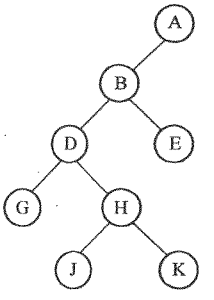


图 5-14　二叉树 T

根据三种遍历策略，得到的二叉树 T 的先序遍历序列为 A,B,D,G,H,J,K,E，中序遍历序列为 G,D,J,H,K,B,E,A，后序遍历序列为 G,J,K,H,D,E,B,A。

可以看出，在二叉树的先序遍历序列中，根在最前面；在二叉树的后序遍历序列中，根在最后面。

很容易采用递归方式实现二叉树的三种遍历算法。二叉树的先序遍历算法如下。

```
void PreorderTraverse(BTree root)    //二叉树先序遍历
{   if (root==NULL) return;
    printf("%d\t",root->data);
    PreorderTraverse(root->left);
    PreorderTraverse(root->right);
}
```

二叉树的中序遍历算法如下。

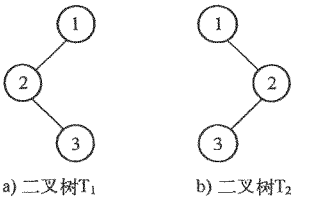
```
void InorderTraverse(BTree root)     //二叉树中序遍历
{   if (root==NULL) return;
    InorderTraverse(root->left);
    printf("%d\t",root->data);
    InorderTraverse(root->right);
}
```

二叉树的后序遍历算法如下。

```
void PostorderTraverse(BTree root)     //二叉树后序遍历
{   if (root==NULL) return;
    PostorderTraverse(root->left);
    PostorderTraverse(root->right);
    printf("%d\t",root->data);
}
```

在给定二叉树后，按照遍历规则，其先序遍历、中序遍历和后序遍历的序列都是确定的。反过来，如果给定了遍历序列，能不能确定二叉树呢？如果遍历序列是空串，则树是空树。如果遍历序列只含有一个元素，则对应的树只含有一个结点。这两种情况都属于特例。下面针对非特例的情况来讨论，即二叉树中至少含有两个结点。

以先序遍历序列为例，图 5-15 中的两棵二叉树的先序遍历序列相同，都是 1,2,3，但它们的树形是不同的，说明同一个先序遍历序列可能对应于不同的二叉树，即给定某棵二叉树的先序遍历序列，并不能唯一确定该二叉树。



a) 二叉树T_1　　b) 二叉树T_2

图 5-15　两棵二叉树有相同的先序遍历序列

对于中序遍历序列和后序遍历序列，也有这样的结论，即给定某棵二叉树的中序遍历序列或后序遍历序列，均不能唯一确定该二叉树。由此得出结论，仅给出一种遍历序列，无论是先序、中序还是后序遍历序列，均不能唯一确定该二叉树。

给出二叉树的两种遍历序列，是不是能唯一确定二叉树？如果回答这个问题，就要看给定的是二叉树的哪两种遍历序列。如果两种遍历序列中包含中序遍历序列，则答案是肯定的，即先序遍历序列加中序遍历序列，或后序遍历序列加中序遍历序列，均能唯一确定二叉树。但先序遍历序列加后序遍历序列，不能唯一确定二叉树。实际上，图 5-15 中的两棵二叉树的先序遍历序列相同，后序遍历序列也相同，但它们是不同的二叉树。

当给定二叉树包括中序遍历序列在内的两种遍历序列后，如何还原这棵二叉树呢？例 5-7 以先序遍历序列加中序遍历序列为例，给出二叉树的重建过程。

【例 5-7】 设二叉树 T 的先序遍历序列是 A,B,D,G,H,J,K,E，中序遍历序列是 G,D,J,H,K,B,E,A，画出二叉树 T。

由先序遍历序列可知，T 的根是 A。在中序遍历序列中查找 A 的位置，位于 A 前面的是其左子树的中序遍历序列，位于 A 后面的是其右子树的中序遍历序列。从而将原问题的求解（对整棵树的还原）分解为两个更小问题的求解（对两棵子树的还原）。现在的问题是，

需要从整棵树的先序遍历序列中抽取出两棵子树的先序遍历序列，从整棵树的中序遍历序列中抽取出两棵子树的中序遍历序列。

对于中序遍历序列，已经得到了两棵子树的中序遍历序列，而且以根为界。根结点前面的部分是左子树的中序遍历序列，后面的部分是右子树的中序遍历序列。

而整棵树的先序遍历序列中也隐藏着左子树的先序遍历序列和右子树的先序遍历序列，它们都在根结点的后面，只是两个序列连在了一起，中间没有明显的分界。从左子树的中序遍历序列中，已经知道了左子树中的结点个数，实际上，结点本身也是知道的。所以，在先序遍历序列中，找到两棵子树遍历序列的分界易如反掌。

剩余要解决的问题和初始问题是一样的，只是问题规模更小了。使用递归处理就可以实现。

观察中序遍历序列，根的后面没有结点，可知 A 的右子树为空，除 A 以外的所有结点都在 A 的左子树中。

再由条件所知，A 的左子树的前序遍历序列是 B,D,G,H,J,K,E，中序遍历序列是 G,D,J,H,K,B,E，可得到该子树的根为 B，B 的右子树中有结点 E，其余的结点在 B 的左子树中。使用递归思想可以将大的问题化解为小的问题，从而可以得到原问题的答案。还原的二叉树 T 如图 5-16 所示。

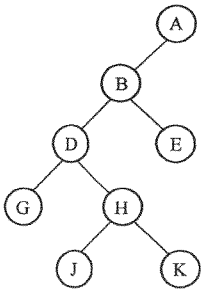


图 5-16　还原的二叉树 T

既然给定包括中序遍列序列在内的两种遍历序列，能唯一确定二叉树，那么，给出二叉树的先序、中序及后序遍历序列，可以证明能够唯一确定二叉树。

【例 5-8】 编写程序，统计以二叉链表表示的二叉树 T 中叶结点的个数。

T 中叶结点的个数等于根左子树中叶结点的个数加上根右子树中叶结点的个数。所以，如果已经通过递归调用得到了两棵子树中叶结点的个数，那么将结果相加即求解。

需要处理的例外情况是，如果树是空树，则返回值 0，表示没有叶结点。空树也是递归出口。

```
int leafNumber(BTree root)
{   if (root==NULL) return 0;
    if (root->left==NULL&&root->right==NULL) return 1;
    return leafNumber(root->left)+leafNumber(root->right);
}
```

【例 5-9】 编写程序，返回二叉树 T 的高度。

仍使用递归的思想编写程序。如果左子树的高度和右子树的高度都已知，则二叉树 T 的高度就可求得，树的高度是两棵子树中较高者的高度再加 1。

递归出口也是空树，空树的高度为 0。

```
int high(BTree root)
{   if (root==NULL) return 0;
    return max(high(root->left),high(root->right))+1;
}
```

除了先序、中序及后序3种典型的遍历方式以外，对二叉树的遍历还有从根开始，按层遍历的方法，称为层序遍历，也称按层遍历。所谓二叉树的层序遍历，即从根结点开始逐层向下遍历，直到最后一层为止。对于同一层的结点，由左至右遍历各结点。同一层中的结点相继被访问，同时，它们之间的相对次序也决定着它们子结点的相对次序，即对于同一层中的结点 u 和结点 v，若先遍历 u 再遍历 v，则对 u 的子结点的遍历早于对 v 的子结点的遍历。

【例 5-10】 给出图 5-17 所示的二叉树 T 的层序遍历结果。

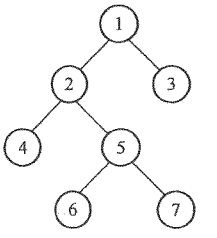


图 5-17　二叉树 T

按照层序遍历的算法描述，从二叉树的根开始按层访问，先访问最上面一层（即 0 层）中结点的内容。该层只有根结点，输出 1。然后依次访问 1 的子结点 2 和 3，同层中依先左后右的次序。这一层访问完毕，再继续访问 2 的子结点和 3 的子结点。3 是叶结点，没有子结点。所以，这一层的访问结果是 4 和 5。接下来访问 4 的子结点和 5 的子结点。结点 4 是叶结点，所以，下一层只访问结点 6 和 7。这样得到的层序遍历结果是：1,2,3,4,5,6,7。

在遍历过程中，需要有一个辅助结构来记录当前已经涉及但尚未遍历的结点。例如，在层序遍历二叉树 T 的过程中，在访问结点 2 时，要记录它的子结点 4 和 5，但这两个结点的遍历要晚于结点 3 的遍历。此时，不能直接输出结点 4 和 5，只能暂时保存起来。在访问结点 2 的兄弟结点（结点 3）时，如果它有子结点，则也要暂时保存起来，而且保存在结点 5 的后面。这正与队列的特性相符，故在实现层序遍历时，使用队列作为辅助结构。

二叉树层序遍历的算法如下。

```
void LevelTraverse(BTree root)                    //二叉树层序遍历
{   BinTNode temp;
    SeqBTreeQueue myq;                            //修改第三章中实现的队列的类型
    initQueue(&myq);                              //初始化队列 myq
    if (root==NULL) return;
    enqueue(&myq,root);                           //根入队
    while (isEmptyQ(&myq)==FALSE){                //队列不空，表明还有结点待遍历
        dequeue(&myq,&temp);
        printf("%d \t",temp.data);
        if (temp.left!=NULL) enqueue(&myq,temp.left);
        if (temp.right!=NULL) enqueue(&myq,temp.right);
    }
}
```

【例 5-11】 借助于二叉树的遍历过程，对二叉树的结点从 1 开始进行连续编号，要求每个结点的编号大于其左子结点、右子结点（若存在）的编号，且其左子结点的编号小于其右子结点的编号（若存在），则遍历算法是（　）。

A. 先序遍历　　B. 中序遍历　　C. 后序遍历　　D. 层序遍历

答案为 C。

根据题意，二叉树中任一结点 v 的编号要大于其子结点的编号，这意味着要先遍历子结

点，再遍历结点 v。对于 v 的子结点，先遍历其左子结点，再遍历其右子结点。这个过程与后序遍历是吻合的。

2. 二叉树遍历的非递归实现

仍然将一棵二叉树分为三部分：根、左子树和右子树。在从根开始遍历二叉树时，根的左子树和右子树的遍历不能同时进行，只能先遍历一棵子树，同时保存二叉树中尚未遍历的信息，留待一棵子树处理完毕后再处理。

分析二叉树的遍历过程可知，使用栈来保存相关信息。

1）先序遍历算法

在先序遍历过程中，当遍历到某个结点时，输出该结点的信息并进入其左子树继续遍历。在进入左子树进行处理之前，需要在栈中保存右子树的相关信息，以便当左子树处理完毕，能够根据保存的线索找到右子树的根。所以，栈中保存右子树的根即可。

非递归先序遍历二叉树的实现如下。

```
void PreorderTraverseNonRecursive(BTree root)
{   SeqBTreeStack S;
    BinTNode temp;
    initStack(&S);
    while (1){
      while (root!=NULL){
            printf("%d \t",root->data);
            if (root->right!=NULL) push(&S,root->right);
            root=root->left;
        }
        if (isEmptyS(&S)==TRUE) return;
        else{
            pop(&S,&temp);
            root=&temp;
        }
    }
}
```

2）中序遍历算法

在中序遍历二叉树的过程中，也因为每个结点都可能有两个分支且目前只能进行一个分支的遍历，而需要在栈中保存另一个分支的相关信息。

在中序遍历时，先进行左子树的遍历，然后遍历根，最后遍历右子树。所以，栈中保存根的信息，遍历完左子树后，从栈中找到根，访问根，同时能够容易地找到右子树。

非递归中序遍历二叉树的实现如下。

```
void InorderTraverseNonRecursive(BTree root)
{   SeqBTreeStack S;
    BinTNode temp;
    initStack(&S);
```

```
    while (1){
        while (root!=NULL){
            push(&S,root);
            root=root->left;
        }
        if (isEmptyS(&S)==TRUE) return;
        else{
            pop(&S,&temp);
            printf("%d \t",temp.data);
            root=temp.right;
        }
    }
}
```

3）后序遍历算法

相比于二叉树的先序遍历过程和中序遍历过程，后序遍历过程的非递归实现要复杂一些。类似于中序遍历过程，在进入左子树遍历之前，需要先保存根的信息。当左子树遍历结束时，从栈中找到根，但此时因为右子树尚未遍历，所以根并不能从栈中弹出，仍需要保存在栈中，只有当右子树也遍历结束后，才能访问根。

栈中保存的结点需要入栈两次、出栈两次。为了能有所区别，二叉树结点入栈时，附带一个标记位 flag。在第一次出栈时，并不访问结点。第二次出栈时才访问它。

为此，将二叉树结点和标记组合在一起，定义新的栈元素的类型，专用于非递归实现二叉树的后序遍历过程。

```
typedef struct SNode                    //入栈带标记结点
{   BinTNode  btreeNode;                //二叉树结点
    int flag;                           //标记
}StackTNode;
typedef struct{                         //用于后序遍历的栈
    StackTNode element[maxSize];
    int top;                            //栈顶位置
}SeqBTreeStackforPostT;
```

栈的基本操作与第三章第一节给出的类似，只是参数的类型需要进行相应的修改如下。

```
int clear(SeqBTreeStackforPostT *mys);                  //将栈 mys 清空
int pop(SeqBTreeStackforPostT *mys,StackTNode *x);      //返回栈顶元素
int push(SeqBTreeStackforPostT *mys,StackTNode *x);     //将元素 x 入栈
int initStack(SeqBTreeStackforPostT *mys);              //初始化栈
int isEmpty(SeqBTreeStackforPostT *mys);                //如果栈 mys 为空，则返回 1，否则返回 0
int isFull(SeqBTreeStackforPostT *mys);                 //如果栈 mys 为满，则返回 1，否则返回 0
```

非递归后序遍历二叉树的实现如下。

```
void PostorderTraverseNonRecursive(BTree root)
{
    SeqBTreeStackforPostT S;
    StackTNode stacktemp;
    initStack(&S);
    while (1){
        while (root!=NULL){
            stacktemp.btreeNode=*root;
            stacktemp.flag=0;
            push(&S,&stacktemp);
            root=root->left;
        }
        if (isEmpty(&S)==TRUE) return;
        else{
            pop(&S,&stacktemp);
            if (stacktemp.flag==0){
                stacktemp.flag=1;
                push(&S,&stacktemp);
                root=stacktemp.btreeNode.right;
            }
            else{
                printf("%d \t",stacktemp.btreeNode.data);
            }
        }
    }
}
```

三、二叉树的应用

二叉树有很多应用。计算机科学中常用的表达式树就是一个代表性的例子。第三章第三节介绍了表达式的表示方式及后缀表达式的计算，实际上，还可以使用二叉树来表示一个表达式，这样的二叉树称为表达式树。

仍以包含双目运算符的表达式为例。所谓表达式树就是使用一棵二叉树来表示一个表达式，从而将线性表示的代数式非线性化，变成一个树形结构。树具有层次结构，不同的层代表不同的计算次序。

表达式由运算符和操作数组成。假设表达式中包括双目运算符加、减、乘、除（+、-、*、/）和操作数（常量或变量），使用二叉链表表示时，在分支结点中存储运算符，在叶结点中存储操作数。图 5-18 所示的表达式树表示表达式 2*y*(a+3*y)-b。操

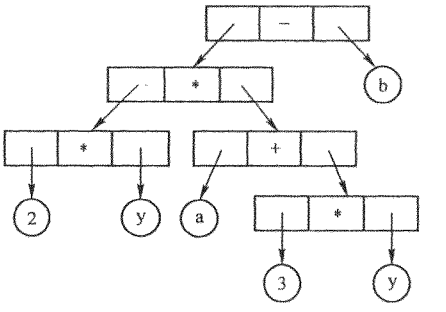


图 5-18　表示 2*y*(a+3*y)-b 的表达式树

作数都在叶结点的位置，运算符都在分支结点的位置。为了更形象化，将运算符与操作数画成两种不同的形状，并且忽略了叶结点中的指针域。

在后缀表达式中，运算符出现的次序决定着其运算的次序。在表达式树中，运算符对应的运算次序一定程度上由运算符所处的层数决定。比如，在从根到叶结点 3 的路径上，包含 4 个运算符，分别是-、*、+和*。在一条路径上，层数深的运算符早于层数浅的运算符进行计算。

以表达式 2*y*(a+3*y)-b 为例，介绍对应于中缀表达式的表达式树的创建过程。

① 根据运算符的优先级对表达式加括号，得到(((2*y)*(a+(3*y)))-b)。

② 去掉最外层括号，得到((2*y)*(a+(3*y)))-b，中间的运算符“-”为根，前后两部分分别是((2*y)*(a+(3*y)))和 b，分别对应于左子树和右子树。

③ 对左子树递归执行步骤②。

④ 对右子树递归执行步骤②。

⑤ 当遇到空串时，递归结束。

实际上，对表达式树进行中序遍历可得到不带括号的中缀表达式。进行先序遍历，可得到前缀表达式。进行后序遍历，可得到后缀表达式。

【例 5-12】 已知算术表达式的中缀形式为 A+B*C-D/E，后缀形式为 ABC*+DE/-，其前缀形式为（　）。

A. -A+B*C/DE　　B. -A+B*CD/E　　C. -+*ABC/DE　　D. -+A*BC/DE

答案为 D。

可以使用表达式的中缀形式与后缀形式还原表达式树，如图 5-19 所示，然后对表达式树进行先序遍历，即可得到表达式的前缀形式。

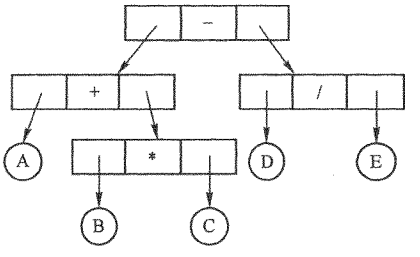


图 5-19　中缀形式为 A+B*C-D/E 的表达式树

第四节　堆及优先队列

堆是一种非常有用的结构，堆中的部分元素之间具有有序性。堆采用数组保存，同时具有层次的特性。借助于堆结构，还可以实现堆排序。

一、堆的定义及其基本操作

定义 5-3　一个数据序列 $a_0,a_1,\cdots,a_{n-1}$，当且仅当满足下列关系时，称之为堆：

$$\begin{cases} a_i \leqslant a_{2i+1} \\ a_i \leqslant a_{2i+2} \end{cases} \text{或} \begin{cases} a_i \geqslant a_{2i+1} \\ a_i \geqslant a_{2i+2} \end{cases} \quad (i=0,1,\cdots,\lfloor (n-2)/2 \rfloor) \tag{5-8}$$

将数据序列 $a_0, a_1, \cdots, a_{n-1}$ 依次保存在一维数组中，将该数组看作一棵二叉树的顺序存储结构，则它对应于一棵完全二叉树。假设堆中的元素是 $a_0, a_1, \cdots, a_7$，对应的完全二叉树如图 5-20 所示。

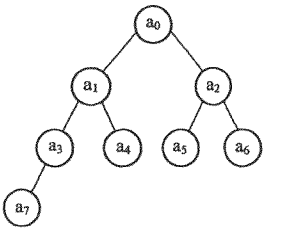


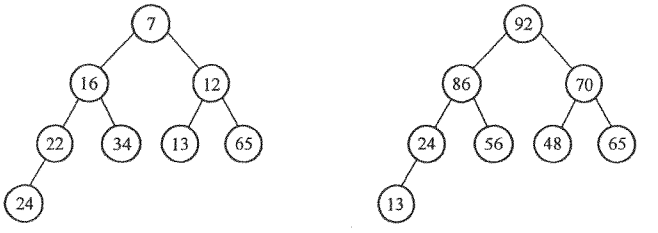
图 5-20　堆对应的完全二叉树

堆定义中要求的数据之间的大小关系，对应在完全二叉树中的直观含义是什么？

根据完全二叉树的定义，a_i 是 a_{2i+1} 和 a_{2i+2} 的父结点，a_{2i+1} 是左子结点，而 a_{2i+2} 是右子结点。堆中所规定的大小关系即二叉树中父、子结点之间的大小关系。式（5-8）中的前一种关系表明，任何一个分支结点的值都不大于它的子结点的值，所以树的根结点的值是全部结点中的最小值。这样的堆称为最小堆或小根堆。式（5-8）中的后一种关系表明，任何一个分支结点的值都不小于它的子结点的值，故树的根结点的值是全部结点中的最大值。这样的堆称为最大堆或大根堆。树的根结点称为堆顶。

堆的定义中还隐含了递归的含义，当用完全二叉树的形式表示堆时，树中的任意一棵子树都可以构成堆，并且保持与原来同样的性质，即最大堆中任何结点的子树仍是最大堆，最小堆中任何结点的子树仍是最小堆。

图 5-21 所示的两棵完全二叉树均满足堆的定义，图 5-21a 是最小堆，堆顶 7 是最小元素，其中以 16 为根的子树也是一个最小堆，16 是这个子堆中的最小元素。而图 5-21b 是最大堆，堆顶 92 是最大元素，其中以 70 为根的子树也是一个最大堆，70 是其中的最大元素。



a) 最小堆　　b) 最大堆

图 5-21　最小堆与最大堆

堆的有序性仅在部分元素之间保持，具体来说，具有祖先和后代关系的元素之间是有序的，即任何一个结点到堆顶的路径上的各结点之间是有序的，如果是最小堆，则是降序的，如果是最大堆，则是升序的。例如，图 5-21a 所示的最小堆中，从 34 到堆顶 7 的路径上含有的 3 个值 34、16、7 呈降序关系，而图 5-21b 所示的最大堆中，从 13 到堆顶 92 的路径上的 4 个值 13、24、86、92 是升序的。堆中各层元素之间未必保持有序性。例如，在图 5-21a 中，第 3 层的元素 24 小于第 2 层的元素 34。

堆的类型定义如下。

```
#define HeapSize 30                    //堆的容量
typedef int ELEMType;                  //元素类型
typedef struct Heap{
```

```
    ELEMType heap[HeapSize];      //存放元素的数组
    int n;                        //堆中当前元素个数
};
```

对于堆，除了有通常的判空、判满以外，基本操作主要有创建堆、删除堆顶元素、在堆中插入一个新元素等，定义如下。

```
//Heap(堆的基本操作)
int createHeap(maxHeap *H,ELEMType arr[], int n);  //创建堆
int insertHeap(maxHeap *H, ELEMType x);            //在堆中插入元素 x
int deleteHeap(maxHeap *H, ELEMType *x);           //删除堆顶元素
int isHeapEmpty(maxHeap *H);                       //如果堆为空，则返回 1，否则返回 0
int isHeapFull(maxHeap *H);                        //如果堆为满，则返回 1，否则返回 0
```

其中用到了调整操作，定义如下。

```
void ShiftDown(ELEMType *myarr,int i,int n);  //调整堆
```

以最大堆为例，实现堆上的基本操作，实现如下。

最大堆的类型定义如下。

```
#define HeapSize 30              //堆的容量
typedef int ELEMType;            //元素类型
typedef structmaxHeap{
    ELEMType heap[HeapSize]; //存放元素的数组
    int n;                       //堆中当前元素个数
};
```

先来实现基本的判空、判满操作。

在堆的定义中，堆中元素保存在一维数组 heap 中，同时将当前堆中元素的个数保存在变量 n 中。根据 n 的值，可判定堆是否为空或是否为满。

```
int isHeapEmpty(maxHeap *H)        //如果堆为空，则返回 1，否则返回 0
{
    if (H->n==0) return TRUE;
    else return FALSE;
}
int isHeapFull(maxHeap *H)         //如果堆为满，则返回 1，否则返回 0
{
    if (H->n==HeapSize) return TRUE;
    else return FALSE;
}
```

堆中的初始元素保存在一个数组中，将它创建为一个最大堆是一个逐渐变化的整理过程。

回忆一下，堆有一个重要的特性，即可以将堆看作一棵完全二叉树，而且任何一棵子树仍是堆。那么，将初始数据整理成堆的过程，可以利用递推的办法来实现。先从树的底层的叶结点开始进行整理，将子树整理成堆，逐层向上，将扩大后的子树再整理成堆，直到整棵树整理成堆为止。

叶结点本身已是堆，仅需要处理分支结点。根据完全二叉树的性质及二叉树顺序存储规则，数组后一半（下标从$(n-1)/2+1$到$n-1$）的数据都是叶结点，前一半（下标从0到$(n-1)/2$）的数据都是分支结点。对分支结点从后向前依次进行处理。先选择最后一个分支结点，将以该结点为根的子树整理成堆。然后处理这个结点的前一个结点，这个结点也是分支结点，将以它为根的子树整理成堆。接下来处理更前一个结点，依此类推，直到处理完树根为止。此时，整棵完全二叉树转化为堆。初始堆建立完毕。

那么，子树的具体整理过程是什么样的呢？先看第一步要整理的子树。一棵子树的高度为2，最多有3个结点。在这2或3个结点中，选出其中值最大者放到子树的根中。如果这个值原本就在根中，则只进行比较而不用进行数据交换。如果这个值在某个子结点中，则进行比较后还需要进行数据交换。这样，子树变为子堆。

再看所含结点多于3个的子树的堆的整理过程。元素整理成新堆，可以称为堆的整理，简称为整堆。假定子树的根是R，它的左、右两个子结点分别是Cl和Cr。整堆过程是自下而上进行的，当整理到以R为根的子树时，以Cl或Cr为根的子树已经满足堆的性质，它们已经是子堆了，即Cl是它这棵子树中的最大值，而Cr是它这棵子树中的最大值。要想将以R为根的子树整理成堆，堆顶元素必定出自R、Cl和Cr三者之中。和刚才的过程一样，选择R、Cl和Cr中的最大者，放到子树的根的位置。这也分两种情况。如果R是三者中最大者，则这个过程完毕。否则，将从Cl和Cr中选出的较大者与R进行交换，较大者作为子树的根，而R是下一级子树的根。在进行数据交换后，下一级子树的大小有序关系可能被破坏，这导致R在这棵子树中可能需要继续进行调整，也就是看R是不是仍比它的新子结点都大，只要小于它的子结点，就必须进行互换，直到R大于等于它所有的子结点，不再发生互换，或R到达叶结点，它没有子结点可以交换为止。

设初始数据保存在数组arr中，使用数组arr创建最大堆的实现如下。

```
void ShiftDown(ELEMType * arr,int i,int n)
{
    int child;
    for ( ; i<=(n-1)/2;i=child)
    { child=i*2+1;
      if ((child!=(n-1))&&(arr[child+1]>arr[child]))
          child++;
      if (arr[i]<arr[child])
          swap(&arr[i],&arr[child]);
    }
}
int createHeap(maxHeap *H,ELEMType arr[ ],int n) //创建最大堆
{
```

```
    int i;
    for (i=(n-1)/2;i>=0;i--)
        ShiftDown(arr,i,n);                    //调整为最大堆
    for (i=0;i<n;i++) H->heap[i]=arr[i];       //保存到堆结构中
    H->n=n;
    return TRUE;
}
```

在最大堆中插入新元素 x 时，先将 x 保存在堆中第一个空位置，然后将 x 向上调整到合适的位置。根据堆的有序性，x 与其父结点中的值进行比较，若父结点中的值较大，则插入完成；若父结点中的值较小，则互换两个元素，并继续比较 x 及其新的父结点。这个过程可能是一系列的，直到 x 小于其父结点中的值，或 x 已经调整到堆顶为止。直观地看，就是 x 沿到根的路径一路进行比较或交换，直到满足堆的特性为止。

```
int insertHeap(maxHeap *H,ELEMType x)      //在堆中插入元素 x
{   int curr;
    if (isHeapFull(H)==TRUE) return FALSE;
    H->heap[H->n++]=x;
    curr=H->n-1;
    while (curr>=0 && H->heap[curr]>H->heap[(curr-1)/2]){
        swap(&H->heap[curr],&H->heap[(curr-1)/2]);
        curr=(curr-1)/2;
    }
    return TRUE;
}
```

堆仅支持删除堆顶元素，其他位置的元素不允许删除。在删除堆顶元素后，使用堆中最后位置的元素来顶替堆顶。与插入新元素类似，这个元素可能破坏了堆的有序性，所以还需要进行调整。堆顶与其较大子结点的值相比较，若堆顶值较小，则堆顶与较大子结点的值互换，然后继续将这个结点与其较大子结点中的值相比较，并进行必要的交换。当结点中的值大于其较大子结点中的值，或结点已经是叶结点时，调整过程结束。相关实现如下。

```
int deleteHeap(maxHeap *H, ELEMType *x)//删除堆顶元素
{   if (isHeapEmpty(H)==TRUE) return FALSE;
    *x=H->heap[0];
    H->heap[0]=H->heap[H->n-1];
    H->n--;
    ShiftDown(H->heap,0,H->n);
    return TRUE;
}
```

上述系列算法进行简单的修改后，即可实现最小堆的基本操作。

【例 5-13】 建立最大堆示例。

给定数据：70,13,65,24,56,48,92,86，将其建成最大堆。

先将这些数据依次放到一棵完全二叉树中，如图 5-22a 所示。

排在最后的分支结点是 24，首先调整以它为根的子树。24 只有左子结点 86，且 86 大于 24，故交换两个值。得到的结果如图 5-22b 所示。

下一个要调整的分支结点是 65，以 65 为根的子树的高度也是 2，65 的较大的子结点是 92，92 大于根 65，于是互换两个元素。得到的结果如图 5-22c 所示。

接下来要调整的子树的根是 13，这棵子树的高度为 3。13 的较大的子结点是 86，86 大于 13，所以互换两个元素，得到的结果如图 5-22d 所示。这个调整还没有结束，因为以 13 为根的子树还没有满足堆的性质，继续进行调整，再判断 13 与它的子结点之间的大小关系是否满足堆的定义，不满足时仍需要进行交换，所以 13 与 24 互换，得到的结果如图 5-22e 所示。

接下来是对树根的调整了。根结点是 70，它的两个子结点都比它大，较大的一个是 92，所以互换 70 与 92。在 70 所处的新子树中，它是最大的，不需要继续调整。调整过程结束，得到的最大堆如图 5-22f 所示。

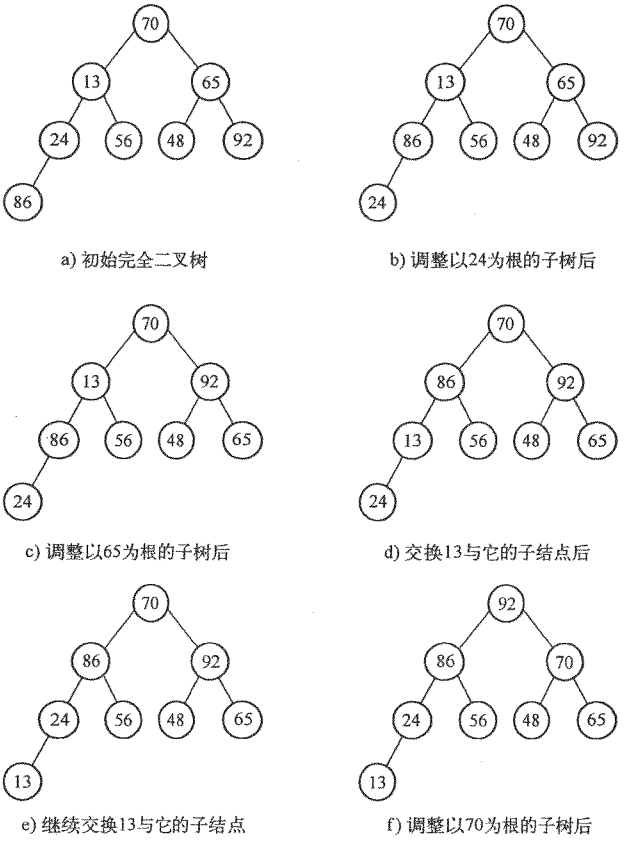
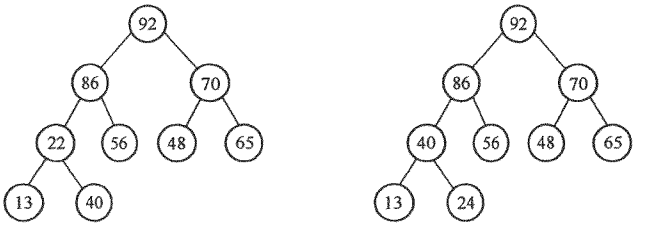


图 5-22　最大堆的建立过程

【例 5-14】 在最大堆中插入新元素示例。

在图 5-22f 所示的最大堆中，插入元素 40。

先将 40 放到完全二叉树的最后位置，如图 5-23a 所示。40 与其父结点相比较，因为 40 大于 24，所以两者互换。交换后，40 小于其父结点的值 86，不需要再交换。由此得到新的最大堆，如图 5-23b 所示。



a) 将元素40放到最后位置　　b) 40与24交换

图 5-23　在最大堆中插入新元素

实际上，对于同一组初始数据，调整过程不同，创建的最大堆可能也不同，但堆顶是唯一的。例如，对于例 5-13 中所给的初始数据，采用例 5-14 的方法，各值依次插入到初始为空的最大堆中，得到的结果如图 5-24 所示。

【例 5-15】 删除最大堆的堆顶示例。

在图 5-22f 所示的最大堆中，删除堆顶。

删除堆顶 92，此时原堆顶的空位置由数组中最后一个元素填充，结果如图 5-25a 所示。

新放置的元素 13 与其子结点进行比较，较大的子结点是 86，而 13<86，所以需要进行交换，结果如图 5-25b 所示。

将元素 13 与其子结点继续进行比较，两个子结点中的较大者是 56，而 13<56，继续交换，结果如图 5-25c 所示。

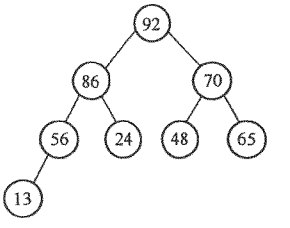
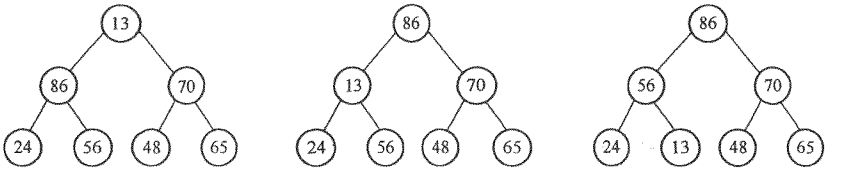


图 5-24　同一组数据得到的另一个最大堆



a) 删除堆顶且放置13　　b) 将13与86交换　　c) 将13与56交换

图 5-25　删除最大堆的堆顶后重新整堆的过程

此时，13 已经到达叶结点位置，不再进行调整。此时得到的结果即新的最大堆。

在堆中进行插入时，新元素与祖先结点进行比较及必要的交换，最多是调整到堆顶，即操作的步骤最多与树高成比例，所以时间复杂度为 $O(\log n)$。类似地，删除堆顶元素时，新补充的堆顶值也最多沿一条路径调整到叶结点，最多与树高成比例，所以时间复杂度也为 $O(\log n)$。

二、优先队列

队列具有先进先出的特性，这个特性是与时间相关的，可以看作时间优先的。在一些应用中，各元素本身还被赋予一个优先值，可以用一个非负整数表示优先值。优先值也称为优

先权。具有优先值的各元素保存在一个结构中，这个结构称为优先队列。输出时，选择最优先的元素输出。“最优先”取决于优先值的定义，可以是优先值最小，也可以是优先值最大。

优先队列不同于第三章中介绍的队列，可以使用堆作为优先队列的存储结构。将具有优先值的各元素按优先值建成最大堆，输出堆顶元素即得到优先队列中的最优先元素，优先队列中的剩余元素再重新整理成最大堆，等待下一次的输出。

优先队列中也可以随时插入新元素 x，借助于堆的插入操作，可以将 x 调整到优先队列中的合适位置。

第五节　树和森林

当不限定树中每个结点的孩子结点个数时，可以得到一般意义下的树。多棵树组成森林。本节讨论树的存储结构，并讨论树及森林与二叉树之间的转换。不特别指明的话，本节所讨论的树均为有序树。

一、树的存储结构

二叉树中每个结点的孩子结点的个数不超过两个，与此不同，树中每个结点的子结点个数可以是任意的。如何保存树将很有挑战性。

树的存储表示方式，既要方便于树的基本操作，即进行这些操作时不能太麻烦，导致需要很多的时间，又要考虑空间复杂度，即不要浪费过多的空间。

那么，树有哪些基本操作呢？与二叉树的基本操作类似，树也需要遍历其中的所有结点。另外，树中也需要插入或删除结点，以及寻找某个结点的父结点、孩子结点等。

在实际应用中，树的存储表示方法有很多种，下面介绍三种常用的表示法。

1. 父结点表示法

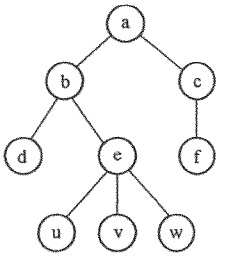
除根结点以外，树中每个结点都只有一个父结点，这个数目是确定的。树中每个结点都至少含两个域，一个域用来保存结点本身的值，另一个域用来保存结点之父结点的相关信息。树的根结点没有父结点，所以对应于父结点的这个域中可以保存一个特殊的值，表示没有父结点。还可以在根结点的这个域中保存树中的结点个数，为了与实际的父结点信息相区别，域中保存的是这个数的相反数。这样的表示法称为父结点表示法，有时也称为父指针表示法或双亲表示法。

父结点表示法可以使用顺序存储方式，即使用一个一维数组保存一棵树，数组的每个单元都对应于树中的一个结点，它含有两个域，其中 data 域用来保存结点的相关信息，parent 域保存父结点在数组中的下标，根结点的 parent 域为负数，表示没有父结点。对于图 5-26a 所示的树 T，其对应的顺序存储结构如图 5-26b 所示。

在这种表示法中，求某个结点的父结点是非常容易的。例如，现在要求结点 e 的父结点。在图 5-26b 所示的数组中，找到 data 域为 e 的元素，它的 parent 域的值为 1，则数组下标为 1 的位置保存的就是它的父结点，即父结点为 b。

求树的根也很方便。数组中 parent 域的值为负数的元素即树的根结点。

在父结点表示法中，查找某个结点的所有子结点要麻烦一些，这通常需要遍历数组中的



a) 一棵树T

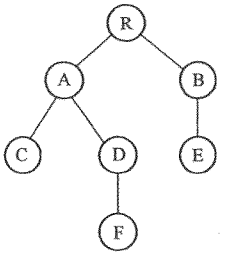
下标	0	1	2	3	4	5	6	7	8
data	a	b	c	d	e	f	u	v	w
parent	-9	0	0	1	1	2	4	4	4

b) 使用父结点表示法表示的树T

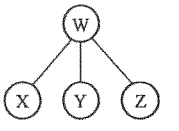
图 5-26　树 T 及其父结点表示

所有元素。例如，要查找结点 e 的所有子结点，先在数组中查找结点 e 所在的位置，得知它存储在下标为 4 的地方，然后在数组中从前向后查找，找出 parent 域的值等于 4 的所有元素，它们都是 e 的子结点，即 e 的子结点有 3 个，分别是 u、v 和 w。

使用一个数组也可以表示多棵树。如图 5-27a 所示的树 T_1 和图 5-27b 所示的树 T_2，采用父结点表示法表示时，如图 5-27c 所示。



a) 树T_1



b) 树T_2

下标	0	1	2	3	4	5	6	7	8	9	10
data	R	A	B	C	D	E	F	W	X	Y	Z
parent	-7	0	0	1	1	2	4	-4	7	7	7

c) 使用父结点表示法表示的树T_1和T_2

图 5-27　两棵树及其父结点表示

2. 孩子结点表示法

父结点表示法中保存的是父结点的相关信息，不方便寻找孩子结点的信息，为此提出孩子结点表示法，其中保存的是孩子结点的相关信息。

在二叉树中，每个结点的孩子结点不会多于两个，所以二叉链表中只需要为孩子结点预留两个指针域。但对于树来说，树中每个结点的孩子结点的个数是任意的，当不能确定具体的个数时，就不能像二叉链表那样，为每个结点预留保存指向其孩子结点指针的域。

如果能确定树中孩子结点的最大个数，如 m，则可以构造一个 m 叉链表来保存树。每个结点，除保存自身信息以外，还有 m 个指针域，分别指向其孩子结点。如果一个结点的孩子结点个数是 k，则结点中有 k 个非空指针域和 $m-k$ 个空指针域。

在使用 m 叉链表保存含 n 个结点的树时，树中有 $m \times n$ 个指针域，其中非空指针域有 $n-1$ 个，空指针域有 $m \times n-n+1$ 个。当 n 确定时，m 越大，空指针域的个数越多。

还可以使用链表来保存结点的所有孩子结点。每个结点的全部孩子结点构成一个单链表，若树中含有 n 个结点，则会得到 n 个单链表。为方便操作，将这些单链表的头指针保存在一个一维数组中，数组的元素含有两个域，一个域 data 是结点的信息，另一个域 link 是指针。

仍以图 5-26a 所示的树 T 为例，采用孩子结点表示法来表示它。

先将树中所有的结点保存在一维数组中，每个结点都对应唯一的下标。例如，结点 a、b、c 的下标分别是 0、1、2。

在树中，根结点 a 有两个孩子结点，分别是结点 b 和 c。创建含两个结点的单链表，链表结点中的数据部分分别是 1 和 2，也就是结点 b 和 c 在数组中的下标。在数组下标 0 的单元中，link 域保存指向这个单链表表头的指针。

其他单链表的创建情况与此类似。例如，结点 d 和 e 是结点 b 的孩子结点，创建一个含两个结点的单链表，表结点中的数据分别是 3 和 4（分别代表结点 d 和 e），头指针保存在下标为 1 的 link 域中。又如，结点 d 是叶结点，没有孩子结点，所以下标 3 的 link 域中保存一个空指针，表示这个结点没有孩子结点。孩子结点表示法的结果如图 5-28 所示。

从图 5-28 可以看出，结点的孩子结点数目不同，所对应的链表长度也不同。孩子结点个数多，则链表也长。如果没有孩子结点，则数组中相应的位置保存一个空指针，代表一个空链表。

在孩子结点表示法中，可以很方便地查找某个结点的所有孩子结点，但查找某个结点的父结点就不太方便了。当在某个应用中，既需要访问结点的孩子结点，也需要访问父结点时，可以将这两种表示法结合起来，即在数组中添加父结点的信息，形成父结点-孩子结点表示法，从而实现所需的功能。仍以图 5-26a 所示的树 T 为例，其父结点-孩子结点表示法的结果如图 5-29 表示。

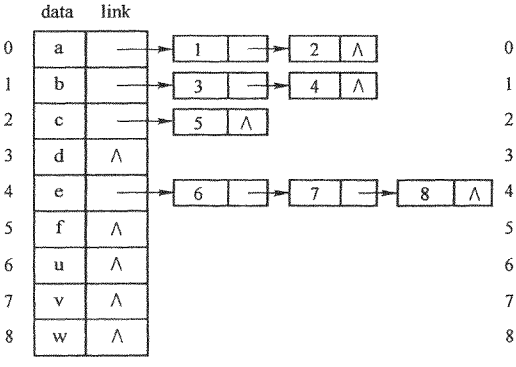


图 5-28　使用孩子结点表示法表示树 T

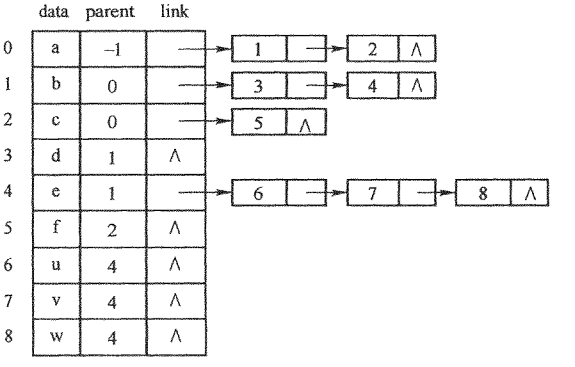


图 5-29　使用父结点-孩子结点表示法表示树 T

3. 孩子-兄弟表示法

树中每个结点的孩子结点个数不定，给树的表示带来了一定的难度。但树中也有个数固定的信息。例如，每个结点的第一个孩子结点（如果有的话）只能有一个，每个结点的下一个兄弟结点（如果有的话）也只能有一个。可以借用这些信息来表示树，使用的是与二叉链表类似的一种结构。

为树中的每个结点定义一个存储结点，其中有左、右两个指针域，左指针域指向这个结点的第一个孩子结点，右指针域指向这个结点的下一个兄弟结点。由于每个结点都含有两个指针域，故它很像是用来表示二叉树的二叉链表结构。树的这种表示法又称为二叉链表表示法，即以二叉链表的形式作为树的存储结构，这个二叉链表也称为孩子-兄弟链表。

虽然它的结构类似于二叉链表的结构，但表中两个指针域的含义与二叉链表中两个指针域的含义是不相同的。另外，树中各个孩子结点是没有次序的，为了方便起见，在使用这种方式存储树时，将孩子结点排好次序，人为规定哪个孩子结点是第一个，哪个孩子结点是第二个，依此类推。因为规定了孩子结点的次序，所以任何结点的下一个兄弟结点也不会再有歧义了。

如图 5-26a 所示的树 T，它的孩子-兄弟表示法如图 5-30 所示。

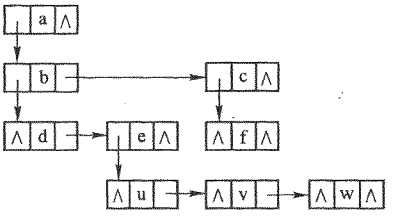


图 5-30　使用孩子-兄弟表示法表示树 T

按照图 5-30 所示的次序，根结点 a 有两个孩子结点，规定结点 b 是它的第一个孩子结点，则结点 c 为结点 b 的下一个兄弟结点。

因为根结点是没有兄弟结点的，所以根结点的右指针域总是空的。在这种表示法中，求任一结点的所有孩子结点也很方便。首先从该结点的左指针域中找到它的第一个孩子结点，然后从第一个孩子结点的右指针域中找到第二个孩子结点，再从第二个孩子结点的右指针域中找到第三个孩子结点，依此类推，直到某孩子结点的右指针域为空时停止，这表明它没有下一个兄弟结点了，由此找到了全部孩子结点。例如，从图 5-30 中可以查找结点 e 的 3 个孩子结点，从结点 e 的左指针找到结点 u，从结点 u 的右指针找到结点 v，从结点 v 的右指针找到结点 w，结点 w 的右指针为空，表示查找结束。

想象一下，将图 5-30 沿顺时针方向旋转 45°，得到的图很像二叉链表的形式，故此得名。

二、树、森林与二叉树的转换

前面已经看到，树和二叉树都可以用二叉链表作为存储结构。同一个二叉链表，若按树的存储含义来解释，则可以还原为树；若按二叉树的存储含义来解释，则可以还原为二叉树。正是因为这一点，可以在树与二叉树之间建立一种对应关系，这种对应关系的基础就是二叉链表。

首先用树的孩子-兄弟表示法将树表示为一个二叉链表，然后将这个二叉链表解释成一棵二叉树，这就建立了树与二叉树的对应关系。例如，对于图 5-30 所示的二叉链表，可以解释为图 5-31 所示的二叉树，它是与图 5-26a 所示树对应的二叉树。

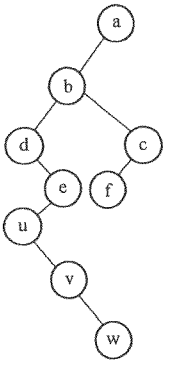


图 5-31　对应于图 5-26a 所示树 T 的二叉树

从图 5-31 中可以看出，一棵树所对应的二叉树的根结点没有右子树。某个结点的所有孩子结点在树中处在同一层，但在对应的二叉树表示中将分属于相邻的不同层中，且越靠右的孩子结点所处的层越深。

多棵树组成的森林也可以转换为一棵二叉树。第一棵树的转换规则与单棵树的转换规则

相同，第二棵树的根被看作第一棵树的根结点的右兄弟结点，第三棵树的根被看作第二棵树的根结点的右兄弟结点，依此类推。

可以得到下列森林（包括单棵的树）与二叉树的转换对应规则。

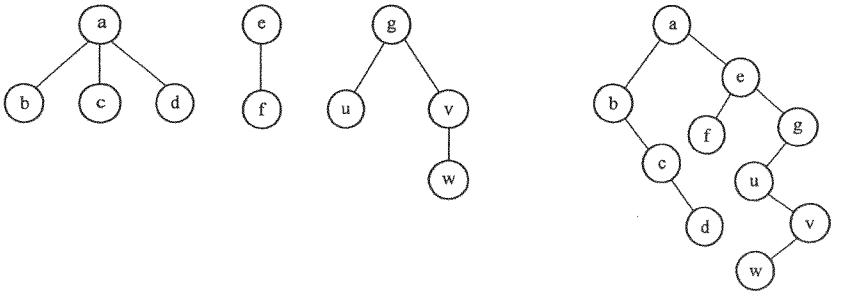
规则 1：将森林转换成二叉树。

设 $F=\{T_1,T_2,\cdots,T_m\}$ 是森林，可按如下规则将森林 F 转换为一棵二叉树 $B=(root,B_L,B_R)$。

（1）若 F 为空（$m=0$），则 B 为空树。

（2）若 F 非空（$m>0$），则森林中 T_1 的根作为二叉树 B 的根 root；T_1 中各子树组成的森林 $F_1=\{T_{11},T_{12},\cdots,T_{1s}\}$ 转换成的二叉树作为 B 的左子树 B_L；森林 $F'=\{T_2,T_3,\cdots,T_m\}$ 转换成的二叉树作为 B 的右子树 B_R。

这个规则也是递归的，为了便于理解，使用一个具体例子来解释。森林及其转换结果如图 5-32 所示。



a) 三棵树组成的森林F　　b) 对应于森林F的二叉树

图 5-32　森林与二叉树的转换示例

根据规则 1，F 的第一棵树的根结点 a 为二叉树的根结点，a 的三棵子树组成的森林为 $F_1=\{\{b\},\{c\},\{d\}\}$，将 F_1 按同样的转换规则转换成 a 的左子树。第一棵树的根结点 b 成为子树的根结点，其余两棵树成为它的右子树。所以 c 是 b 的右孩子结点，d 是 c 的右孩子结点。第一棵树转换完毕。

接下来看森林中另外两棵树。包含剩余这两棵树的森林也按同样的规则进行转换，它们组成的二叉树成为 a 的右子树。第一棵树的根结点是 e，它成为 a 的右孩子结点。而 e 的各子树（实际上只有一棵）转换后得到的二叉树（结点 f 构成的单结点二叉树）是 e 的左子树。以 g 为根的树转换后得到的二叉树是 e 的右子树。按照这个过程继续下去，各森林中所含树的棵数越来越少，为空时就可以结束转换过程。

规则 2：将二叉树还原为森林。

设 $B=(root,B_L,B_R)$ 是一棵二叉树，则可按如下规则转换成森林 $F=\{T_1,T_2,\cdots,T_m\}$。

（1）若 B 为空，则 F 为空。

（2）若 B 非空，则 B 的根 root 作为 F 中第一棵树 T_1 的根；B 的左子树 B_L 还原得到的森林作为 T_1 的各子树；B 的右子树 B_R 还原得到的森林作为 F 中的 $T_2,T_3,\cdots,T_m$。

按照规则 2，分析将图 5-32b 所示的二叉树还原为图 5-32a 所示的森林的过程。首先，将二叉树的根结点 a 作为第一棵树 T_1 的根结点；然后，将 a 的左子树按同样规则转换为 T_1

的三棵子树 {b}、{c} 和 {d}；最后，将根结点 a 的右子树按照同样的规则还原为森林的另外两棵树 T_2 和 T_3。

可以断言，按规则 1 将森林转换成的二叉树，一定能按规则 2 还原为原来的森林。有了这种规律作为保证，就可以将对森林（包括单棵树）的操作转变为在转换后的二叉树上的操作，必要时，再将此二叉树还原为森林。

【例 5-16】 已知由 4 棵树组成的森林 F，各棵树中所含的结点个数依次是 2、3、4、5，则在 F 对应的二叉树 T 中，根的右子树中结点的个数是（　）。

A. 4　　B. 12　　C. 13　　D. 14

答案为 B。

根据转换规则，森林 F 中后 3 棵树转换为 T 的右子树，相应的结点个数=3+4+5=12。

【例 5-17】 给出树 T，如图 5-33 所示，试画出其对应的二叉树 BT，并分别给出二叉树 BT 的先序、中序和后序遍历序列。

将树 T 转换为二叉树 BT，如图 5-34 所示。

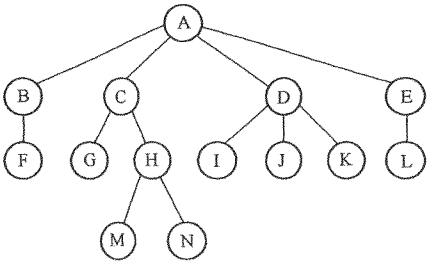


图 5-33　树 T

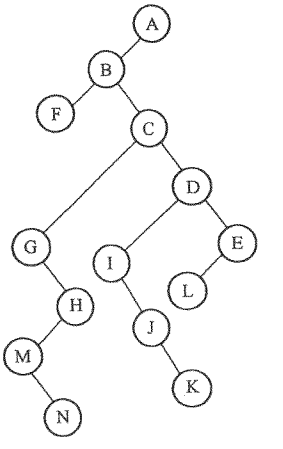


图 5-34　对应于树 T 的二叉树 BT

得到的二叉树 BT 的先序遍历序列是 A,B,F,C,G,H,M,N,D,I,J,K,E,L，中序遍历序列是 F,B,G,M,N,H,C,I,J,K,D,L,E,A，后序遍历序列是 F,N,M,H,G,K,J,I,L,E,D,C,B,A。

实际上，树和森林的遍历序列与其对应的二叉树的遍历序列是有一定关系的。

三、树和森林的遍历

在有些应用中，需要对树或森林中的所有结点进行处理，这时需要对树或森林进行遍历。森林是由一棵棵树组成的，对森林的遍历过程即按照某种策略依次对各棵树进行遍历的过程。它们的遍历策略只存在数量上的差别。先讨论树的遍历策略。

与二叉树的先序、中序和后序遍历策略类似，树的遍历策略有两种：一是先序遍历，也称为先根遍历或前序遍历；二是后序遍历，也称为后根遍历。

（1）先序（根）遍历：先访问树的根结点，再依次先序遍历根结点的各棵子树。

（2）后序（根）遍历：若树的根结点有子树，则首先后序遍历各棵子树，然后访问根

结点；否则（根结点无子树），只访问根结点。

在先序遍历中，根结点在各棵子树遍历之前被遍历。在后序遍历中，根结点在各棵子树遍历之后被遍历。

【例 5-18】 对图 5-33 所示的树 T 进行先序遍历和后序遍历，并给出遍历结果。

在先序遍历树时，先输出根结点 A，然后输出 A 的 4 棵子树的先序遍历序列。

遍历第一棵子树：得到根结点 B 及 B 的子树 F，结果是 B,F。

遍历第二棵子树：得到根结点 C、子树 1 的结果和子树 2 的结果。子树 1 的结果是 G。在子树 2 中，得到根结点 H，以及 H 的子树 1 的结果 M 和子树 2 的结果 N。得到结果：C,G,H,M,N。

遍历第三棵子树：得到根结点 D 和三棵子树的结果，每棵子树都只含一个结点。结果是 D,I,J,K。

第四棵子树很简单，遍历结果是 E,L。

综上所述，树 T 的先序遍历结果是：A,B,F,C,G,H,M,N,D,I,J,K,E,L。

结合例 5-17 的结果可知，树 T 的先序遍历结果与对应的二叉树 BT 的先序遍历序列相同。

再看 T 的后序遍历。先后序遍历 A 的 4 棵子树，最后访问 A。

遍历第一棵子树：先遍历其子树，再遍历根结点 B。结果是 F，B。

遍历第二棵子树：先遍历其两棵子树，再遍历根结点 C。遍历 C 的第一棵子树，得到 G。遍历 C 的第二棵子树，得到 M,N,H。结果是 G,M,N,H,C。

遍历第三棵子树：先遍历其三棵子树，再遍历根结点 D。由于其子树都是单结点树，故得到 I,J,K,D。

遍历第四棵子树：得到 L,E。

综上，T 的后序遍历结果是 F,B,G,M,N,H,C,I,J,K,D,L,E,A。树 T 的后序遍历结果与对应的二叉树 BT 的中序遍历序列相同。

将树的遍历策略应用于森林，可以得到森林的遍历策略。基于树的遍历策略，森林的遍历策略也有两种，一是先序遍历森林，二是后序遍历森林。

森林的先序遍历过程是，按照树的排列次序，先序遍历各棵树。森林的后序遍历过程是，按照树的排列次序，后序遍历各棵树。

可以看出，在森林的遍历序列中，各棵树中的结点不会混杂在一起。

【例 5-19】 分别对图 5-32a 所示的森林 F 进行先序遍历和后序遍历，并给出遍历结果。

先看先序遍历。森林 F 共有 3 棵树，分别先序遍历 3 棵树的结果即对森林 F 的先序遍历结果。

按照树的先序遍历策略，森林 F 的第一棵树的先序遍历结果是 a,b,c,d，第二棵树的先序遍历结果是 e,f，第三棵树的先序遍历结果是 g,u,v,w。由此得到对森林 F 进行先序遍历的结果：a,b,c,d,e,f,g,u,v,w。

再看后序遍历。按照树的后序遍历策略，森林 F 的第一棵树的后序遍历结果是 b,c,d,a，第二棵树的后序遍历结果是 f,e，第三棵树的后序遍历结果是 u,w,v,g。由此得到对森林 F 进行后序遍历的结果：b,c,d,a,f,e,u,w,v,g。

可以验证，图 5-32b 所示的二叉树的先序遍历序列和中序遍历序列与本例中得到的两个遍历序列分别相同。

设森林 F 对应的二叉树为 T，而 F 的后序遍历结果与 T 的中序遍历结果相同，所以，在有些教材中，也将森林的后序遍历称为树的中序遍历。

第六节　哈夫曼树及哈夫曼编码

本节介绍哈夫曼树及哈夫曼编码，基于这种编码方法可以实现简单的文件压缩算法。

一、编码

使用二叉树不仅可以表示一个表达式，还可以表示字符集中各字符的编码。

日常使用的字符，包括自然语言中的字符和程序设计时用到的字符，在计算机内部需要表示为二进制串，所以需要在字符与二进制串之间建立一一对应的关系，即对字符进行编码。编码是信息从一种形式或格式转换为另一种形式或格式的过程。转换前的文本称为原文，转换后的文本称为译文。字符编码有不同的体系，比如，ASCII 编码和 Unicode 编码都是计算机内使用的编码体系。日常生活中也会使用编码，比如电报码。具体来说，ASCII 编码的字符集包括英文字母、数字、标点符号和一些控制字符，使用 8 位二进制表示一个符号。Unicode 编码的字符集不但包含 ASCII 编码的字符集，而且包含一些自然语言使用的字符，如汉字等，使用 16 位二进制表示一个字符。电报码使用 4 位十进制数表示一个汉字。这些都属于定长编码，也就是所有字符的编码长度都相等。表示编码的数字可以是二进制的，也可以是十进制的。

定长编码在处理时算法简单，效率高，在编码长度确定之后，译文的长度完全取决于原文的长度。比如，当用 m 位二进制表示一个字符时，若原文含有 n 个字符，则译文的二进制位数是 $n \times m$。在信息的存储和传输过程中，为追求高效率，人们希望表示信息的译文总长度越短越好。定长编码机制显然做不到这一点。

自然语言中各字符出现的频度是不一样的，可以借助这个特性来设计不等长编码方案，即变长编码方案，目的是得到尽可能短的译文。在译文缩短后，后续的操作将更有效率。采用不等长编码的原则是使要处理的文本中出现次数较多的字符采用较短的编码。

以二进制编码为例。可以这样设计编码方案，将字符集中的字符按其使用频率从高到低进行排序，将编码按其长度从短到长进行排序，并将这两个序列进行一一对应。

不失一般性，以含 6 个字符的字符集 S={a,b,c,d,e,f} 为例，各字符的使用频率从高到低排列，给出长度最短的 6 个二进制编码：{0,1,00,01,10,11}。字符的使用频率定义为该字符的权值。字符集与编码集中的元素一一对应，得到编码方案。这个方案看似可行，但译码时会存在二义性。比如，对应于原文 abcdd 的译文是 01000101。根据编码方案，这个结果是确定的。但给定译文 01000101，能够还原的原文却不是确定的。除 abcdd 以外，daaadd、dcdd 也是对应于 01000101 的原文。也就是说，同一个译文会对应于多个原文。这样的编码方案在实际中是不可用的。一个编码方案能够实用，必须满足前缀特性。

在一个编码方案中，任何一个字符的编码都不是该方案中任何其他字符编码的前缀，这样的编码称为具有前缀特性。正是因为编码方案具有前缀特性，才能够保证译码过程的正确性和唯一性。

可以验证，前面给出的字符集 S 的编码方案不具有前缀特性，字符 a 的编码 0 是字符 c

和 d 的前缀。所以，在译文中遇到 0 时，既可以将它还原为字符 a，也可以将它和后面遇到的 0 一起还原为字符 c，或和后面遇到的 1 一起还原为字符 d。从而产生了二义性。

可以使用二叉树来表示一个编码方案。在二叉树的分支上标注 0 或 1，从根结点到某结点的路径上，各分支标注的 0 或 1 组成一个编码。通常，左分支标注 0，右分支标注 1；也可以反过来。以字符集 S 及其编码为例，对应的二叉树如图 5-35 所示。

从图 5-35 可以看出，从根结点到某结点的路径上所标注的 0、1 串正好对应于该结点的编码。比如，从根结点到结点 a 的路径上所标注的 0 代表字符 a 的编码，从根结点到结点 d 的路径上所标注的 01 代表字符 d 的编码。

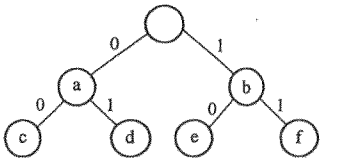


图 5-35　对应于集合 S 的编码树

在使用二叉树表示编码方案时，很容易验证编码方案是否具有前缀特性。图 5-35 所示的编码树所代表的编码方案显然不满足前缀特性，因为从根结点到叶结点的路径上，遇到了多于一个的字符。由此可知，如果一个编码方案满足前缀特性，那么在对应的编码树中，字符只能出现在叶结点中。这个特性很容易判断。

在二叉树中，层越高，编码越短。这表明，如果字符 1 出现的频度比字符 2 高，则在编码树中，字符 1 的层数不应大于字符 2 的层数。这为设计编码方案提供了一个思路。

二、哈夫曼树

在给定字符集及各字符的权值后，如何设计具有前缀特性的编码方案呢？哈夫曼给出一种对字符集进行编码的一般性规则。首先构造一棵哈夫曼（Huffman）树，然后给出各字符的哈夫曼编码。哈夫曼编码不仅满足前缀特性，还能保证在满足前缀特性的前提下，得到的译文是最短的。

在二叉树中，叶结点的带权路径长度定义为叶结点的权值和其到根结点的路径长度之积。二叉树的外部带权路径长度就是树中所有叶结点的带权路径长度之和。二叉树的外部带权路径长度记为 WPL，有：

$$\mathrm{WPL} = \sum_{k=1}^{n} w_k l_k \tag{5-9}$$

其中，n 为叶结点个数，w_k 为第 k 个叶结点的权值，l_k 为第 k 个叶结点的路径长度。

可以证明，哈夫曼树的 WPL 是最小的，即哈夫曼树是带权路径长度最短的树，因此它又称为最优二叉树。相关的证明可参考其他书籍。

【例 5-20】 计算图 5-36 中 4 棵二叉树的 WPL。其中，叶结点左侧标注的是其权值。

先看图 5-36a 所示的二叉树，其 WPL=30×1+25×2+12×3+6×3=134。其他 3 棵树的计算过程类似。4 棵二叉树的 WPL 分别为 134、195、139 和 146。图 5-36a 所示二叉树的 WPL 最小。可以验证，这是使用这个权值集合能够构造的具有最小 WPL 的二叉树。

仔细观察图 5-36 中的 4 棵二叉树，树中叶结点的权值集合是相同的。也就是说，如果有一个含 4 个字符的字符集，各字符出现的频度分别是 6、12、25、30，那么可以构造不同的均满足前缀特性的编码树，各编码树的 WPL 可能不同。现在的任务是构造具有最小 WPL 且满足前缀特性的编码树。

设有 n 个权值 $\{w_1, w_2, \cdots, w_n\}$，构造具有 n 个叶结点的二叉树，每个叶结点带有一个权

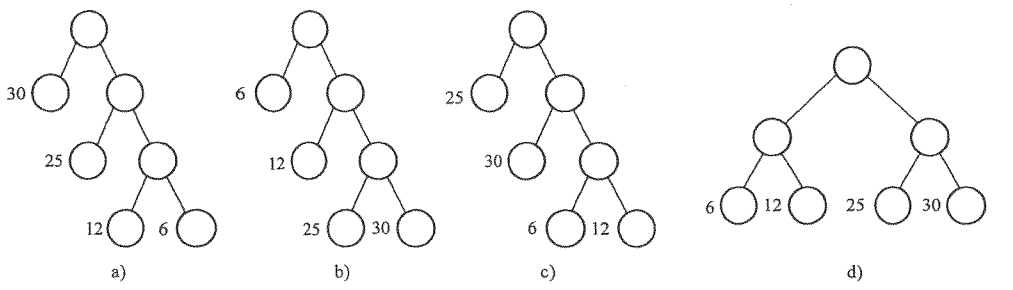


图 5-36　使用相同的权值集合构造的 4 棵二叉树

值 w_i。在所有这样的二叉树中，WPL 最小的一棵二叉树称为哈夫曼树。图 5-36a 所示二叉树就是一棵哈夫曼树。

构造哈夫曼树的算法是由哈夫曼首先提出的，所以又称为哈夫曼算法。算法步骤如下。

（1）根据给定的 n 个权值$\{w_1, w_2, \cdots, w_n\}$（$n \geqslant 2$），构造含 n 棵二叉树的集合 $F=\{T_1, T_2, \cdots, T_n\}$，其中每棵二叉树 T_i（$1 \leqslant i \leqslant n$）只有一个根结点，它带有权值 w_i（$i=1,2,\cdots,n$）。

（2）在 F 中选出两棵根结点权值最小（如果有相等的权值，则可任意选择）的二叉树，将它们分别作为左、右子树，新增加一个根结点，从而构造一棵新的二叉树，新二叉树中根结点的权值为其左、右子树的根结点的权值之和。然后从 F 中删去选出的这两棵树，加入这棵新构造的树。

（3）重复步骤（2），直到 F 中只有一棵树为止，这棵树就是哈夫曼树。

在构造哈夫曼树，选出的具有最小权值的两棵二叉树组成一棵更大的二叉树时，谁是左子树、谁是右子树可以是随意的。这意味着，对于同一个权值集，哈夫曼算法构造的哈夫曼树不是唯一的。虽然树形不同，但它们的 WPL 相同。在选择树时，从权值相等的两棵树中选择一棵，也可以随意，选择的结果也不影响树的 WPL。

【例 5-21】 已知字符集 $S=\{a,b,c,d,e\}$，对应的权值为$\{17,25,12,6,14\}$，试构造哈夫曼树，并计算其 WPL。

在叶结点中，将所给权值列在结点左侧；在分支结点中，将权值写在结点内。

初始时，5 个权值分别对应 5 棵单结点的树，并按权值从小到大排序，叶结点表示为方形结点，如图 5-37 所示。

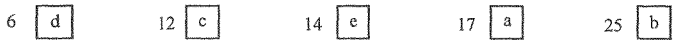


图 5-37　构造哈夫曼树的初始步骤

选择权值最小和次小的两棵树进行合并，新增加的根结点表示为圆形结点。新树的根结点的权值是 18，选择权值小的树作为左子树，权值大的树作为右子树。当然，左子树和右子树可以互换。将选中的两棵树删除，添加新合并的树，仍按权值从小到大排序。合并两棵树后的树的集合如图 5-38 所示。

继续选择权值最小和次小的两棵树进行合并。新树的根结点的权值是 31。将选中的两棵树删除，增加新合并的树。又合并两棵树后的树的集合如图 5-39 所示。

现在，还剩余 3 棵树。继续这个过程。最后两步合并的结果分别如图 5-40 和图 5-41 所示。

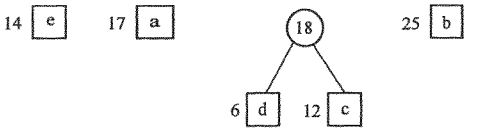


图 5-38　合并两棵树后的树的集合

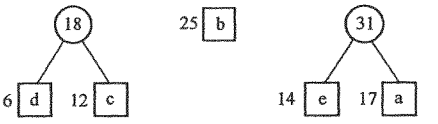


图 5-39　又合并两棵树后的树的集合

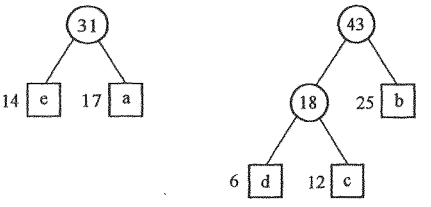


图 5-40　再合并两棵树后的树的集合

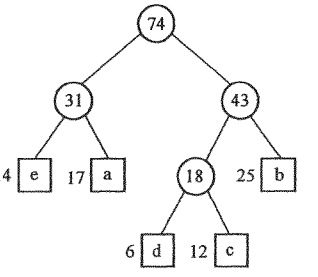


图 5-41　得到的哈夫曼树

WPL=14×2+17×2+6×3+12×3+25×2=166。

可以看到，由哈夫曼算法得到的哈夫曼树中没有度为 1 的结点，这是因为每次合并时都选择了两棵树分别作为左子树和右子树。初始时，如果给定的权值个数为 n，则哈夫曼树的构建过程需要 $n-1$ 步。在得到的哈夫曼树中，叶结点个数为 n，分支结点个数为 $n-1$，结点总数为 $2n-1$。

可以验证，哈夫曼树任一分支结点的左子树和右子树进行交换并不会改变哈夫曼树的 WPL 的值，即对相同权值集合构造的不同哈夫曼树的 WPL 是相同的。

三、哈夫曼编码

在哈夫曼树中，对于所有的分支结点，其左分支标为 0，右分支标为 1。在这样标记以后，将从根结点到叶结点的路径上标记的 0 和 1 依次收集起来，可以得到叶结点对应字符的具体编码，这就是哈夫曼编码。

【例 5-22】 求出例 5-21 所给字符集 S 中各字符的编码。

根据字符集 S，得到图 5-41 所示的哈夫曼树，在哈夫曼树中添加 0、1 标记，如图 5-42 所示。

字符集 S 中各字符的编码见表 5-1。

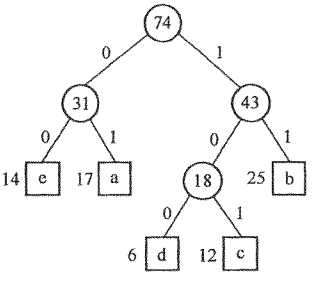


图 5-42　加标记后的哈夫曼树

表 5-1　字符集 S 中各字符的哈夫曼编码

字　符	a	b	c	d	e
哈夫曼编码	01	11	101	100	00

哈夫曼树的结果不是唯一的，甚至对同一棵哈夫曼树，将左分支标记为 1、右分支标记为 0 也是合理的，所以哈夫曼编码也不是唯一的。对于图 5-42 所示的哈夫曼树，如果将 0、1 互换，则得到字符集 S 中各字符的另一种哈夫曼编码，见表 5-2。

表 5-2 字符集 S 中各字符的另一种哈夫曼编码

字　符	a	b	c	d	e
哈夫曼编码	10	00	010	011	11

哈夫曼树的 WPL 及根结点中的权值有什么实际含义呢?

以字符集 S 为例，各字符的权值是其出现的频度。当这个权值解释为字符出现的次数时，即在原文中，a、b、c、d、e 分别出现了 17、25、12、6、14 次，则原文中总的字符数=17+25+12+6+14=74，这即根结点中的权值。

原文总字符数是 74，那么译文的总位数是多少呢? 表示字符 a 时使用 2 位，a 出现了 17 次，即译文中需要使用 17×2=34 位 0 或 1 来表示全部的 a。34 即叶结点 a 的带权路径长度。如果将原文中的 74 个符号（5 种字符）表示为 0、1 串，则需要 17×2+25×2+12×3+6×3+14×2=166 位，这即哈夫曼树的 WPL 的值。

编码的带权平均码长=译文总位数/原文字符总数=166/74 ≈2.24。

通常来讲，当字符集中各字符的权值差异较大时，使用变长编码得到的带权平均码长会比使用定长编码得到的码长短。但这个结论并不总是成立的，比如，对于只含有两个字符的字符集 S，无论是定长编码还是变长编码，平均码长都是 1。

在哈夫曼树上进行译码很简单。译码的策略是：从前向后逐位扫描编码的每一位，根据编码的值，找到从哈夫曼树的根结点到叶结点的路径。从根开始，如果编码的当前位是 0，则沿左分支到下一层，否则沿右分支到下一层。继续看编码的下一位，并决定下一步的走向。当到达树中的叶结点时，叶结点中所保存的数据值就是对应于刚刚扫描过的这几位编码的字符。继续这个过程，扫描剩余编码位，根据各编码位的情况，寻找从根结点到叶结点的一条路径，并输出叶结点中保存的相应字符，完成又一个字符的译码。当全部的二进制编码扫描完毕时，译码过程即告结束。

【例 5-23】 对于表 5-1 中给定的字符集编码方案，已知译文是 110010101111000011，其原文是什么?

从哈夫曼树的根结点开始，根据编码的各位决定选择的分支，前两位是 11，故得到字符 b。接下来，00 对应于字符 e。依此类推，最终得到的原文是 becabdeb。

哈夫曼树是一棵特殊的二叉树，其中没有度为 1 的结点。在扫描二进制编码时，无论遇到的是 0 还是 1，都可以保证从当前结点进入到下一层，除非到达了叶结点。如果到达叶结点，则表明完成了一个字符的译码。

内 容 小 结

树是一种层次结构，适合用来表示具有层次关系的数据。本章介绍了树的基本概念及常用的几种表示方法，着重介绍了二叉树的概念、性质及两种不同的实现方式，针对二叉树的链式存储方式，给出了二叉树遍历算法的递归实现及非递归实现。

本章还介绍了堆和优先队列的概念，实现了堆的基本操作。

在此基础上，本章介绍了树、森林与二叉树之间的关系，给出了树、森林与二叉树进行相互转换的规则，介绍了树、森林的遍历算法。本章最后介绍了二叉树的一个重要应用实例，即哈夫曼树及哈夫曼编码。

习　题

一、单项选择题

1. 下列关于二叉树的叙述中，正确的是_____。

A. 二叉树的度为 2　　B. 二叉树的度可以小于 2

C. 二叉树中至少有一个结点的度为 2　　D. 二叉树中任何一个结点的度都为 2

2. 下列存储形式中，不是树的存储形式的是_____。

A. 父结点表示法　　B. 孩子结点表示法

C. 孩子-兄弟表示法　　D. 顺序存储表示法

3. 下列关于二叉树的叙述中，正确的是_____。

Ⅰ. 只有一个结点的二叉树的度为 0

Ⅱ. 二叉树的度为 2

Ⅲ. 二叉树的左、右子树可任意交换

Ⅳ. 深度为 K 的完全二叉树的结点个数小于或等于深度相同的满二叉树

A. Ⅰ、Ⅳ　　B. Ⅱ、Ⅳ　　C. Ⅰ、Ⅱ、Ⅲ　　D. Ⅱ、Ⅲ、Ⅳ

4. 一棵二叉树共有 20 个结点，其中度为 1 的结点的个数是 7，则叶结点个数是_____。

A. 4　　B. 7　　C. 9　　D. 13

5. 在深度为 5 的二叉树中，结点个数最多是_____。

A. 10　　B. 16　　C. 31　　D. 32

6. 对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，根结点的编号为 1，则对树中任意结点，若其编号为 i 则正确的是_____。

A. 若 $2i \geqslant n$，则结点 i 无左子结点

B. 若 $2i+1 \geqslant n$，则结点 i 无右子结点

C. 若 i 为偶数，则 i 是其父结点的左子结点

D. 若 i 为奇数，则 i 是其父结点的右子结点

7. 在采用顺序存储方式保存一棵二叉树时，根结点保存在数组的 $A[0]$ 中，若编号为 i 的结点有右子结点，则该子结点的下标为_____。

A. $2i-1$　　B. $2i$　　C. $2i+1$　　D. $2i+2$

8. 在一棵非空二叉树的中序遍历序列中，根结点的右边_____。

A. 只有左子树上的所有结点　　B. 只有右子树上的所有结点

C. 只有左子树上的部分结点　　D. 只有右子树上的部分结点

9. 若一棵满二叉树有 m 个叶结点、n 个结点，其高度为 h，则下列关系式中正确的是_____。

A. $h+m==2n$　　B. $n==h+m$　　C. $m==h-1$　　D. $n==2^h-1$

10. 若一棵二叉树的高度为 h，所有结点的度为 0 或 2，则这棵二叉树的结点数最少为_____。

A. $h+1$　　B. 2^h-1　　C. 2^h　　D. 2^h+1

11. 设 n、m 为二叉树 T 中的两个结点，在中序遍历序列中，n 出现在 m 之前的条件是

_____。

A. n 在 m 左侧　B. n 在 m 右侧　C. n 是 m 的子孙结点　D. n 是 m 的祖先结点

12. 由 3 个结点可以构造出的不同形态的二叉树的数量是_____。

A. 3 个　B. 4 个　C. 5 个　D. 6 个

13. 下列 4 个序列中，能构成最大堆的是_____。

A. 75,65,30,15,25,45,20,10　B. 75,65,45,10,30,25,20,15

C. 75,45,65,30,15,25,20,10　D. 75,45,65,10,25,30,20,15

14. 在 13 题得到的最大堆中删除堆顶后，得到的最大堆是_____。

A. 65,30,15,25,45,20,10　B. 65,45,25,30,15,10,20

C. 65,45,30,15,25,20,10　D. 65,45,10,25,30,20,15

15. 在 13 题得到的最大堆中插入新元素 55 后，得到的最大堆是_____。

A. 75,55,65,45,15,25,20,10,30　B. 75,65,55,45,10,30,25,20,15

C. 75,55,45,65,30,15,25,20,10　D. 75,45,65,55,10,25,30,20,15

16. 与树的后序遍历序列相同的是对应二叉树的_____。

A. 先序遍历序列　B. 中序遍历序列

C. 后序遍历序列　D. 按层遍历序列

17. 设森林 F 中有 3 棵树，第一棵、第二棵和第三棵树的结点个数分别为 M1、M2 与 M3（M1、M2 和 M3 皆大于 0）。与森林 F 对应的二叉树的根结点的右子树上的结点个数是_____。

A. M1　B. M1+M2　C. M3　D. M2+M3

18. 若一棵非空二叉树的先序遍历序列与后序遍历序列正好互为逆序序列，则该二叉树的形状应为_____。

A. 从树的根结点开始，左、右子树相互对称

B. 所有非叶结点只有向左的一个分支

C. 所有非叶结点只有向右的一个分支

D. 所有非叶结点都是度为 1 的结点

19. 设给定权值总数为 n，用它构造的哈夫曼树的结点总数为_____。

A. 不确定　B. $2n-1$　C. $2n$　D. $2n+1$

20. 下列编码集中，不具有前缀特性的是_____。

A. {0,10,110,1111}　B. {00,10,011,110,111}

C. {00,010,0110,1000}　D. {11,10,001,101,0001}

21. 若一棵哈夫曼树共有 215 个结点，则树中编码的字符个数是_____。

A. 107　B. 108　C. 215　D. 214

二、填空题

1. 具有 18 个结点的完全二叉树的高度是_____。

2. 已知完全二叉树的第 5 层（根在第 0 层）有 8 个叶结点，则整个二叉树的结点个数最多是_____。

3. 已知一棵二叉树的先序遍历结果为 A,B,C,D,E,F，中序遍历结果为 C,B,A,E,D,F，则其后序遍历结果为_____。

4. 二叉树的先序遍历结果是 E,F,H,I,G,J,K，中序遍历结果是 H,F,I,E,J,K,G，该二叉树的根结点的右子树的根结点是_____。

5. 若一棵二叉树具有 10 个度为 2 的结点，5 个度为 1 的结点，则度为 0 的结点个数是_____。

6. 设树 T 的度为 4，其中度为 1、2、3 和 4 的结点的个数分别为 4、2、1 与 1，则 T 中的叶结点个数为_____。

7. 与算术表达式 a+b*(c+d/e)对应的后缀表达式为_____。

8. 设森林 F 对应的二叉树为 B，它有 m 个结点，B 的根结点为 p，p 的右子树的结点个数为 n，则森林 F 中第一棵树的结点个数是_____。

9. 已知二叉树的后序遍历序列为 D,G,E,B,F,C,A，中序遍历序列为 D,B,G,E,A,C,F，则其先序遍历序列是_____。

10. 对于任意一棵二叉树，若它有 n_0 个叶结点，n_2 个度为 2 的结点，则 n_0=_____。

11. 如果深度为 k 的二叉树为满二叉树，则树中结点的个数是_____。

12. 若用二叉链表存储二叉树，则含有 n 个结点的二叉树中的空指针个数为_____。

13. 设有含 n 个结点的完全二叉树，如果对结点按照自上到下、从左到右的次序从 1 开始进行连续的编号，则第 i（$2 \leqslant i \leqslant n$）个结点的父结点的编号为_____。

14. 设有含 n 个结点的完全二叉树，如果对结点按照自上到下、从左到右的次序从 1 开始进行连续的编号，则第 i（$2 \leqslant i \leqslant n$）个结点有左子结点的条件是_____。

15. 假设用孩子-兄弟表示法存储一个森林，则对森林进行的先序遍历等同于对其孩子-兄弟链表进行的_____。

16. 假设用孩子-兄弟表示法存储一个森林，则森林进行的后序遍历等同于对其孩子-兄弟链表进行的_____。

三、解答题

1. 请使用一棵树表示你所在学校的院系结构。

2. 当使用一棵树描述本章目录（如第五章、节）时，请回答下列问题：

（1）树中共有多少个结点？

（2）每个结点的度分别是多少？

（3）树的深度是多少？

3. 有 4 个结点的二叉树共有多少种不同的树形？分别画出。

4. 若某棵树有 n_1 个度为 1 的结点、n_2 个度为 2 的结点……n_m 个度为 m 的结点，则这棵树共有多少个叶结点？给出推导过程。

5. 使用同样结构的结点构造的双向链表和二叉链表有什么不同？

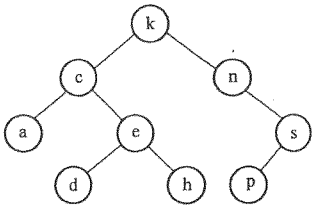


图 5-43　解答题 6 的图

6. 分别给出图 5-43 所示二叉树的先序、中序、后序遍历序列。

7. 将图 5-43 所示的二叉树还原为森林。

8. 树的集合表示法能直接用来表示二叉树吗？若不能，则修改集合表示法，使之能表示二叉树，以图 5-43 所示的二叉树为例，给出其集合表示。

9. 现有一棵二叉树，其先序遍历序列是 A,B,C,D,E,F,G,H,I，中序遍历序列是 B,C,A,E,D,G,H,F,I，画出该二叉树。

10. 给定一组权值：15,22,14,6,7,5,7，构造一棵哈夫曼树，并计算树的外部带权路径长度。

11. 假设字符 a、b、c、d、e、f 在某文本中出现的频率分别为 0.07、0.09、0.12、0.22、0.23、0.27，求这些字符的哈夫曼编码。

12. 画出算术表达式((a+b)+c*(d+e)+f)*(g+h)的表达式树。

13. 已知一棵二叉树 T，请回答下列问题：

（1）如果知道 T 的先序遍历序列和层序遍历序列，是否可以唯一还原 T？为什么？

（2）如果知道 T 的中序遍历序列和层序遍历序列，是否可以唯一还原 T？为什么？

14. 将图 5-27 所示的两棵树合并为一棵树，令 W 为 R 的第三个孩子结点，给出保存该树的存储结构。

四、算法阅读题

1. 二叉链表中结点类的定义及二叉树的定义如下。

```
typedef int ELEMType;
typedef struct BNode                    //二叉树结点
{   ELEMType data;                      //数据域
    struct BNode *left, *right;         //指向左孩子结点、右孩子结点的指针
}BinTNode;
typedef BinTNode *BTree;                //二叉树
```

以下程序实现了二叉树的若干基本操作，请在空白处填上适当内容以将算法补充完整。

（1）返回二叉树的高度。

```
int high(BTree root)
{
    int i,j;
    if ( (1) ) return 0;
    i= (2) ;
    j= (3) ;
    if (i>=j) return i+1;
    else return j+1;
}
```

（2）返回二叉树中度为 1 的结点个数。

```
int oneDNodeNumber(BTree root)
{
    if (root==NULL) (1) ;
    if ( ( (2) )||(root->left!=NULL && root->right==NULL))
    return (3) ;
    else return oneDNodeNumber(root->left)+oneDNodeNumber(root->right);
}
```

2. 二叉链表中结点类的定义及二叉树的定义如下。

```
typedef int ELEMType;
typedef struct BNode                    //二叉树结点
{   ELEMType data;                      //数据域
    struct BNode *left, *right;         //指向左孩子结点、右孩子结点的指针
}BinTNode;
typedef BinTNode *BTree;                //二叉树
```

说明 change 函数的功能。

```
void change(BTree root){
    BinTNode *temp;
    if (root==NULL) return;
    change(root->left);
    change(root->right);
    temp=root->left;
    root->left=root->right;
    root->right=temp;
    return;
}
```

3. 函数 CountLeave 的功能是计算二叉树中叶结点的数量，请在空白处填上适当内容以将算法补充完整。

```
typedef struct BiTNode
{
    ElemType data;
    struct BiTNode   *lchild, *rchild;
} *BiTree;                          //二叉树中结点定义
int CountLeave( BiTree T )          //返回值为二叉树 T 中叶结点的个数
{
    if (!T)  return  (1) ;
    if (T->lchild==NULL && T->rchild==NULL)
        return  (2) ;
    else
        return  (3) ;
}
```

五、算法设计题

1. 设二叉树以二叉链表的形式存储，试设计算法，返回二叉树中度为 2 的结点的个数。

2. 设二叉树以二叉链表的形式存储，每个结点的数据域中存放一个整数。试设计算法，求二叉树中所有结点内保存的整数之和。

3. 将二叉树中每层的结点个数定义为该层的宽度，各层中最大的宽度定义为二叉树的宽度。设二叉树以二叉链表的形式存储，试设计算法，返回二叉树的宽度。

第六章 图 结 构

学习目标：

1. 理解图的定义和相关的基本概念。
2. 掌握图的顺序存储结构和链式存储结构。
3. 掌握图的基本操作的实现。
4. 掌握并实现图的深度优先和广度优先搜索算法，理解图的连通性及连通分量概念。
5. 理解图的生成树概念，掌握求图最小代价生成树的两个算法。
6. 理解有向无环图的概念，掌握图的拓扑排序算法，掌握关键路径的求解过程。
7. 理解最短路径概念，掌握迪杰斯特拉算法和弗洛伊德算法的求解过程。
8. 了解各算法的时间复杂度。

建议学时： 13 学时。

教师导读：

1. 有些概念在树和图中都出现了，其含义有的相同，有的不同。既要讲明它们的联系，也要指明它们的区别。比如，边（分支）在树中没有强调方向，但在图中，既有有向边，也有无向边，边上还可能带有权值。

2. 图的两种存储方式涉及数组及链表的概念，在两种存储方式下实现图的基本操作时，既可以借助前面的代码，也可以从零开始实现。

3. 图中有很多经典算法，要求考生重点掌握。

4. 在学习完本章后，应要求考生完成实习题目 5。

图结构（简称为图）是一种重要的数据结构，它的应用领域十分广泛，常用于通信网络描述分析、电路分析、项目规划、控制论、城市交通等，甚至还会应用于一些社会科学领域。

本章将介绍图的概念，定义图的基本操作，并讨论图的主要存储方式及在顺序存储方式下图的基本操作的实现。同时，本章还将介绍图的一些经典问题，给出相应的算法。

第一节 图的基本概念与基本操作

与树结构一样，图结构也是非线性结构。图中的结点可以与许多其他结点相连接，并且没有像树中那样的父子关系。在线性结构中，元素之间的关系可以看成一对一的。在树结构中，元素之间的关系可以看成一对多的。在图结构中，元素之间的关系可以看成多对多的。图结构比树的层次结构又复杂一些，它没有像树根那样的一个特殊结点。树可以看成图的简单特例。

一、图的基本概念

图（graph）由顶点和边组成，一般地，用 G=(V,E)来表示，其中 V 表示顶点（vertex）

集，是一个有限非空集合；E 表示边（edge）集，E 中的每条边都是 V 中某一对顶点的连接。当顶点分别是 u 和 v 时，连接这两个顶点的边可以表示为一个二元组(u,v)，有时也将边称为顶点的偶对。图中任意两个不同顶点之间允许有边，但不能超过一条。

在图 G=(V,E)中，顶点的总数记为|V|，边的总数记为|E|。如果图中边的数目较少（相对于顶点数来说），则图称为稀疏图；反之，边数较多的图称为密集图或稠密图。至于边数多到多少是密集图或少到多少是稀疏图，并没有严格的界定。当图中边数的数量级不高于顶点数的数量级时，就可以认为图是稀疏图。

在有些情况下，可以限定图中边(u,v)的方向，既可以从 u 指向 v，也可以从 v 指向 u。当图中的边限定为从一个顶点指向另一个顶点时，称为有向边，或称为弧（arc）。不限定方向的边称为无向边。实际上，一条无向边可以看成两条方向相反的有向边。组成有向边的顶点二元组是有序的，而组成无向边的顶点二元组是无序的。例如，有向边(u,v)表示从顶点 u 指向顶点 v 的边，弧的方向是从 u 指向 v，u 称为弧尾（tail），v 称为弧头（head）。对于有向边，(u,v)与(v,u)不同。而无向边(u,v)既可以表示从顶点 u 指向顶点 v，也可以表示从顶点 v 指向顶点 u。对于无向边，(u,v)与(v,u)是等价的。

含有有向边的图称为有向图（directed graph，或简写为 digraph）。如果图中的边都是无向边，则图称为无向图（undirected graph，或简写为 undigraph）。如果一个图中既含有有向边，又含有无向边，则可以将其中所有的无向边表示为一对方向相反的有向边。所以，提到有向图，则表明其中所含的边全部都是有向边。

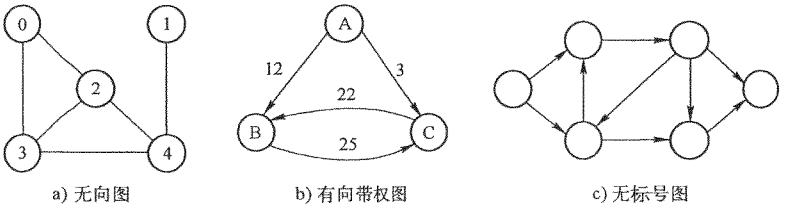
若无向图 G 中含有边(u,v)，则 u 和 v 互为邻接（adjacent）点。边(u,v)称为与顶点 u 和 v 相关联（incident），也可以说边(u,v)依附于顶点 u 和 v。对于有向图中的有向边(u,v)，称顶点 v 是顶点 u 的邻接点，具体来说，顶点 u 邻接到顶点 v，或顶点 v 邻接自顶点 u。

在有些应用中，为了表明图中边的某些特性，往往给边赋予一个非负的数值，这个非负数值称为边的权（weight），相应的图称为带权图（weighted graph）或加权图，有些教材称之为网（network）。在实际应用中，权值可以根据需要自行定义其代表的意义。

为了明确表示图中的所有顶点，可以让各顶点带有标号，这样的图称为标号图（labeled graph）。本章讨论的图大多是标号图，标号可以作为顶点的名称。

通常，以带标号或不带标号的圆表示图中的顶点，以直线或弧线表示图中的边，箭头表示有向边的方向，从弧尾指向弧头。对于带权图，权值通常写在边的旁边。一般地，所画的图是一个逻辑图，边或弧只表示两个顶点之间是否存在邻接关系，其长短并不代表权值的大小。

【例 6-1】 图 6-1 中给出了 3 个图的示例。



a) 无向图　　b) 有向带权图　　c) 无标号图

图 6-1　图的示例

图 6-1a 是一个含 5 个顶点、6 条边的无向图。顶点集 V={0,1,2,3,4}，边集 E={(0,2),(0,3),(1,4),(2,3),(2,4),(3,4)}。图 6-1b 是一个含 3 个顶点、4 条弧的有向图，且是带权图，弧的权值均为正整数，顶点集 V={A,B,C}，弧集 E={(A,B,12),(A,C,3),(B,C,25),(C,B,22)}。这两个图都是标号图。图 6-1c 是一个含 6 个顶点、9 条弧的有向图，这是一个无标号图，也不是带权图。

再看图 6-1a 中顶点与边的关系，顶点 2 分别与顶点 0、顶点 3、顶点 4 互为邻接点，但与顶点 1 不是邻接点。边(0,2)与顶点 0 及顶点 2 相关联，边(1,4)与顶点 1 及顶点 4 相关联。其他关系不再赘述。

在图 6-1b 所示的有向图中，顶点 B 及顶点 C 之间存在两条弧，分别是(B,C)和(C,B)。弧(B,C)的弧尾是顶点 B，弧头是顶点 C。弧(C,B)的弧尾是 C，弧头是 B。与无向图不同，有向图中顶点相同、但方向相反的两条弧是两条不同的有向边。例如图 6-1b 中的(B,C)和(C,B)，它们的权值分别是 25 和 22。在实际中，这样的情况有很多，如两个海拔不同的城市之间相互输水的成本可能不同。

图中两个顶点之间的边不能有重复，也就是说，无向图中两个顶点之间最多只能有一条边，有向图中两个顶点之间最多有两条方向相反的弧。图中不包含(i,i)形式的边，即没有顶点自身到自身的边。

在无向图中，与顶点 v 相关联的边的数目称为顶点的度（degree）。在有向图中，顶点的度分为出度和入度。以某顶点为弧头的弧称为该顶点的入边，入边的数目称为该顶点的入度；以某顶点为弧尾的弧称为该顶点的出边，出边的数目称为该顶点的出度。一个顶点的出度与入度之和称为该顶点的度。在图 6-1a 所示的图中，顶点 2 的度为 3，而在图 6-1b 所示的图中，顶点 B 的入度为 2，出度为 1，其度为 3。

在有 n 个顶点的无向图中，其边数最多可达 $n(n-1)/2$；而在有向图中，由于边具有方向性，因此可能的最大边数比无向图多一倍，含 n 个顶点的有向图的边数最多为 $n(n-1)$。包含所有可能边的图称为完全图（complete graph）。

【例 6-2】 完全图示例。

含 4 个顶点的无向完全图和含 3 个顶点的有向完全图如图 6-2 所示。在完全图中，各顶点的度均达到最大。在图 6-2a 中，所有顶点的度均为 3。在图 6-2b 中，每个顶点的入度和出度也达到最大，都为 2，顶点的度是入度与出度之和，为 4。

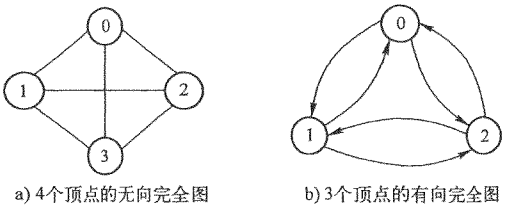
假设无向图 G=(V,E)中含有 n 个顶点，e 条边，每个顶点的度为 $d_i(0 \leqslant i \leqslant n-1)$，则下列关系式成立

$$e = \frac{1}{2}\sum_{i=0}^{n-1} d_i \qquad (6\text{-}1)$$

例如，图 6-1a 中 5 个顶点的度分别为 2、1、3、3、3，图中含有的边总数

$$e = \frac{1}{2}\sum_{i=0}^{4} d_i = \frac{1}{2}(2+1+3+3+3) = 6$$

式（6-1）对有向图也成立，如图 6-1b 中 3 个顶点的度分别为 2、3、3，图中含有的边总数



a) 4个顶点的无向完全图　　b) 3个顶点的有向完全图

图 6-2　完全图示例

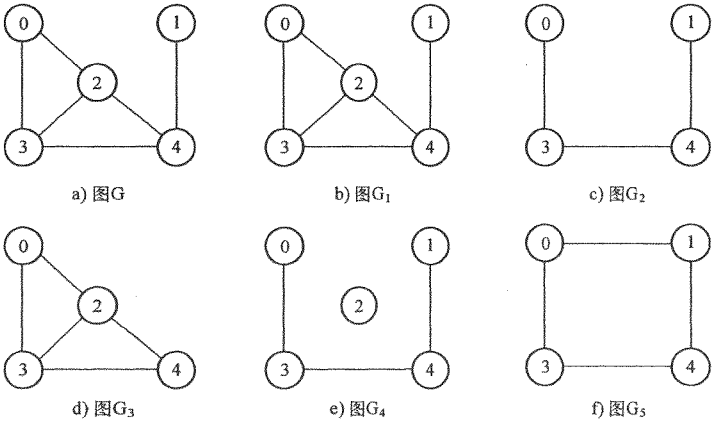
$$e=\frac{1}{2}\sum_{i=0}^{2}d_i=\frac{1}{2}(2+3+3)=4$$

所有顶点的出度之和=$\sum_{i=0}^{2}$ 顶点 i 的出度=2+1+1=4。所有顶点的入度之和=$\sum_{i=0}^{2}$ 顶点 i 的入度=0+2+2=4。可以验证，有向图中所有顶点的出度之和与入度之和相等，并且等于图中所含的边数。

若两个图 G=(V,E)和 G′=(V′,E′)满足条件：V′⊆V，E′⊆E，且 E′中边依附的顶点均属于 V′，则称 G′为 G 的子图（subgraph）。实际上，一个图 G 的子图是指由图 G 中选出其顶点集的一个子集 V_s以及仅与 V_s中顶点相关联的一些边所构成的图。

【例 6-3】 子图示例。

找出图 6-3 中有子图关系的图。



a) 图G　b) 图G_1　c) 图G_2　d) 图G_3　e) 图G_4　f) 图G_5

图 6-3　子图示例

在图 6-3 中，对于图 G，图 G_1、G_2、G_3和 G_4都是它的子图；特别地，图 G_1与图 G 完全一样，它也是图 G 的子图，即图本身也是它自身的子图；同时，图 G_2是图 G_4的子图。而图 G_5不是图 G 的子图，因为边(0,1)并不是图 G 中的一条边。从本例中可知，一个图的子图可能有多个。

在图 G=(V,E)中，如果$(v_{i0},v_{i1}),(v_{i1},v_{i2}),\cdots,(v_{im-2},v_{im-1})$都是 E 中的边，则顶点序列$(v_{i0},v_{i1},\cdots,v_{im-1})$称为从顶点 v_{i0}到顶点 v_{im-1}的路径（path）。若图 G 是有向图，则路径也要求是有向的，且有向边必须是方向一致的，即有向路径$(v_{i0},v_{i1},\cdots,v_{im-1})$是由 E 中的弧$(v_{i0},v_{i1}),(v_{i1},v_{i2}),\cdots,(v_{im-2},v_{im-1})$组成的。路径$(v_{i0},v_{i1},\cdots,v_{im-1})$中包含的边或弧的数目 m 称为路径长度。如果路径上各顶点均不同，则称此路径为简单路径（simple path）。第一个顶点和最后一个顶点相同的路径称为回路（cycle），也称为环。如果一个回路中除第一个顶点和最后一个顶点以外，其余顶点均不相同，则称为简单回路（simple cycle）或简单环。不带回路的图称为无环图。不带回路的有向图称为有向无环图（Directed Acyclic Graph，DAG）。

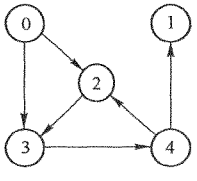


图 6-4　路径示例

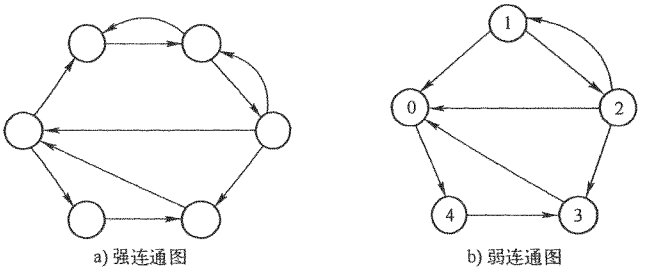
【例 6-4】 路径示例（见图 6-4）。

由图 6-4 可知，从顶点 0 到顶点 3，再到顶点 4，最后到顶点 1，构成一条有向简单路径(0,3,4,1)，路径长度为 3；从顶点 2 到顶点 3，

再到顶点4，最后回到顶点2，构成一个简单有向回路(2,3,4,2)。对于从顶点2到顶点0，再到顶点3，虽然其中均有边相连，但由于边的方向不一致，所以不能构成有向路径。

对于无向图G，如果顶点u和顶点v之间有路径，则称这两个顶点是连通的；如果无向图G中任意一个顶点到其他任意顶点都至少存在一条路径，也就是说，图中任意两个顶点都是连通的，则称图G为连通图（connected graph）。无向图中的极大连通子图称为连通分量（connected component）。例如，在图6-3中，图G_4是一个含有两个连通分量的非连通图，其他5个都是连通图，连通分量个数为1。

在有向图G中，如果每对顶点u与v之间均有从u到v的有向路径，则称G为强连通图（strongly connected graph），有向图的最大强连通子图称为强连通分量；如果对于每对顶点u和v，存在顶点序列$v_{i0},v_{i1},\cdots,v_{im-1}$，这里$u=v_{i0}$，$v=v_{im-1}$，并且$(v_{ij},v_{ij+1})\in E$或$(v_{ij+1},v_{ij})\in E(0\leqslant j\leqslant m-2)$，则称图G为弱连通图（weakly connected graph）。图6-5a是一个强连通图；而图6-5b是一个弱连通图，从顶点0到顶点1没有有向路径。



a) 强连通图　　b) 弱连通图

图6-5　有向连通图

【例6-5】设无向图G含有10个顶点和6条边，则G中连通分量的个数最多可能是多少？最少可能是多少？

要让G中连通分量最多，可以让某些顶点与尽可能多的边相关联，且这些顶点组成尽可能少的连通分量。6条边可以让4个顶点组成完全图，其余的6个顶点均为孤立顶点，则G含有7个连通分量。

二、图的基本操作

图的定义加上对它的操作组成图的抽象数据类型（ADT）的定义，表示为MGraph。图的基本操作中有对顶点的操作及对边的操作，具体如下。

```
int CreateGraph(MGraph *g)                          //创建图
void PrintGraph(MGraph g)                           //输出图信息
WType GetWeight(MGraph g,VType u,VType v)           //返回边(u,v)的权值
VType FirstNeighbor(MGraph g, VType u)              //返回顶点u的第一个邻接点
VType NextNeighbor(MGraph g,VType u,VType w)        //返回顶点u排在邻接点w后的下一个邻接点
int getNumVertices(MGraph g)                        //返回图的顶点数
int getNumEdges(MGraph g)                           //返回图的边数
```

VType表示顶点类型，MEdge表示边的类型。在图的基本操作中，常常需要找到与一个顶点相关联的边，因此定义方法FirstNeighbor()和NextNeighbor()，通过这两个方法，可以

找到一个顶点的所有邻接点，即找到相关联的所有边。

顶点使用单字符表示，保存在顶点表 verticesList[MaxVtxNum]中，使用两个辅助方法，在顶点字符与顶点表下标之间进行转换。

```
int VerToNum(MGraph g, VType u)      //返回顶点 u 在顶点表 verticesList 中的下标值
VType NumToVer(MGraph g, int i)      //返回顶点表 verticesList 中下标 i 对应的顶点
```

第二节　图的存储结构

图的结构较复杂，相关的信息包括顶点和边等，边即顶点之间的关系。要保存图的信息，首先需要保存这些内容。从图的存储结构中，能够获取图中所有顶点的信息，同时，该存储结构能够反映顶点之间的相互关系。另外，在图的基本操作中，需要能够方便地得到图中的顶点数和边数，需要查找某个顶点的邻接点等，所以图的存储结构要方便这些操作的实现。

与其他数据类型类似，图也有两类基本的存储方式，即顺序存储结构及链式存储结构。顺序存储结构以邻接矩阵（adjacency matrix）为代表，链式存储结构以邻接表（adjacency list）为代表。

一、顺序存储结构

设图 G=(V,E)，|V|=n，图的邻接矩阵是一个 $n\times n$ 矩阵，矩阵元素表示图中各顶点之间的邻接关系。邻接矩阵也称为相邻矩阵。设各顶点为 $v_0,v_1,\cdots,v_{n-1}$，如果从 v_i 到 v_j 存在一条边，则邻接矩阵中位于 i 行 j 列的元素值为 1，否则值为 0。这样的一个矩阵可以表示图中所有顶点之间的邻接关系。邻接矩阵的存储空间与顶点个数有关，为 $O(|V|^2)$。如果还需要保存图中各顶点的信息，则可用一维数组来保存。

定义邻接矩阵如下，它是一个二维数组：

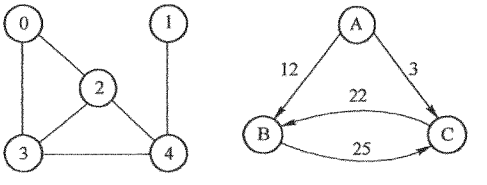
$$A[i][j]=\begin{cases}1 & \text{若}<v_i,v_j>\in E,\quad 0\leqslant i,j\leqslant n-1\\ 0 & \text{若}<v_i,v_j>\notin E,\quad 0\leqslant i,j\leqslant n-1\end{cases}\tag{6-2}$$

对于带权图，可以修改邻接矩阵的定义。如果图中从 v_i 到 v_j 存在一条边，则邻接矩阵中位于 i 行 j 列的元素值为边 (v_i,v_j) 的权 w_{ij}，否则值为∞，这里∞ 表示计算机中可表示的一个最大数。修改定义式（6-2），得到式（6-3）：

$$A[i][j]=\begin{cases}w_{ij} & \text{若}<v_i,v_j>\in E,\quad 0\leqslant i,j\leqslant n-1\\ \infty & \text{若}<v_i,v_j>\notin E,\quad 0\leqslant i,j\leqslant n-1\end{cases}\tag{6-3}$$

【例 6-6】邻接矩阵示例。

对于无向图 G_1 和有向带权图 G_2，分别给出其邻接矩阵，如图 6-6 所示。



a) 图G_1　　b) 图G_2

$$\begin{bmatrix}0&0&1&1&0\\0&0&0&0&1\\1&0&0&1&1\\1&0&1&0&1\\0&1&1&1&0\end{bmatrix}\qquad\begin{bmatrix}\infty&12&3\\\infty&\infty&25\\\infty&22&\infty\end{bmatrix}$$

c) 图G_1的邻接矩阵　　d) 图G_2的邻接矩阵

图 6-6　图及其邻接矩阵

从邻接矩阵的定义可知，如果图中含有边 (v_i,v_j)，则 $A[i][j]$ 为 1 或者 w_{ij}。在无向图中，由于边没有方向性，因此，当边 (v_i,v_j) 存在

时，边(v_j,v_i)也必定存在，即$A[i][j]$和$A[j][i]$同为1或w_{ij}；而如果边(v_i,v_j)不存在，则边(v_j,v_i)也不存在，即$A[i][j]$和$A[j][i]$同为0或∞。这些表明，邻接矩阵是一个对称矩阵。图6-6c所示的邻接矩阵是对称矩阵。有向图的邻接矩阵不能保证对称性，因为有向图中有向边(v_i,v_j)与(v_j,v_i)是两条不同的弧，不能保证同时存在或同时不存在，因此不能保证$A[i][j]$与$A[j][i]$的值相等。图6-6d所示的邻接矩阵不是对称矩阵。

【例6-7】若无向图G中含有n个顶点和e条边，则它的邻接矩阵中0的个数是多少？

根据图的邻接矩阵的定义，因为图G中含有n个顶点，所以邻接矩阵中元素个数为n^2。无向图的邻接矩阵是对称矩阵，对于G中的任一条边(v_i,v_j)，在邻接矩阵中都会对应两个1，即$A[i][j]$和$A[j][i]$均为1。所以，图G的邻接矩阵中有$2e$个1，0的个数为n^2-2e。

当图G含有n个顶点时，一般地，其邻接矩阵需要n^2个存储位置。考虑到无向图的邻接矩阵的对称性，当使用数组存储对称矩阵时，可以只存储其下三角（或上三角）部分的元素。同时，图中不含有(i,i)形式的边，所以邻接矩阵的对角线元素全为0或∞，也不需要存储。因而无向图的存储空间可以压缩到$n(n-1)/2$个存储位置。

在用邻接矩阵表示图时，可以很容易地判定图中任意两个顶点之间是否存在边（或弧）。具体来说，若矩阵元素$A[i][j]$为1或w_{ij}，就表示从v_i到v_j有一条边（或弧），否则从v_i到v_j没有边（或弧）存在。

借助于邻接矩阵，也很容易得到图中各顶点的度。对于无向图G，邻接矩阵i行或i列中非0元素的个数即顶点v_i的度。对于有向图G，邻接矩阵i行中非0（也不等于∞）元素的个数为顶点v_i的出度，而i列中非0（也不等于∞）元素的个数为顶点v_i的入度，i行非0（也不等于∞）元素的个数加上i列非0（也不等于∞）元素的个数为顶点v_i的度。

设用邻接矩阵表示有n个顶点的图G，在计算G中有多少条边时，需要检查矩阵中的所有元素，因此时间复杂度为$O(n^2)$。另外，其存储空间仅与图G中的顶点数有关，而与边数无关，即$O(n^2)$。因此，当图中边的数目远远小于n^2，也就是邻接矩阵为稀疏矩阵时，检查图中边的数目的时间复杂度及存储邻接矩阵的空间复杂度都将有极大的浪费。为解决这个问题，可以采用另外一种存储结构，即邻接表。

二、链式存储结构

链式存储结构包括邻接表和逆邻接表。

邻接表是图的动态存储表示方法，它使用单链表存储图中顶点的邻接点，每个顶点的所有邻接点存储在一个单链表中。为了便于管理，使用一个数组保存这些单链表的表头，根据数组下标就可以快速找到相应的单链表。实际上，这种表示方法是动态表示法与静态表示法的结合，因为它既使用了链表结构，也使用了数组结构。

设图$G=(V,E)$，则G的邻接表由一个一维数组和$n=|V|$个链表组成，一维数组包含n个元素，每个元素都包含表示顶点信息的域和一个指针域。与顶点$v_i(0\leqslant i\leqslant n-1)$相邻接的所有顶点组成一个单链表，其头指针保存在一维数组下标为i的元素的指针域中。单链表的每个结点都有两个域，一个是顶点域，存储邻接顶点在数组中的下标；另一个是指针域，指向下一个邻接顶点，单链表最后一个结点的指针域为空。因为该单链表的每个结点中保存的是顶点v_i的邻接点，所以有些教材中将该表称为邻接点表；而顶点v_i与邻接点构成图中的一条边，所以有些教材中将该表称为边链表。

邻接点表的表结点的结构如下。

顶点下标	指针

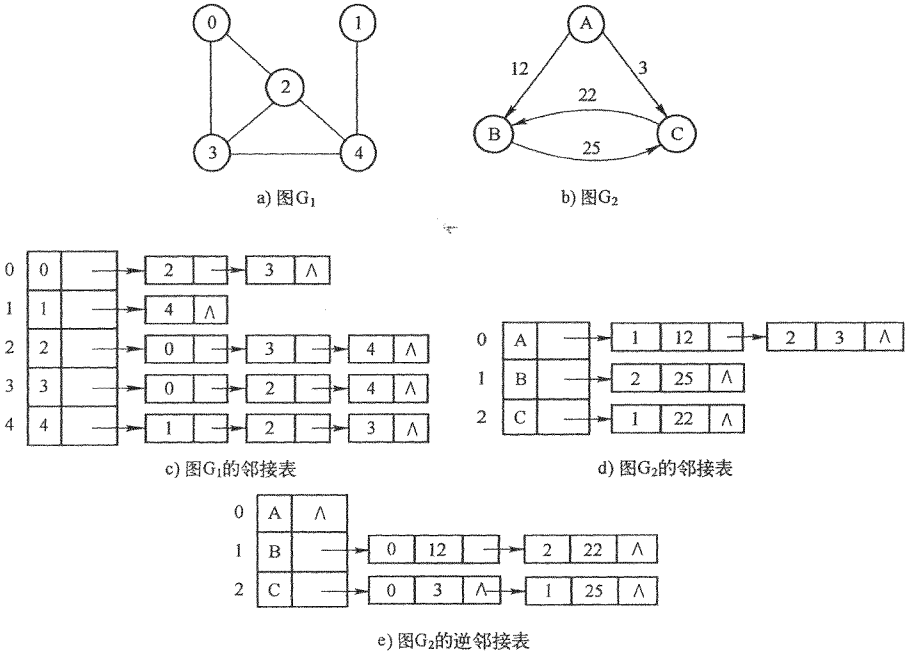
对于带权图，可以扩展邻接点表的结构，在邻接点表的每个表结点中都增加一个域，用来存储两个顶点间这条边的权。其表结点的结构如下。

顶点下标	权值	指针

对于有向图，如果将所有有向弧的方向转向，则得到的新图的邻接表称为原图的逆邻接表。

【例 6-8】 邻接表和逆邻接表示例。

对于无向图 G_1，给出其邻接表；对于有向带权图 G_2，分别给出其邻接表和逆邻接表，如图 6-7 所示。



a) 图G_1　　b) 图G_2

c) 图G_1的邻接表

d) 图G_2的邻接表

e) 图G_2的逆邻接表

图 6-7　图及其邻接表（和逆邻接表）

对于无向图，使用邻接表可以很方便地求出顶点的度，数组下标为 i 的元素指向的单链表中结点的个数即顶点 v_i 的度。

图 G_1 共有 6 条边，由图 6-7c 可知，对应的邻接表内共有 12 个表结点，为边数的两倍。对于每条边 (v_i, v_j)，v_i 结点出现在 v_j 对应的单链表中，同时 v_j 结点又出现在 v_i 对应的单链表中，即每条边的信息都出现在这条边所关联的两个顶点所对应的单链表中。

对于有向图，数组下标为 i 的元素指向的单链表中结点的个数为顶点 v_i 的出度；如果要计算顶点 v_i 的入度，则需要遍历邻接表中所有的 n 个单链表，找出顶点为 v_i 的结点个数，这个过程比计算顶点的出度要费时一些。比如，在图 6-7d 中，顶点 A 指向的单链表中含有两个表结点，说明顶点 A 的出度是 2。所有的表结点中都没有出现下标 0，出现两个下标 1 和

两个下标2，说明顶点A的入度是0，而顶点B的入度是2，顶点C的入度也是2。表结点的个数是4，说明有向图中含有4条有向边。

使用逆邻接表求顶点入度的过程，类似于在邻接表中求顶点出度的过程，非常方便。仍以求图G_2各顶点的入度为例。在图6-7e所示的逆邻接表中，顶点A、顶点B和顶点C对应的邻接点表的长度分别是0、2、2，所以顶点A、顶点B和顶点C的入度分别是0、2、2，这与通过图6-7d计算的结果是一样的。

从例6-8可以看出，在用邻接表表示有n个顶点和e条边的无向图时，需要一个由n个元素组成的顺序表（表头结点表）和由总共$2e$个结点组成的n个单链表。而在表示有n个顶点、e条弧的有向图时，需要一个由n个元素组成的顺序表和由总共e个结点组成的n个单链表。邻接表的空间复杂度为$O(n+e)$。当图中边的数目远远小于顶点的数目时，邻接表所需的存储空间要比邻接矩阵的少。

在使用邻接表存储方式时，判定两个顶点间是否存在边的操作将比使用邻接矩阵费时，因为此时需要遍历邻接表中的一个单链表。例如，如果要判定图中从v_i到v_j是否存在边，则要在数组第i个元素所含的指针指向的单链表（即第i个单链表）中查找是否存在含v_j的结点，如果存在，则表示有边存在，否则，表示没有边存在。

当图中顶点个数经常变化时，为便于顶点的插入和删除，也可以将图的全部顶点保存在一个单链表中，而不是一维数组中。单链表中的结点结构如下。

顶点信息	指向下一顶点的指针	指向邻接点表的指针

相应地，邻接表的表结点的结构也要做相应的修改。将原来的“顶点下标”改为一个指针，指向在单链表中保存该顶点的相应结点。例如，对于图6-7b所示的图G_2，采用单链表保存图的顶点信息的邻接表如图6-8所示。

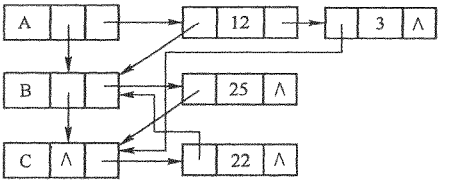


图6-8　图6-7b中G_2的邻接表

第三节　图的基本操作的实现

本节给出邻接矩阵存储结构下图的基本操作的实现。图的定义如下。

```
#define MaxVtxNum 10          //最大顶点个数
#define MaxEdgesNum 30        //最大边数
#define NA MinInterger        //MinInterger 表示最小整数
#define NB MaxInterger        //MaxInterger 表示最大整数
typedef char VType;           //顶点数据类型
typedef int WType;            //边上权值类型
```

```
typedef struct{                                    //边数据类型
    VType u,v;
}MEdge;
typedef struct {                                   //邻接矩阵表示
    VType verticesList[MaxVtxNum];                 //顶点表
    WType AdjMatrix[MaxVtxNum][MaxVtxNum];         //邻接矩阵
    int numVertices,numEdges;                      //当前顶点个数与边数
}MGraph;
```

顶点保存在顶点表 verticesList 中，这是一维数组，最大容量是 MaxVtxNum，图中实际的顶点个数使用 numVertices 来记录。AdjMatrix 是邻接矩阵，保存的是顶点间的边。统计矩阵中相应元素的个数可以得知边数。但因为可能会频繁地获取边数，故为了程序中处理方便，使用 numEdges 记录图中的当前边数。

需要经常查询顶点表，根据相应的条件获取相应的信息。例如，给定顶点 u，返回 u 在顶点表 verticesList 中的下标值，或返回顶点表 verticesList 中下标 i 处保存的顶点。相应的查询操作实现如下。

```
int VerToNum(MGraph g,VType u){            //返回顶点 u 在顶点表 verticesList 中的下标值
    int i=0;
    while (g.verticesList[i]!=u&&i<getNumVertices(g)) i++;
    if (i!=getNumVertices(g))return i;
    return -1;
}
VType NumToVer(MGraph g,int i){            //返回顶点表 verticesList 中下标 i 对应的顶点
    if (i>=0&&i<getNumVertices(g)) return g.verticesList[i];
    return '#';
}
```

程序中还需要查询边的权值，实现如下。

```
WType GetWeight(MGraph g,VType u,VType v){     //返回边(u,v)的权值
    int i,j;
    i=VerToNum(g,u);
    j=VerToNum(g,v);
    if (i!=-1&&j!=-1) return g.AdjMatrix[i][j];
    else return NB;
}
```

要实现图的操作，需要先建立一个图，从键盘输入相关的信息。图可以是无向图，也可以是有向图。创建无向图的程序实现如下。

```
void CreateGraph(MGraph *g){               //创建无向图
    int i,j,k,w;
    VType u,v,temp;
```

```
    printf("请输入图的顶点数及边数:");
    scanf("%d %d", &g->numVertices, &g->numEdges);
    printf("请输入图的顶点信息\n");
    scanf("%c", &temp);
    for (i=0;i<getNumVertices(*g);i++)
        scanf("%c", &g->verticesList[i]);
    for (i=0;i<getNumVertices(*g);i++)
        for (j=0;j<getNumVertices(*g);j++)
            g->AdjMatrix[i][j]=0;
    printf("请输入图的边信息,顶点1 顶点2:\n");
    for (k=0;k<getNumEdges(*g);k++){
        scanf("%c",&temp);
        scanf("%c %c",&u,&v);
        i=VerToNum(*g,u);
        j=VerToNum(*g,v);
        if (i!=-1 && j!=-1){
            g->AdjMatrix[i][j]=1;
            g->AdjMatrix[j][i]=1;          //创建有向图时无此语句
        }
    }
    return;
}
```

例如，图6-7a所示的图G_1含有5个顶点、6条边，顶点名依次是0、1、2、3和4，6条边依次是(0,2)、(0,3)、(2,3)、(1,4)、(2,4)和(3,4)，则执行CreateGraph()时输入的信息如下：

```
5 6
01234
0 2
0 3
2 3
1 4
2 4
3 4
```

创建带权有向图的程序实现如下。

```
int CreateGraph1(MGraph *g){                    //创建带权有向图
    int i,j,k,w;
    VType u,v,temp;
    printf("请输入图的顶点数及边数:");
    scanf("%d %d",&g->numVertices,&g->numEdges);
```

```
    printf("请输入图的顶点信息\n");
    scanf("%c",&temp);
    for (i=0;i<getNumVertices(*g);i++)
        scanf("%c",&g->verticesList[i]);
    for (i=0;i<getNumVertices(*g);i++)
        for (j=0;j<getNumVertices(*g);j++)
            g->AdjMatrix[i][j]=NB;          //NB表示带权有向图中的特殊值,表示无穷大
    printf("请输入图的边信息,顶点1 顶点2 权值:\n");
    for (k=0;k<getNumEdges(*g);k++){
        scanf("%c",&temp);
        scanf("%c %c %d",&u,&v,&w); //创建带权有向图时要多读入一个表示权值的参数
        i=VerToNum(*g,u);
        j=VerToNum(*g,v);
        if (i!=-1&&j!=-1){
            g->AdjMatrix[i][j]=w;
            //创建带权无向图时还需要加: g->AdjMatrix[j][i]=w;
        }
    }
}
```

输出图的信息，以验证图构建函数的正确性。相关程序实现如下。

```
void PrintGraph(MGraph g){          //输出图信息
    int i,j;
    printf("图的顶点:");
    for (i=0;i<getNumVertices(g);i++)printf("%c ",g.verticesList[i]);
    printf("\n");
    printf("图的边:\n");
    for (i=0;i<getNumVertices(g);i++)
        for (j=0;j<getNumVertices(g);j++)
            if (g.AdjMatrix[i][j]!=NB)
            printf("%c %c %d\n",NumToVer(g,i),NumToVer(g,j),g.AdjMatrix[i][j]);
    printf("图的邻接矩阵:\n");
    for (i=0;i<getNumVertices(g);i++){
        for (j=0;j<getNumVertices(g);j++)
            printf("%d ",g.AdjMatrix[i][j]);
        printf("\n");
    }
}
```

找到某顶点的第一个邻接点，然后继续查找这个邻接点的下一个邻接点，从而能够找到所有的邻接点。相关程序实现如下。

```
VType FirstNeighbor(MGraph g,VType u){                    //返回顶点 u 的第一个邻接点
    int i=0,j;
    i=VerToNum(g,u);
    if (i!=-1){
        for (j=0;j<getNumVertices(g);j++)
            if (g.AdjMatrix[i][j]!=NB) return g.verticesList[j];
            //顺序检测顶点 u 所在行,寻找第一个邻接点
    }
    return '#';
}
VType NextNeighbor(MGraph g,VType u,VType w){   //返回顶点 u 的排在邻接点 w 后的邻接点
    int i=0,j=0,col;
    i=VerToNum(g,u);
    j=VerToNum(g,w);
    if (i!=-1&&j!=-1){
        for (col=j+1;col<getNumVertices(g);col++)
            if (g.AdjMatrix[i][col]!=NB) return g.verticesList[col];
                                                    //在 i 行顺序寻找下一个邻接点
    }
    return '#';
}
```

获取顶点个数及边数的程序实现如下。

```
int getNumVertices(MGraph g){
    return g.numVertices;
}
int getNumEdges(MGraph g){
    return g.numEdges;
}
```

第四节　图的搜索

图已经被应用于很多领域，使用图解决实际问题的一个著名实例是 18 世纪数学家欧拉（Euler）解决了经典的哥尼斯堡七桥问题。

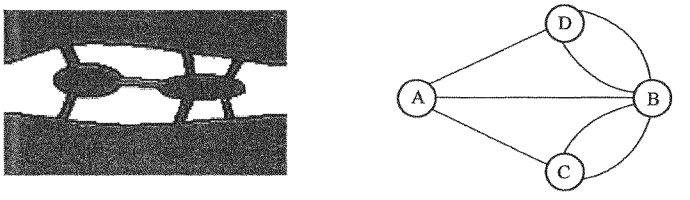
哥尼斯堡（今俄罗斯加里宁格勒）是欧洲的一座名城，是哲学家康德和数学家希尔伯特的故乡。

小城内，Pregel 河贯穿其中，河中心有两个小岛，有七座桥把两个小岛与河的两岸连接起来。一天又一天，这七座桥上走过无数的人，在桥上，可以听到悠扬的钟声。

当时，城中的人热衷于做一件事，即从自己的家中出发，找出一条经过所有的桥且每座桥只经过一次，最终回到家中的路线。可是一天天过去了，全城竟然没有一人能够找到这样

的路线。在消息传遍全欧洲之后，大数学家们也解不出这个难题。这就是著名的哥尼斯堡七桥问题。

使用抽象的图来表示哥尼斯堡的七座桥与陆地，如图 6-9a 所示，其中的 A、B、C、D 分别表示四块陆地。将七座桥连同陆地简化为如图 6-9b 所示的一个图，图中的顶点表示原来的陆地，七座桥分别表示为图中的 7 条边。因为没有限制走路的人经过七座桥的方向和次序，所以图中的边均为无向边。哥尼斯堡七桥问题可以描述为：从图中的任何一个顶点出发，能否走过图中的所有边，然后回到起始点，并且每条边只经过一次。



a) 哥尼斯堡的七桥　　b) 七座桥的逻辑结构

图 6-9　哥尼斯堡七桥问题

欧拉将此问题归结为图形的一笔画问题，即能否从某一点开始用一笔画出这个图形，最后回到原点，而不重复？欧拉在 1736 年发表图论方面的第一篇论文，解决了这个著名的哥尼斯堡七桥问题。

日常生活中的很多实际问题也可以归结为图的问题，例如，邮差要将信件送达多个目的地，每个目的地只需要到达一次，并且必须到达一次，他需要选择一条路径，这条路径可以覆盖所有目的地，且所走的路途总长度最短。

这两个例子有类似的地方。七桥问题是要走过每座桥，即要经过图中的每条边，而邮差问题是要经过每个目的地，即要经过图中的每个顶点。它们都属于遍历问题，前者是边的遍历，后者是顶点的遍历。本节所说的图的搜索是指顶点的遍历，即访问图中所有的顶点一次且仅访问一次。

定义 6-1　给定一个连通图 G=(V,E)，从图中的某个顶点出发，经过一定的路线，访问图中的所有顶点，使每个顶点被访问且只访问一次，这一过程称为图的搜索（traversal）。

图的搜索过程类似于树的遍历过程，即从一个顶点出发，沿着边访问下一个顶点。图的搜索也有不同于树遍历的地方，表现在以下三个方面。首先，树的遍历一般都从根结点开始，而图的搜索可以从图中任一顶点出发。其次，对于图，任何两个顶点之间都可能存在边，因此图中可能包含回路。从一个顶点出发，在访问若干顶点之后，可能又回到了前面已访问过的顶点，这是重复访问的问题，需要有一个机制来防止对顶点的重复访问。第三，由于图可能是不连通的，因此，从一个顶点出发，可能永远到达不了图的另一部分（另一连通分量）中的顶点，这样还必须从另一连通分量开始继续遍历。

为了解决重复访问及图的不连通这两个问题，在遍历图的顶点时，对每个顶点增加一个访问标志，标明该结点是否被访问过。可以使用一维数组 visited 记录图顶点的访问标志，数组元素的个数等于图中顶点的个数。初始时，所有顶点的访问标志置为 UNVISITED 或 0，当访问过后，顶点的访问标志置为 VISITED 或 1。在遍历时，检查图中所有的顶点，如果它的标志是 UNVISITED，则输出相关信息，进行相应的处理，同时将访问标志置为 VISITED。

否则，跳过该顶点，继续对下一顶点的访问。有了这个数组，既能防止对顶点的重复访问，也不会漏掉哪个顶点。

与树的遍历类似，图的搜索也需要依据某种规则进行，不会漏掉某个顶点或多次访问一个顶点，同时还能保证算法的有效性。遍历的方式主要有两种，分别是深度优先搜索（Depth First Search，DFS）及广度优先搜索（Breadth First Search，BFS）。

一、深度优先搜索

深度优先搜索类似于树的先序遍历，遍历过程如下。

（1）选择图中任意指定的顶点作为起始顶点，将其设为当前顶点。

（2）访问当前顶点 v，输出关于 v 的信息。

（3）将 v 的访问标志置为 VISITED。

（4）如果 v 的邻接点中存在未被访问的顶点 w，则将 w 设为当前顶点，转到（2）继续；否则转到（5）。

（5）回退到最近被访问过的且仍有未被访问邻接点的顶点 u，转到（4）继续；在不能回退时，转到（6）。

（6）如果所有顶点均已访问，则遍历结束，否则，选择未被访问的另一个顶点作为起始顶点，继续上述过程。

退出遍历过程时有两种可能的情况，一种情况是，在退出时，图中所有的顶点都被访问过，这表明图是连通图；另一种情况是，在退出时，图中还有未被访问的顶点，这表明从起始顶点开始，找不到可以到达那里的路径，也就是说，图是不连通的。遍历过的这些顶点都在同一个连通分量上，未被遍历的顶点都处在其他的连通分量上。这时，只能再从新的未被访问过的顶点出发，重复这个过程，遍历另一个连通分量上的各个顶点。当图中存在未被访问的顶点时，继续这个过程，直到图中的所有顶点都被访问过为止。每次退出时，都得到一个连通分量。

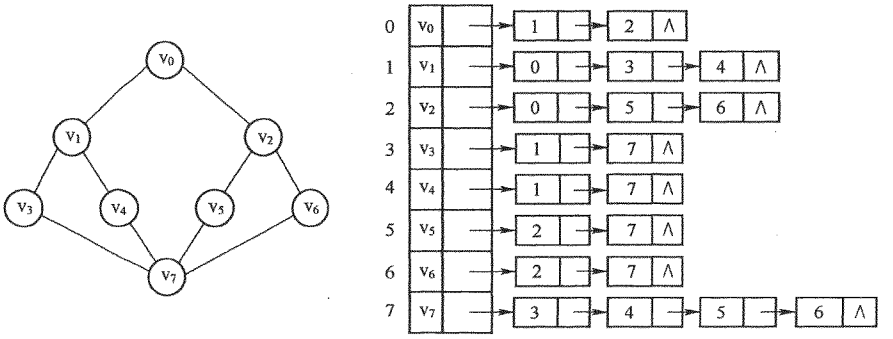
直观地看，深度优先搜索即从起始顶点开始，沿着一条路径尽可能地向前搜索，在不能再向前时（这个顶点的所有邻接点都已被访问过），就往回退，回退过程也称为回溯。回溯时沿着与访问次序相反的顺序，当回溯到仍有未被访问的邻接点的一个顶点时，把这个点当成起始顶点，再找一条路径以继续向前搜索，这个过程类似刚才的过程。因此，图的深度优先搜索过程是一个递归过程。

回溯时，需要回退到当前顶点之前被访问的顶点，因此需要记录顶点的访问顺序，以便回溯的时候能够正确地找到前一个顶点。从深度优先搜索的过程可知，顶点的访问顺序与回溯的顺序刚好相反，即“后进先出”顺序，因此，在实际操作时，选择栈来保存访问顶点的顺序信息。

【例 6-9】 深度优先搜索示例。

设有无向图 G，如图 6-10a 所示，图 G 的邻接表如图 6-10b 所示，从顶点 v_0开始，进行深度优先搜索。

从 v_0开始，访问 v_0，根据邻接表找到 v_0的第一个邻接点 v_1，访问 v_1；继续从 v_1开始，寻找 v_1的邻接点，v_1的邻接点有 v_0、v_3和 v_4，找到未曾访问过的邻接点 v_3并访问之。重复这样的过程。在依次访问 v_0、v_1、v_3、v_7、v_4后，由于 v_4的所有邻接点均已被访问过，因此，



a) 无向图G　　b) 图G的邻接表

图 6-10　图的深度优先搜索示例

退回到 v_4的前一个顶点 v_7，寻找 v_7的下一个未曾被访问的邻接点 v_5；从 v_5开始，继续这个过程。接下来访问的顶点依次为 v_5、v_2、v_6。到此，所有的顶点都被访问过。图 6-10a 的深度优先搜索访问序列为：$v_0, v_1, v_3, v_7, v_4, v_5, v_2, v_6$。

在深度优先搜索过程中，系统中使用一个栈来记录顶点访问顺序。当访问某个顶点时，将该顶点入栈，当回退时，将顶点从栈顶弹出，新的栈顶就是要回退到的顶点。使用栈很容易找到回退的位置。在上述深度优先搜索过程中，栈的变化情况如图 6-11 所示。

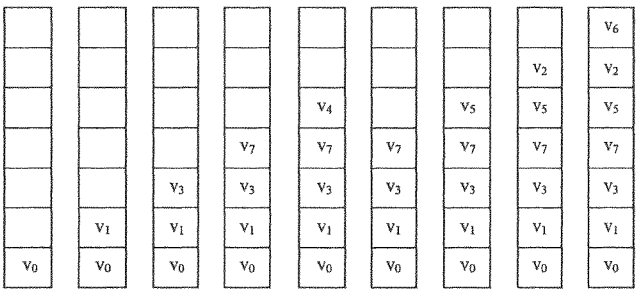


图 6-11　深度优先搜索过程中栈的变化情况

当访问完 v_6后，依次出栈，最终栈为空。

在对图进行深度优先搜索时，需要借助图的存储结构找到当前顶点的未曾被访问的邻接点，如求解例 6-9 时，在邻接表中可以找到某个顶点的所有邻接点，并借助于访问标志数组确定下一个要访问的顶点。

如果邻接表中邻接点的存储顺序改变了，则寻找的下一个未曾被访问的顶点也将有所变化。这表明，图的深度优先搜索过程在某种程度上将依赖于邻接表的存储次序及邻接点的选择算法，遍历的顺序可能有多个结果。另外，选择的初始顶点不同，遍历的结果也不同。

例如，$v_0, v_1, v_4, v_7, v_5, v_2, v_6, v_3$也是图 6-10a 从 v_0开始的深度优先搜索结果。$v_1, v_0, v_2, v_5, v_7, v_3, v_4, v_6$是从 v_1开始的一种深度优先搜索结果。

图的深度优先搜索算法如下。

```
void DFS(MGraph g){                          //图的深度优先搜索
    int number,i;
    int visited[MaxVtxNum];                  //定义 visited[]
    number=getNumVertices(g);
    for (i=0;i<number;i++)                   //初始化 visited[]
        visited[i]=0;
    for (i=0;i<number;i++)
        if (visited[i]==0)
            DFS1(g,NumToVer(g,i),visited);
}
void DFS1(MGraph g,VType v,int *visited){
    VType w;
    printf("%c ",v);                         //访问顶点 v
    visited[VerToNum(g,v)]=1;                //将 visited[v]置为 1,表示已经访问过
    w=FirstNeighbor(g,v);                    //取 v 的第一个邻接点 w
    while (w!='#'){
        if (!visited[VerToNum(g,w)])         //若顶点 w 还未曾被访问
            DFS1(g,w,visited);               //递归调用
        w=NextNeighbor(g,v,w);               //取下一个邻接点
    }
}
```

实现的遍历算法包含两个方法：DFS(MGraph g)和 DFS1(MGraph g,VType v,int *visited)。在 DFS 方法中，进行初始准备，设置访问标志。而 DFS1 方法将实际遍历图中各顶点，并且递归调用。visited[]的初始设置不能放在 DFS1 方法中，否则，在反复调用时，会改变已得到的访问标志。

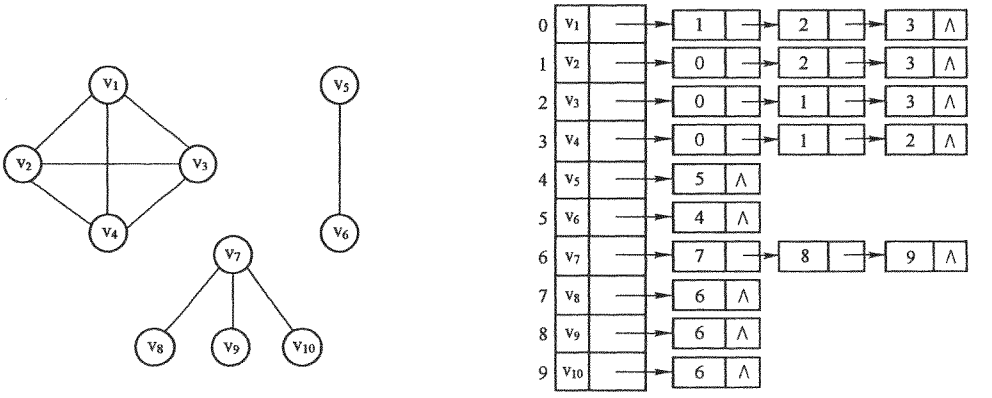
当图是连通图时，使用 DFS 算法从某个顶点出发，可以访问到图中的所有顶点。如果图不连通，则一次调用 DFS 方法不能访问图中的全部顶点，只能访问初始顶点所在的连通分量中的所有顶点。在从 DFS 方法退出后，需要再从图中其他未曾被访问的顶点开始，继续深度优先搜索过程以访问其他连通分量上的顶点。因此，在 DFS 方法中使用一个循环来调用 DFS1 方法。如果某个顶点已被访问过，则不需要再从它开始进行深度优先搜索。实际上，当图的边集非空时，循环体的执行次数严格少于图中的结点数。

【例 6-10】 利用 DFS 得到连通分量示例。

图 6-12a 是一个包含三个连通分量的非连通图 G，图 G 的邻接表如图 6-12b 所示，使用深度优先搜索策略求出图 G 的所有连通分量。

根据图 G 的邻接表，对图 6-12a 进行深度优先搜索，三次调用 DFS 函数，选择每次的起始顶点分别是 v_1、v_5和 v_7，得到 3 个访问序列，对应于 3 个连通分量：$\{v_1,v_2,v_3,v_4\}$、$\{v_5,v_6\}$和$\{v_7,v_8,v_9,v_{10}\}$。

在深度优先搜索图的算法中，用到两个主要操作，一个是寻找某顶点的第一个邻接点，另一个是寻找这个顶点的下一个邻接点。此外，遍历过程中访问顶点的操作可以通过编写方法 visit(v)来完成，函数 DFS 中使用 printf 语句来替代。在遍历算法中，时间主要用在以上



a) 有三个连通分量的图G　　b) 图G的邻接表

图 6-12　求无向图 G 的连通分量示例

三部分操作上，因为访问顶点的操作因应用的不同而不同，所以仅讨论前两种操作所花费的时间，以此作为深度优先搜索图的时间代价。

深度优先搜索图的时间代价因存储结构不同而有所不同。当将邻接矩阵（即二维数组）作为图的存储结构时，查找与每个顶点相关联的边的操作，需要遍历邻接矩阵中某一行的信息，时间复杂度为 $O(|V|)$，而对图中每个顶点，最多只调用一次 DFS()，则遍历所有顶点的时间复杂度为 $O(|V|^2)$，其中 $|V|$ 是图中顶点的个数；当将邻接表作为存储结构时，邻接表各单链表中的所有结点共有 $2\times|E|$ 个，这里，$|E|$ 是图中的边数。这样，遍历边的时间复杂度为 $O(|E|)$。遍历每个顶点的次数最多为 1，深度优先搜索的时间复杂度为 $O(|V|+|E|)$。

二、广度优先搜索

广度优先搜索是遍历图的另一种常用方法，又称为宽度优先搜索，它类似于树的层序遍历。广度优先搜索从图中任意指定的起始顶点 v 开始，访问顶点 v；然后访问与 v 相邻接的所有尚未被访问过的顶点，之后再依次访问与这些新近被访问过的那些顶点相邻接的尚未被访问过的顶点。继续这个过程，直到图中所有的顶点均被访问过为止。

广度优先搜索的过程如下。

（1）选择图中指定的顶点 v 作为起始顶点，将其入队列，且将其访问标志置为 VISITED。

（2）队列为空时，转到（5）；队列不为空时，出队列，设出队列的顶点为 w。

（3）输出 w 的相关信息。

（4）找到与顶点 w 相邻接的且访问标志不是 VISITED 的顶点序列 $w_1, w_2, \cdots, w_k$，依次入队列，转到（2）。

（5）如果所有顶点均已被访问，则遍历结束，否则，选择未被访问的一个顶点作为起始顶点，继续上述过程。

在广度优先搜索过程中，使用一个队列作为辅助存储结构，顶点是一批批加入队列的。加入队列的次序也就是被遍历的次序。如果顶点 v 早于顶点 w 被访问，则访问顶点 v 的邻接点，也会早于访问顶点 w 的邻接点。既是 v 的邻接点也是 w 的邻接点的顶点

除外。

与深度优先搜索类似，如果图是不连通的，则这个过程也不能访问图中的全部顶点，而只能遍历图中某个连通分量中的所有顶点。当广度优先搜索结束时，需要选择另外一个连通分量中的顶点作为起始顶点，开始新的广度优先搜索。

【例 6-11】 广度优先搜索示例。

仍以图 6-10a 为例，它的邻接表如图 6-10b 所示，从顶点 v_0开始，对图进行广度优先搜索。

从顶点 v_0开始，入队列且给 v_0打标记，出队列，访问它；由其邻接表得知，v_0的邻接点为 v_1、v_2，将这两个顶点依次入队列并打标记；然后出队列，得到顶点 v_1，访问它。v_1的邻接点有 v_0、v_3和 v_4，而 v_0已经打了标记，因此只将它的未标记的邻接点依次入队列，此时，队列中的顶点有 v_2、v_3、v_4；再出队列，得到的顶点是 v_2，访问它，将它的所有未标记的邻接点依次入队列且打标记，此时队列中增加两个新顶点 v_5和 v_6；继续这个过程；当队列中剩下 v_4、v_5、v_6、v_7时，出队列且访问 v_4，它没有未标记的邻接点了，所以不再需要入队列，v_5、v_6、v_7的情况与此类似。当访问完 v_7后，队列为空，图中的所有顶点也全部访问完毕。得到的广度优先搜索的顶点序列为：$v_0, v_1, v_2, v_3, v_4, v_5, v_6, v_7$。

从本例可以看出，在广度优先搜索过程中，先入队列的顶点将先被访问，某顶点的各邻接顶点入队列的顺序依赖于它在图的邻接表中的次序及相应的选择算法，因此，与深度优先搜索一样，广度优先搜索序列可能不唯一，但当邻接表及入队列算法确定后，这个顺序就确定了。$v_0, v_2, v_1, v_6, v_5, v_4, v_3, v_7$也是图 6-10a 的一种广度优先搜索序列，当然，使用的邻接表不同于图 6-10b。选择不同的起始顶点，广度优先搜索的序列也会有变化，如 $v_1, v_0, v_3, v_4, v_2, v_7, v_5, v_6$是从 v_1开始的一种广度优先搜索序列。

图的广度优先搜索算法如下所示。

```
void BFS(MGraph g,VType v)                    //图的广度优先搜索
{   int number,i;
    SeqQueue q;                               //创建队列 q
    VType w;
    int visited[MaxVtxNum];                   //定义 visited[]
    initQueue(&q);                            //初始化队列 q
    number=getNumVertices(g);
    for (i=0;i<number;i++) visited[i]=0;      //初始化 visited[]
    printf("%c ",v);                          //访问顶点 v
    visited[VerToNum(g,v)]=1;                 //表示 v 已经被访问过
    enqueue(&q,v);                            //将顶点 v 入队列
    while (!isEmpty(q)){                      //若 q 不为空
        dequeue(&q,&v);                       //出队列
        w=FirstNeighbor(g,v);                 //取 v 的第一个邻接点 w
        while (w!='#'){
            if (!visited[VerToNum(g,w)]){
                printf("%c ",w);              //访问顶点 w
```

```
            visited[VerToNum(g,w)]=1;          //表示 w 已经被访问过
            enqueue(&q,w);                     //将 w 入队列
        }
        w=NextNeighbor(g,v,w);                 //取下一个邻接点
    }
}
```

直观地看，从一个顶点开始，广度优先搜索图的过程即由近及远，依次访问和该顶点有路径相通且路径长度为 1、2……的顶点的过程。

广度优先搜索算法由二重循环实现，当队列不为空时，反复执行外层 while 循环，最多执行|V|次，这里 V 是图的顶点集合。内层循环中的主要操作是顶点入队列及寻找下一个邻接点。图中每个顶点入队列一次且出队列一次，因此内层 while 循环最多执行|V|次。当用邻接矩阵表示图时，检查每个顶点的所有邻接点的时间复杂度为 $O(|V|)$，总的时间复杂度为 $O(|V|^2)$。当用邻接表表示图时，对于一个顶点，要将其邻接点入队列。入队列操作最多要进行 2×|E|次判断，入队列|E|个。广度优先搜索算法的时间复杂度为 $O(|V|+|E|)$。

第五节　图的生成树与图的最小代价生成树

在无向连通图中，任何顶点都可以有到达图中其他任何顶点的路径，这正是连通图的连通性所保证的。甚至，从一个顶点到达另一个顶点的路径可能不止一条，从而构成回路。在有些情况下，需要在多条冗余的路径中去掉回路中的边。更进一步地，当边具有权值时，还需要有选择地选出满足条件的边。这就涉及生成树及最小代价生成树问题。本节讨论无向图的生成树概念及求最小代价生成树的两个经典算法。

一、图的生成树

某公司要在一个小区内铺设电缆，要求电缆能通达每幢楼，以保证各楼之间都能传递电信号。楼与楼之间的距离不等，铺设所费工时可能不同，铺设电缆的成本也可能各不相同。如何设计电缆铺设方案以使总成本最低呢?

可以用顶点表示各个楼，边表示楼之间的电缆，权表示两幢楼之间的电缆铺设成本。要保证各楼之间都能传递电信号，这个图必须是连通的，也就是说，从任何一幢楼到其他任何楼必然有路径存在。显然，任何两幢楼之间只需要一条路径就可以保证电信号的传递。要使总成本最低，两幢楼之间存在两条或两条以上的路径是多余的。比如，如果两幢楼之间存在两条路径，那么这两条路径必然构成回路。显然，去掉回路中的任何一条边，仍能保证图的连通性，总成本也随之降低。

设 G=(V,E)是一个连通的无向图，包含图中全部顶点的极小连通子图称为图的生成树。极小连通子图是指含有图中所有顶点并使图仍保持连通性的最小子图。由生成树的定义可以看出，生成树中包含两个要素，一个是要包含图中的全部顶点，另一个是在保证图连通的前提下所含的边数最少。

如何得到连通图的生成树呢？实际上，借助于本章第四节介绍的图的搜索算法，可以得

到连通图的生成树。

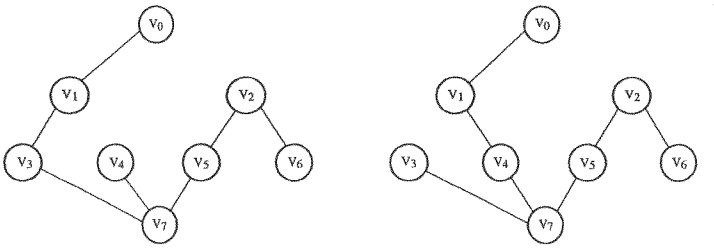
从连通图中任一顶点出发，按照深度优先搜索策略或广度优先搜索策略，可以访问到图中的全部顶点。在遍历的过程中，所经过的边集设为 T(G)，没有经过的边集设为 B(G)。T(G)即构成 G 的一棵生成树。

进行深度优先搜索时得到的生成树称为深度优先生成树，进行广度优先搜索时得到的生成树称为广度优先生成树。我们已经知道，在进行遍历时，可能会得到不同的顶点序列，那么经过的边可能是不一样的，表明图的生成树也可能是不唯一的。进一步地，即使是同一个图的深度优先生成树，也可能不是唯一的。对于广度优先生成树，也有同样的结论。

使含 n 个顶点的无向图连通的最少边数是 $n-1$，所以生成树中所含的边数必定是 $n-1$。边数少于这个值，图肯定是不连通的；边数多于这个值，则必定存在回路，也满足不了边数最少这个条件。也就是说，若在生成树中任意增加一条 B(G)中的边，则必然形成回路。而从生成树中去掉任何一条边，都破坏了图的连通性。

【例 6-12】 仍以图 6-10a 为例，求其生成树。

在对图 6-10a 所示的无向图从顶点 v_0开始进行深度优先搜索，并得到遍历序列 $v_0, v_1, v_3, v_7, v_4, v_5, v_2, v_6$时，经过的边组成如图 6-13a 所示的深度优先生成树。在得到遍历序列 $v_0, v_1, v_4, v_7, v_5, v_2, v_6, v_3$时，经过的边组成如图 6-13b 所示的深度优先生成树。



a) 深度优先生成树1　　b) 深度优先生成树2

图 6-13　图 6-10a 对应的两棵深度优先生成树

在对图 6-10a 所示的无向图从顶点 v_0开始进行广度优先搜索，并得到遍历序列 $v_0, v_1, v_2, v_3, v_4, v_5, v_6, v_7$时，经过的边组成如图 6-14 所示的一棵广度优先生成树。

如果一个图中含有回路，则图的生成树并不唯一。这是因为遍历图时选择的起始点不同，遍历的策略不同，因此遍历所经过的边就可能不同，故而产生包含不同边的生成树。

对于不连通的无向图，从任意顶点出发，一次调用 DFS 或 BFS 算法不能访问到图的所有顶点，只能得到连通分量的生成树。一个图的所有连通分量的生成树组成图的生成森林。

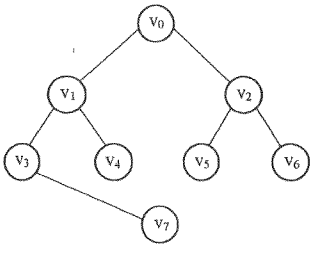


图 6-14　图 6-10a 对应的一棵广度优先生成树

二、图的最小代价生成树

带权图的每条边上都有一个非负的权值，各边的权值可能不全相等，而现在又知道一个连通图的生成树可能有多棵。虽然生成树所包含的边数是一样的，但对于生成树中所有边的

权值之和，可能会因选择的边不同而导致结果不等。其中权值之和最小的那棵生成树具有特殊意义。将边的权值之和最小的生成树称为最小代价生成树（Minimum-cost Spanning Tree，MST），简称为最小生成树。

给定一个带权无向连通图 G，则 MST 是一个包括 G 的所有顶点及其边子集的图，边的子集满足下列条件：

（1）这个子集中所有边的权值之和为所有满足条件的子集中最小的。

（2）子集中的边能够保证图是连通的。

最小生成树问题在实际中很有意义。比如，铺设电缆的问题实际上可以转化为求图的最小生成树问题。

含有 n 个顶点的连通图 G 的最小生成树有下列性质：

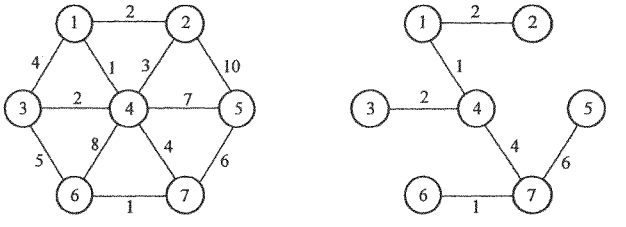
（1）它含有图 G 中所有的 n 个顶点。

（2）它没有回路，因为从构成回路的各边中去掉一条边，仍能保证其连通性，而所得的权值总和可以进一步减少。

（3）它含有的边数为 $n-1$。

（4）去掉最小生成树中的一条边，换上不在最小生成树中的另外一条边，在仍要求图连通的前提下，所得的权值总和都不会小于原最小生成树的权值总和。

【例 6-13】 对于如图 6-15a 所示的带权无向连通图，它的 MST 如图 6-15b 所示。



a) 带权无向连通图G　　b) 图G的MST

图 6-15　带权无向连通图及其 MST

求带权无向连通图的最小生成树有两个经典的算法，分别是普里姆（Prim）算法和克鲁斯卡尔（Kruskal）算法。

1. 普里姆算法

普里姆给出的一个算法可以构造图的最小生成树。该算法先定义一棵树 T 和一个顶点集合 U，T 用来表示已得到的构成最小生成树的边，初始时为空，结束时将包含求得的最小生成树中的所有边。U 表示到目前为止 T 所涉及的顶点，初始时只含有初始顶点，初始顶点可以选为图中的任何顶点，结束时应包含图中的所有顶点。

普里姆算法从图中任选的初始顶点开始，不断地将符合条件的边并入目标集合 T 中，同时将边所依附的顶点并入 U 中，直到 U 含有图中的所有顶点为止。每一步选择一条边并入 T，同时 U 中增加一个顶点。算法共需 $n-1$ 步。

那么，如何选择进入 T 中的边呢？将与 U 中任何顶点关联的所有边按权值进行排序，选择权最小的边，设为(v,w)，此时 $v\in U$。如果 $w\notin U$，则(v,w)就是被选中的进入 T 中的边，w 加入到集合 U 中；否则，这条边依附的两个顶点都在集合 U 中，舍弃该边，选择下

一条满足条件的权最小的边。在这个过程中，每次满足条件时选择一条边、一个顶点，分别加入 T 和 U 中。

普里姆算法的具体过程如下。

设连通的带权图 G=(V,E)，V 是顶点的集合，E 是边的集合。

（1）初始化。设 T 是图 G 的最小生成树的边集合，U 是最小生成树的顶点集合，由顶点 v_1 开始，初始时，T=ϕ，U=$\{v_1\}$。

（2）在所有 $u\in U$，$v\in V-U$ 的边$(u,v)\in E$ 中，选择一条权最小的边(u_i,v_j)，将 v_j 并入 U 中，将边(u_i,v_j)并入 T 中。

（3）重复步骤（2），直到 U=V 为止。

在该算法的实现步骤（2）中，需要从众多的边中选择满足条件的边。若 V-U 中的一个顶点与 U 中的多个顶点间均有边，则只需要让其中权最小的边参与选择，即在步骤（2）中选择权最小的边时，参与选择的边数最多为|V-U|。为此，设置一个辅助数组 closedge[vtxnum]，用它来记录一个顶点在 U 中，而另一个顶点在 V-U 中的具有最小权的边。数组的每个元素都有两个域，分别是 vex 和 lowcost，其中 vex 存储该边依附在 U 中的顶点，lowcost 存储该边的权。

【例 6-14】 对于如图 6-16a 所示的带权无向图 G，它的 MST 如图 6-16b 所示。图 6-17 是最小生成树的构造过程，其中数组 closedge 的变化情况见表 6-1。

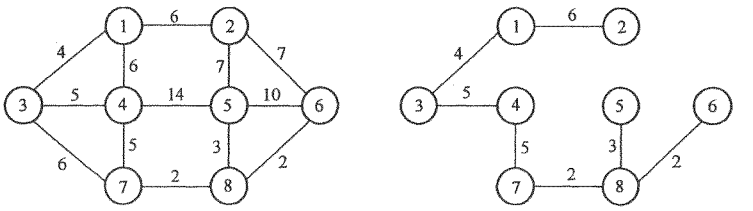


图 6-16　一个带权无向图及其 MST

表 6-1　普里姆算法中 closedge 数组中各分量的值

V / closedge	2	3	4	5	6	7	8	U	V-U
vex lowcost	1 6		1 6					{1}	{2,3,4,5,6,7,8}
vex lowcost	1 6					3 6		{1,3}	{2,4,5,6,7,8}
vex lowcost	1 6			4 14				{1,3,4}	{2,5,6,7,8}
vex lowcost	1 6			4 14				{1,3,4,7}	{2,5,6,8}
vex lowcost	1 6			8 3				{1,3,4,7,8}	{2,5,6}
vex lowcost	1 6							{1,3,4,6,7,8}	{2,5}
vex lowcost								{1,3,4,5,6,7,8}	{2}
vex lowcost								{1,2,3,4,5,6,7,8}	Φ

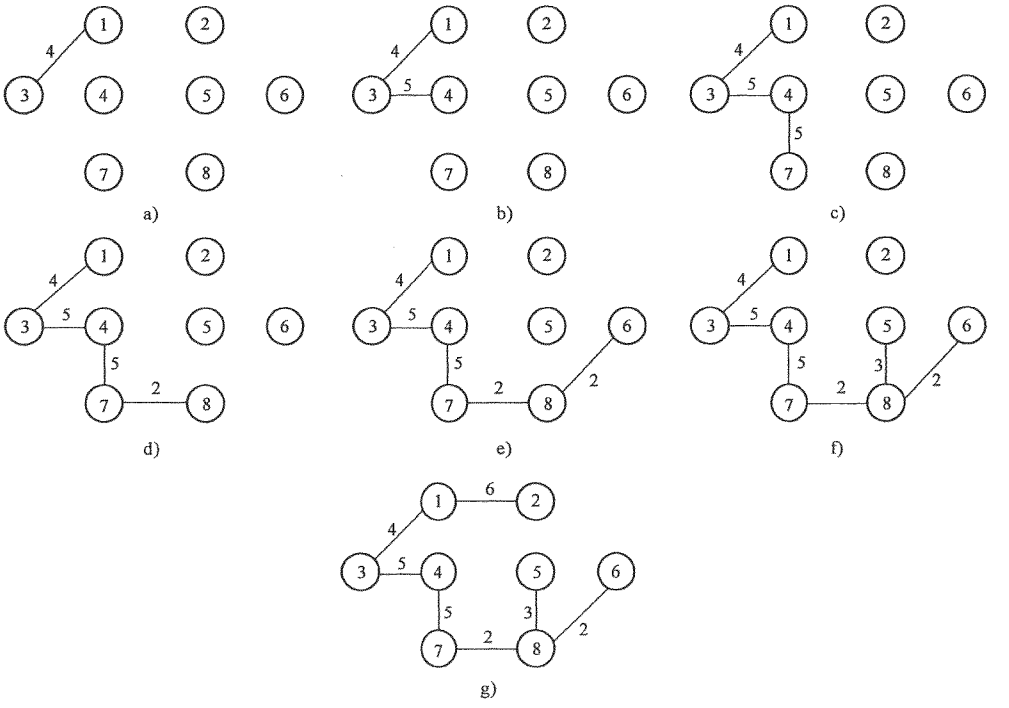


图 6-17　普里姆算法构造最小生成树的过程

第一步，选择顶点 1 作为初始顶点，其两个端点分别处于 U 和 V-U 两个集合的边只有 3 条，分别是(1,2)、(1,3)和(1,4)，选择其中权最小的边(1,3)，见表 6-1 中第一行。将顶点 3 加入 U 中。

第二步，其两个端点分别处于 U 和 V-U 两个集合的边有 4 条，分别是(1,2)、(1,4)、(3,4)和(3,7)。对于顶点 4，它与 U 中两个顶点都有边相连，按照选择权值最小的原则，只需要保留一条，选择边(3,4)，而不必保留(1,4)。这样，在表 6-1 中，只记录 3 条边的信息，见表中第二行。此时，选中的顶点是顶点 4，将它加入 U 中。现在，U 中含有顶点 1、顶点 3 和顶点 4。

第三步，对顶点 7 的处理类似于前面对顶点 4 的处理，也只需要保留一条边，即(4,7)。此时，有 3 条候选边，分别是(1,2)、(4,5)和(4,7)，选中的顶点是顶点 7，加入 U 中。

第四步，对于顶点 2、顶点 5 和顶点 8，候选边分别是(1,2)、(4,5)、(7,8)，选中的顶点是顶点 8，加入 U 中。

第五步，对于顶点 2、顶点 5 和顶点 6，候选边分别是(1,2)、(5,8)和(6,8)，选中的顶点是顶点 6，加入 U 中。

第六步，对于顶点 2 和顶点 5，候选边分别是(1,2)和(5,8)，选中的顶点是顶点 5，加入 U 中。

最后一步，选中顶点 2。到此为止，得到最小生成树。

按照所选的次序，构成最小生成树的边依次是(1,3)、(3,4)、(4,7)、(7,8)、(6,8)、(5,8)和(1,2)，最小生成树所含边的权值总和为 27，可以验证，这是权值总和最小的一棵

生成树。

普里姆算法是贪心算法的一个实例，每次选出一条连接 U 中顶点及 V-U 中顶点的具有最小权值的边，并逐步生成最小生成树。

最小生成树的类型如下。

```
typedef struct MSTEdgeNode{                    //带权边
    VType ver1,ver2;                           //边的两个端点
    WType key;                                 //边上的权值
}MSTEN;
typedef struct MinSpanTree{                    //最小生成树
    MSTEN edgeArray[MaxVtxNum];                //MST 中边的数组
}MinST;
```

算法中使用的辅助数组 closedge 的类型定义如下。

```
typedef struct {
    VType vex;          //某顶点与已构造好的部分生成树的顶点之间权值最小的顶点
    int lowcost;        //某顶点与已构造好的部分生成树的顶点之间的最小权值
}ClosEdge;              //普里姆算法求最小生成树时的辅助数组
```

算法实现如下。

```
int Prim(MGraph g,int u,ClosEdge closedge[ ],MinST *T)
{
    /* 从顶点 u 开始构造图 g 的最小生成树，计算过程体现在辅助数组 closedge 中，最小生成树保存在 T 中 */
    int i,j,w,k,count=0;
    int n=g.numVertices;                       //结点个数
    T->totalcost=0;
    for (i=0;i<n;i++)                          //辅助数组初始化
        if (i!=u){
            closedge[i].vex=NumToVer(g,u);
            closedge[i].lowcost=g.AdjMatrix[u][i];
        }
    closedge[u].lowcost=0;                     //初始时，U={u}
    for (i=0;i<n-1;i++){                       //选择其余的 n-1 个顶点
        w=NB;
        for (j=0;j<n;j++)                      //在辅助数组 closedge 中选择权值最小的顶点
            if (closedge[j].lowcost!=0 && closedge[j].lowcost<w){
                w=closedge[j].lowcost;
                k=j;
            }                                  //找到生成树的下一个顶点 k
        closedge[k].lowcost=0;                 //第 k 个顶点并入集合 U
        T->edgeArray[count].ver1=NumToVer(g,k);
```

```
            T->edgeArray[count].ver2=closedge[k].vex;
            T->edgeArray[count].key=w;
            T->totalcost+=w;
            count++;
            for (j=0;j<n;j++)                    //在新顶点 k 并入 U 后，修改辅助数组
                if (g.AdjMatrix[k][j]<closedge[j].lowcost){
                    closedge[j].vex=NumToVer(g,k);
                    closedge[j].lowcost=g.AdjMatrix[k][j];
                }
        }
        for (i=0;i<n-1;i++)
            printf("%c %c %d\n",T->edgeArray[i].ver1, T->edgeArray[i].ver2,T->edgeArray[i].key);
        printf("MST = %d\n",T->totalcost);
        return T->totalcost;
    }
```

在主函数中，定义如下变量，然后调用 Prim 函数，选择图 g 的第一个顶点作为初始顶点。

```
MinST T;
ClosEdge closedge[ MaxVtxNum ];
int u=0;
Prim(g,u,closedge,&T);
```

对于含 $n=|V|$ 个顶点的图，普里姆算法共进行 $n-1$ 轮的选边操作，每轮选边时，从最多 $n-1$ 条边里选中权值最小的边。算法的时间复杂度为 $O(|V|^2)$。当图是一个边稠密的图时，适合使用普里姆算法来求图的最小生成树。

2. 克鲁斯卡尔算法

克鲁斯卡尔算法是构造图的最小生成树的另一种经典算法。

初始时，将图中的所有边按权值不减的顺序排列，并将图中的所有顶点加入生成树中，但不包含任何的边。此时，生成树是只有顶点而没有任何边的非连通图，这些顶点分别自成一个连通分量，连通分量的个数 $n=|V|$。循环做以下事情：从边集中选择并删去权值最小的边，如果这条边所依附的两个顶点分别在生成树的两个连通分量上，则将这条边添加到生成树中，并令边所依附的两个顶点所在的两个连通分量合并成一个，生成树中连通分量的个数减 1；如果这条边所依附的两个顶点在同一个连通分量上，则舍弃这条边。重复这个过程。当生成树中含有 $|V|-1$ 条边时，循环结束。此时，原来的 $|V|$ 个连通分量最终合并成一个。

设 T 是无向连通图 $G=(V,E)$ 的最小生成树，T 的顶点集合为 V，边集为 E_1，构造最小生成树的克鲁斯卡尔算法的步骤如下。

（1）初始化。设 E_1 的初值：$E_1=\Phi$，T 中只含有图中的所有顶点，顶点个数为 $n=|V|$。

（2）当 E_1 中边数小于 $n-1$ 时，重复执行下面的步骤：

① 在图 G 的边集 E 中选择权值最小的边(u,v)，并从 E 中删除它。

② 如果 u 和 v 分别属于 T 中不同的连通分量，则将边(u,v)加入 E_1 中，否则丢掉该边，选择 E 中的下一条权值最小的边。

在克鲁斯卡尔算法中，每次只选取权值最小的边，可以将这些边先排序，并保存在一个线性表中，组成一个有序表。每次从有序表中找到排在第一个的边，并进行处理。

初始时，T 中保存的是 n 个顶点，可以看作 n 个单结点的树构成的森林。在算法执行过程中，T 中每增加一条边，都会使两棵树合并为一棵。当构造过程结束时，T 中只剩下一棵树，这就是所求的最小生成树。

对于稀疏图，其边数远小于$|V|^2$，此时适合采用克鲁斯卡尔算法求图的 MST。

【例 6-15】 仍以例 6-14 中使用的带权无向连通图为例，使用克鲁斯卡尔算法求它的最小生成树，具体过程如图 6-18 所示。

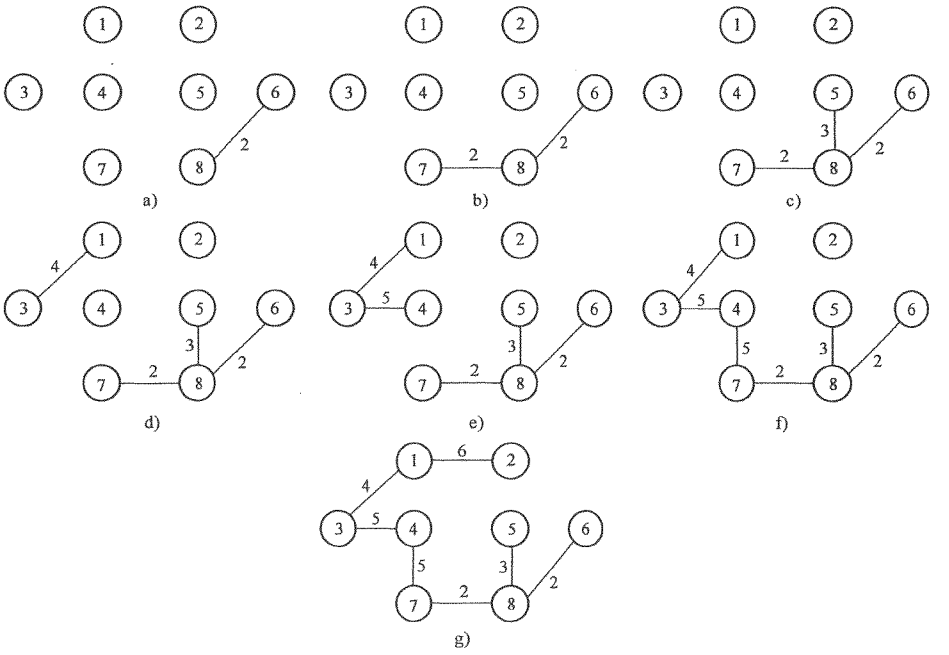


图 6-18　克鲁斯卡尔算法构造最小生成树的过程

图 6-16a 所示的图中有 13 条边，先将各边按权值从小到大排序，列在表 6-2 中。

表 6-2　图 6-16a 所示图中的全部边

边	权　值	边	权　值
(6,8)	2	(1,2)	6
(7,8)	2	(3,7)	6
(5,8)	3	(2,5)	7
(1,3)	4	(2,6)	7
(3,4)	5	(5,6)	10
(4,7)	5	(4,5)	14
(1,4)	6		

依次选择权值最小的边，并判断边所依附的两个顶点是否在同一个连通分量上。显然，前两条边必定会被选中，因为它们是权值最小的两条边，且不会属于同一个连通分量。

接下来看第三条边(5,8)，它的两个顶点也不在同一个连通分量上，所以被选中。按照同样的原则，在后面三步中，依次被选中的边是(1,3)、(3,4)和(4,7)。

第七条边是(1,4)，它的两个顶点已经属于同一个连通分量，所以舍弃。接下来，选中边(1,2)，由此得到最小生成树。

可以看出，因为权值相同的边的排列次序可以对换，所以选择时可能会得到不同的选择结果。但无论如何，最小生成树的权值是一定的。

为了不破坏原图的存储结构，使用一个临时数组保存各条边。数组元素的类型和最小生成树的类型如下。

```
typedef struct MSTEdgeNode{                    //带权边
    VType ver1,ver2;                           //边的两个端点
    WType key;                                 //边上的权值
}MSTEN;
typedef struct MinSpanTree{                    //最小生成树
    MSTEN edgeArray[MaxVtxNum];                //MST 中边的数组
}MinST;
```

在实现克鲁斯卡尔算法时，首先要对图中的各条边按权值进行排序，算法实现如下。

```
void edgeSort(MGraph g,MSTEN edge[])           //边排序
{   int i,j,k=0;
    MSTEN temp;
    int v=getNumVertices(g);                   //得到图中的边数
    for (i=0;i<v;i++)                          //获取各条边的信息
        for (j=i+1;j<v;j++)
            if (g.AdjMatrix[i][j]!=NB){
                edge[k].ver1=NumToVer(g,i);
                edge[k].ver2=NumToVer(g,j);
                edge[k].key=g.AdjMatrix[i][j];
                k++;
            }
    for (i=1;i<k;i++){                         //排序
        for (j=i;edge[j].key<edge[j-1].key && j>0;j--){
            temp=edge[j-1];
            edge[j-1]=edge[j];
            edge[j]=temp;
        }
    }
    return;
}
```

在算法中，需要根据权值依次挑选权值最小的边，因为边数组 edge 中已经按权值从小到大排序，所以依次进行遍历，挑选满足条件的边即可。

设当前的边是 edge[i]，如果边的两个顶点不在同一个连通分量上，则该边将被选中。为了表示顶点所属的连通分量，使用另一个临时数组 flag。初始时，每个顶点都自成连通分量，相应的元素值设置为顶点的编号。对于当前边 edge[i]，使用函数 VerToNum 可以获取它的两个端点，代码如下。

```
end1=VerToNum(g,edge[i].ver1);
end2=VerToNum(g,edge[i].ver2);
```

然后根据 flag 数组中的信息，判断它们是否属于同一个连通分量。如果不属于同一个连通分量，则将边 edge[i]添加到 MST 中，同时，边的两个端点合并为一个连通分量，即修改 flag 数组中的值。

克鲁斯卡尔算法的程序实现如下。

```
void Kruskal(MGraph g,MinST *T)                    //利用克鲁斯卡尔算法求 MST
{
    int i,n,temp;
    int k=0,k1=0;
    MSTEN *edge;                                   //保存边的临时数组
    int *flag,end1,end2;                           //用于标识连通分量
    int edgecount=g.numEdges;                      //实际的边数
    n=getNumVertices(g);                           //实际的顶点数
    edge=(MSTEN *)malloc(edgecount*sizeof(MSTEN));
    flag=(int *)malloc(n*sizeof(int));
    for (i=0;i<n;i++) flag[i]=i;                   //初始时，各顶点自成连通分量
    edgeSort(g,edge);                              //边排序
    k=0;                                           //记录选中的边数，需要选择足够的 n-1 条边
    T->totalcost=0;
    for (i=0;i<edgecount;i++,k<n-1){
        end1=VerToNum(g,edge[i].ver1);
        end2=VerToNum(g,edge[i].ver2);
        if (flag[end1]!=flag[end2]){               //当前边的两个顶点不在同一连通分量上
            T->edgeArray[k].ver1=edge[i].ver1;     //选中该边
            T->edgeArray[k].ver2=edge[i].ver2;
            T->edgeArray[k].key=edge[i].key;
            T->totalcost+=T->edgeArray[k].key;
            k++;
            temp=flag[end2];                       //合并连通分量
            for (k1=0;k1<n;k1++)
                if (flag[k1]==temp) flag[k1]=flag[end1];
        }
    }
```

```
    for (i=0;i<k;i++)
        printf("%c %c %d\n",T->edgeArray[i].ver1, T->edgeArray[i].ver2,T->edgeArray[i]
.key);
    printf("MST = %d\n",T->totalcost);
    free(edge);
    free(flag);
}
```

在主函数中，调用语句如下。

```
Kruskal(g,&T);
```

在克鲁斯卡尔算法中，从|E|条边中选出权值最小的边(u,v)，并检测 u 和 v 是不是在同一连通分量上，时间复杂度为 $O(|E|\log|E|)$。所以，克鲁斯卡尔算法适合求稀疏图的最小生成树。

第六节　有向无环图及其应用

一些实际的问题可以表示为有向无环图。例如，用有向无环图描述一个工程中各个子工程之间的关系，还可以表示含有公共子表达式的表达式等。有向无环图可用来表示有先后关系的事件集合。本节介绍有向无环图的概念及其两个应用，一个是图的拓扑排序，另一个是图的关键路径。

一、有向无环图

图中不存在回路的有向图称为有向无环图，简称为 DAG 图。前面已经介绍过，可以使用二叉树表示一个表达式，这样的二叉树称为表达式树。实际上，也可以使用有向图来表示表达式。例如，图 6-19 是一个有向图，由于它具有层次关系（没有回路），因此也可以被看成一棵有向树；它表示的是算术表达式((a+b)×c+((a+b)+e)×(e+g))×((a+b)×c)。

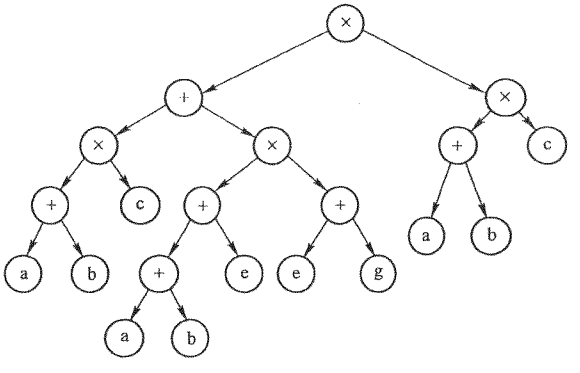


图 6-19　用有向图表示算术表达式

这个表达式中有很多重复的部分，如有三个(a+b)、两个(a+b)×c 等，这些称为公共子表达式。在进行实际存储时，这些公共部分没有必要重复存储。对于重复内容，只需要存储一个版本，在其他使用的地方仅保存一个引用，这个引用指向所保存的内容。在使用时，通过引用就可以找到实际的内容。将图 6-19 进行修改，去掉重复部分，得到图 6-20，它是一个有向无环图。

从图 6-20 中可以看出，表达式中相同的子表达式在图中只画了一次，避免了在有向树中的重复出现，方便了某些操作的实现和执行。

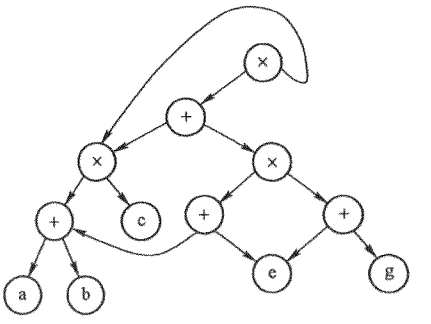


图 6-20　用有向无环图表示算术表达式

二、拓扑排序

1. 拓扑排序简介

有向无环图 G=(V,E)的拓扑序列是 G 中顶点的一个线性序列，并且满足以下关系：对于图 G 中的所有顶点 v 和 w，如果(v,w) ∈ E，则在线性序列中，v 排在 w 之前。求有向无环图拓扑序列的过程称为拓扑排序。

由拓扑序列的定义可以看出，如果有向图中存在回路，则不能给出图的拓扑序列。因此，本节仅对有向无环图进行讨论。

拓扑排序有其实际的应用背景。例如，可以用有向无环图表示一个工程，图中用顶点表示工程中的各个子任务，顶点之间的边表示任务之间的先后顺序。若两个任务之间存在先后关系，则在这两个任务对应的顶点之间连接一条弧。具有先后关系的任务必须按照约定决定执行的次序，在完成了先导任务之后，才能开始后继任务；而有些任务之间没有按直接或间接方式约定任何的先后关系，这样的两个任务的执行顺序可以是任意的。在实际应用中，经常需要在不违反原来先后次序的前提下，列出所有任务之间的先后顺序，这就是拓扑排序的过程。

在大学里，可以应用拓扑排序来安排课程的选修次序。把这些课程看作有向图的一个个顶点，如果课程 A 是课程 B 的先导课，则画一条从 A 指向 B 的有向边，表示必须先修完课程 A 才能选修课程 B。这样，这些课程连同它们之间的先后关系组成一个有向图。对这个图进行拓扑排序，可以得到选修课程的顺序，从而了解选修一门课程之前应该先掌握哪些知识。

在工程管理中，有些大的工程往往分解成若干个子工程，这些子工程被称为活动(activity)。当所有的活动都完成时，整个工程即告结束。有些活动可以并行进行，而有些活动必须在另外一些活动完成后才能开始。如何安排这些活动？哪个活动可以开始？哪个活动必须等待？类似这样的问题，都要用到拓扑排序。

在有向图中，以顶点表示活动，有向边表示活动之间的优先关系，这样的有向图称为顶点表示活动的网络（activity on vertex network），简称为 AOV 网。在 AOV 网中，若从顶点 v_i 到 v_j有一条有向路径，则称 v_i是 v_j的前驱，v_j是 v_i的后继；若(v_i,v_j)是 AOV 网中的一条弧，则称 v_i是 v_j的直接前驱，v_j是 v_i的直接后继。如果 v_i是 v_j的前驱或直接前驱，则 v_i活动必须在 v_j活动开始之前结束，即 v_j活动必须在 v_i活动结束之后才能开始。

在 AOV 网中，不允许出现回路，如果有回路，则表示某个活动是以自己为先决条件的，即存在一个活动 v_i，v_i既是其本身的前驱，又是其后继，这显然是矛盾的。因此，AOV 网必是有向无环图。

在 AOV 网中，如果从顶点 v_i到顶点 v_j存在有向路径，则在拓扑序列中，v_i必定排在 v_j的前面；如果从顶点 v_i到顶点 v_j没有有向路径，则在拓扑序列中，v_i与 v_j的先后次序可以任意。

例如，可考查计算机系软件专业学生的 5 门课程之间的关系。课程代号、课程名称及先导课的情况见表 6-3。

表 6-3　课程之间的关系

课程代号	课程名称	先导课
C_1	高等数学	无
C_2	程序设计基础	无
C_3	数据结构	C_2
C_4	编译原理	C_3
C_5	算法分析	C_1、C_2

以顶点表示课程，弧尾对应的课程是弧头对应课程的先导课。表 6-3 中课程之间的关系可以用图 6-21 表示。

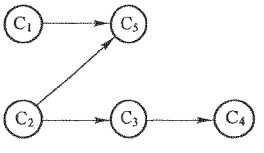


图 6-21　各门课程的选修顺序

没有先导课的课程没有入边，如 C_1和 C_2，它们可以最先被选修；而有先导课的课程都有入边，必须在学完其先导课程之后才能选修，如 C_3必须在学完 C_2之后再学，C_5必须在学完 C_1和 C_2之后再学；没有先后关系的课程，其选修的先后次序任意，如 C_5和 C_4，可以先选修 C_5，也可以先选修 C_4。上述 5 门课程排成的拓扑序列为 C_1,C_2,C_3,C_4,C_5，还可以排列成 C_1,C_2,C_5,C_3,C_4。

由于 C_3与 C_5、C_4与 C_5之间没有先后次序约定，因此可以先选修 C_3，再选修 C_5（如第一种排序），也可以先选修 C_5，再选修 C_3（如第二种排序）。可见，AOV 网的拓扑序列可能并不是唯一的。

2. 基于广度优先搜索策略实现拓扑排序

基于广度优先搜索策略，可以对 AOV 网进行拓扑排序。

拓扑排序遵循的原则是，在拓扑序列中，每个顶点都必须排在它的后继顶点之前。那么排在最前面的顶点一定不能是其他任何顶点的后继。而有向图中一个顶点的直接前驱数即顶点的入度，可以使用一个一维数组来记录各个顶点的入度。如果一个顶点不是其他任何顶点的后继，就意味着该顶点的入度为 0。初始化时，数组中填入各顶点的入度，可以将入度看作一个顶点的约束条件个数。例如，入度为 2，表明它有两个直接前驱顶点；入度为 3，表明它有 3 个直接前驱顶点。每当从图中输出一个顶点时，表明该顶点的所有直接后继顶点均少了一个约束，即入度减 1。

基于广度优先搜索策略求 AOV 网的拓扑序列的步骤如下。

（1）初始化：记录 AOV 网中所有顶点的入度。

（2）选择一个没有前驱（入度为 0）的顶点，输出之。

（3）从图中删除该顶点和以它为尾的所有的弧，即将输出顶点的所有直接后继顶点的入度减 1，然后转（2）。

步骤（2）、（3）重复执行，每输出一个顶点，就修改入度数组，然后选择满足条件的顶点输出，再修改数组，直至输出全部顶点，或者还没有输出全部顶点，但已找不到没有直接前驱的顶点（即入度为 0 的顶点）为止，第一种情况表示拓扑排序已经完成，第二种情况表明原有向图中含有回路。

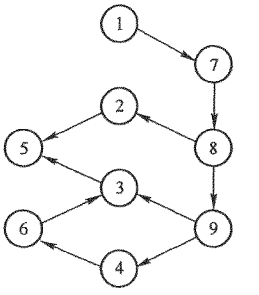


图 6-22　AOV 网

【例 6-16】求图 6-22 所示 AOV 网的拓扑序列。

在拓扑排序过程中，始终寻找入度为 0 的顶点，并输出之。为得到拓扑排序结果，先建立图中各顶点的入度表，见表 6-4。

表 6-4　图 6-22 所示图中各顶点的入度

顶　点	入　度	顶　点	入　度
1	0	6	1
2	1	7	1
3	2	8	1
4	1	9	1
5	2		

初始时，图中入度为 0 的顶点是顶点 1，输出后，将它的直接后继顶点的入度减 1，即顶点 7 的入度由 1 变为 0。图中余下 8 个顶点，如图 6-23a 所示。

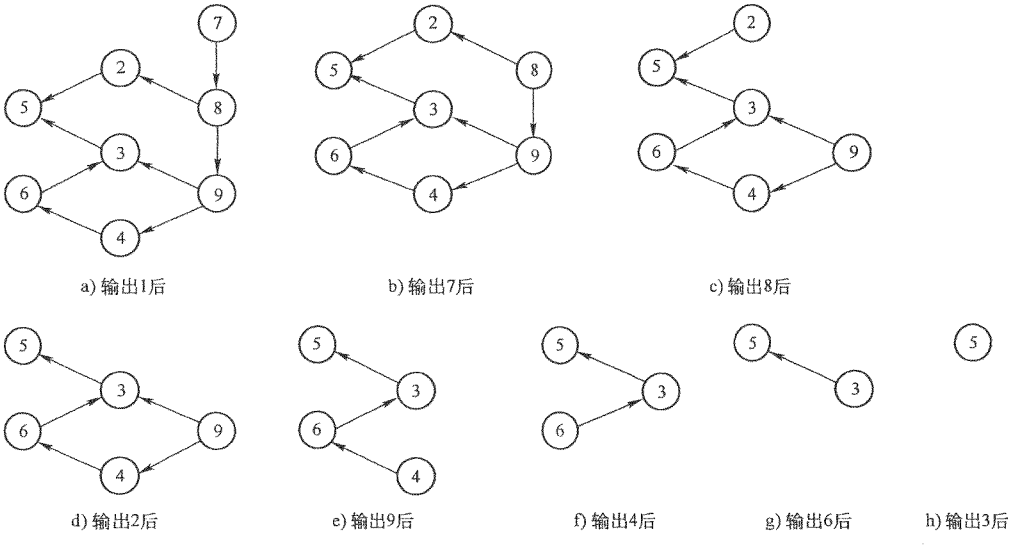


图 6-23　拓扑排序过程

此时，图中入度为 0 的顶点是顶点 7，输出 7，再将它的直接后继顶点的入度减 1，则顶点 8 的入度也由 1 变为 0。图中剩余 7 个顶点，如图 6-23b 所示。

继续输出顶点 8，并将它的两个后继顶点的入度均减 1，则顶点 2 和顶点 9 的入度均由 1

变为0。现在的图中剩余6个顶点，如图6-23c所示。

现在图中有两个顶点的入度均为0，分别是顶点2和顶点9。下一步选择哪个顶点输出都可以。假定按照顶点编号从小至大的顺序选择，则输出顶点2。输出后的图如图6-23d所示。

顶点2的直接后继顶点是顶点5，它有两个约束条件，输出顶点2后，顶点5的入度由2变为1。所以，在这一步的选择中，只能选择顶点9。输出9。得到的图如图6-23e所示。

接下来，依次输出顶点4、6、3、5。此时图中的顶点全部输出完毕，表明图是一个有向无环图，并且得到了它的拓扑序列：1,7,8,2,9,4,6,3,5。

当图中剩余顶点中存在多个入度为0的顶点时，说明这几个顶点之间并不存在先后关系，因此在拓扑排序时可以有多种选择。拓扑排序可能有多种结果，如对于图6-22，正确的拓扑序列还可以是：1,7,8,9,2,4,6,3,5。此外，1,7,8,9,4,2,6,3,5和1,7,8,9,4,6,2,3,5也是正确的拓扑序列。

在进行拓扑排序时，每当选择一个入度为0的顶点并输出后，都需要根据图的存储结构修改图的入度表，并再次选择下一个入度为0的顶点并输出。通常情况下，选择入度为0的顶点，需要遍历入度表，逐点筛选。

观察表6-4，在入度表中，那些入度为0的顶点的“入度”域的值都是0，所以可以用这个域在入度表中建立一个静态链表，专门将入度为0的顶点链接起来，方便后面的选择。

在初始建立入度表时，将入度为0的顶点逐一插入到这个静态链表中。在入度表建立完成时，静态链表的初始化也会完成。当同时有多个入度为0的顶点时，拓扑排序并不强制输出其中的哪一个，所以，在将满足要求的顶点插入静态链表时，可以插入在表头位置，这样的插入效率最高。

在输出入度为0的顶点时，即从静态链表的表头位置删除一个结点，输出其中保存的顶点。在根据存储结构寻找输出顶点的邻接点并修正入度表时，若遇到修正后入度为0的顶点，则将其插入到静态链表的表头。

入度表的结构定义如下。

```
typedef struct{
    VType ver;
    int indeg;
}IndegreeV;
Indegree VindegreeTable[MaxVtxNum];
```

建立入度表的实现如下，其中first是入度为0的静态链表的表头指针。

```
int CreateIndegrTable(MGraph g,IndegreeV *indegreeTable,int *first){
    int i,j;
    for (i=0;i<g.numVertices;i++){
        indegreeTable[i].indeg=0;
        indegreeTable[i].ver=g.verticesList[i];
        for (j=0;j<g.numVertices;j++)
            indegreeTable[i].indeg+=g.AdjMatrix[j][i];
```

```
        }
        *first=-1;
        for (j=0;j<g.numVertices;j++){
            if (indegreeTable[j].indeg==0){
                indegreeTable[j].indeg=*first;
                *first=j;
            }
        }
        return TRUE;
    }
```

基于入度表的拓扑排序算法实现如下。

```
void Topo(MGraph g,IndegreeV *indegreeTable,int first)
{   int i,temp;
    while (first!=-1){
        printf("%c ",g.verticesList[first]);
        temp=first;
        first=indegreeTable[first].indeg;
        for (i=0;i<g.numVertices;i++){
            if (g.AdjMatrix[temp][i]>0){
                indegreeTable[i].indeg--;
                if (indegreeTable[i].indeg==0){
                    indegreeTable[i].indeg=first;
                    first=i;
                }
            }
        }
    }
}
```

3. 基于深度优先搜索策略实现拓扑排序

在对图进行深度优先搜索时，改变顶点输出的条件，不是在遇到一个顶点时即刻输出顶点，而是等到该顶点的所有邻接点都输出后才输出该顶点，将得到的顶点序列逆序，即可得到图的拓扑排序。

修改本章第四节中实现的 DFS 算法，基于深度优先搜索策略实现的拓扑排序如下。

```
void TopoDFS(MGraph g)
{
    int number,i;
    int visited[MaxVtxNum];              //定义 visited[]
    number=getNumVertices(g);
    for (i=0;i<number;i++)               //初始化 visited[]
```

```
        visited[i]=0;
    for (i=0;i<number;i++)                    //输出 visited[ ]
        printf("%d ",visited[i]);
    printf("\n");
    for (i=0;i<number;i++)
        if (visited[i]==0)
            TopoDFS1(g,NumToVer(g,i),visited);
}
void TopoDFS1(MGraph g,VType v,int *visited)
{   VType w;
    visited[VerToNum(g,v)]=1;                 //将 visited[v]置为 1，表示已经访问过
    w=FirstNeighbor(g,v);                     //取 v 的第一个邻接点 w
    while (w!='#')
    {
        if (!visited[VerToNum(g,w)])          //若顶点 w 还未被访问
            TopoDFS1(g,w,visited);            //递归调用
        w=NextNeighbor(g,v,w);                //取下一个邻接点
    }
    printf("%c ",v);                          //访问顶点 v
}
```

在输出的序列逆置后，得到拓扑序列。

三、关键路径

可以使用带权有向图表示一个含多个活动的工程。在这样的图中，用顶点表示事件，用弧表示活动，边上的权值表示活动持续的时间，这样组成的网称为以边表示活动的网，简称为 AOE 网。在 AOE 网中，通常只有一个入度为 0 的点和一个出度为 0 的点，这是因为一个工程只有一个开始点和一个完成点。入度为 0 的点称为起始点或源点，出度为 0 的点称为结束点或汇点。一个工程中的某些子活动是可以并行进行的；从源点到汇点的最长路径的长度，即该路径上所有活动持续时间之和，也就是完成整个工程所需的最少时间。将从源点到汇点具有最大长度的路径称为关键路径。关键路径上的所有活动称为关键活动。求关键路径是带权有向无环图的一个重要应用。

例如，图 6-24 表示一个含有 8 个活动的工程，其中顶点 1 是源点，顶点 6 是汇点。假设图中活动的持续时间的单位是天，则活动 a_1需要 3 天完成，活动 a_2需要 2 天完成，活动 a_3需要 2 天完成，等等。在从顶点 1 到顶点 6 的所有路径中，路径长度最大为 8，表明这项工程至少需要 8 天才能完成。

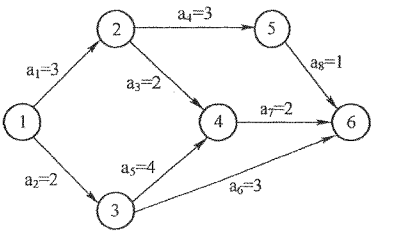


图 6-24　表示工程的 AOE 网

在 AOE 网中，从源点 v_1到任意顶点 v_i的最长路径长度称为事件 v_i的最早开始时间。这个时间决定了所有以 v_i为尾的弧所表示的活动的最早开始时间。另外，在不推迟整个工程完成的前提下，活动 a_i最迟必须开

始进行的时间称作活动 a_i 的最迟开始时间。

用 $e(i)$ 表示活动 a_i 的最早开始时间，$l(i)$ 表示其最迟开始时间。两者之差 $l(i)-e(i)$ 表示完成活动 a_i 的时间余量。a_i 的实际开始时间可以在 $e(i)\sim l(i)$ 之间任意调整，而丝毫不会影响整个工程的完成时间。把 $l(i)=e(i)$ 的活动称作关键活动，即关键活动的最早开始时间和最迟开始时间相等，它们没有时间余量。显然，关键路径上的所有活动都是关键活动，关键活动的推延将直接导致整个工程的推延，而提前完成非关键活动并不能加快整个工程的进度。

由以上分析可知，辨别关键活动就是要找 $e(i)=l(i)$ 的活动。为了求得 AOE 网中活动的 $e(i)$ 和 $l(i)$，先求出事件的最早开始时间 $ve(j)$ 和最迟开始时间 $vl(j)$。

设活动 a_i 由弧 (j,k) 表示，其持续时间记为 $dut(j,k)$，事件的最早开始时间为 $ve(j)$，最迟开始时间为 $vl(j)$，则有如下关系：

- $e(i)=ve(j)$
- $l(i)=vl(k)-dut(j,k)$

求 $ve(j)$ 和 $vl(j)$ 时将采用递推方法，分两步进行：

（1）从 $ve(1)=0$ 开始向前递推

$$ve(j)=\max\{ve(i)+dut(i,j)\} \qquad (i,j)\in T,2\leqslant j\leqslant n$$

其中，T 是所有以顶点 j 为头的弧的集合。

（2）从 $vl(n)=ve(n)$ 起向后递推

$$vl(i)=\min\{vl(j)-dut(i,j)\} \qquad (i,j)\in S,1\leqslant i\leqslant n-1$$

其中，S 是所有以顶点 i 为尾的弧的集合。

计算 ve 时，从事件 1 开始向前递推；计算 vl 时，从最后一个事件开始向后递推。这两个递推公式的计算必须分别在拓扑排序和逆拓扑排序的前提下进行。也就是说，在事件 v_j 的所有直接前驱的最早开始时间都确定下来之后，才能求得事件 v_j 的最早开始时间 $ve(j)$；同样，在事件 v_j 的所有直接后继的最迟开始时间求得之后，才能确定 v_j 的最迟开始时间。求逆拓扑排序的过程与求拓扑排序的过程类似：检查所有出度为 0 的顶点，输出之并删除所有以这些点为头的弧，重复这个过程，直到结束。如果已经得到了拓扑序列，那么将这个序列倒排，即可得到逆拓扑序列。

由此得到以下求关键路径的算法。

（1）输入 e 条弧 (j,k)，建立 AOE 网的存储结构。

（2）从源点 v_1 出发，令 $ve(1)=0$，按拓扑序列求其余各顶点（事件）的最早发生时间 $ve(i)(2\leqslant i\leqslant n)$。如果得到的拓扑序列中顶点个数小于 AOE 网中顶点数 n，则说明网中存在环，不能求关键路径，算法终止；否则执行步骤（3）。

（3）从汇点 v_n 出发，令 $vl(n)=ve(n)$，按逆拓扑序列求其余各顶点（事件）的最迟发生时间 $vl(i)(n-1\geqslant i\geqslant 1)$。

（4）根据各顶点（事件）的 ve 和 vl 值，求每条弧 s（活动）的最早开始时间 e(s)和最迟开始时间 l(s)。若某条弧（活动）满足条件 e(s)=l(s)，则为关键活动。

【例 6-17】 求图 6-24 所示工程的关键路径，列出关键活动。

根据求关键路径的算法，要先求出图 6-24 的拓扑序列，结果是 1,2,3,4,5,6。根据拓扑序列，求出图中各事件的最早开始时间，然后再按逆拓扑序列的次序，求出各事件的最迟

开始时间。

先看源点，即时刻1。通常，整个工程应尽早完成，所以源点的最早开始时间为0。再看时刻2，它只有一个入边(1,2)，所以时刻1的最早开始时间加上入边表示的活动a_1的持续时间3，即为时刻2的最早开始时间。同样，可以求出时刻3的最早开始时间为2。

再看时刻4。它有两条入边，表明到时刻4时，活动a_1和a_3已经完成，且a_2和a_5也已经完成。也就是到达时刻2后，再用两天时间完成a_3，到达时刻3时，再用4天时间完成a_5。时刻2的最早开始时间加上2天是一条分支所用的时间，时刻3的最早开始时间加上4天是另一条分支所用的时间。两者取最大值，得6，即时刻4的最早开始时间为6。

时刻5的最早开始时间等于时刻2的最早开始时间加上a_4的持续时间，为6。计算时刻6的最早开始时间类似于计算时刻4的最早开始时间。将时刻6的3个直接前驱结点的最早开始时间分别加上各自入边对应活动的持续时间，在3个结果中取最大值，得到8，即为时刻6的最早开始时间，也是关键路径的长度，即完成整个工程所需的时间。

各事件的最早开始时间结果见表6-5第二列。

现在按逆拓扑序列计算各事件的最迟开始时间。时刻6的最迟开始时间即它的最早开始时间，为8。时刻5的最迟开始时间是其直接后继结点的最迟开始时间减去a_8的持续时间，为7。同样，得到的时刻4的最迟开始时间是6。再看时刻3，因为它有两条出边，所以从两个后继结点分别计算得到的值中取最小值，得到2。时刻2的计算是类似的，得到的最迟开始时间为4。源点的最迟开始时间为0。各事件最迟开始时间的计算结果见表6-5第三列。

表6-5　各事件的最早开始时间及最迟开始时间

事　件	ve	vl
1	0	0
2	3	4
3	2	2
4	6	6
5	6	7
6	8	8

根据事件的最早开始时间及最迟开始时间，可计算各活动的最早开始时间及最迟开始时间。

先看活动a_1和a_2，它们都是以时刻1为弧尾的，所以，它们的最早开始时间都是0。活动a_3和a_4的最早开始时间与时刻2的最早开始时间相同，都是3。其他活动的最早开始时间的计算过程与上面类似。各活动的最早开始时间见表6-6第二列。

再计算各活动的最迟开始时间。活动a_8需要1天时间完成，所以它的最迟开始时间是7。类似地，a_7和a_6各需要2天与1天的完成时间，所以最迟开始时间分别是6和5。

a_5需要4天的完成时间，而时刻4的最迟开始时间是6，所以a_5的最迟开始时间是2。类似地，a_4的最迟开始时间是4，a_3的最迟开始时间是4。时刻2的最迟开始时间是4，a_1的最迟开始时间是1。时刻3的最迟开始时间是2，a_2的最迟开始时间是0。各活动的最迟开始时间的计算结果见表6-6第三列。

表 6-6　各活动的最早开始时间及最迟开始时间

活　动	最早开始时间 e	最迟开始时间 l	时间余量 l-e
a_1	0	1	1
a_2	0	0	0
a_3	3	4	1
a_4	3	4	1
a_5	2	2	0
a_6	2	5	3
a_7	6	6	0
a_8	6	7	1

再计算各活动的时间余量 l-e，计算结果见表 6-6 第四列。时间余量为 0 的活动即关键活动，这些活动是 a_2、a_5和 a_7。相应地，关键路径是 1,3,4,6，路径长度是 8。

【例 6-18】 图 6-25 所示的 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干活动的进度，可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（　）。

A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

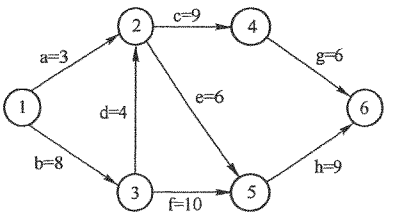


图 6-25　含 8 个活动的 AOE 网

答案为 C。

先计算关键路径，结果见表 6-7。

表 6-7　例 6-18 中各活动的最早开始时间及最迟开始时间

活　动	最早开始时间 e	最迟开始时间 l	时间余量 l-e
a	0	9	9
b	0	0	0
c	12	12	0
d	8	8	0
e	12	12	0
f	8	8	0
g	21	21	0
h	18	18	0

由表 6-7 可知，共有三条关键路径，路径上的三组关键活动分别是 b,f,g、b,d,c,g 和 b,d,e,h，路径长度是 27。关键活动有 b,c,d,e,f,g,h。活动 a 不是关键活动。

三条关键路径的开头部分是重合的，后面的部分不完全相同。如果提前完成重合部分的关键活动，则能使三条关键路径均缩短。具体来说，如果提前完成活动 b，则整个工程的工期会缩短。但所给的 4 个选项中，没有这样的答案。

如果提前完成后面部分的关键活动，则必须同时缩短三条路径的长度才能满足要求，即每条路径上选择一个活动。选项中仅给出了两个活动，则一定至少有一个活动是在两条关键路径上的。提前完成关键活动 f，会缩短关键路径 b,f,g 的长度。提前完成关键活动 d，会缩短关键路径 b,d,c,g 和 b,d,e,h 的长度。

【例 6-19】 下列关于 AOE 网的叙述中，错误的是（　）。

A. 关键活动不按期完成就会影响整个工程的完成时间

B. 任何一个关键活动提前完成，那么整个工程将会提前完成

C. 所有的关键活动提前完成，那么整个工程将会提前完成

D. 某些关键活动提前完成，那么整个工程将会提前完成

答案为 B。

整个工程的完成时间既取决于关键活动的完成时间，也取决于非关键活动的完成时间。如果关键活动拖期了，则关键路径的长度一定增加，所以选项 A 是正确的。如果关键活动提前完成了，是不是会让整个工程提前完成呢？这要看关键路径的组成情况。如果只有一条关键路径，那么，关键活动提前完成一定会使关键路径的长度缩短，工程可以提前完成。但如果有多条关键路径，那么，一个关键活动提前完成，不一定能让所有关键路径的长度都缩短，所以选项 B 不正确。选项 C 的正确性显而易见。在选项 D 中，没有指明哪些关键活动已提前完成，当然该选项也是正确的，可以像选项 C 那样选择所有的关键活动，也可以在每条关键路径上均选择一些活动，只需要保证所有关键路径的长度均缩短，整个工程势必提前完成。

第七节　最 短 路 径

在研究交通网络时，有时需要了解甲地和乙地之间是否有路可通？在有多条路可通的情况下，走哪条路最好？“最好”的评判标准可以不同，有时是指路程最短，有时是指花费最少，有时可以指所用时间最少，等等。无论如何，它们都可以表示为一个数值，相应的问题使用一个带权图来表示，每条边上有相应的权值。

一般地，可以用图中的顶点表示地点，带权的边表示两地之间的距离等一些具有实际意义的量。对于带权图，修改路径长度概念：两个顶点之间的路径长度定义为两顶点间路径上各边的权值之和。路径的开始顶点称为源点，目的顶点称为终点。

给定图中的两个顶点 A 和 B，它们之间可能存在多条路径，每条路径都有相应的路径长度，其中长度最短的那条路径称为两个顶点之间的最短路径。此时，A 即源点，B 即终点。

1. 单源最短路径

将求从某个源点到图中其他各顶点的最短路径的问题称为单源最短路径（single-source shortest path）问题，即已知图 G=(V,E)，找出从某个给定源点 $s \in V$ 到 V 中其他任一顶点的最短路径。

图中任意两个顶点之间并不一定有边存在，为符合实际情况，如果两个顶点之间没有边相连，则权值记为 ∞，程序中可以使用一个计算机能表示的最大值来代表 ∞。为简化问题，规定所有边上的权值均为非负值。

两个指定顶点之间的最短路径可能有边直接相连，也可能要经过其他顶点。可以肯定，这样的最短路径一定不包含回路，因为如果路径中包含回路，则去掉回路后可以得到长度更短的路径。

在实际应用中，很可能只需要知道一对顶点（如 s 与 e）之间的最短路径，但它的求解难度与单源最短路径的求解难度相当，因此本节先讨论单源最短路径问题。

【例 6-20】 给定如图 6-26 所示的一个无向带权图，其源点为 a，从源点到其他任何顶点的最短路径见表 6-8。

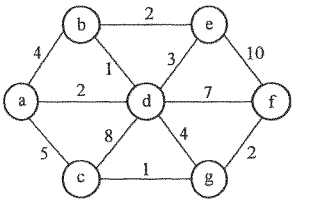


图 6-26　无向带权图

表 6-8　从源点 a 到其他各顶点的最短路径

终　点	最短路径	路径长度
b	(a,d,b)	3
c	(a,c)	5
d	(a,d)	2
e	(a,d,e)	5
f	(a,d,g,f)	8
g	(a,d,g)	6

求带权图单源最短路径的算法称为迪杰斯特拉（Dijkstra）算法，它按照路径长度不减的次序产生最短路径，也就是生成的各条路径的长度越来越长。

Dijkstra 算法的基本思想是：把图中的所有顶点分成两个集合，令 S 表示已求出最短路径的顶点集合，其余尚未确定最短路径的顶点组成另一个集合 V-S。初始时，S 中仅含有源点。按最短路径长度不减的次序逐个把第二个集合 V-S 中的顶点加入到 S 中，不断扩大已求出最短路径的顶点集合，直到从源点出发可以到达的所有顶点都在 S 中为止。

给图中每个顶点定义一个距离，分两种情况：S 集合中顶点具有的距离就是从源点到此顶点的最短路径长度；集合 V-S 中顶点具有的距离是从源点到此顶点且路径中仅包括 S 中的顶点为中间顶点的最短路径的长度。当有 V-S 中的顶点加入 S 时，对 S 中已有顶点的距离没有影响，而有可能使 V-S 中顶点的距离变小。对于 V-S 中的所有顶点，如果距离确实变小了，则以变化后的值替代原来的值；如果距离没有变小，则保持原值不变。

循环进行上述过程，每次从 V-S 集合中选中一个顶点加入到 S 中，必要时，修正 V-S 集合中顶点的距离，直到所有顶点都加入到 S 中为止。选择加入 S 中的顶点的规则是：以目前 V-S 中距离最小的顶点为选中的目标。根据定义，S 中顶点的距离即已求出的最短路径的长度。当 S 包含了除初始顶点以外的所有可达顶点时，Dijkstra 算法结束。

在求最短路径的过程中，总保持从源点到 S 中各顶点的距离都不大于从源点到 V-S 中任何顶点的距离。也就是说，从源点到 V-S 中任何一个顶点的距离都大于从源点到 S 中任何一个顶点的最短路径长度。在每次循环时，选取 V-S 中距离最小的一个顶点加入 S 就可以保证这一点。

设图 G=(V,E)，源点为 v_0，引入辅助数组 dist，它的每个分量 dist[i]表示对应于顶点 i 的距离，这个距离的含义如前所述。dist 的初值为：

$$\text{dist}[i]=\begin{cases}\text{弧上的权值} & \text{若从源点到 } v_i \text{ 有弧}\\ \infty & \text{其他情况}\end{cases}$$

S 的初态只包含源点 v_0，dist[0]=0。其余所有顶点都在 V-S 中，v_0到 V-S 中各顶点 v_i 的边的权值为各顶点的距离 dist[i]。接下来，循环进行下列操作：每次从 V-S 集合的顶点中选取距离最小的一个顶点 v_i加入到 S 中，并对 V-S 中剩余各顶点的距离重新计算，若源点 v_0到顶点 v_k的现有路径长度 dist[k]大于加进中间点 v_i后的路径（v_0,v_i,v_k）长度，即 dist[k]>dist[i]+边(v_i,v_k) 的权值，则令 dist[k]=dist[i]+边(v_i,v_k)的权值。反复进行上述运算，直到再也没有可加入到 S 中的顶点为止。

Dijkstra 算法的步骤如下。

（1）初始化：

① 设用带权的邻接矩阵 cost 表示带权有向图 G =（V,E）。

② cost[i][j]为弧(v_i,v_j)上的权值（若弧(v_i,v_j)不存在，则该值为∞，一般地，取计算机中能表示的最大值）。

③ S 为已找到从源点 v_0出发的最短路径的终点集合，其初态只含有源点 v_0。

④ 从 v_0出发到图中其余各顶点 v_i的距离为：dist[i]=cost[v_0][v_i](v_i∈V)。

（2）选择 v_j，使得 dist[j]=min{dist[i] |v_i∈V-S}，v_j即当前求得的一条从 v_0出发的最短路径的终点。令 S=S∪{v_j}。

（3）修改从 v_0出发到集合 V-S 中任一顶点 v_k可达的最短路径长度。如果 dist[j]+cost[j][k]<dist[k]，则 dist[k]=dist[j]+cost[j][k]。

（4）重复执行步骤（2）、（3），直到再也没有可加入到 S 中的顶点为止。

下面仍以图 6-26 为例，介绍最短路径的求解过程。

图 6-26 对应的带权邻接矩阵 **cost** 为：

$$\mathbf{cost}=\begin{bmatrix}\infty & 4 & 5 & 2 & \infty & \infty & \infty\\ 4 & \infty & \infty & 1 & 2 & \infty & \infty\\ 5 & \infty & \infty & 8 & \infty & \infty & 1\\ 2 & 1 & 8 & \infty & 3 & 7 & 4\\ \infty & 2 & \infty & 3 & \infty & 10 & \infty\\ \infty & \infty & \infty & 7 & 10 & \infty & 2\\ \infty & \infty & 1 & 4 & \infty & 2 & \infty\end{bmatrix}$$

初始时，S 中仅含有源点 a，各顶点的距离等于 cost 中第一行的值：dist[1]=4,dist[2]=5,dist[3]=2，相应的路径即从 a 到 b、c、d 的边见表 6-9 第二列。

选择其中 dist 值最小的顶点，d 被选中。将 d 加入到 S 中，这意味着从 a 到 d 的最短路径已经生成，即 a→d，长度为 2。

重新计算从 a 到其余各顶点可达的当前长度最短的路径。注意，在计算时，在到达目标顶点之前，路径仅能经过 S 中的顶点。例如，在到 c 的路径中，除目标顶点 c 以外，仅能包含顶点 a 和 d。在选中 d 加入 S 之前，到 c 的路径长度是 5。在 d 加入 S 后，到 c 的路径并没有缩短，仍为 5。再看到 b 的路径，原来的长度是 4，现在多了一条从 a 到 d，再到 b 的路径，长度是 3，所以到 b 的当前最短路径的信息需要修改，路径为（a→d→b），长度为 3。类似地，到 e 的路径从原来的不可达变为可达，路径为（a→d→e），长度为 5。到 f 的路径

为 a→d→f，长度为 9，到 g 的路径为 a→d→g，长度为 6，见表 6-9 第三列。从得到的新路径值（表 6-6 第三列）中选择最小长度值的路径，此时选中的顶点为 b，将其加入到 S 中。从 a 到 b 的最短路径已经生成，即 a→d→b，路径长度为 3。

再次重新计算从 a 到其余各顶点{c,e,f,g}的路径长度。实际上，到顶点 c、e、f、g 的路径均没有变化，见表 6-9 第四列。选择最小长度值的路径，此时选中的顶点为 c，将其加入到 S 中。从 a 到 c 的最短路径已经生成，即 a→c，路径长度为 5。注意，此时有两条路径的长度是一样的，选择哪条都是可以的。

再次重新计算从 a 到其余各顶点{e,f,g}的路径长度。到顶点 e、f、g 的路径仍没有变化，见表 6-9 第五列。接下来，选中顶点 e，将其加入到 S 中。从 a 到 e 的最短路径已经生成，即 a→d→e，路径长度为 5。

继续这个过程，直到结束为止。

在求最短路径的过程中，dist 值的变化过程及所求的最短路径见表 6-9。

表 6-9　求最短路径过程中 dist 值的变化过程和所求的最短路径

终点	从 a 到其余各顶点的 dist 值和最短路径					
b	4 (a,b)	3 (a,d,b)				
c	5 (a,c)	5 (a,c)	5 (a,c)			
d	2 (a,d)					
e	∞	5 (a,d,e)	5 (a,d,e)	5 (a,d,e)		
f	∞	9 (a,d,f)	9 (a,d,f)	9 (a,d,f)	9 (a,d,f)	8 (a,d,g,f)
g	∞	6 (a,d,g)	6 (a,d,g)	6 (a,d,g)	6 (a,d,g)	
v_j	d	b	c	e	g	f
S	ad	adb	adbc	adbce	adbceg	adbcegf

除初始化部分以外，Dijkstra 算法的主要操作包括从 V-S 中挑选一个顶点加入到 S 中，然后需要修改 V-S 中剩余顶点的距离，该算法总的时间复杂度为 $O(|V|^2)$。

2. 所有顶点间的最短路径

在 Dijkstra 算法中，将源点设为图中的不同顶点，算法执行|V|次，可以得到图中任意两点间的最短路径。还有一个更简洁的算法可以求得所有顶点间的最短路径，这个算法称为弗洛伊德（Floyd）算法。

考虑图中从顶点 v_i到顶点 v_j的一条最短路径，中间经过或不经过一些顶点。如果最短路径不经过任何中间顶点，则这条最短路径就是从 v_i到 v_j的边。反之，假设中间经过顶点 k，则 v_i到 v_j的最短路径分为从 v_i到 v_k及从 v_k到 v_j的两段，并且这两段分别都是相应顶点间的最短路径，否则可以用更短的路径来分别替代它们，从而组成从 v_i到 v_j的更短的路径。显然，从 v_i到 v_j的最短路径不包含回路，否则可以去掉回路，找到一条更短的路径。

设图 G 中顶点序列为 $v_1,v_2,\cdots,v_n$，带权邻接矩阵为 $\boldsymbol{A}$，$A[i][j]$ 中保存从 v_i 到 v_j 的边的权值，仍设权值为非负值。现在限制最短路径所经过中间顶点的编号。初始时，规定最短路径中间不允许经过任何顶点，则从 v_i 到 v_j 的路径就是从 v_i 到 v_j 的边，其路径长度为 $A[i][j]$。现在允许最短路径中间经过编号不大于 v_1 的顶点，如果从 v_i 到 v_j 有最短路径，则最短路径只有以下两种可能：

（1）从 v_i 到 v_j 的原有最短路径，即从 v_i 到 v_j 的边，$A[i][j]$ 为路径长度。

（2）从 v_i 到 v_1 的路径加上从 v_1 到 v_j 的路径，$A[i][v_1]+A[v_1][j]$ 为路径长度。

判断上述两值，取较小者作为新的 $A[i][j]$ 值，此时 $\boldsymbol{A}$ 中保存的是允许经过 v_1 后顶点之间最短路径的长度。为了与原来的 $\boldsymbol{A}$ 相区别，修正值后的矩阵标记为 $\boldsymbol{A}^{(1)}$，初始的带权邻接矩阵可以表示为 $\boldsymbol{A}^{(0)}$。

这样，增加 v_1 中间顶点后，有可能缩短从任意顶点 v_i 到任意顶点 v_j 的最短路径长度。继续这个过程，再允许增加编号不大于 v_2 的顶点作为中间顶点，然后再来判定从 v_i 到 v_j 的最短路径。

一般地，若已得到 $A^{(k-1)}[i][j]$，即从顶点 v_i 到顶点 v_j 的中间顶点序号不大于 v_{k-1} 的最短路径的长度，现在增加中间顶点 v_k，这样，中间顶点可以是 v_1 到 v_k 中的任何顶点。分析从 v_i 到 v_j 的最短路径，有以下两种可能。

（1）不经过 v_k 点的路径，即保存在 $A^{(k-1)}[i][j]$ 中的原有路径。

（2）经过 v_k 点的路径。路径分为两段，即从 v_i 到 v_k 的最短路径和从 v_k 到 v_j 的最短路径。因为这条路径允许经过的中间顶点序号不大于 v_k，所以从 v_i 到 v_k 的最短路径和从 v_k 到 v_j 的最短路径所经过的中间顶点序号不会大于 v_{k-1}，这两条最短路径的长度分别为 $A^{(k-1)}[i][k]$ 和 $A^{(k-1)}[k][j]$，两者之和为从顶点 v_i 到顶点 v_j 的中间顶点序号不大于 v_k 的最短路径的长度。

比较上述两种情况，取较小者作为从顶点 v_i 到顶点 v_j 的中间顶点序号不大于 v_k 的最短路径的长度，记作 $A^{(k)}[i][j]$，即由 $\boldsymbol{A}^{(k-1)}$ 可以得到 $\boldsymbol{A}^{(k)}$。

设已得到 $\boldsymbol{A}^{(n)}$，按照定义，$A^{(n)}[i][j]$ 中保存的是从 v_i 到 v_j 中间允许经过的顶点序号不大于 v_n 的最短路径长度，由于图中所含顶点集合为 $\{v_1,v_2,\cdots,v_n\}$，不存在大于 v_n 的顶点，也就是说，$A^{(n)}[i][j]$ 中保存的是从 v_i 到 v_j 中间允许经过任意顶点的最短路径的长度，这就是要求的每对顶点之间的最短路径。

由此得到 Floyd 算法如下。

定义 $A^{(k-1)}[i][j]$ 表示从顶点 v_i 到顶点 v_j 的中间顶点序号不大于 v_{k-1} 的最短路径的长度，$A^{(0)}[i][j]$ 即 cost$[i][j]$。从 $A^{(0)}[i][j]$ 出发，逐次计算从顶点 v_i 到顶点 v_j 的中间顶点序号不大于 $k(1\leqslant k\leqslant n)$ 的最短路径的长度，即在 $\boldsymbol{A}^{(k-1)}$ 的基础上计算 $\boldsymbol{A}^{(k)}$，递推地产生一个矩阵序列：$\boldsymbol{A}^{(0)},\boldsymbol{A}^{(1)},\cdots,\boldsymbol{A}^{(k)},\cdots,\boldsymbol{A}^{(n)}$，$A^{(n)}[i][j]$ 表示从顶点 v_i 到 v_j 的最短路径的长度。

设目前已求出 $A^{(k-1)}[i][j]$，求 $A^{(k)}[i][j]$ 时分以下两种情况。

（1）如果从顶点 v_i 到 v_j 的最短路径不经过顶点 v_k，则由 $A^{(k)}[i][j]$ 的定义可知，从 v_i 到 v_j 的中间顶点序号不大于 k 的最短路径的长度就是 $A^{(k-1)}[i][j]$，即 $A^{(k)}[i][j]=A^{(k-1)}[i][j]$。

（2）如果从顶点 v_i 到 v_j 的最短路径经过顶点 v_k，则由 v_i 到 v_k 和由 v_k 到 v_j 两条路径组成从 v_i 到 v_j 的路径。由于 $A^{(k-1)}[i][k]$ 和 $A^{(k-1)}[k][j]$ 分别代表从 v_i 到 v_k 与从 v_k 到 v_j 的中间顶点序

号不大于 v_{k-1} 的最短路径的长度，则 $A^{(k)}[i][j]=A^{(k-1)}[i][k]+A^{(k-1)}[k][j]$。由此得到 $A^{(k)}[i][j]$ 的迭代计算公式：

$$\begin{cases} A^{(0)}[i][j]=\text{cost}[i][j] \\ A^{(k)}[i][j]=\min\{A^{(k-1)}[i][j],A^{(k-1)}[i][k]+A^{(k-1)}[k][j]\} & 1\leqslant k\leqslant n \end{cases}$$

最后得到的 $A^{(n)}[i][j]$ 就是从顶点 v_i 到 v_j 的最短路径的长度。

【例 6-21】 仍以图 6-26 所示的无向带权图为例，利用 Floyd 算法求所有顶点间的最短路径。

图 6-26 对应的带权邻接矩阵 **cost** 如下。

$$\textbf{cost}=\begin{bmatrix} \infty & 4 & 5 & 2 & \infty & \infty & \infty \\ 4 & \infty & \infty & 1 & 2 & \infty & \infty \\ 5 & \infty & \infty & 8 & \infty & \infty & 1 \\ 2 & 1 & 8 & \infty & 3 & 7 & 4 \\ \infty & 2 & \infty & 3 & \infty & 10 & \infty \\ \infty & \infty & \infty & 7 & 10 & \infty & 2 \\ \infty & \infty & 1 & 4 & \infty & 2 & \infty \end{bmatrix}$$

Floyd 算法中的核心部分是一个三重循环。例如，对于 MGraph g，循环如下。

```
int i,j,k,n=g->numVertices;
for (k=0;k<n;k++)
    for (i=0;i<n;i++)
        for (j=0;j<n;j++)
            if (g->AdjMatrix[i][k]+g->AdjMatrix[k][j]<g->AdjMatrix[i][j])
                g->AdjMatrix[i][j]=g->AdjMatrix[i][k]+g->AdjMatrix[k][j];
```

计算后得到的矩阵结果如下。

$$\textbf{cost}=\begin{bmatrix} 4 & 3 & 5 & 2 & 5 & 8 & 6 \\ 3 & 2 & 6 & 1 & 2 & 7 & 5 \\ 5 & 6 & 2 & 5 & 8 & 3 & 1 \\ 2 & 1 & 5 & 2 & 3 & 6 & 4 \\ 5 & 2 & 8 & 3 & 4 & 9 & 7 \\ 8 & 7 & 3 & 6 & 9 & 4 & 2 \\ 6 & 5 & 1 & 4 & 7 & 2 & 2 \end{bmatrix}$$

矩阵第一行是顶点 a 到其余各顶点的最短路径的长度，这个结果与 Dijkstra 算法计算的结果一致。矩阵第二行是顶点 b 到其余各顶点的最短路径的长度，例如，从顶点 b 到顶点 c 的最短路径的长度为 6，路径是 b,d,g,c。矩阵第三行至第七行分别对应顶点 c、d、e、f、g 到其余各顶点的最短路径的长度。

内 容 小 结

图是非常重要的非线性结构。本章介绍了图的基本概念，给出了邻接矩阵、邻接表及逆邻接表的存储结构，实现了邻接矩阵下的各种基本操作。

本章着重介绍了图的几个经典问题，包括图的搜索、有向无环图的应用、生成树及最短路径问题。对于图的搜索问题，介绍了深度优先搜索算法及广度优先搜索算法。对于有向无环图，介绍了相关的概念、拓扑排序及关键路径的算法，实现了基于深度优先搜索策略和广度优先搜索策略的拓扑排序算法。对于生成树问题，介绍了求最小生成树的普里姆算法和克鲁斯卡尔算法。对于最短路径问题，介绍了迪杰斯特拉算法和弗洛伊德算法。本章还对各个算法的效率进行了简单分析。

习 题

一、单项选择题

1. 设无向图的顶点个数为 n，则该图所含的边数最多是_____。

A. $n-1$　　B. $n(n-1)/2$　　C. $n(n+1)/2$　　D. n^2

2. 在含 n 个顶点的连通无向图中，边数至少是_____。

A. $n-1$　　B. n　　C. $n+1$　　D. $n\log n$

3. 下列叙述中，错误的是_____。

A. 图的深度优先搜索是一个递归过程

B. 图的深度优先搜索不适用于有向图

C. 深度优先搜索和广度优先搜索是图搜索的两种基本算法

D. 图的搜索是指，从给定的顶点出发，每一个顶点仅被访问一次

4. 无向图 G=(V,E)，其中：V={a,b,c,d,e,f}，E={(a,b),(a,e),(a,c),(b,e),(c,f),(f,d),(e,d)}，对该图进行深度优先搜索，得到的顶点序列是_____。

A. a,b,e,c,d,f　　B. a,c,f,e,b,d　　C. a,e,b,c,f,d　　D. a,e,d,f,c,b

5. 已知有向图 G=(V,E)，其中 $V=\{v_1,v_2,v_3,v_4,v_5,v_6,v_7\}$，$E=\{(v_1,v_2),(v_1,v_3),(v_1,v_4),(v_2,v_5),(v_3,v_5),(v_3,v_6),(v_4,v_6),(v_5,v_7),(v_6,v_7)\}$，G 的拓扑序列是_____。

A. $v_1,v_3,v_4,v_6,v_2,v_5,v_7$　　B. $v_1,v_3,v_2,v_6,v_4,v_5,v_7$

C. $v_1,v_3,v_4,v_5,v_2,v_6,v_7$　　D. $v_1,v_2,v_5,v_3,v_4,v_6,v_7$

6. 在有向图 G 的拓扑排序中，若顶点 v_i 在顶点 v_j 之前，则下列情形中不可能出现的是_____。

A. G 中有弧(v_i,v_j)　　B. G 中有一条从 v_i 到 v_j 的路径

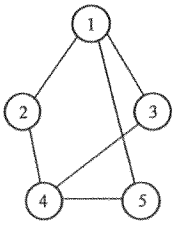
C. G 中没有弧(v_i,v_j)　　D. G 中有一条从 v_j 到 v_i 的路径

二、解答题

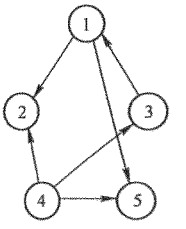
1. 带权图中的权值可以表示什么含义？举例说明。

2. 分别给出图 6-27a、图 6-27b 的邻接矩阵和邻接表。

3. 求图 6-27a 各顶点的度，以及图 6-27b 各顶点的入度和出度。
4. 给出图 6-27a 的几个不同的子图。
5. 给出图 6-28 所示带权 G 的邻接矩阵和邻接表。



a) 无向图



b) 有向图

图 6-27　一个无向图和一个有向图

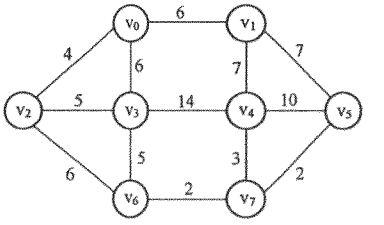


图 6-28　带权图 G

6. 对于图 6-29，从顶点 1 开始进行深度优先搜索，试给出 5 种不同的遍历序列。
7. 对于图 6-29，从顶点 1 开始进行广度优先搜索，试给出 5 种不同的遍历序列。
8. 对于图 6-29，给出它的几棵不同的深度优先生成树。
9. 对于图 6-29，给出它的几棵不同的广度优先生成树。
10. 对于图 6-30，使用普里姆算法求最小生成树。

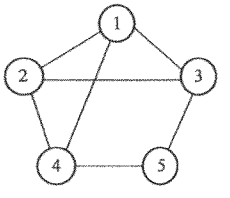


图 6-29　无向图

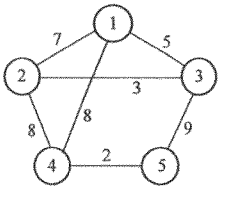


图 6-30　带权无向图

11. 对于图 6-30，使用克鲁斯卡尔算法求最小生成树。
12. 试写出图 6-31 所示有向无环图的 6 种不同的拓扑排序。
13. 求图 6-32 中从顶点 1 到其余各顶点的最短路径的长度。

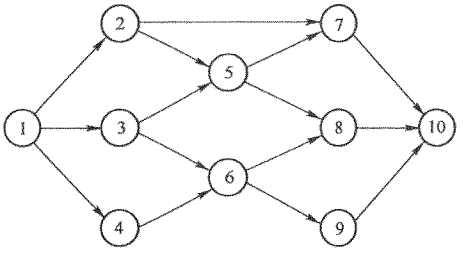


图 6-31　一个有向无环图

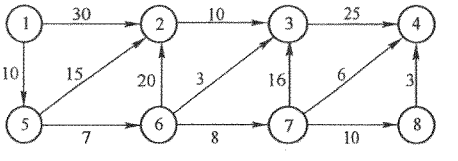


图 6-32　一个带权有向图

14. 求图 6-33 所示 AOE 网中各事件的最早开始时间和最迟开始时间。

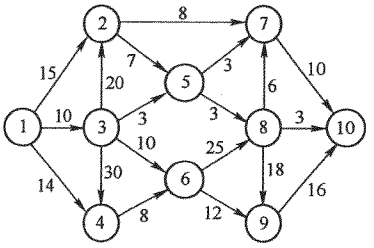


图 6-33　AOE 网

三、算法设计题

1. 对于给定的无向图 G，采用邻接矩阵存储，试设计算法 int getD(MGraph g,VType u)，返回顶点 u 的度。

2. 对于给定的无向图 G，采用邻接矩阵存储，试设计算法 isPath(MGraph g,VType * path)，判定给定的顶点序列 path 是否是图中的路径。

第七章　内部排序

学习目标：

1. 理解排序的基本概念。
2. 掌握各种排序方法及其特点。
3. 掌握各种排序算法的实现过程，能够对其进行稳定性及复杂度分析。
4. 能够对各种排序方法进行比较，理解各排序方法的使用条件。

建议学时：8 学时。

教师导读：

1. 排序是非常基本且重要的概念。要让考生理解大部分排序方法的关键操作有比较和交换，进而理解排序算法的复杂度的分析过程。

2. 要让考生理解各排序算法的特点，理解排序过程中影响排序结果的关键步骤。例如，插入排序过程中已有序子段和待排序子段的变化情况；希尔排序如何借助插入排序的特点完成排序；基数排序中没有关键字之间的比较等。

3. 要让考生理解不同排序算法之间的异同点。

4. 在学习完本章后，应要求考生完成实习题目 6。

当数据的排列有序时，可以提高某些操作的效率。例如，查找学生的学号，进而可以很方便地知道学生的姓名，因为班级中学生通常是按学号排好序的。又如，在一本英语字典中查找某个单词也是很快的，因为字典中的单词是按照字典序排好序的。

有研究数据表明，排序与查找是计算机内部频繁进行的操作，目前已经出现了很多经典的排序算法。在数据结构中，排序的概念与日常生活中使用的含义是一样的，就是将数据按照某种标准来排定次序，或为升序，或为降序。

本章将介绍排序的概念，详细介绍几类常用的排序方法，包括：插入排序、交换排序、选择排序、归并排序和分配排序等。同时分析各排序方法的效率，说明它们各自的适用条件，并对它们进行简单的比较。

不失一般性，本章假定数据存放在数组中，所有排序方法都是对数组中保存的数据进行的。

第一节　排序的基本概念

实际应用中常常需要对一组记录按照规定的次序进行排列，这是计算机程序中的一种重要操作，称为排序。待排序的每个记录内有一个或多个特殊的域，用来存放该记录的关键字。它的类型必须是可比较的，如字符型或数值型。对于字符型数据，一般按其对应的 ASCII 码的顺序排列，而数值型数据可按数值的大小顺序排列。记录的排列次序实际上是由这个关键字的排列次序决定的。

定义 7-1 给定一组记录 $r_1, r_2, \cdots, r_n$，其关键字分别为 $k_1, k_2, \cdots, k_n$，将这些记录排成顺序为 $r_{s1}, r_{s2}, \cdots, r_{sn}$ 的序列 S，满足条件 $k_{s1} \leqslant k_{s2} \leqslant \cdots \leqslant k_{sn}$ 或 $k_{s1} \geqslant k_{s2} \geqslant \cdots \geqslant k_{sn}$，此过程称为排序（sorting）。换句话说，排序就是要重排一组记录，使其关键字域的值具有非递减或非递增的顺序。

经过排序的数据，若关键字值依从小到大的顺序排列，则称为升序；若依从大到小的顺序排列，则称为降序。如果不特别指明，本章所讨论的排序算法将数据按升序排列。

对不同类型的关键字而言，“大”和“小”的含义可能不一样。例如，对于数值型关键字，可按一般意义的“大”和“小”来理解；对于字符型关键字，往往按其对应的 ASCII 码来定义大小次序。在排序算法中，关键字也称为排序码。

在有些应用中，一个记录内可能含有不止一个关键字，而且这种情况往往更常见。此时，把能唯一标识记录的关键字称为主关键字，其余的称为次关键字。一般情况下，主关键字值不允许重复，因为若关键字相等，则不能根据关键字唯一识别出记录。而次关键字值可以有相同的值。上述定义中的关键字 k_i 可以是记录的主关键字，也可以是记录的次关键字，甚至可以是若干数据项的组合。在待排序的记录中，如果按主关键字排序，则记录的任何一个无序序列经排序后得到的结果都是唯一的；如果按次关键字排序，由于允许有多个记录具有相同的关键字值，则得到的排序结果可能不是唯一的。

计算机中存放待排序数据的存储介质不完全一样。存储介质可以分为内存和外存两大类。当待排序的数据量不大，全部数据都可以放入内存，排序操作也完全在内存中进行时，相应的排序称为内部排序或内排序（internal sorting）。因为内排序不涉及内、外存数据交换的问题，所以排序速度较快。

当待排序的数据量大，全部数据不能同时放在内存中，需要借助外存完成排序过程时，相应的排序称为外部排序或外排序（external sorting）。在外排序中，排序的具体操作在内存中完成，且仅能对部分数据进行。排序过程中需要多次分批在内存、外存之间进行数据交换，从而完成全部数据的排序任务。数据交换所占用的时间远多于内存中完成排序所花费的时间。一般来讲，外排序除了要检查是否能得到正确的结果以外，更注重的是减少数据内、外存交换次数。

定义 7-2 具有相等关键字的两个记录 R_1 和 R_2，在排序之前，R_1 位于 R_2 之前，在排序之后，R_1 仍然位于 R_2 之前，即排序并没有改变具有相等关键字值的两个记录的相对次序，这样的排序方法称为稳定的（stable）。反之，如果排序方法不能保证具有相等关键字值的两个记录的相对次序在排序前后不被改变，则称排序方法是不稳定的。

当两个记录中关键字的大小次序与记录的相对次序不一致时，称两个记录是逆序的（inversion）。实际上，排序就是不断地调整逆序数据对的过程。

为简单起见，在本章的讨论中，只关心关键字值，而忽略记录中除关键字以外的其他域的值。不失一般性，将待排序的数据存放在一维数组中。待排序的数组定义为 data，数组中元素的类型是 ELEMType，数组中实际的元素个数为 currentNum。数组及元素个数定义为记录 myRcd。在实际应用中，可以使用具体的类型来替代 ELEMType，如对整数进行排序时，可以使用 int 来替代。

相关的类型定义如下。

```
#define maxSize 30                    //最大记录数
#define TRUE 1
```

```
#define FALSE 0
typedef int ELEMType;                    //元素类型
typedef struct{
    ELEMType data[maxSize];              //数组
    int currentNum;                      //元素个数
}myRcd;
```

为数组分配的空间大小可以根据需要而定，如使用整型变量 maxSize 来表示数组的最大容量，即在数组中可以保存的最多个数。数组中实际保存的元素个数 currentNum 需要满足：currentNum≤maxSize，待排序的数据存放在 myRcd. data[0]~myRcd. data[currentNum-1]中。

排序是一项常用的操作。对于内排序算法，主要衡量排序算法的时间复杂度和空间复杂度，看它在各种情况下运行时的时间开销和空间开销。

排序算法中的主要操作包括关键字的比较及记录的交换，交换意味着数据的移动。所以，在时间开销上，主要统计执行这两项操作所花费时间的多少。

另外，数据的初始排列次序不同，进行比较及移动的次数可能也会不同。在所有情况中，执行次数最少的情形的时间复杂度称为最优时间复杂度，执行次数最多的情形的时间复杂度称为最差时间复杂度，平均情况下的执行次数称为平均时间复杂度。在不引起歧义的情况下，排序算法的时间复杂度是指其平均时间复杂度。

而对于空间，原始数据存放在数组中，这对每种排序算法都是一样的。排序过程中可能需要使用除这个数组以外的其他空间，这就是排序算法的空间开销。

为叙述方便，排序算法的描述中仅关注待排序数据记录的关键字，而忽略记录中的其他内容。有时以关键字作为其所属记录的代表，将其称为元素。

第二节　插入排序

插入排序（insertion sort）是一类很常用、很简单的排序方法，它用到的一个基本操作是将一个待排序的元素插入到一个已有序的子序列中，从而形成一个更长的子序列。

插入排序算法的一般策略是：初始时，表中第一个元素自然有序，它形成有序子序列。从第二个元素开始，将该元素插入到前面的有序子序列中，形成含两个元素的新有序子序列。接下来处理第三个元素，将其插入到前面长度为 2 的有序子序列中，形成含 3 个元素的有序子序列。继续这个过程，直到最后一个元素插入到前面含 $n-1$ 个元素的有序子序列中，由此得到含有全部元素的有序序列。

将待排序元素插入到有序子序列中的操作，涉及插入位置。根据查找这个位置的方式不同，插入排序又分为直接插入排序和折半插入排序。本书只介绍直接插入排序。插入排序经过改进后，得到希尔排序。

一、直接插入排序

参与排序的 n 个数据均保存在数组 A 中。A 的前面部分（S 部分）是已有序的子序列，后面部分（U 部分）是待排序的部分。初始时 S 部分中仅含有一个元素 e，U 部分中含有除 e 以外的其他 $n-1$ 个元素。每进行一趟排序，S 部分增加一个元素，相应地，U 部分减少一

个元素。在 $n-1$ 趟排序后，S 部分中包含全部的 n 个元素，而 U 部分变为空。排序结束。

假设现在要进行第 $i(1 \leqslant i \leqslant n-1)$ 趟排序。在该趟排序前，$A[0] \sim A[i-1]$ 中的元素已有序，要处理的元素是 $A[i]$，在该趟排序后，$A[0] \sim A[i]$ 中的元素有序。一言以蔽之，第 i 趟直接插入排序就是将 $A[i]$ 插入到有序子序列 $A[0] \sim A[i-1]$ 中。这个插入过程非常简单。将 $A[i]$ 看作当前元素，并将它与前面的元素进行比较，如果两者呈逆序，则交换两者。当前元素前移一个位置，继续与它前面的元素进行比较，必要时进行交换。在当前元素大于等于它前面的元素，或当前元素已经到达 $A[0]$ 时，这个过程结束。

【例 7-1】 直接插入排序示例。

使用直接插入排序方法，对数据序列 42,68,35,1,70,25,79,59,63,65 进行排序。

用下画线表示已排好序的有序子序列，后面是待排序的序列。在每趟排序时，选择待排序序列的第一个元素作为本趟排序的待处理元素。

初始时，第一个记录 42 是已排好序的有序子序列。第一趟排序处理 68。68 大于 42，所以不需要进行交换，满足当前元素大于等于它前面元素的条件，该趟排序过程结束，形成含两个记录的有序子序列 42,68。

第二趟排序将 35 插入到有序子序列 42,68 中，目标是形成含 3 个记录的有序子序列 35,42,68。相应的过程是：35 小于它前面的元素 68，所以两个元素交换。此时 35 前面的数据是 42，继续交换，此时 35 已经到达 $A[0]$，该趟排序过程结束。在第三趟排序时，要将 1 插入到前面的有序子序列中，因为 1 小于有序子序列中的所有元素，所以它与每一个元素进行交换。这个过程持续到最后一个记录 65 插入到适当位置中为止。整个排序过程如图 7-1 所示。

```
i=1:    42 68 35 1  70 25 79 59 63 65
i=2:    42 68 35 1  70 25 79 59 63 65
i=3:    35 42 68 1  70 25 79 59 63 65
i=4:    1  35 42 68 70 25 79 59 63 65
i=5:    1  35 42 68 70 25 79 59 63 65
i=6:    1  25 35 42 68 70 79 59 63 65
i=7:    1  25 35 42 68 70 79 59 63 65
i=8:    1  25 35 42 59 68 70 79 63 65
i=9:    1  25 35 42 59 63 68 70 79 65
结果:   1  25 35 42 59 63 65 68 70 79
```

图 7-1　直接插入排序的排序过程

直接插入排序的实现如下。

```
int insertSort(myRcd * myarr)
{   int i,j;
    for (i=1;i<myarr->currentNum;i++){
        if (myarr->data[i]<myarr->data[i-1]){
            for (j=i;myarr->data[j]<myarr->data[j-1] && j>0;j--){
                swap(&myarr->data[j],&myarr->data[j-1]);
            }
        }
    }
    return TRUE;
}
```

在直接插入排序的每趟扫描中，当待排序元素不再向前移动时，有序子序列中它前面的元素（如果有的话）必定小于等于它，而它后面的元素（如果有的话）必定大于它。如果当前元素与它前面的元素值相等，则不需要交换。所以直接插入排序可以保证具有相等关键字值的两个元素在排序前后的相对次序保持不变，即直接插入排序是稳定的。

当待排序元素与它前面的元素进行交换时，需要进行 1 次比较操作和 3 次元素移动操作。在统计时间复杂度时，元素的交换与移动可以看成等价的指标，不失一般性，可以只统计元素交换的次数。另外，待排序元素的最后一次操作是比较操作，即当它大于等于前面的元素时，不再需要与前面的元素进行交换。因此，在每趟排序中，比较的次数比交换的次数多 1。

如果待排序记录要前移的位置较多，则可以采用另外一种策略以减少数据移动次数。将待排序元素保存到临时变量 temp 中，然后将有序子序列中大于 temp 的元素依次后移，后移完毕再将 temp 中保存的值放回数组里面移动元素后出现的空位置中。实现如下。

```
int insertSort1(myRcd * myarr)
{   int i,j,temp;
    for (i=1;i<myarr->currentNum;i++){
        temp=myarr->data[i];
        j=i-1;
        while (myarr->data[j]>temp && j>=0)
          myarr->data[j+1]=myarr->data[j--];     //后移
        myarr->data[j+1]=temp;
    }
    return TRUE;
}
```

除原来的数据所占用的数组空间以外，直接插入排序仅需要临时工作单元，所以它的空间复杂度为 $O(1)$。从时间角度来看，它的主要操作是进行两个关键字的大小比较和必要的元素移动，元素移动的次数与关键字原来的次序有关。当待排序序列中记录按关键字非递减有序排列时，要插入的记录只需要和前一位置的记录进行比较，并且不发生交换，总的交换次数为 0。这是直接插入排序算法能够达到的最优情况。在对 n 个元素进行排序时，总的比较次数为

$$\sum_{i=1}^{n-1} i = n - 1$$

即时间复杂度为 $O(n)$。当待排序记录按关键字递减排列时，因为是逆序排列，所以每个待插入元素都需要移动到有序子序列的最前面，待插入的元素需要和有序子序列中的所有元素相比较，比较的次数最多，n 个元素排序时的比较次数为

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

相应地，有序子序列中的所有记录都需要向后平移一个位置，此时元素移动的次数也最多。这是直接插入排序算法能够达到的最差情况，即时间复杂度为 $O(n^2)$。

在平均情况下，待排序元素只和有序子序列中约一半的元素相比较，有序子序列中要移动的元素个数也约为一半，此时的时间代价为前述最差情况的一半，即时间复杂度也为 $O(n^2)$。

直接插入排序非常简单，但是效率不高。即使大部分数据都已插入到有序子序列中，仍有可能因新插入的数据最小而让有序子序列中的所有数据都再次被移动。另外，插入排序中

所有的移动都只在相邻单元中进行，如果一个元素要插入到与它相隔 m 个位置的单元中，就需要移动 m 次。当数组中待排序数量 n 很大时，这些移动所花费的时间将会非常多。

在每趟直接插入排序中，当前元素边比较、边移动，找到本趟插入位置。可以采用第八章介绍的折半查找方法直接找到插入位置，然后进行元素交换，得到折半插入排序。

二、希尔排序

希尔（shell）排序又称缩小增量排序（diminishing increment sort），它是插入排序的一种改进，由 Donald L. Shell 于 1959 年提出。它采用了一个很巧妙的技巧，因此在时间复杂度上比直接插入排序要好。

直接插入排序在两种情况下性能较好，一种情况是当数据序列基本有序时，另一种情况是当数据量 n 很小时。希尔排序就是利用了直接插入排序的这两个特点，先把长数据段划分成短数据段，对短数据段进行直接插入排序（利用的是数据量少的特性），再将这些短数据段逐渐扩成基本有序的长数据段，并对长数据段进行直接插入排序（利用的是数据基本有序的特性），直到所有数据都处于同一数据段，并排好序为止。

那么如何将全部数据划分为短数据段呢？又如何合并所有这些短数据段？

划分数据段是按照一定的间隔数进行的，将相距等间隔数的元素放在同一个数据段内。例如，间隔数为 3 时，所有数组下标等于 $3i$（$i\geq 0$）的元素都分在第一个数据段中，所有下标等于 $3i+1$（$i\geq 0$）的元素都分在第二个数据段中，所有下标等于 $3i+2$（$i\geq 0$）的元素都分在第三个数据段中。一般，在间隔数为 k（$k\geq 1$）时，全部数据会分成 k 个数据段。

在采用这样的机制划分数据段后，合并过程会变得易如反掌。

下面来看希尔排序的过程。

首先划分数据段。初始时，数据不能保证是基本有序的，应该划分为较短的数据段，也就是说，数据段内所含的数据较少，而数据段数比较多。第一次应该选择一个较大的 k 值，将全部的数据分成 k 个组。

当数据段划分完毕后，每一个数据段可看成一个组，在组内进行直接插入排序。数据划分为 k 个组的话，需要对这 k 个组分别进行各自的插入排序。这些插入排序的过程可以调整各组内呈逆序的数据对，排序的结果是减少了原数据中的逆序对数量，即提高了数据的有序性。

依同样的机制，将全部数据划分为更长的数据段，数据段内的数据个数增多，段数减少，也就是选择一个比前一次更小的 k 值。同样地，在各组内分别进行直接插入排序，使得各组内的数据有序。

继续这个过程。重新划分组，并在组内进行直接插入排序。当然，划分的组不应该再和上一次的划分一样，因为各组内已经不存在呈逆序的数据对了。

从选一个 k 值开始，对数据进行分组，并在各组内进行插入排序，到各组内的排序结束，这个过程称为一趟希尔排序过程。在每一趟希尔排序中，所选择的 k 值都是不同的。从刚才的分析中知道，k 值应该越来越小。k 称为间隔，也称为增量。所以希尔排序也称为缩小增量排序。最后一趟排序的 k 值为 1，也就是全部元素都在同一个组内，对所有元素进行直接插入排序。此时排序过程结束。

在希尔排序中，增量序列 d_i 的取法有很多种，比如：

① 取 $d_0=m$，$d_{i+1}=\lfloor d_i/2 \rfloor$

② 取 $d_0=m$，$d_{i+1}=\lfloor (d_i+1)/2 \rfloor$

③ 取 $d_0=m$，$d_{i+1}=\lfloor (d_i-1)/3 \rfloor$

④ 取 $d_0=m$，$d_{i+1}=\lfloor (d_i-1)/2 \rfloor$

在③中，若 $m=121$，则相应的增量序列为：40,13,4,1。在④中，若 $m=63$，则相应的增量序列为：31,15,7,3,1。

【例 7-2】 有一组记录，相应的关键字序列为：

42,68,35,1,70,25,79,59,63,65,26,80,17,36

采用希尔排序法对它进行排序，增量序列为 5，3，1。

初始时数据为：42,68,35,1,70,25,79,59,63,65,26,80,17,36，增量为 5，数据分为 5 组，同一组数据列在同一行中，仅对同组内的数据进行排序，5 个组内的排序共同组成第一趟排序。各组内的排序结果放在原始数据的后面，用斜体表示。

d=5	42	*25*			25	*26*			26	*42*					第一组
	68	*68*			79	*79*			80	*80*					第二组
	35	*17*			59	*35*			17	*59*					第三组
	1	*1*			63	*36*			36	*63*					第四组
	70	*65*			65	*70*									第五组

第一趟排序的结果是：25,68,17,1,65,26,79,35,36,70,42,80,59,63。

接下来，增量为 3，数据分为 3 组，3 个组内的排序共同组成第二趟排序。

d=3	25	*1*		1	*25*		79	*59*		70	*70*		59	*79*		第一组
	68	*35*		65	*42*		35	*63*		42	*65*		63	*68*	第二组
	17	*17*		26	*26*		36	*36*		80	*80*				第三组

第二趟排序的结果是：1,35,17,25,42,26,59,63,36,70,65,80,79,68。

最后一趟使用增量 1，数据均在同一组内。这是第三趟排序。排序后，得到的最终排序结果是：1,17,25,26,35,36,42,59,63,65,68,70,79,80。

从例 7-2 中可以看出，每一趟排序都试图让关键字更接近于它的最终位置，这样，在最后一趟排序开始之前，数据都非常接近各自的最终位置，所以最后一趟排序是在基本有序的数据序列上进行的，能很快地得到结果。

【例 7-3】 在用希尔排序方法对一个数据序列进行排序时，若第一趟排序结果为 9,1,4,13,7,8,20,23,15，则该趟排序采用的增量（间隔）可能是（　）。

A. 2　　B. 3　　C. 4　　D. 5

答案为 B。使用排除法来分析。

在希尔排序中，将相隔增量整数倍的关键字组成一组，并使用直接插入排序在组内进行排序。一趟排序后，各组内关键字已有序。

若增量为 2，则两个组内的关键字序列分别是 9,4,7,20,15 和 1,13,8,23。显然，两个组内关键字均无序。

若增量为 4，则 4 个组内的关键字序列分别是 9,7,15、1,8、4,20 和 13,23，第一组

无序。

若增量为5，则5个组内的关键字序列分别是9，8、1，20、4，23、13，15和7，第一组无序。

以上分析排除了选项A、C和D。下面验证选项B是否符合要求。

若增量为3，则3个组内的关键字序列分别是9,13,20、1,7,23和4,8,15。3个组内的关键字均有序。

实际上，排序中用到的增量序列的取法很有讲究。如果增量序列取2的幂次，如增量序列为8,4,2,1，虽然计算增量时简单，但在前一趟扫描中互相比较过的两个关键字，在后一趟扫描中还会遇到，从而又比较一次，显然这些比较操作是多余的。一般地，增量之间最好不是倍数关系，其目的是每一趟扫描都会尽可能地让没有同过组的记录分在一组内，即让没有互相比较过的两个关键字相互比较，从而能够得到更多的有序信息。

有研究建议，增量最好都取奇数；也有研究表明，使用形如2^k-1或2^k+1的增量比较好；其他的还有自然序列$(2^k-(-1)^k)/3$、$(3^k-1)/2$等。尽管如此，目前对增量的选择还没有一个很成熟的结论，也没有找到一个增量序列可以适用于任何情况，即不同情况需要使用不同的增量序列。如果增量序列递减得比较慢，则每趟扫描中进行的比较和移动操作将会较少，但扫描的趟数将较多。反之，如果增量序列递减得很快，则意味着扫描的趟数较少，但每趟中比较和移动的操作将较多。注意，最后一趟的增量一定是1。

希尔排序算法如下。

```
void ShellSort(myRcd *myarr,int d[],int t)      //希尔排序
{   ///进行t趟希尔排序，d[0]≤currentNum,d[0]>d[1]>…>d[t],d[t]=1
    int k=0,gap=d[k];
    while (k<t){
        insertSortforShell(myarr,gap);
        k++;
        gap=d[k];
    }
}
```

在实现希尔排序时，每趟扫描都对每组数据进行直接插入排序，可以借用前面的直接插入排序算法，但需要修改两个地方：其一是数据的起始位置可能不是数组的第一个元素，根据不同的分组情况，各组的起始位置依次是数组最前面的几个元素；另一个要修改的地方是，进行比较及交换的两个记录不再是相邻记录，而是间隔为增量的两个记录。

在直接插入排序算法insertSort(myRcd *myarr)的基础上进行修改，实现如下。

```
int insertSortforShell(myRcd *myarr,int gap)
{   int i,j;
    for (i=gap;i<myarr->currentNum;i++){
        if (myarr->data[i]<myarr->data[i-gap]){
            for (j=i;myarr->data[j]<myarr->data[j-gap] && j-gap>=0;j=j-gap){
                swap(&myarr->data[j],&myarr->data[j-gap]);
```

```
            }
        }
    }
    printf("d=%d:\t",gap);
    display(*myarr,0);
    return TRUE;
}
```

针对前面实现的第二个直接插入排序算法 insertSort1(myRcd *myarr)，也给出了用于希尔排序的修改版，实现如下。

```
void insertSortforShell1(myRcd *myarr,int gap)      //以 gap 为步长进行直接插入排序
{   int tmp,i,j;
    for (i=gap;i<myarr->currentNum;i++){
        j=i;
        tmp=myarr->data[i];
        while (j>=gap &&tmp<myarr->data[j-gap]){
            myarr->data[j]=myarr->data[j-gap];
            j-=gap;
        }
        myarr->data[j]=tmp;
        printf("d=%d:\t",gap);
        display(*myarr,0);
    }
}
```

在希尔排序中，数据移动的“步幅”较大，确实改善了直接插入排序的效率，但也使算法丢失了稳定性，即希尔排序算法不是稳定的。希尔排序的效率分析是一个复杂的问题，到目前为止仍在研究中。

第三节　交换排序

交换排序算法是另一类常用、重要的排序方法，其基本思想是：在待排序的序列中，找到逆序的两个关键字，直接交换它们；重复这个过程，直到该序列中不再出现逆序的关键字时为止。本节介绍两种常用的交换排序：起泡排序和快速排序。

一、起泡排序

起泡排序（bubble sort）也称为冒泡排序，该算法非常简单，通常也会在程序设计语言类的课程中介绍。

参与排序的 n 个数据均保存在数组 A 中，起泡排序算法的一般策略是：扫描整个数组，依次比较相邻的两个元素，如果它们是逆序的，则交换它们，然后去查看后面的相邻元素。持续地进行这个过程，直到所有元素都有序时为止。

从前至后扫描一遍数组的执行结果是将原始数据中的最大值“起泡”到数组的最后位置，这也正是它的最终位置。这样的过程称为一趟起泡排序。当完成第一趟起泡排序后，选出了最大值，它不需要再参与后续的排序过程。所以后面的排序只需要对前面剩余的元素进行。在第二趟扫描后，可以不去考虑最后两个元素，依此类推。当数组中只剩余一个元素待排序时，这个元素自然有序，排序过程结束。每趟起泡排序都会将一个元素移动到它的最终位置，剩余元素越来越少，在执行了若干步之后，算法一定会结束。一般地，对含有 n 个元素的数组进行起泡排序，需要进行 $n-1$ 趟起泡排序过程。

【例 7-4】 采用起泡排序对一组记录进行排序，相应的关键字序列为：

42,68,35,1,70,25,79,59,63,65

用下画线表示已到达最终位置的元素。初始时，当前位置在下标 0，当前位置的元素 42 与其后继元素 68 进行比较，维持现状，不进行交换。当前位置前进到下标 1，元素 68 与后继 35 进行比较并交换。当前位置前进到下标 2，元素 68 与其后继 1 进行比较并交换。当前位置前进到下标 3，元素 68 与后继 70 进行比较，但不交换。当前位置前进到下标 4，元素 70 与后继 25 进行比较并交换。当前位置前进到下标 5，元素 70 与后继 79 进行比较，但不交换。当前位置前进到下标 6。之后元素 79 分别与 3 个后继进行比较并交换。在这一趟排序后，最大元素 79 位于最后位置，即下标 9 处。

i=1: 42 35 1 68 25 70 59 63 65 79
i=2: 35 1 42 25 68 59 63 65 70 79
i=3: 1 35 25 42 59 63 65 68 70 79
i=4: 1 25 35 42 59 63 65 68 70 79
i=5: 1 25 35 42 59 63 65 68 70 79
i=6: 1 25 35 42 59 63 65 68 70 79
i=7: 1 25 35 42 59 63 65 68 70 79
i=8: 1 25 35 42 59 63 65 68 70 79
i=9: 1 25 35 42 59 63 65 68 70 79

图 7-2 起泡排序过程
（从前向后扫描）

第二趟排序仍从下标 0 开始。因为最后一个位置已经是最大元素，不需要再进行比较，所以结束的位置是下标 8 处。这一趟将次大元素 70 调整到倒数第二个位置，即下标 8 处。

后几趟的排序过程类似。完整的过程如图 7-2 所示。

选取并调整本趟最大值的起泡排序的算法如下。

```
int bubbleSortMAX(myRcd *myarr)     //将最大值放到最后面
{   int i,j;
    for (i=0;i<myarr->currentNum-1;i++){
        for (j=0;j<myarr->currentNum-i-1;j++){
            if (myarr->data[j]>myarr->data[j+1]){
                swap(&myarr->data[j],&myarr->data[j+1]);
            }
        }
    }
    return TRUE;
}
```

例 7-4 所示的每一趟起泡排序都是从数组的最前面开始，比较相邻的两个元素，将较大值放到较小值的后面。其结果是将本趟排序中的最大值调整到相应的位置。实际上，起泡排序也可以从数组的最后面开始，比较相邻的两个元素，将较小值放到较大值的前面。这样的处理结果是将本趟排序中的最小值调整到相应的位置。

仍以例 7-4 中的数据为例，从后向前扫描的排序过程如图 7-3 所示。

```
          42, 68, 35, 1, 70, 25, 79, 59, 63, 65
i=1:    1  42  68 35 25 70  59  79 63  65
i=2:    1  25  42 68 35 59  70  63 79  65
i=3:    1  25  35 42 68 59  63  70 65  79
i=4:    1  25  35 42 59 68  63  65 70  79
i=5:    1  25  35 42 59 63  68  65 70  79
i=6:    1  25  35 42 59 63  65  68 70  79
i=7:    1  25  35 42 59 63  65  68 70  79
i=8:    1  25  35 42 59 63  65  68 70  79
i=9:    1  25  35 42 59 63  65  68 70  79
```

图 7-3　起泡排序过程（从后向前扫描）

选取并调整本趟最小值的起泡排序的算法如下。

```
int bubbleSortMIN(myRcd *myarr)          //将最小值放到最前面
{   int i,j;
    for (i=myarr->currentNum-1;i>0;i--){
        for (j=myarr->currentNum-1;j>myarr->currentNum-i-1;j--){
            if (myarr->data[j]<myarr->data[j-1]){
                swap(&myarr->data[j],&myarr->data[j-1]);
            }
        }
    }
    return TRUE;
}
```

以第一趟起泡排序为例来说明排序过程。从数组的最后面（即下标 9 处）开始，比较当前元素 65 和它的前驱 63，65>63，不做交换。然后当前位置位于下标 8，比较当前元素 63 和其前驱 59，也不进行交换。接下来比较当前元素 59 和其前驱 79，59<79，进行交换。此刻，最后面的 4 个元素依次是 59、79、63 和 65。当前位置在下标 6，仍是元素 59，比较 59 与其前驱 25，不进行交换。然后比较 25 与 70，进行交换。此刻，最后面的 6 个元素依次是 25、70、59、79、63 和 65。后面的排序过程是：25 与 1 进行比较且不交换，1 与 35 进行比较且交换、1 与 68 进行比较且交换、1 与 42 进行比较且交换。这趟排序的结果是，1 交换到了最前面的位置。

起泡排序算法共进行 $n-1$ 趟扫描。在每一趟扫描中，从前向后（或从后向前）交换逆序的数据记录。例如，在例 7-4 中，共有 10 个初始数据，需要进行 9 趟扫描。

【例 7-5】 若用起泡排序方法对序列 10,14,26,29,41,52 进行降序排序，则需要进行比较操作的次数是（　）。

A. 3　　B. 10　　C. 15　　D. 25

答案为 C。

数据元素个数 $n=6$，进行比较的次数是 $n\times(n-1)/2=6\times5/2=15$。

实际上，在某一趟扫描中，如果不发生数据交换，则表示已不存在逆序记录，那么后续的每趟扫描也不会再交换任何记录。在如图 7-2 所示的排序过程中，当 $i=4$ 时，数组中保

存的已经是有序序列了，后面的几趟扫描都没有发生数据交换，这几趟扫描完全可以不进行。也就是说，起泡排序趟数可能少于 $n-1$ 次。

可以修改算法，使用一个变量来记录一趟扫描中是否有数据交换。当发现没有数据交换时，起泡排序可以立即结束。

对算法进行如下修改：设置一个变量 flag，用它来标识一趟排序过程中是否有记录交换。在每趟排序之前，flag 的值为 0，每次交换记录之后，flag 的值修改为 1。在每趟排序之后，判别 flag 的值，若为 1，则继续下一趟排序；若为 0，则表明该趟没有交换任何记录，意味着再没有逆序的记录存在，排序结束。

在算法实现时，还可以记录每趟排序时进行交换的记录的最后位置，这个位置就是下趟排序中要比较的关键字的最后位置。如果一趟扫描要处理的记录范围内，最后几个记录已经有序，则这样处理后可以减少后面这些有序关键字之间的无效比较次数。可以使用变量 right 来记录这个位置，初值为 $n-1$。

仍以例 7-4 中的数据为例，从前向后扫描，改进的排序过程如图 7-4 所示。

```
        42, 68, 35, 1, 70, 25, 79, 59, 63, 65      right=9
i=1:  42 35 1  68 25 70 59 63 65 79                right=8
i=2:  35 1  42 25 68 59 63 65 70 79                right=7
i=3:  1  35 25 42 59 63 65 68 70 79                right=6
i=4:  1  25 35 42 59 63 65 68 70 79                right=1
i=5:  1  25 35 42 59 63 65 68 70 79
```

图 7-4　改进的起泡排序过程

在进行第一趟扫描时，元素 79 经过 3 次比较与交换后，到达最后面的位置，right 的值为 8，即下一趟扫描的结束位置是 8。在第二趟扫描中，元素 70 也经过 3 次比较与交换，最后到达下标为 8 的位置，right 的值为 7，即下一趟扫描要进行到 65 所在的位置。在第三趟扫描中，元素 68 交换到下标为 7 的位置，right 的值是 6。此时，right 之后的所有元素，即最后面的 3 个元素，已经处于最终的排序位置。在第四趟扫描中，元素 35 与 25 进行了比较，并进行了交换。此时 right 的值是 1，也就是位置 2 及之后的所有值都已经有序。在第五趟扫描中，没有交换任何记录，flag 的值为 0，排序结束。

改进的起泡排序算法如下。

```
int bubbleSortMAXFLAG(myRcd *myarr)          //改进的起泡排序——设置界限
{   int i,j;
    int flag=1;                              //flag=1 标志在本趟扫描过程发生过元素交换
    int right=myarr->currentNum-1;
    int temp=myarr->currentNum-1;
    for (i=0;i<myarr->currentNum-1 && flag;i++){
        flag=0;
        right=temp;
        for (j=0;j<right;j++){
            if (myarr->data[j]>myarr->data[j+1]){
                flag=1;
                swap(&myarr->data[j],&myarr->data[j+1]);
                temp=j;
            }
```

```
        }
    }
    return TRUE;
}
```

外层循环控制扫描的趟数，当一趟扫描中没有数据交换时，flag 的值变为 0，循环立即结束。在内层循环中，由循环变量 j 控制本趟扫描中数据比较的范围。从最前面的元素开始，一直到 right 所标记的位置结束。使用变量 temp 记录做数据交换的下标值。然后，将这一趟扫描中记录的最后一个 temp 值作为下一趟扫描的右边界。

在基本起泡排序算法中，数据比较的次数是确定的，第一趟要比较 $n-1$ 次，第二趟要比较 $n-2$ 次……第 i 趟要比较 $n-i$ 次，总的比较次数为 $n\times(n-1)/2$。因此，起泡排序在最优、最差及平均情况下的比较次数是相同的，时间复杂度均为 $O(n^2)$。

在数据记录比较之后，根据一个结点与它前面结点关键字的大小，可能发生数据交换，也可能不发生交换。在最差情况下，每次比较之后都要发生数据交换，则交换次数也为 $n\times(n-1)/2$。在最优情况下，只进行记录比较而不发生交换。在平均情况下，记录交换的次数约为最差情况下交换次数的一半。

对于改进的起泡排序算法，当初始数据已有序时，出现最优情况，仅需要一趟扫描即能完成排序任务，时间复杂度为 $O(n)$。

在起泡排序中，始终是相邻元素之间进行比较及交换，排序方法是稳定的。

二、快速排序

本章讨论的直接插入排序和起泡排序都比较简单，算法容易实现，但它们的效率不高，平均时间复杂度都为 $O(n^2)$。

快速排序（quick sort）算法正如它的名称一样，是目前为止所发现的最快速的排序算法之一。它是由 C. A. R. Hoare 在 1962 年提出的。

快速排序算法基于分治思想。分治思想是指，对一个大的难解问题，将其分解为几个小的问题，如果这些小问题仍不能求解，则再继续将它们分解为更小的问题，依此类推，逐步缩减问题尺度。直到问题变为可解时，一一解决。然后将小问题的解整合起来，从而得到原问题的求解结果。正所谓“大而化小，分而治之”。

快速排序算法将一个含多数据的大数据段的排序问题，分解为两个或一个含更少数据的小数据段的排序问题，小的数据段又继续分解为更小的数据段，分解过程依此类推。这样的分解过程称为划分。每次划分，都会得到比原来的数据段更小的数据段，经过多次划分后，总会得到只含一个数据的数据段，而这样的数据段自然有序。再将这些有序的小数据段组合起来成为有序的大数据段，从而得到初始数据的排序结果。

在快速排序中，需要解决两个主要问题。一个是如何将一个大数据段划分为小数据段，另一个是有序的小数据段如何组合为大数据段，并保证大数据段仍是有序的。

快速排序从初始数据中选择一个元素，用它作为基准元素，称为枢轴（pivot）。将初始数据中所有小于枢轴的数据分在一个组内，设为组 1；将所有大于枢轴的数据分在另一个组内，设为组 2。这就是快速排序划分数据段的机制。从分组过程可以看出，组 1 中的所有数

据全部都小于组 2 中的所有数据，这个性质称为整体有序。

为简单起见，假设初始数据各不相同，则划分过程中，不会出现与枢轴相等的数据。实际上，若划分时有数据等于枢轴，则该数据可以放置在组 1 或组 2 的任一个中。

考虑整体有序的两个子段，因为一个子段中的所有数据值都小于另一个子段中的所有数据值，所以在任何一个子段中进行排序不会影响到另一个子段中的数据。也就是说，两个子段可以独立进行排序。如果组 1、组 2 内各数据都已经有序，则按组 1、枢轴、组 2 这样的次序将数据排列起来，就可以得到原始数据的有序序列。也就是说，将两个有序的小数据段分放在枢轴的前后，关键字值较小的一组放在前面，关键字值较大的一组放在后面，就会得到更长的有序数据段。这就是快速排序的合并机制。

快速排序的枢轴也称为划分元素，虽然它的选择不影响排序的结果，但直接影响快速排序的执行效率。如果每次枢轴都能把数据划分为数量上大致相等的两部分，则会出现最好的情形。如果枢轴选择得不好，可能导致划分的两部分数据的个数相差很大，最极端的情况是，组 1 或组 2 是空的，不含任何元素，而除枢轴以外的其他元素全部都在另外一个组内。这种情况下执行效率是最低的，当然算法仍能得到正确的排序结果。

每次的划分都能保证组 1 与组 2 之间数据的整体有序。只要组 1 内的数据小于枢轴，组 2 内的数据大于枢轴即可，组 1 和组 2 内各数据元素的次序并不重要。之后，分别对组 1 和组 2 内的数据进行递归处理。为简单起见，可以选择第一个元素当作枢轴，并将它与最后一个元素交换，然后对前面的元素进行处理。

下面来看看具体的划分过程。

首先定义两个变量：left 和 right，用它们来记录数组下标。初始时，left 从最左位置开始向右依次扫描，找到第一个大于枢轴的元素，并停在这个位置。接下来，right 从最右位置开始向左依次扫描，找到第一个小于枢轴的元素，也停在相应的位置。此时交换 left 和 right 所指向的元素。可以看出，交换后数组中位于 left 及左侧的元素都是小于枢轴的元素，位于 right 及右侧的元素都是大于枢轴的元素，而 left 到 right 之间的元素都是待筛选的元素。继续这个过程，筛选并处理从 left 到 right 之间的元素。left 从左向右，继续查找大于枢轴的元素。然后 right 从右向左，继续查找小于枢轴的元素。在找到这样一对元素后，交换它们。这样的交换到什么时候结束呢？很显然，当 left>right 时，就结束了。

结束时，两个下标之间满足的关系是：right+1 = left。left 所指的位置是大于枢轴的所有元素中的第一个位置，而 right 所指的位置是小于枢轴的所有元素的最后一个位置。

按照快速排序的分治思想，枢轴应该放回这两段数据的中间。最初，枢轴被放在了最后一个位置，故可以将大于枢轴的数据段中的第一个元素与枢轴进行交换，也就是将 left 位置的元素与枢轴互换。

到这里，完成了一趟划分过程。假定枢轴位于数组的第 k 个位置，下标小于 k 的位置上的元素值均小于枢轴，下标大于 k 的位置上的元素值均大于枢轴。在对两个子段内的数据进行排序时，任何一个记录都不会越过枢轴从一个子段转到另一个子段中。因此，枢轴所在的这个位置就是它排序后的最终位置。接下来，继续将这个机制分别应用于枢轴左、右两侧的数据段，将这两个数据段划分为更小的数据段，直到数据段中只含有一个元素时，它自然有序。这个问题可以递归解决。

【例 7-6】 划分示例一。

仍以例 7-4 中的数据为例，一趟划分的过程如下。

初始数据：42 68 35 1 70 25 79 59 63 65

选择第一个元素作为枢轴并将枢轴放到最后的位置：

pivot：42　　65 68 35 1 70 25 79 59 63 42

此时，left 对应于 65，right 对应于 63，数据序列如下。

65　68　35　1　70　25　79　59　63　42

↑　　↑

left　　right

接下来，从 left 开始向右寻找大于枢轴的元素，停在 65 处。然后从 right 开始向左寻找小于枢轴的元素，停在 25 处。

65　68　35　1　70　25　79　59　63　42

↑　　↑

left　　right

当找到这样一对元素后，进行交换。交换后，得到的数据序列如下。

25　68　35　1　70　65　79　59　63　42

↑　　↑

left　　right

继续这个过程，从 left 开始向右寻找大于枢轴的元素，停在 68 处，然后从 right 开始向左寻找小于枢轴的元素，停在 1 处，数据序列如下。

25　68　35　1　70　65　79　59　63　42

↑　　↑

left　　right

再交换由 left 和 right 指向的这两个元素，得到的数据序列如下。

25　1　35　68　70　65　79　59　63　42

↑　　↑

left　　right

第三次寻找的结果如下。

25　1　35　68　70　65　79　59　63　42

↑　↑

right　left

left 停在 68 处，而 right 停在 35 处。此时，right<left，这两个元素不做交换，同时这一趟扫描也结束。剩下的工作是将枢轴归位，要放到两段整体有序子段的中间。为此，将枢轴与后面一段的第一个元素，即 68，进行交换，结果如下。

25　1　35　42　70　65　79　59　63　68

↑　↑

right　left

此时，一趟划分完成。得到的结果具有如下性质：

① 枢轴 42 在最终的位置上；

② 枢轴前面的元素均小于枢轴；

③ 枢轴后面的元素均大于枢轴。

接下来，对枢轴前、后两个子段递归执行上述操作，最终能得到排序结果。

划分算法如下。

```
int partition(myRcd *myarr,int mostleft,int mostright)                    //对调划分
{
    int left,right;
    ELEMType pivot;
    if (mostleft<mostright){
        pivot=myarr->data[mostleft];                                      //将第一个元素选为枢轴
        swap(&myarr->data[mostleft],&myarr->data[mostright]);             //将枢轴放到最后位置
        left=mostleft;                                                    //left从左向右找
        right=mostright-1;                                                //right从右向左找
        while(left<=right){                                  //循环查找，从左向右找大于枢轴的元素，
                                                             //从右向左找小于枢轴的元素
            while (myarr->data[left]<=pivot && left<=mostright-1){
                left++;
            }
            //从左向右找到第一个大于枢轴的记录
            while (myarr->data[right]>=pivot && right>=mostleft){
                right--;
            }
            //从右向左找到第一个小于枢轴的记录
            if (left<right)                                               //交换刚找到的两个元素
                swap(&myarr->data[left],&myarr->data[right]);
        }
        swap(&myarr->data[mostright],&myarr->data[left]);     //将枢轴放到正确的位置
    }
    return left;
}
```

快速排序算法如下。

```
void QuickPass(myRcd *myarr,int left,int right)          //递归实现
{
    int k;
    if (left<right){
        k=partition(myarr,left,right);
        QuickPass(myarr,left,k-1);                       //递归快速排序枢轴左侧的数据记录
        QuickPass(myarr,k+1,right);                      //递归快速排序枢轴右侧的数据记录
    }
```

```
}
void QuickSort(myRcd *myarr)
{    QuickPass(myarr,0,myarr->currentNum-1);
}
```

【例 7-7】对 7 个关键字进行快速排序，最好情况下需要比较多少次？最坏情况下需要比较多少次？

快速排序的最好情况是指在进行划分时，枢轴左、右两边的两个子序列的长度相等或接近相等，而最坏情况是指两个子序列的长度相差最大（其中一个长度为 0）。因此，对于 7 个关键字，最好情况是：第一趟排序，枢轴左、右两边的两个子序列的长度各为 3，需要进行 6 次比较；第二趟排序再将两个长度为 3 的子序列进一步划分，得到 4 个长度为 1 的子序列，需要进行 2×2=4 次比较。所以总的比较次数是 10。最坏情况是：第一趟排序，枢轴左、右两边的两个子序列的长度分别为 0 和 6，比较了 6 次；第二趟排序再将长度为 6 的子序列进一步划分，得到的两个子序列的长度分别为 0 和 5，比较了 5 次；第三趟、第四趟、第五趟及第六趟排序分别进行了 4 次、3 次、2 次及 1 次比较，所以，总的比较次数为 6+5+4+3+2+1=21。

划分还可以有其他的实现过程，见例 7-8。

【例 7-8】划分示例二。

仍以例 7-4 中的数据为例，另一种划分的过程如下。

初始数据：42 68 35 1 70 25 79 59 63 65

初始时，left=0，right=9。选择第一个元素作为枢轴，保存到临时变量 pivot 中。枢轴所在的位置 left 腾空了，可用来保存其他的值。

从 right 开始，向左寻找小于枢轴的第一个元素，找到后将该元素保存到位置 left 中。现在位置 right 腾空了。从 left 开始向右寻找大于枢轴的第一个元素，找到后保存到位置 right 中。继续这个过程，将找到的小于枢轴的元素放到 left 中，大于枢轴的元素放到 right 中。在此过程中，left 和 right 相向移动。当它们交错时，过程停止。最后将枢轴放回腾出的空位置中。这个划分过程如图 7-5 所示。

划分完成。相应的实现程序如下。

```
int partition1(myRcd *myarr,int mostleft,int mostright)          //转移划分
{
    int left=mostleft,right=mostright,flag=0;
    ELEMType pivot;
    if (mostleft<mostright){
        pivot=myarr->data[mostleft];          //将第一个元素选为枢轴
        while(left<=right){                   //循环查找左边大于枢轴、右边小于枢轴的元素
            while (myarr->data[right]>=pivot && right>mostleft){
                right--;
                flag=0;
            }
            // 找到右边第一个小于枢轴的记录
```

```
        if (left<right) myarr->data[left++]=myarr->data[right];
        while (myarr->data[left]<=pivot && left<mostright){
            left++;
            flag=1;
        }
        // 找到左边第一个大于枢轴的记录
        if (left<right) myarr->data[right--]=myarr->data[left];
    }
    if (flag==1) myarr->data[right]=pivot;
    else myarr->data[left]=pivot;
}
return right;
}
```

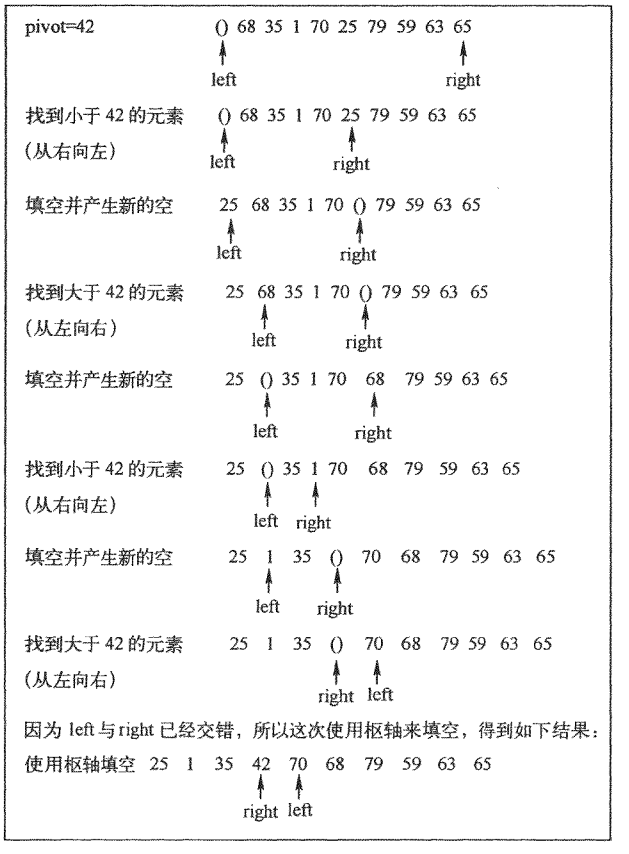


图 7-5　快速排序的另一种划分过程

下面分析快速排序的时间复杂度。如果每次都能把数组划分成元素个数大致相等的两个子段，则快速排序能达到最优情况。在每个子段内对数组扫描一遍，每一次划分中，对所有

子段的扫描长度之和不超过原数组的长。由于每次都对半划分，因此数据的划分过程类似于一棵较平衡的二叉树。当数据元素个数为 n 时，二叉树的层数近似为 $\log n$，也就是需要进行 $\log n$ 趟的划分，而每趟划分后要扫描的数据个数不会多于 n，这样总的时间复杂度为 $O(n\log n)$。

与之相对的是每次划分都不平衡，不能将数组分为元素个数大致相等的两个子段，而是出现另一种极端现象，即一个子段极大（含 $n-1$ 个元素），另一个子段极小（含 0 个元素）。这样的情况是快速排序的最差情况。在这种情况下，因为每次只能划分出一个枢轴，需要对数组进行 $n-1$ 趟扫描，而第 i 趟要扫描 $n-i$ 个元素，所以总的时间复杂度为 $O(n^2)$。

平均情况介于最优与最差之间，在理想状态下，时间复杂度为 $O(n\log n)$。

导致快速排序效率不高的主要原因是划分时选择的枢轴不好。如果选择的枢轴是待排序序列中的最小值（或最大值），则必然会使快速排序出现最差情形。例如，在选第一个元素为枢轴时，如果数据序列是有序的，反而会让排序花费更多时间。避免这个问题的一种办法是，在最左位置元素、中间位置元素及最右位置元素中选择中间值作为枢轴，划分后，两个部分都不会为空，每个部分中最少含有一个元素。枢轴的这种选择方法称为三元取中方法。

另外，当 n 值很小时，快速排序的优越性并不突出。实际应用中采取的措施是，当数据项数目非常少时，使用直接插入排序来替代快速排序。

快速排序不是稳定的排序方法。例如，对初始数据 $42,68,75,20_1,20_2$（20_1 和 20_2 指包含两个 20，以下标来区分）进行快速排序，采用例 7-6 所示的划分方法，选择 42 作为枢轴来进行一趟划分。将枢轴 42 与 20_2 相交换，并将前 4 个元素分组，再将枢轴放置到位，得到的划分结果是 $20_2,20_1,42,68,75$。两个 20 的相对位置发生了变化，表明快速排序不是稳定的。

本节中给出的快速排序程序是递归实现的。我们已经知道，递归函数在执行时需要使用系统内部的栈。在函数调用时，会占用系统资源，即有隐性的系统开销。在调用完毕并返回时，也需要系统做很多额外的工作。可以使用非递归方法代替递归实现，一般来讲，非递归的快速排序算法在执行时要优于递归的快速排序算法。

无论是递归方式实现还是非递归方式实现，快速排序都需要一个栈作为辅助空间，故最坏情况下的空间复杂度为 $O(n)$。

第四节　选 择 排 序

选择排序（selection sort）算法重复地选择特定值，然后将其放到最终位置，从而完成一组值的排序，即对于数组中的每个位置，算法选出应该处在每个位置的值，并将它们一次性地放置到位。

根据扫描数组的方式及放置元素的位置的不同，可以得到两种选择排序方法，分别是简单选择排序和堆排序。在不引起歧义的情况下，简单选择排序也称为选择排序。本节分别介绍这两种方法。

一、简单选择排序

顾名思义，在简单选择排序（simple selection sort）中，元素选择的过程比较简单。它从数组的最前面扫描到最后面，通过不断比较，记录最小值的位置，并将最小值与第一个元

素进行交换。这是第一趟简单选择排序过程。

除最小值以外，剩余的元素全部参与第二趟简单选择排序过程，即从数组的第二个位置到最后位置的元素都参与第二趟排序。对于本趟排序，数组的第二个位置保存的是参与排序的最小元素。仍使用这样的扫描机制，从参与本趟排序的第一个元素扫描到最后一个元素，记录的最小值与本趟第一个元素互换。实际上，本次扫描得到的这个最小值是全部原始数据中的次小值，所处的最终位置也是全部元素中的第二个位置。依此类推。

在第一趟扫描时，从全部 n 个元素中找到最小值，并将其放到数组的第一个位置；在第二趟扫描时，从剩余的 $n-1$ 个元素中找到最小值，并将其放到数组的第二个位置。一般地，在第 k（$k \geqslant 1$）趟扫描时，从 $n-k+1$ 个元素中找到最小值，并将其放到数组的第 k 个位置。直到只剩余一个元素时，不需要再做任何处理，排序过程结束。

每次扫描时都能从待排序的元素范围内选出一个关键字值最小的记录放置到适当的位置，当待排序记录个数有限时，简单选择排序可以在有限步内完成。

如何选择最小值？又如何记录它的位置呢？很简单，可以只使用一个临时变量记录最小值所处的数组下标，知道数组下标，也就知道了最小值是多少。在扫描开始时，这个下标值是本趟扫描区间最前面的位置。然后依次用后面的元素和这个元素进行比较，如果发现有更小的元素，则更新这个下标值，让它修正为新的下标值。

还可以使用两个临时变量，一个变量用来记录下标值，另一个变量用来记录该下标位置的元素值。虽然多用了一个变量，但每次进行比较时，数组下标的计算量减少了。在找到最小元素后，更新的工作相应地也要多一步，既需要更新用来记录下标的变量，也需要更新用来记录元素值的变量。

简单选择排序的算法如下。

```
void SmpSelSort (myRcd * myarr)                          //简单选择排序
{   int i,j,k;
    for (i=0;i<myarr->currentNum-1;i++){                 //选择第 i 个记录
      k=i;
      for (j=i+1;j<myarr->currentNum-1;j++)              //找最小值
        if (myarr->data[j]<myarr->data[k])
          k=j;
        if (k!=i)
          swap(&myarr->data[i],&myarr->data[k]);         //找到的最小值与第 i 个记录相交换
    }
}
```

简单选择排序算法使用一个双重循环，内层循环负责在剩余的元素中找出关键字值最小的记录，并记下其位置；外层循环将找到的最小关键字值记录调整到适当的位置。

【例 7-9】简单选择排序示例一。

给定一组数据：42 68 35 26 70 25 79 59 63 65，简单选择排序的执行过程如图 7-6 所示。

在简单选择排序中，每趟排序记录最小关键字所处的位置，然后让它与最终的目标位置元素进行交换。对于有 n 个数据的序列，它共需 $n-1$ 趟排序。在每趟排序中，只有将所有剩余元素的关键字都扫描一次，才能找到有最小关键字的记录，比较操作次数较多；而将找

初始数据：	42 68 35 26 70 25 79 59 63 65
i=1:	25 68 35 26 70 42 79 59 63 65
i=2:	25 26 35 68 70 42 79 59 63 65
i=3:	25 26 35 68 70 42 79 59 63 65
i=4:	25 26 35 42 70 68 79 59 63 65
i=5:	25 26 35 42 59 68 79 70 63 65
i=6:	25 26 35 42 59 63 79 70 68 65
i=7:	25 26 35 42 59 63 65 70 68 79
i=8:	25 26 35 42 59 63 65 68 70 79
i=9:	25 26 35 42 59 63 65 68 70 79

图 7-6　简单选择排序示例

到的记录移动到最终位置最多只需要 3 次移动操作。总体来说，简单选择排序的时间复杂度为 $O(n^2)$。

【例 7-10】 简单选择排序示例二。

对于由 a、b、c、d、e、f 和 g 这 7 个符号任意组成的序列，如果采用简单选择排序算法进行排序，那么，在最好和最坏情况下，分别需要进行多少次的比较操作和交换操作？

对于简单选择排序，其比较次数与待排序序列的初始状态无关。所以，无论这 7 个关键字如何排列，比较次数均为 6+5+…+1 = 21。排序过程中的交换次数则与初始状态有关。若初始序列有序，则交换次数为 0，这是最好情况。若初始序列逆序，则交换次数为 6，这是最坏情况。

可以看出，虽然简单选择排序的比较次数并不比插入排序少，但每趟排序最多只需要进行一次数据交换，交换次数明显少于插入排序。

另外，因为简单选择排序中发生了不相邻关键字之间的交换，所以它是不稳定的。

二、堆排序

堆排序（heap sort）也是选择排序，它借助于堆的结构，在选择最大值元素或最小值元素时，不需要进行全部元素的顺序查找，比简单选择排序的效率要高。

回顾第五章的内容，堆中的元素存储在一个数组中，同时也可以将堆对应为一棵完全二叉树。假设堆中的元素是 $a_0, a_1, \cdots, a_7$，依二叉树的顺序存储规则，对应的完全二叉树如图 7-7 所示。

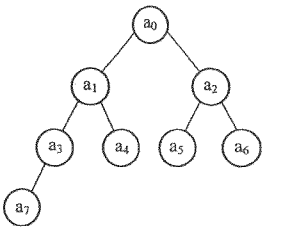


图 7-7　堆对应的完全二叉树

可以利用堆的特性对数据进行排序。不失一般性，以最大堆为例。最大堆的堆顶元素是全部数据中的最大值，输出这个值。再将剩余元素整理成新的最大堆，此时，堆顶元素是剩余元素中的最大值，再输出这个值。继续这个过程，每次输出堆顶元素，并将剩余元素重新整理成新堆。直到输出全部数据，没有剩余元素时为止。这就是堆排序的过程。每次输出堆顶并将剩余元素重新整理成新堆的过程即一趟堆排序过程。

从堆排序过程中可以看出，每趟排序中，都会选出剩余元素中的最大值并输出，实际上就是选出最大值并输出到最终位置。所以堆排序也是选择排序的一种。

堆排序需要解决的问题有这样几个：第一个是如何将数据排列成堆的形式，这称为初始堆的建立，简称为建堆；第二个是输出堆顶元素后，剩余元素如何再整理成新堆，这是堆的整理过程，简称为整堆；第三个是输出的元素放在什么地方。

第五章第四节中实现了建初始堆的算法，第一个问题迎刃而解。同时，该节还实现了删除堆顶元素的算法，所以整堆问题也解决了。在删除堆顶元素后，堆中元素个数减 1，最后一个位置正好用来放置刚删除的堆顶元素，所以第三个问题也解决了。故在堆排序中，不需要为依次输出的各个堆顶元素再分配存储空间，它们可以借用堆空出的位置，从后向前依次排放，即最大值放在最后位置，次大值放在倒数第二个位置，依此类推，最小值放在第一个位置。由此看来，使用最大堆得到的排序结果是升序的。类似地，使用最小堆得到的排序结果是降序的。

【例 7-11】 建最大堆示例。

设有数据序列：44,97,76,29,13,7,50,9,20，将它们建成最大堆，给出最后得到的堆的结果。

建成的最大堆如图 7-8 所示。

当初始最大堆建立后，堆顶即数据中的最大值，可以输出它。根据前面的分析知道，输出的堆顶元素放到堆的最后面的位置，如在例 7-11 所示的最大堆中，堆顶元素 97 输出后，将放置在 20 目前所在的位置。实际上，就是将堆顶与最后一个元素进行互换。20 交换到树根位置后，因为它并没有大于它的所有孩子结点，即得到的这个新结构不再满足最大堆的性质，所以需要进行整堆过程。

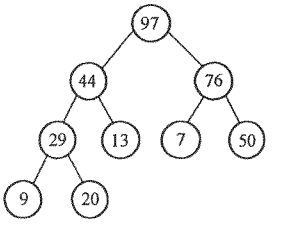


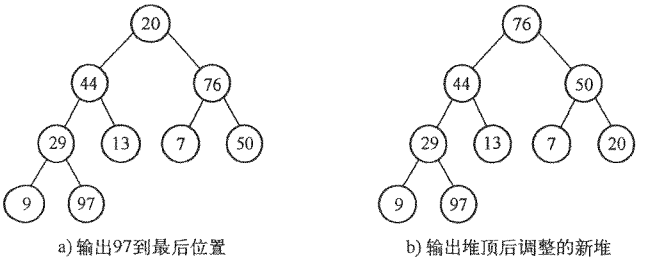
图 7-8　建成的最大堆

此时，最后一个元素是原来的堆顶，它只是暂时存放在这个位置的元素，剩余元素个数减 1。在输出堆顶后，原堆顶的子树没有变化，依然保持堆的特性，只有新的树根可能不再满足堆的特性。所以只需要对树根进行调整就可以了。这个过程实际上类似于建立初始堆时各分支结点调整的最后一步。

【例 7-12】 整堆示例。

基于例 7-11 得到的初始最大堆，输出堆顶 97，给出调整后的新堆。

将初始堆的堆顶元素 97 输出到数组的最后一个位置，交换后的完全二叉树如图 7-9a 所示。新的树根 20 小于其孩子结点的值，将它与其较大的孩子结点进行互换，76 调整到树根中。而 20 仍需要继续向下调整，又与 50 互换。此时，20 已是叶结点，不需要再进行调整。此时得到新的堆，如图 7-9b 所示。



a) 输出97到最后位置　　b) 输出堆顶后调整的新堆

图 7-9　输出一个元素后调整新堆

在将最大值输出到数组最后一个位置后，这个位置不再算作堆的位置。实际上，堆中剩余元素的个数减 1，堆在数组中占用的元素个数也减 1。若原数据元素个数为 n，则一趟堆排序后，元素个数剩余 $n-1$，已有序的数据个数为 1，元素个数总和仍是 n。

继续这个过程，将堆顶值输出到堆空间的最后位置，调整剩余元素构成新的堆。在进行 $n-1$ 趟后，得到升序排序的数据序列。

堆的调整算法如下。

```
void ShiftDown (myRcd *myarr,int i,int n)          //堆的调整算法
{   int child;
    for (;i<=(n-1)/2;i=child){
      child=i*2+1;
      if ((child!=(n-1))&&(myarr->data[child+1]>myarr->data[child]))
        child++;
      if (myarr->data[i]<myarr->data[child])
        swap(&myarr->data[i],&myarr->data[child]);
    }
}
```

堆的排序算法如下。

```
void HeapSort (myRcd *myarr)                  //堆的排序算法
{   int i;
    for (i=(myarr->currentNum/2-1);i>=0;i--)
      ShiftDown (myarr,i,myarr->currentNum);
    for (i=myarr->currentNum-1;i>=1;i--){
      swap (&myarr->data[0],&myarr->data[i]);
      ShiftDown (myarr,0,i);
    }
}
```

在算法 HeapSort 中，第一个循环将建立初始堆；第二个循环将堆顶元素放到数组的最后，并再次调用 ShiftDown 函数进行整堆。排序输出的结果仍在原数组中。对最小堆进行排序后，数组中关键字将从大到小排列。

【例 7-13】 堆排序示例。

给定无序序列：46,20,17,40,52，使用堆排序对其进行降序排序。

如果要进行降序排序，则应该将初始数据建成最小堆。建立最小堆的过程如图 7-10 所示。降序排序的过程如图 7-11 所示。

【例 7-14】 下列关键字序列能构成一个堆的是（　）。

A. 90,31,53,23,16,48　　B. 90,48,31,53,16,23

C. 16,53,23,90,31,48　　D. 16,31,23,90,53,48

答案为 A 和 D。

给出对应于题目所给 4 个选项的 4 棵完全二叉树，如图 7-12 所示。

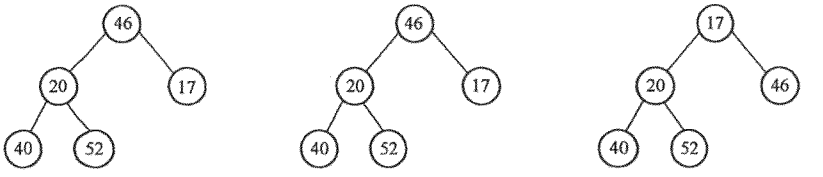


图 7-10　建立最小堆的过程

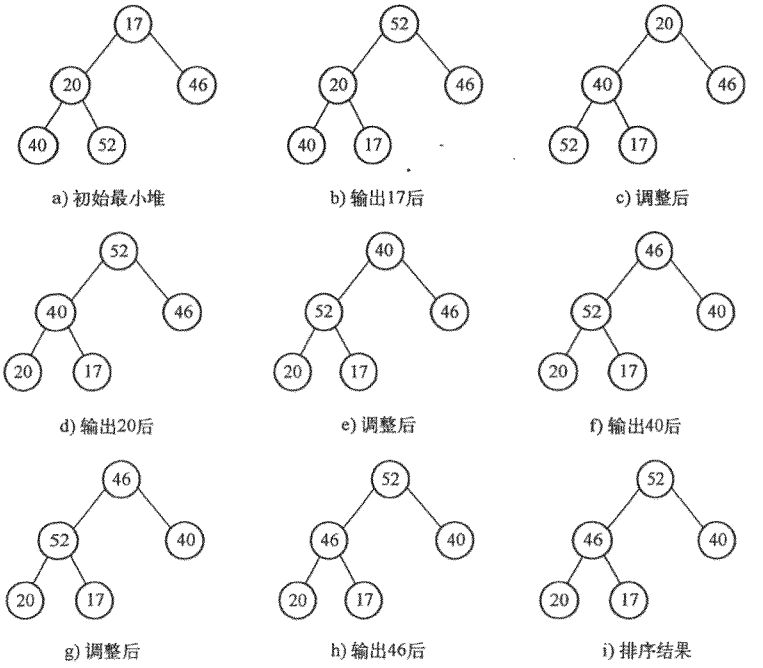


图 7-11　降序排序的过程

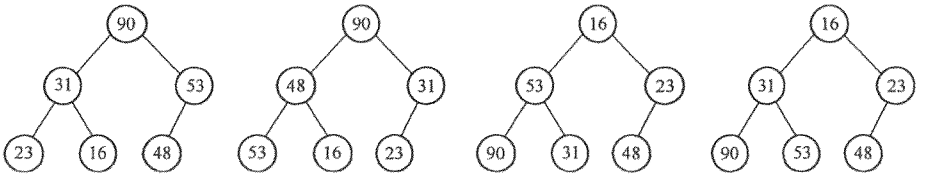


图 7-12　4 棵完全二叉树

可以看出，在选项 A 对应的完全二叉树中，每个元素都大于其所有孩子结点，故它是一个堆，且是大根堆。在选项 D 对应的完全二叉树中，每个元素都小于其所有孩子结点，所以它也是一个堆，且是小根堆。

在选项 B 中，元素 48 小于其父结点的元素 90，也小于其左子结点的元素 53。在选项 C 中，元素 53 大于其父结点的元素 16，也大于其右子结点的元素 31。所以它们都不是堆。

【例 7-15】 已知最小堆为 8,15,10,21,34,16,12，删除关键字 8 并将它放置到数组的最

后位置，之后进行整堆，在此过程中，关键字之间的比较次数是（ ）。

A. 1　　B. 2　　C. 3　　D. 4

答案为 C。

题目中所给的最小堆如图 7-13 所示。

在删除关键字 8 之后，将它与元素 12 互换，12 会被调整到堆顶。在这一步中，没有进行关键字之间的比较。12 的两个孩子结点相比较（第一次），较小者为 10，12 与较小的孩子结点相比较（第二次），因为 12 大于 10，故两者交换，10 成为堆顶。12 再与其孩子结点 16 进行比较（第三次），不需要交换。堆调整完毕。

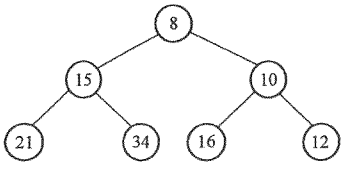


图 7-13　初始最小堆

对于堆结构，不仅支持删除堆顶元素，也支持元素的插入。新元素先放置在堆结构的最后，然后进行整堆。

【例 7-16】 已知元素序列 25,13,10,12,8 是最大堆（大根堆），在序列尾部插入新元素 18，将其再调整为最大堆。

给出的最大堆如图 7-14 所示。要在堆中插入新元素，需要将新元素放置在完全二叉树的第一个空闲位置，结果如图 7-15 所示。

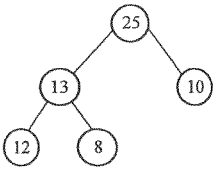


图 7-14　最大堆

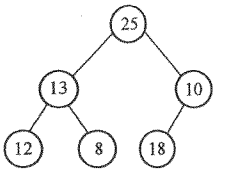


图 7-15　放置新元素 18

新元素的插入破坏了堆的特性，需要进行整堆。此时，新元素 18 与其父结点相比较。如果新元素小于父结点，则插入完毕，得到新的最大堆。否则，新元素与其父结点交换，然后再与新的父结点进行比较，可能的话，进行必要的交换。直到它小于父结点，或交换到树根，成为新的堆顶。插入完毕。

具体来说，18 与父结点 10 进行比较（第一次），18 大于 10，故进行交换，结果如图 7-16 所示。18 继续与新的父结点（25）进行比较（第二次），18 小于 25，故不进行交换。得到了新的最大堆，如图 7-17 所示。

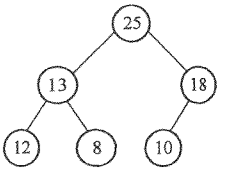


图 7-16　交换 18 与其父结点后

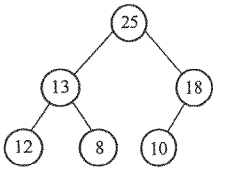


图 7-17　插入 18 后的新的最大堆

堆排序的最佳、平均及最差时间复杂度均为 $O(n\log n)$。堆排序不受初始数据有序性的影响，因此，在选择一种任何情况下都要求时间复杂度较好的排序方法时，堆排序可以是候选方法。

第五节　归并排序

归并（merge）是指将若干个有序序列合并为一个有序序列的操作，也称为合并。若参与合并的序列的个数为 2，则称为二路归并。借助于归并操作完成的排序就是归并排序（merging sort）。归并排序采用的是二路归并操作。

参与二路归并的两个有序序列的长度可以相同，也可以不同。归并操作还需要一个辅助数组，用来存储归并的结果。假设参与归并的两个有序序列的长度分别为 n 和 m，则辅助数组的长度就是 $n+m$。

归并排序可以使用非递归方式实现，也可以使用递归方式实现。

从数组头开始，依次扫描各数据，将单个数据两两合并为长度为 2 的有序子段。再从数组头开始，依次扫描各有序子段，将长度为 2 的有序子段两两合并为长度为 4 的有序子段。这个过程持续进行到所有数据都在一个有序序列中时为止。在此过程中，可能会出现长度不足 2 的幂次的有序子段。长度不影响合并的结果，也不影响排序的结果。这是非递归实现方式。

也可以将待排序数据等分为两个子段并分别进行排序，然后对两个有序序列进行二路归并操作，从而得到最终的排序结果。这属于递归方式实现。

一、两个有序序列的合并操作

二路归并排序中用到的基本操作就是两个有序序列的合并。一般地，有序序列也称为有序段，有些教材中也称其为归并段或顺串。归并排序就是反复归并有序序列的过程。先介绍两个有序序列的归并操作。

设在数组 A 中依次保存两个递增有序序列，它们的数据元素分别是 $a_1, a_2, \cdots, a_m$ 和 $b_1, b_2, \cdots, b_n$。使用两个变量 i 和 j 分别指示两个序列当前元素的下标，初始时均指向各自的第一个元素，即 $i=0$，$j=m$。使用数组 B 保存合并后的数据，使用变量 k 表示对应的下标，初始时为 0。

归并过程如下。比较 $A[i]$ 和 $A[j]$，若 $A[i]<A[j]$，则 $B[k]=A[i]$，然后 i 和 k 均加 1；否则，$B[k]=A[j]$，然后 j 和 k 均加 1。持续这个过程，直到 $i=m$ 或 $j=m+n$ 时，表示有一个有序序列的元素已经全部保存到 B 中。接下来，将另一个序列中全部的剩余元素依次保存到 B 中。归并完毕。

在两个非空有序序列进行归并时，必定有一个序列先变为空，而不可能两个同时为空。

【例 7-17】 两个有序序列的归并示例。

数组 A 中依次保存了两个有序序列：13,22,41,53 和 1,5,30,46，将它们归并为一个有序序列，结果保存到数组 B 中。归并过程如图 7-18 所示。

在归并过程中，每一次均从两个有序序列之一向结果序列中复制一个记录。若两个已知序列中元素个数总共为 n，则归并过程中总共需要复制 n 个记录，它的时间复杂度为 $O(n)$。同时，在归并过程中，还需要与待归并记录等量的存储空间。

下面实现两个有序段的归并操作。两个有序段相邻保存在 myarr 中，第一个有序段保存在下标从 1 到 m 的地方，第二个有序段保存在下标从 $m+1$ 到 n 的地方。归并后的有序段保

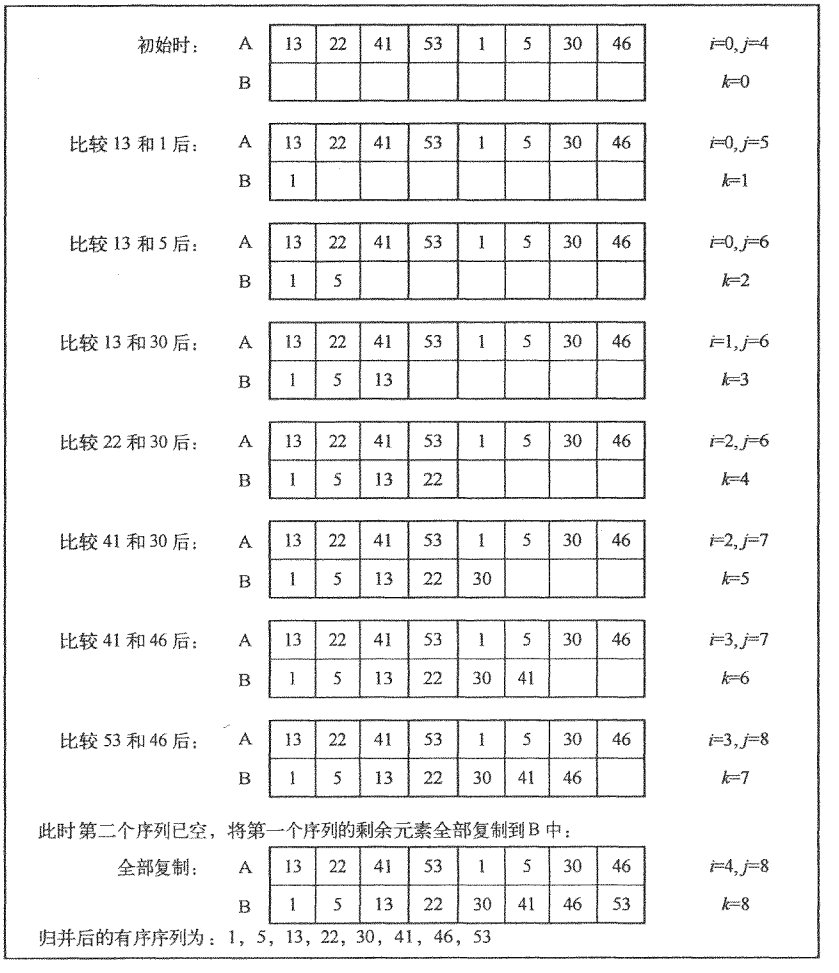


图 7-18　两个有序序列归并为一个有序序列的过程

存在 tmplist 中，下标从 k 开始。

```
void merge (myRcd * myarr,myRcd * tmplist,int l,int m,int n)
{   int i=l,j=m+1,k=l-1,t;
    while (i<=m && j<=n){
      if (myarr->data[i]<=myarr->data[j])      //将两个子段中较小记录移到临时空间中
        tmplist->data[++k]=myarr->data[i++];
      else
        tmplist->data[++k]=myarr->data[j++];
    }
    if (i<=m)
      for (t=i; t<=m; t++)                     //将第一个子段中的剩余元素移到临时空间中
        tmplist->data[++k]=myarr->data[t];
```

```
    if (j<=n)
      for (t=j;t<=n;t++)  //将第二个子段中的剩余元素移到临时空间中
        tmplist->data[++k]=myarr->data[t];
    for (i=1;i<=n;i++)    //将临时空间中的记录移回到数组 list 中
      myarr->data[i]=tmplist->data[i];
  }
```

二、归并排序的两种实现方式

归并排序是一个分治算法，可以使用非递归或递归的方式实现。

1. 非递归实现的归并排序

每个元素都可以看成长度为 1 的有序段，这是最短的有序段。从最短的有序段开始，逐渐将相邻的两个短有序段归并为一个长度为它们两个长度之和的长有序段。持续进行这个过程，直到所有的元素都排好序为止。

利用归并操作进行排序的具体过程是：设待排序序列中含有 n 个记录，初始时将它们看成 n 个有序子序列，即 n 个有序段。每个子序列的长度为 1。然后两两归并。如果 n 是偶数，则得到 $n/2$ 个长度为 2 的有序子序列；如果 n 是奇数，则得到 $(n-1)/2$ 个长度为 2 的有序子序列和一个长度为 1 的有序子序列。总之，这个过程得到的有序子序列的个数是 $\lceil n/2 \rceil$。

再把得到的这些有序子序列两两归并，得到 $\lceil \lceil n/2 \rceil /2 \rceil$ 个有序子序列。如果 n 不是 4 的倍数，则最后一个子序列的长度小于 4。

继续这个过程，直到得到一个长度为 n 的有序序列为止。在进行归并时，不断地将两个有序子序列归并为一个有序序列，所以归并过程中，有序子序列的个数每次减半，而长度倍增。最后一个子序列除外。

【例 7-18】 二路归并的非递归过程示例。

给定关键字序列：42,68,35,1,70,25,79,59,63,65，使用非递归方法进行二路归并排序的过程如图 7-19 所示。括号表示有序子序列。

初始：	(42)	(68)	(35)	(1)	(70)	(25)	(79)	(59)	(63)	(65)
第一趟归并后：	(42	68)	(1	35)	(25	70)	(59	79)	(63	65)
第二趟归并后：	(1	35	42	68)	(25	59	70	79)	(63	65)
第三趟归并后：	(1	25	35	42	59	68	70	79)	(63	65)
第四趟归并后：	(1	25	35	42	59	63	65	68	70	79)

图 7-19　非递归的归并排序过程

初始时，$n=10$ 个元素都被看成有序段，有序段的个数为 10，长度为 1。

在第一趟归并时，将 10 个有序段两两归并为 $n/2=5$ 个有序段，每个有序段的长度为 2。在第二趟归并时，第一个、第二个有序段归并为长度为 4 的有序段，第三个、第四个有序段归并为长度为 4 的有序段，第五个有序段不变，即得到 $\lceil 5/2 \rceil = 3$ 个有序段，最后一个有序段的长度为 2。

在第三趟归并时，前两个长度为 4 的有序段归并为长度为 8 的有序段，第三个（即上一

趟的第五个）有序段仍不变，即得到⌈3/2⌉=2 个有序段，长度分别是 8 和 2。

在第四趟归并时，长度为 8 的有序段和长度为 2 的有序段进行归并，得到长度为 10 的有序段，即全部数据排序完毕，得到排序结果。

非递归方式实现的归并排序的算法如下。

```
void mSort (myRcd *myarr,myRcd *tmplist,int left,int right)
{
    int i,j;
    if (left==right){
      tmplist->data[left]=myarr->data[left];
      return;
    }
    i=1;
    while (i<myarr->currentNum){
      for (j=0;j<right;j+=2*i){
        merge(myarr,tmplist,j,j+i-1,min(j+2*i-1,right));
      }
      i=i*2;
    }
}
```

在 mSort 函数中，使用变量 i 表示要归并的有序段的长度，在每一趟扫描后，有序段的长度倍增。同时，外层的 while 循环也用它来控制归并的趟数。内层的 for 循环，用来对本趟扫描中两两相邻的有序段进行归并，每次归并时，第一个有序段的起始位置是 j，有序段的长度是 i，所以有序段的结束位置是 $j+i-1$。最后一个有序段的长度可能小于 i，以 right 为界。

调用时，需要先声明临时变量：myRcd tmplist，然后按下列方式调用函数：

```
mSort(myarr,&tmplist,0,myarr->currentNum-1);
```

2. 递归实现的归并排序

在递归处理的初始，以待排序数据的中间位置为界，将待排序序列一分为二。如果数据个数为偶数，则前、后两部分的元素个数相等；如果数据个数为奇数，则前、后两部分的元素个数相差 1。不失一般性，让前半部分的元素个数多 1。

对划分的两部分数据进行递归处理。处理的结果是分别得到两个有序序列。再调用 merge 方法，将两个有序子序列归并为一个。

【例 7-19】 二路归并的递归过程示例。

给定关键字序列：35,70,79,25,59,63,65，使用递归法进行二路归并排序的过程如图 7-20 所示。

待排序数据元素个数为 7，是奇数，故划分的两段分别含 4 个和 3 个元素。先处理含 4 个元素 35,70,79,25 的子段，后处理含 3 个元素 59,63,65 的子段。

当处理 35,70,79,25 时，又将它划分为两个各含 2 个元素的子段。这两个子段分别是

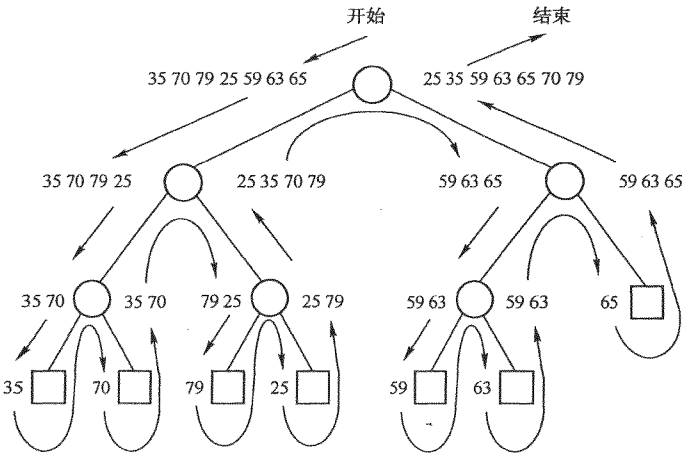


图 7-20　递归的归并排序过程

35,70 和 79,25。这两个段的处理都早于子段 59,63,65 的处理。

对于子段 35,70，又将它划分为各含 1 个元素的两个子段，并进行递归调用。含一个元素的子段自然有序，故不必再递归调用，递归返回。将两个各含 1 个元素的子段归并为含 2 个元素的子段。

接下来处理子段 79,25，处理的过程类似于对 35,70 的处理，即划分为两个子段，并进行递归调用。递归调用后，将两个子段归并为含 2 个元素的子段。

此时，已有两个各含 2 个元素的子序列，可以完成它们的归并了。归并的结果是得到有序序列 25,35,70,79。

对含 3 个元素 59,63,65 的子段的处理与对含 4 个元素的子段的处理类似。当得到它的处理结果后，就可以将前 4 个元素与后 3 个元素的两个有序子段进行归并了，最终得到 7 个元素的有序序列。

递归的归并排序算法的实现如下。

```
void mSort (myRcd *myarr,myRcd *tmplist,int left,int right)   //递归实现
{   int center;
    if (left==right)
      tmplist->data[left]=myarr->data[left];
    else{
      center=(left+right)/2;
      mSort (myarr,tmplist,left,center);                    //归并排序数组 myarr 的前半段
      mSort (myarr,tmplist,center+1,right);                 //归并排序数组 myarr 的后半段
      merge (myarr,tmplist,left,center,right);
    }
}
```

归并排序中需要一个等长的工作数组，即辅助数组，并需要在原数组和工作数组中互相复制数据。

归并排序每次二分数组，对于长度为 n 的数组，二分数组的次数应为 $\log n$。因为每一次都需要对数组中所有元素进行一次扫描，所以每次的操作都需要 n 步，这样，总的时间复杂度为 $O(n\log n)$，这也是它的最佳、最差及平均时间复杂度。

第六节　分配排序

前面介绍的几种排序方法的时间复杂度介于 $O(n\log n)$ 与 $O(n^2)$ 之间，有理论证明，排序算法的时间复杂度不会好于 $O(n\log n)$，这个界限是一般意义下的时间复杂度。实际上，时间复杂度和空间复杂度在一定条件下可以互相转换。当排序的数据比较特殊，或者不计空间代价时，排序算法的时间复杂度可能低于 $O(n\log n)$。

一、盒子排序

现在来看一个例子，对下列两位正整数进行排序：

23,98,10,19,28,3,53,29,20,94

因为数据都是不大于 100 的正整数，所以可以使用一个有 100 个元素的整数数组 count 来完成排序，初始时数组元素均为 0。从左至右扫描待排序数据，若数据为 d，则令 count[d]=1。扫描完毕，输出数组信息。使用 for 循环语句处理 count 中的每个元素，若元素值为 1，则输出其下标值。处理完毕，即得到原初始数据的排序结果。这种排序称为盒子排序。每个数组单元称为一个盒子。

如果数据中有重复的元素，则将“count[d]=1”修改为“count[d]++”。如果需要保存排序后的结果，可以根据 count 中各元素的信息，将排序结果保存到原数组中。

盒子排序的算法如下。

```
void BoxSort (myRcd *myarr,int k)          //k 为盒子数
{
  int i,j,m;
  int *count;
  count=(int *)malloc(sizeof(int)*k);
  for(i=0;i<k;i++)
    count[i]=0;
  for(i=0;i<myarr->currentNum;i++)
    count[myarr->data[i]]++;
  j=m=0;
  for(i=0;i<k;i++){
    if(count[i]!=0){
      for(j=0;j<count[i];j++){
        myarr->data[m]=i;
        m++;
      }
    }
```

```
    }
}
```

如果不能确定盒子的个数，则需要先统计数据的范围，然后计算出盒子的个数。程序修改如下。

```
void BoxSort (myRcd * myarr)
{
    int i,j,m,k;
    int min=INT_MAX,max=INT_MIN;
    int * count;
    for(i=0;i<myarr->currentNum;i++){
      if(myarr->data[i]>max) max=myarr->data[i];
      if(myarr->data[i]<min) min=myarr->data[i];
    }
    k=max-min+1;
    count=(int *)malloc(sizeof(int) * k);
    for(i=0;i<k;i++)
      count[i]=0;
    for(i=0;i<myarr->currentNum;i++)
      count[myarr->data[i]-min]++;
    j=m=0;
    for(i=0;i<k;i++){
      if(count[i]!=0){
        for(j=0;j<count[i];j++){
          myarr->data[m]=i+min;
          m++;
        }
      }
    }
}
```

在排序过程中，主要操作是设置数组中某下标位置的元素值，并根据元素值输出相应的下标值。若初始待排序的元素个数为 n，则排序的时间复杂度为 $O(n)$。

盒子排序有一定的限定条件，不能算作通用的排序方法。排序时，直接或间接地将待排序的数据对应为数组下标。对于整数，这很容易进行转换，但如果待排序的数据是字符串，在字符串与下标之间建立一一对应关系是比较困难的。另外，待排序整数的大小范围决定了数组的大小。如果 n 个数据的取值范围是 $1\sim m$，而 $m\gg n$，则需要分配含 m 个元素的数组，且数组中仅有 n 个元素是 1，$m-n$ 个元素都是 0，输出排序结果的时间复杂度是 $O(m)$，而不是 $O(n)$。另外，盒子排序的空间复杂度是 $O(m)$。

二、基数排序

基数排序是一种分配排序，排序过程就是反复进行“分配”和“收集”的过程。在基数排序中，关键字被拆分为若干个子关键字，排序过程中需要分别对每个子关键字进行分配和收集操作。例如，若数组 $K[0],K[1],\cdots,K[n-1]$ 中的关键字均是两位十进制整数，则将关键字拆分为“个位”和“十位”两个子关键字。由于每个子关键字的取值可能有 10 种，即 0~9，因此设置 10 个盒子，编号为 0~9。从左到右扫描数组，对数组中各关键字按个位数进行分配，依次将个位数为 r（$0\leqslant r\leqslant 9$）的关键字放入编号为 r 的盒子中。这称为第一趟分配。之后依次将编号 0~9 的各盒子中的关键字按放入的次序收集起来。这称为第一趟收集。将收集的数据再按十位数分类，十位数为 r（$0\leqslant r\leqslant 9$）的元素放入编号为 r 的盒子中。这称为第二趟分配。之后是第二趟收集。最后的结果即有序序列。

【例 7-20】 基数排序示例。

设有如下关键字序列：27,91,1,97,17,23,84,28,72,4,67,25，给出基数排序过程。

使用 10 个盒子，当前数位相同的关键字保存在同一个盒子里。先对各元素按个位数进行第一趟分配与收集。第一趟分配的结果如图 7-21 所示。

第一趟收集的结果是：91,1,72,23,84,4,25,27,97,17,67,28。再按十位数进行第二趟分配与收集。第二趟分配的结果如图 7-22 所示。

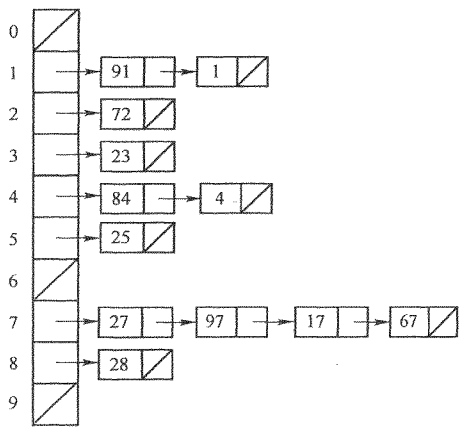


图 7-21　基数排序的第一趟（按个位数进行排序）

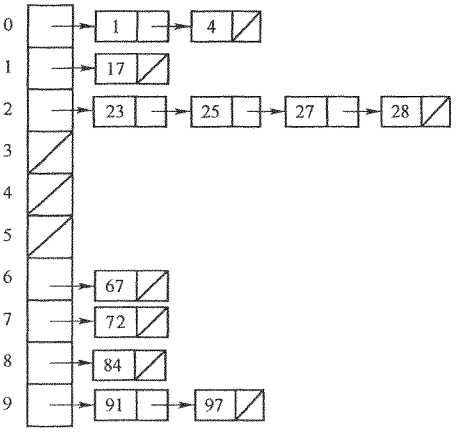


图 7-22　基数排序的第二趟（按十位数进行排序）

第二趟收集的结果是：1,4,17,23,25,27,28,67,72,84,91,97。结果已经有序。

扩展上述排序思想，基数排序可以对任何数制的元素进行排序。若进制为 r，则对应的盒子数为 r，位数为 d，分配与收集的趟数为 d。分配和收集的操作从低位到高位依次进行。基数排序一共进行 d 趟分配和收集，每趟收集需要将 r 个盒子中的关键字收集起来。因为在每趟排序中，每个数据都被处理两次，所以，算法的时间复杂度可以粗略地认为是 $O(d\times n)$。对于待排序数据集，通常要求 r 和 d 固定，即它们不随数据量的大小而改变。因此基数排序的时间复杂度是线性的，且基数排序算法是稳定的。若位数 d 不固定，则基数排序也是线性对数级的。

在实现基数排序时，图 7-21 和图 7-22 中使用单链表表示每个盒子，这能很好地适应盒子中数据个数的不确定性，但是，对单链表访问的不方便性在所难免。

在每趟排序时，每个盒子中保存的数据个数依排序数据而定，所以每个盒子的大小不是固定的。但从另一个角度来看，这些盒子的总量是固定的，即等于给定的初始数据个数。所以可以分配一个与初始数组等大的数组，由这些盒子共用，在这个新数组中给每个盒子留出各自的空间。不妨设新数组为 mybrr，这是基数排序需要占用的额外空间。空间复杂度为 $O(n)$。

那么，mybrr 中每个盒子具体是多大呢？扫描一趟数据，针对当前数位进行计数，就可以得到相应的结果。计数结果可以保存在 count 数组中，当进行十进制数的排序时，count 的大小为 10。count 数组也是排序时占用的额外空间，但因为它并不随排序数据的个数而有所变化，所以空间复杂度是 $O(1)$。总的空间复杂度为 $O(n)$。

以例 7-21 中第一趟排序为例。第一趟排序是针对数据的个位数进行的，统计每个数的个位数，并记录到 count 中，获取个位数的方法是：myarr->data[j] % 10，得到的 count 数组的结果如图 7-23 所示。

	0	1	2	3	4	5	6	7	8	9
count	0	2	1	1	2	1	0	4	1	0

图 7-23　用于第一趟排序的辅助数组 count

从 count 数组中可以直接获取任一个盒子中的数据个数，例如，count[1]是第 1 个盒子的大小，其值 2 表示个位数为 1 的数据有两个，即原始数据中这样的数据有两个（91 和 1）。

对 count 数组进行进一步处理，从下标 1 开始，将前一个元素中的值加到本元素上，即 count[j]=count[j-1]+count[j]，可以得到各个盒子在 mybrr 中的下标范围。处理后 count 数组的结果如图 7-24 所示。

	0	1	2	3	4	5	6	7	8	9
count	0	2	3	4	6	7	7	11	12	12

图 7-24　处理后的辅助数组 count

第 j（$j \geq 1$）个盒子在 mybrr 中的下标范围为 count[$j-1$]～count[j]-1，如个位数为 1 的两个数据（91 和 1）将分别保存在 mybrr[0]和 mybrr[1]中。第 0 个盒子在 mybrr 中的下标从 0 开始。count 中的值与 mybrr 位置的对应如图 7-25 所示。

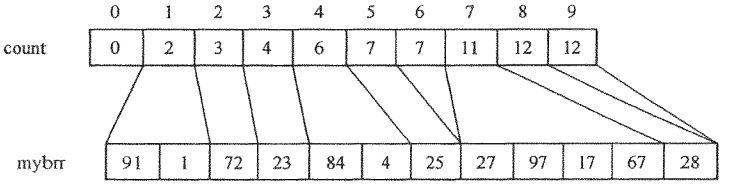


图 7-25　count 中的值与 mybrr 位置的对应

实际上，可以根据 count 中的值将本趟排序中的数据直接放置到 mybrr 中。对于要放置到第 j 个盒子中的若干个数据，最后一个数据要放置到 count[j]-1 中，倒数第二个数据要放置到 count[j]-2 中，依此类推。从后向前依次扫描数据序列，设当前待处理的元素的个位数是 j，将其放置到 mybrr[count[j]-1]中，然后 count[j]中的值减 1。

第二趟排序是针对数据的十位数进行的，获取十位数的方法是：(myarr->data[j] /10) % 10，其他处理与第一趟排序类似。

基数排序的算法如下。

```
void RadixSort (myRcd * myarr,int k,int r,int count[])
//对 k 位数、r 进制的数据进行基数排序，count 是辅助计数数组
{
    int i,j,rtok;
    myRcd mybrr;                                    //辅助空间
    for (i=0,rtok=1;i<k;i++,rtok * =r){
        for (j=0;j<r;j++) count[j]=0;               //赋初值
        for (j=0;j<myarr->currentNum;j++)
            count[ myarr->data[j]/rtok % r]++;      //计数
        for (j=1;j<r;j++)count[j]=count[j-1]+count[j];//计算下标范围
        for (j=myarr->currentNum-1;j>=0;j--)
            mybrr. data[ --count[ myarr->data[j]/rtok%r]]=myarr->data[j];
            //根据 count 的值将数据放入辅助空间中
        for (j=0;j<myarr->currentNum;j++)
            myarr->data[j]=mybrr. data[j];          //将辅助空间中的数据复制回原数组中
    }
}
```

在对数组 myAarray 中保存的不超过 k 位的十进制数进行基数排序时，主程序中需要定义 count 数组，语句如下。

```
int count[10]={0};
```

调用基数排序的语句如下。

```
RadixSort (&myAarray,k,10,count);
```

【例 7-21】对给定的关键字序列 110,119,007,911,114,120,122 进行基数排序，则第二趟分配和收集后得到的关键字序列是（　）。

A. 007,110,119,114,911,120,122　　B. 007,110,119,114,911,122,120

C. 007,110,911,114,119,120,122　　D. 110,120,911,122,114,007,119

答案为 C。

基数排序方法是一种最低位优先的分配排序算法，第一趟排序按最低位（个位）进行排序，分配和收集的结果为：110,120,911,122,114,007,119。

第二趟按十位进行排序，分配和收集的结果为：007,110,911,114,119,120,122。

第七节　有关内部排序算法的比较

本章的前几节讨论了几种内部排序算法，本节将对内部排序算法进行总结。

在介绍排序算法的同时，也给出了算法的时间复杂度及空间复杂度。排序算法的时间复杂度分为三种情况，分别是最优情况时间复杂度、最差情况时间复杂度及平均情况时间复杂度。对不同的具体问题所要求的衡量指标可能不全一样，因此选择的排序算法也不全一样。例如，对于需要日常进行处理的常规排序操作，由于进行排序的次数比较多，一般要看它的平均情况时间复杂度，因此应选择一种平均时间复杂度较好的算法；而对于某些需要在任何情况下都能得到较高效率的问题，应该选择一种最差情况时间复杂度最优的算法。大多数情况下，可以用平均情况时间复杂度来衡量一个排序算法的优劣。平均情况时间复杂度是指，假设待排序序列中各关键字排列的情况是等概率的，此时对所有可能的输入数据进行排序的期望时间复杂度。

在几种内部排序算法中，如果考虑平均时间复杂度，则快速排序算法的性能最好，归并排序次之，堆排序更次之，特别是当待排序数据量 n 很大时，归并排序优于堆排序。

如果考虑最差情况时间复杂度，则堆排序和归并排序的性能都要优于快速排序，但归并排序需要较多的辅助存储空间。如果待排序序列中的数据个数较少或者基本有序，则简单插入排序的性能最佳。

由此可以看出，没有一个绝对的标准来评判排序算法性能的优劣，对于不同的输入序列，各算法表现出的性能有很大差别。在实际应用中，往往结合多种排序算法一起使用。

有算法理论证明，对于有 n 个记录的序列，对它进行内部排序时，没有一种方法的比较次数能够少于 $n\log n$，也就是说，任何一种内部排序算法所能达到的最佳时间复杂度为 $O(n\log n)$。

除了考虑算法的时间复杂度以外，算法的稳定性也是一个重要因素。一般地，平均情况时间复杂度为 $O(n^2)$ 的排序算法大多是稳定的，其中包括直接插入排序、起泡排序等。而时间性能较好的几种排序算法，如希尔排序、快速排序、堆排序等，因为它们进行了不相邻记录之间关键字的比较和记录的移动，所以排序的结果是不稳定的。其中值得一提的是归并排序和基数排序，它们的时间复杂度分别是 $O(n\log n)$ 和 $O(dn)$，并且都是稳定的排序方法。

综合比较已经讨论的几种内部排序方法，有下列结果，见表 7-1。

表 7-1　几种内部排序方法的时间复杂度和稳定性

排序方法	最优情况时间复杂度	平均情况时间复杂度	最差情况时间复杂度	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	稳定
起泡排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	不稳定
希尔排序	$O(n^{3/2})$	—	—	不稳定
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	不稳定
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	稳定
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	不稳定
基数排序	$O(dn)$	$O(dn)$	$O(dn)$	稳定

【例 7-22】 对序列 15,9,7,8,20,-1,4 进行排序，若进行一趟排序后数据的排列变为 4,9,-1,8,20,7,15，则采用的排序方法可能是（ ）。

A. 选择排序　　B. 快速排序　　C. 希尔排序　　D. 起泡排序

答案为 C。

先排除不可能的选项，再验证可能的选项。

选择排序的第一趟，会将原始数据中的最大值或最小值放到最后面或最前面的位置。原始数据中的最大值是 20，最小值是-1，它们都没在相应的位置，说明不会采用选择排序，选项 A 被排除。

起泡排序也是类似的，第一趟会将最大值或最小值放置到最终的位置。选项 D 也被排除。

再看快速排序。在第一趟快速排序后，数据序列中应该有一个元素是枢轴，在枢轴前面的数据都小于枢轴，在枢轴后面的数据都大于枢轴。但在排序结果 4,9,-1,8,20,7,15 中，找不到这样一个元素来充当枢轴。选项 B 被排除。

下面验证这可能是一趟希尔排序的结果。

初始数据：15,9,7,8,20,-1,4

一趟排序：4,9,-1,8,20,7,15

根据最小值-1 在排序前后的位置，可以猜测增量应该大于 2，因为如果增量等于 2，则数据被分为两组，最前面的两个数分别是各自组内的最小值。如果增量为 1，则结果是全部有序的。这两种情况都不满足。先用增量 3 来验证。斜体表示组内采用直接插入排序的结果。

d=3	15	*4*		8	*8*		4	*15*	第一组
		9	*9*		20	*20*			第二组
			7	*-1*		-1	*7*		第三组
一趟排序结果	4	9	-1	8	20	7	15		

结果符合题目要求。

【例 7-23】 下列选项中，均为稳定排序方法的是（ ）。

A. 堆排序和起泡排序　　B. 快速排序和希尔排序

C. 简单选择排序和归并排序　　D. 归并排序和起泡排序

答案为 D。

根据排序方法稳定性的定义可知，堆排序、快速排序、希尔排序、简单选择排序都不是稳定排序方法。而归并排序和起泡排序是稳定排序方法。

内 容 小 结

本章介绍了几种内部排序算法，给出了各排序算法的基本思想，并基于数组存储结构，实现了相关算法。同时，本章重点分析了各排序算法的时间复杂度和空间复杂度，对各个算法进行了比较、分析。

习　题

一、单项选择题

1. 下列关于稳定排序算法的叙述中，正确的是_____。

Ⅰ. 该排序算法可以处理相同的关键字

Ⅱ. 该排序算法不可以处理相同的关键字

Ⅲ. 该排序算法处理相同的关键字时能确定它们的排序结果

Ⅳ. 该排序算法处理相同的关键字时不能确定它们的排序结果

A. Ⅰ、Ⅲ　　B. Ⅰ、Ⅳ　　C. Ⅱ、Ⅲ　　D. Ⅱ、Ⅳ

2. 下列排序方法中，排序趟数与序列的初始状态有关的是_____。

A. 插入排序　　B. 选择排序　　C. 起泡排序　　D. 快速排序

3. 若用起泡排序对关键字序列 18,16,14,12,10,8 进行升序排序，则所需进行的关键字比较的总次数是_____。

A. 10　　B. 15　　C. 21　　D. 34

4. 下列排序算法中，不能保证每趟排序至少能将一个元素放到其最终位置上的是_____。

A. 快速排序　　B. 希尔排序　　C. 堆排序　　D. 起泡排序

5. 一组记录的关键字为 46,79,56,38,40,84，利用 partition 划分算法，以第一个记录为枢轴，得到的一次划分结果为_____。

A. 38,40,46,56,79,84　　B. 40,38,46,79,56,84

C. 40,38,46,56,79,84　　D. 40,38,46,84,56,79

6. 在下面的排序方法中，辅助空间为 $O(n)$ 的是_____。

A. 希尔排序　　B. 堆排序　　C. 选择排序　　D. 归并排序

7. 下列排序算法中，当待排序数据已有序时，花费时间反而最多的是_____。

A. 起泡排序　　B. 希尔排序　　C. 快速排序　　D. 堆排序

8. 数组中有 10000 个元素，如果仅要求求出其中最大的 4 个元素，则最节省时间的算法是_____。

A. 直接插入排序　B. 希尔排序　　C. 快速排序　　D. 选择排序

9. 从未排序序列中依次取出一个元素与已排序序列中的元素依次进行比较，然后将其放在已排序序列的合适位置，该排序方法称为_____。

A. 直接插入排序　B. 选择排序　　C. 希尔排序　　D. 二路归并排序

10. 在使用数据序列 10, 15, 20, 25, 30 建立初始大根堆时，数据对交换的次数是_____。

A. 1　　B. 2　　C. 3　　D. 4

二、填空题

1. 对含 n 个数据的序列进行排序，直接插入排序在最好情况下的时间复杂度为_____。

2. 现有含 9 个关键字的最小堆，排在关键字升序第三位的元素（第三小）所处的位置

个数可能是_____。

3. 一组记录的关键字值为45,79,59,63,65,26,80,17，利用partition划分算法，以第一个记录为枢轴，划分过程中，与元素79相交换的关键字是_____。

4. 已知一组关键字为：45,78,57,30,40,89，利用堆排序对其进行升序排序，建立的初始堆是_____。

5. 设关键字序列为16,15,32,11,6,30,22,46,7，采用基数排序进行升序排序。若关键字序列保存在含9个元素的数组A中，则一趟基数排序后，元素16的下标位置为_____。

三、解答题

1. 设一维整数数组内保存整数序列，回答下列问题：

（1）试写出整数序列2,3,8,6,1中的所有逆序对。

（2）在什么情况下，由$1,2,\cdots,n$组成的序列中逆序对最多？

2. 有关键字序列：38,19,65,13,97,49,41,95,1,73，采用起泡排序方法进行升序排序，请写出每趟排序的结果。

3. 对于数据序列：66,61,200,30,80,150,4,8,100,12,20,31,1,5,44，写出采用希尔排序算法排序的每一趟结果。设增量序列d={5,3,1}。

4. 有关键字序列：38,19,65,13,97,49,41,95,1,73，采用直接插入排序算法进行升序排序，请写出每趟排序的结果。

5. 有关键字序列：38,19,65,13,97,49,41,95,1,73，采用简单选择排序算法进行升序排序，请写出每趟排序的结果。

6. 对于数据序列：31,5,44,55,61,200,30,60,20,1,80,150,4,29，采用改进的起泡排序算法进行降序排序，请写出每趟排序的结果。

7. 对于数据序列：31,5,44,55,61,200,30,60,20,1,80,150,4,29，写出采用快速排序算法排序的过程。

8. 将数据序列70,12,30,1,5,31,44,56,61建成一个最大堆。

9. 对于数据序列：70,12,30,1,5,31,44,56,61，采用堆排序算法进行升序排序，请写出每趟排序的结果。

10. 对于数据序列：31,5,44,55,61,30,60,20,1,4,29，采用基数排序算法进行升序排序，请写出每趟排序的结果。

四、算法阅读题

设一系列正整数存放在一个数组中，算法OddEvenSort将所有偶数存放在数组的前半部分，将所有的奇数存放在数组的后半部分。请在空白处填上适当内容以将算法补充完整。

```
void OddEvenSort (int *data, int n)                    //奇偶排序
{
    int left,right,mostleft=0,mostright=n-1;
    left=mostleft;                                     //left从左向右找
    right=mostright;                                   //right从右向左找
    for (;;){
      while ((1) && left<mostright){
            left++;
```

```
        }
        while (data[right]%2!=0 &&(2)){
                right--;
        }
        if (left<right)
                swap((3));
        else
                break;
    }
    display(data,n);
    return;
}
```

五、算法设计题

1. 将 3 个关键字进行排序，关键字之间的比较次数最多为多少？试实现这样一个排序算法。

2. 设计一个算法，计算含 n 个元素的数据序列中的逆序数据对的个数。

3. 设有 n 个整数的数组，其元素值仅取 0、1、2 三种，试设计一个时间复杂度为 $O(n)$ 的算法，将这个数组中的元素按升序排序。

第八章 查 找

学习目标：

1. 理解查找的基本概念。

2. 掌握顺序查找、折半查找、索引顺序查找方法的基本思想及实现过程，理解各种查找方法的适用条件。

3. 理解二叉查找树的基本概念，掌握二叉查找树操作的定义及实现。

4. 理解 AVL 树的基本概念，掌握 AVL 树基本操作的定义及实现。

5. 理解 B 树的概念，掌握插入及查找操作的定义及实现过程。

6. 理解哈希方法中的基本概念，掌握哈希方法。

7. 能够对各查找方法进行比较、分析。

建议学时： 11 学时。

教师导读：

1. 查找是重要的操作，要让考生理解查找中的关键操作是关键字之间的比较，进而理解查找算法复杂度的分析过程。

2. 要让考生理解选择查找方法时不仅要考虑算法的效率，还要了解查找方法适用的数据结构，领会影响查找方法使用的关键因素。

3. 查找树是重要的知识点，不仅要掌握相关的概念，更要领会查找树的特点，能进行简单的推理。

4. 哈希方法中有些内容很灵活，如哈希函数的设计。要让考生掌握设计的原则，而不仅仅是记忆公式。

5. 在学习完本章后，应要求考生完成实习题目 7。

查找是日常生活中常用的词汇，也是计算机内部最常进行的操作之一。在日常生活中提到查找时，常常是指查找某个具体的对象，其含义并不是太严谨。数据结构中的查找是指给定一个具体的目标，在数据集中进行搜索，寻找与给定目标相符合的对象的一个过程。例如，在数据库中查找关键字等于某个确定值的一个或多个记录，在一段视频中查找一幅满足某个条件的图像等。查找又称为检索。

查找分单值查找及范围查找。单值查找是指给定一个具体的值，查找与该值相等的对象。范围查找是指给定一个值的区间，查找其值位于该区间内的所有对象。本章主要讨论单值查找方法。将这些单值查找方法进行适当修改，可以适用于范围查找。

查找过程是在一个数据集上完成的，数据集有时也称为查找表或查找池。查找表可以是数组，也可以是链表，还可以是树。

查找过程中涉及的主要操作是比较，查找效率高是指在查找目标的过程中进行少量的比较。一般而言，查找表中数据项越多，找到目标需要的比较次数可能就越多。所以，问题的大小由查找表中数据项的个数来决定。

针对不同的查找要求，有不同的策略。本章将讨论这些查找方法，并分析不同方法的适用条件及它们的查找效率。

第一节　查找的基本概念

定义 8-1　设有一个集合 T，其中有 n 个数据项，集合 T 及其中数据项的形式如下：

$$T=\{(k_0,I_0),(k_1,I_1),\cdots,(k_{n-1},I_{n-1})\}$$

其中，$k_0,k_1,\cdots,k_{n-1}$是互不相同的关键字值，$I_j(0\leqslant j\leqslant n-1)$是与关键字值 k_j相关的信息。给定一个特定的关键字值 K，查找问题是在 T 中确定数据项(k_j,I_j)，使得 $k_j=K$。

查找表中的数据项也称为记录，每个记录至少包含一个关键字。查找实际上就是根据给定的某个值，在查找表中找出一个记录，该记录的关键字值恰好等于给定的值。给定的这个值称为目标。

记录中关键字的类型可以是能够进行比较操作的任意类型，如整数、字符串等。能唯一确定某个记录的关键字称为主关键字，各记录的主关键字互不相同。主关键字值一旦确定，它就对应着唯一的一条记录，主关键字与记录一一对应。不能唯一确定记录的关键字称为次关键字，可能存在多个记录的次关键字具有相同值的情况。也就是说，可能会有多条记录，它们的次关键字值都一样。

在实际操作中，不仅可以对主关键字和次关键字进行查找，还可以进行多个关键字的混合查找。不失一般性，本章后面的叙述中提到的关键字，一般是指主关键字，相应的查找过程都是对主关键字的查找。在经过适当的修改后，这些算法同样可以适用于对次关键字的查找。

根据关键字匹配的情况，查找有两种可能的结果：一种是找到了相应的记录，即找到一个关键字值为 k_j的记录，使得 $k_j=K$，这称为成功查找；另一种是找不到相应的记录，称为查找失败，或不成功查找。查找失败意味着在查找表中不存在要找的记录。

例如，一批学生的成绩单构成一个查找表，每名学生的成绩单组成一个记录。每条记录中可以包含学号、姓名、性别、年龄以及各科成绩等信息，这些项目都可以作为关键字。其中学号可以作为主关键字，而姓名等其他信息都可能存在有重复值的情况，所以只能作为次关键字。

根据数据存储介质的不同，查找又分为两种类型：当数据量非常大，需要借助于外存存储数据时，相应的查找称为外部查找；反之，当全部数据可以同时存放在内存中时，相应的查找称为内部查找。

查找表的结构及特性不同，所用的查找方法也可能不同。当查找表是无序表时，可以使用顺序查找方法。当查找表是有序表时，可以使用折半查找方法。索引顺序查找方法结合了顺序查找方法和折半查找方法的特点。还可以使用二叉查找树、AVL 树和 B 树等树结构保存查找表中的记录，进而在树中进行查找。还有一种很特别的基于数组的查找方法，即哈希方法。

研究查找方法的主要目的是提升查找效率，但同时也要考虑节省存储空间。查找的效率与记录的组织方式及查找方法有很大的关系，此外，每次查找时所花费的具体开销也与查找目标在数据中所处的位置有关。所以，在评价查找效率时，需要建立一个评价标准，通常使

用平均查找长度（Average Search Length，ASL）来衡量。

对于一个给定的关键字值 K，在查找表中进行查找，最好的可能是经过一次比较就能查找到，当然也有可能需要多次比较才能查找到。为了表示在平均意义下的查找次数，定义平均查找长度为：

$$ASL = \sum_{i=0}^{n-1} P_i C_i$$

其中，n 为查找表中的记录个数，P_i 为查找记录 i 的查找概率，C_i 为查找记录 i 时关键字与查找目标进行比较的次数。通常情况下，本章讨论等概率情况下的查找。

本章将详细介绍几种常用的查找方法，并分析各自的查找效率。

第二节　顺序表的查找

将数据存储在数组中，从而形成顺序表。数组中保存的数据可能有两种状态，一种是按关键字无序，也就是数据任意放置，此时为无序表；另一种是按关键字有序，即各记录在数组中的存放位置依关键字的大小顺次排列，此时为有序表。

本节讨论 3 种查找方法，分别是顺序查找方法、折半查找方法和索引顺序查找方法。顺序查找方法适用于任意顺序表，而对于有序表，由于其中数据的排列有规律，因此可以使用效率更高的折半查找方法。结合了这两种方法特性的查找方法是索引顺序查找方法。

一、顺序查找方法

顺序查找是最简单、最直接的查找方法之一，它的基本思想是从顺序表头开始，用给定的目标与表中各记录的关键字值逐个进行比较。如果表中存在目标，则进行若干次比较后，一定能够找到目标。如果表中不存在目标，则比较到表尾也不会出现相等的情况。

具体的过程是：初始时，将给定的关键字值 K 与表中第一个记录的关键字值相比较，若两个值相等，则找到目标，查找成功；否则将 K 与表中下一个记录的关键字值继续比较并判断是否相等，依此类推。如果直到最后一个记录的关键字值与 K 都不相等，则表明所存储的数据中没有要查找的目标，查找不成功。

无论顺序表中数据是否有序，都可以使用顺序查找方法。另外，如果数据使用链表来存储，则也可以使用顺序查找方法。

顺序表保存在一个数组中，定义如下。

```
typedef int ELEMType;
typedef struct{
    ELEMType element[maxSize];
    int n;                                  //实际元素个数
}searchList;
```

用到的常量定义如下。

```
#define TRUE 1
#define FALSE 0
#define ERROR -1
```

在顺序表中进行顺序查找的算法如下。

```
//数组表示的顺序表中的顺序查找
int SeqSearch(searchList L,ELEMType target)  //返回 target 在表 L 中第一次出现的位置
{   int i;
    for (i=0;i<L.n;i++)                      //对记录号从小到大逐个查找
      if (L.element[i]==target) return i;    //返回查找结果
    return ERROR;                            //不成功时会返回 ERROR
}
```

SeqSearch 函数在顺序表中查找关键字等于 target 的记录。从第一个记录开始依次向后查找，判断每个记录的关键字值是否等于 target，据此给出查找成功与否的反馈信息。在查找成功后，返回目标在查找表中第一次出现的下标。

实际上，在 for 循环中，除要判断两个值是否相等以外，还需要注意数组下标不能越界。如果在数组中查找失败，则函数返回 ERROR。因为数组下标是非负值，所以-1 显然不是正确的下标值。根据返回值，可以判定在顺序表中的查找是否成功，以及成功时查找目标在数组中的位置。

可以修改这个顺序查找方法，让其也适用于关键字值有重复的情况。当遇到数组中关键字等于目标时，并不直接退出，而是使用一个辅助的结构来保存与目标相等的所有关键字所在的下标值。

顺序查找是从头至尾依次进行的，所以它也称为线性查找。如果是对次关键字进行查找，则顺序查找方法能保证查找到第一个满足条件的记录。顺序查找不要求数据按某种特定的次序保存在数组中，它的查找过程很容易理解，但查找效率不高。

顺序查找的效率如何呢？SeqSearch 需要的空间并不依顺序表的元素个数的变化而变化，空间复杂度为 $O(1)$。

在顺序查找的情况下，最坏的可能是要经过 n 次比较才能查找到。在算法 SeqSearch 中，$C_i=i+1(i\geq 0)$。假设查找表中每个记录的查找概率相等，即 $P_i=1/n(i=0,1,\cdots,n-1)$，则此时

$$\mathrm{ASL}=\frac{1}{n}\sum_{i=0}^{n-1}(i+1)=(n+1)/2$$

这意味着，成功查找的平均查找长度为$(n+1)/2$。显然，不成功查找的查找次数为 n。这个结果表明，顺序查找的平均查找长度与顺序表中的记录个数成正比，无论是成功查找还是不成功查找，时间复杂度皆为 $O(n)$。例如，当 $n=1000$ 时，成功查找的平均查找长度约为 500。

当所有关键字的出现频率不一样时，还可以进一步提升顺序查找方法的效率。可以根据关键字的查找频率从高到低排列记录，先放置查找频率最高的记录，接下来查找频率次高的记录，依此类推。这样存储的表称为自组织线性表。

对于自组织线性表，它的查找仍然是从第一个位置开始顺序进行的。由于各记录依查找概率从大到小排列，因此查找的平均比较次数不会多于$(n+1)/2$，查找需要的预期比较次数是：

$$C_n=1P_0+2P_1+\cdots+nP_{n-1}$$

其中 P_i是访问记录 i 的概率。

在实际应用中，当无法事先知道哪一个记录最经常被访问时，可以使用预期访问频率来

代替查找频率。

二、折半查找方法

当数据按关键字有序存储时，可以改进顺序查找方法。

假设在如表 1-1 所示的学生基本信息表中，根据学号查找学生的相关信息。假设这个表中学生人数非常多，并且已经按学号排好序。现要查找学号为 M2022103003 的学生。当然，仍可以从第一条记录开始，一条条地查找下去。但更有效的方法是先找到中间位置的记录，查看当前记录的学号，与查找目标进行比对，然后决定下一步的查找动作。这个过程体现了折半查找的思想。

折半查找也称为二分查找，其适用条件是数组中各个记录按关键字有序排列，就像是字典中所有的单词按字典序升序排列一样。所以折半查找只适用于有序表，不失一般性，假定有序表是递增表。

折半查找并不是从有序表的一端开始查找，而是从中间开始查找。如果有序表的中间位置元素等于目标，则查找成功，否则继续查找。

那么如何继续查找呢？又在哪个范围内继续查找呢？表是有序的，如果目标在表中存在，那么根据目标与中间元素的大小关系，它一定在数组的这一边或另一边。例如，若中间位置元素大于目标，那么，因为后半部分的所有元素均大于中间位置元素，也即大于目标，所以这部分的元素可以舍弃掉，不必再进行比较，也就是只需要在前半部分进行比较。反之，也是一样的，如果中间位置元素小于目标，则需要继续在后半部分进行查找。

无论是在前半部分还是在后半部分继续查找，这个过程都类似于前面的过程，仍是先确定剩余元素的中间位置，并将该位置的元素与目标进行比较，从而决定下一步动作。

折半查找中的每一步比较，都可以将候选区间缩小一半，直到找到目标元素，或候选区间中没有待选元素为止，这意味着有序表中没有目标元素。

【例 8-1】 折半查找示例一。

给定有序表：10 12 18 22 31 34 40 46 59 67 69 72 80 84 98，使用折半查找方法查找 67，给出查找过程。

注意到，给定的初始数据是有序的，所以可以使用折半查找方法。设有序表保存在一维数组中。初始时，数组中的所有元素都是候选者。

折半查找方法从检测中间元素 46 开始。这个元素不是目标，所以必须继续查找。如果 67 在表中，则它一定在后半段中，因为前半段中的所有数据项都小于等于 46。剩余的候选区间用下画线标识，如下所示：

10 12 18 22 31 34 40 46 59 67 69 72 80 84 98

继续这个过程，检测候选区间的中间元素 72。它也不是目标，所以必须继续查找。可以排除大于 72 的所有元素，仅余下 3 个元素，如下所示：

10 12 18 22 31 34 40 46 59 67 69 72 80 84 98

注意，只用两次比较，就将候选区间从 15 项减少到 3 项。再用同样的方法，选择中间元素 67，发现它正是要查找的。如果它仍不是要查找的目标，则必须继续这个过程，直到找到目标或排除了所有可能的数据为止。

折半查找的每次比较都将遗留的数据排除掉差不多一半（也排除了中间元素）。具体来

说，折半查找在第一次比较时排除一半数据，第二次比较时排除 1/4，第三次比较时排除 1/8，依此类推。

正因为每次比较都保留一半元素，舍弃一半元素，故称为折半查找或二分查找。

因为剩余的候选区间处于数组中的位置依每次比较的结果而定，是不确定的，所以需要使用两个整型变量 low 和 high 来分别记录候选区间的开始位置与结束位置。初始时，设置它们包含数组中的全部元素。每次查找时，取 low 和 high 下标的中间值（使用整除），以计算候选区间的中间点，使用 mid 来保存。

如果 mid 位置保存的记录即查找目标，则返回 mid 值。否则，当要保留的是左侧候选区间时，用中间点 mid 的左邻居调整 high 的值，当要保留的是右侧候选区间时，用中间点 mid 的右邻居调整 low 的值，继续查找。这个查找过程是一个迭代过程。

下标必须是整数，求中间位置时，若遇到 mid 的计算结果不是整数，则 mid 要取整。可以向下取整，即 mid=⌊(low+high)/2⌋，也可以向上取整，mid=⌈(low+high)/2⌉，但必须统一。

【例 8-2】 折半查找示例二。

现有含 11 个元素的有序表，数据元素的关键字依次为 11 15 18 22 33 45 60 65 82 86 97，现在要查找关键字为 22 的数据元素。

使用指针 low 和 high 分别指向查找表中待查区间的下界与上界，指针 mid 指向该区间的中间位置，并使用向下取整，即 mid=⌊(low+high)/2⌋。

low 和 high 的初值分别为 0 与 10。为简单起见，数组中仅列出关键字，初始状态如图 8-1 所示。

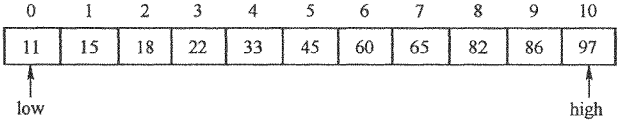


图 8-1　初始有序表及初始边界值

要查找 22，首先要求出初始查找区间的中间位置 mid=5，将这个位置的关键字与查找目标 22 进行比较。这是第一次查找过程，如图 8-2 所示。因为 22<45，所以应该丢弃后半区间，在前半区间继续查找。

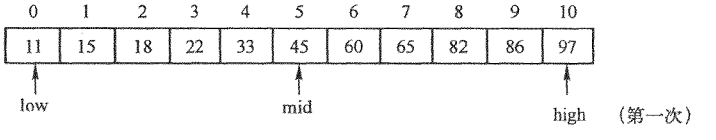


图 8-2　第一次查找过程

此时 low 的值不变，而新的区间上界 high 应为 mid-1=4，也就是在区间[0,4]中继续查找。在这个区间再求出新的中间位置 mid=⌊(0+4)/2⌋=2。保存在位置 2 的值是 18，因为 22>18，所以应该在后半区间查找，新的区间上界 high=4 不变，下界 low 应为 mid+1=3，即在区间[3,4]中继续查找。再求它的中间位置 mid=⌊(3+4)/2⌋=3，这个位置的关键字恰好是 22，因此查找成功。这个过程如图 8-3 所示。

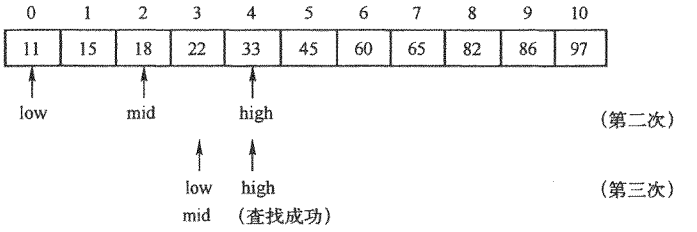


图 8-3　在前半区间查找 22 的过程

如果给定关键字值 85，在同一有序表上的查找过程如图 8-4 所示。

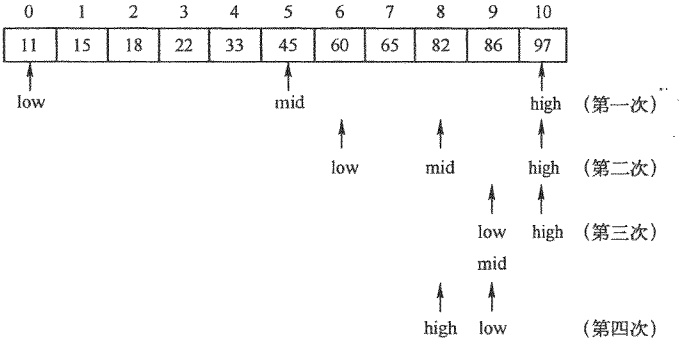


图 8-4　查找 85 的过程

由图 8-4 可以看出，经过三次比较之后，mid=9，该位置的关键字值 86>85，按照规则，应在前半区间[low,mid-1]继续查找，但此时新的区间上界 mid-1=8，它已小于下界 low=9，即新的区间已不存在，这就意味着查找表中根本没有关键字值为 85 的元素，查找失败。

折半查找的具体算法如下。

```
int BinSearch(searchList L,ELEMType target)      //使用数组存储的有序表中的折半查找
{  //若成功，则返回所处下标值；否则返回 ERROR
   int low=0,high=L.n-1;                         //置区间初值
   int found=0,mid;                              //置是否找到标记 found 为 0（未找到）
   while (low<=high&&! found) {                  //当区间下界未超过上界且未找到时，循环查找
     mid=(low+high)/2;                           //求中点
     if (L.element[mid]==target) found=1;        //已找到，置标记 found 为 1
     else if (target>L.element[mid]) low=mid+1;  //在后半区间继续查找
     else high=mid-1;                            //在前半区间继续查找
   }
   if (found)return mid;                         //查找成功，返回找到的记录的位置
   else return ERROR;                            //查找不成功，返回-1 为标记
}
```

【例 8-3】给定有序表：1 3 9 12 32 41 45 62 75 77 82 95 100，在进行折半查找时，若查找关键字 9 时的比较次数为 2，则查找关键字 75 时，比较次数是多少？

答案为4。

在进行折半查找时，关键一步是要求候选区间的中间位置并进行关键字比较。求中间位置mid时，若遇到mid的计算结果不是整数，则mid要取整，既可以向上取整，也可以向下取整。题目中给出的“查找关键字9时的比较次数为2”这个条件，就是用来告诉求mid时如何取整。

先将有序表保存到一维数组中，可以看出，有序表中的数据个数是13。

0	1	2	3	4	5	6	7	8	9	10	11	12
1	3	9	12	32	41	45	62	75	77	82	95	100

第一次求中间位置mid时，mid=6，结果为整数，不需要向上或向下取整。

第二次在下标0和下标5之间进行查找，mid=2.5，取整的结果是2或3。而查找关键字9时的比较次数为2，意味着这一趟查找的是9，且查找成功。mid=2，向下取整。

在查找关键字75时，第一次仍是得到mid=6。关键字75与45进行比较，然后，候选区间变为从下标7到下标12。

第二次，mid=⌊(7+12)/2⌋=9，第二趟关键字77与75进行比较，候选区间变为从下标7到下标8。

第三次，mid=⌊(7+8)/2⌋=7，第三趟关键字62与75进行比较，候选区间变为从下标8到下标8。

第四次，mid=⌊(8+8)/2⌋=8，第四趟关键字75与75进行比较，查找成功。

所以，比较次数是4。

折半查找的平均查找长度可以用二叉树进行分析。仍以上述具有11个元素的有序表为例，从折半查找过程可知，查找到表中位置5的元素仅需一次运算，找到位置2和位置8的元素各需要两次运算，找到位置0、位置3、位置6和位置9的元素各需要三次运算，找到位置1、位置4、位置7和位置10的元素各需要四次运算。这里所谓一次运算包括比较大小和更新查找区间。这个查找过程对应的二叉树称为折半查找判定树，如图8-5所示，树中结点中的值表示位置。

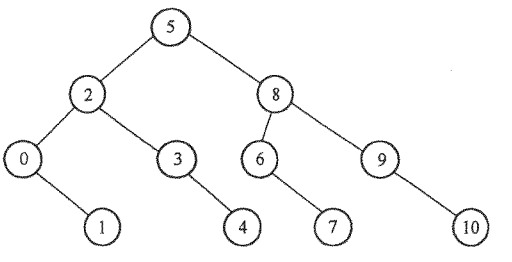
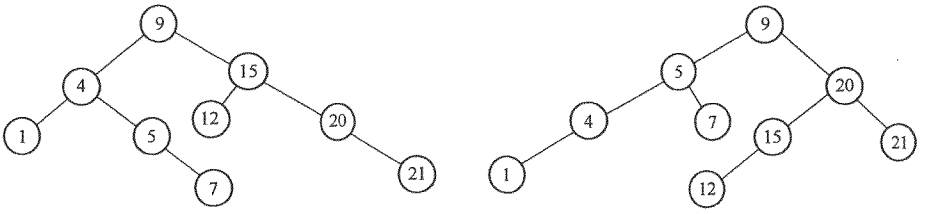


图8-5　折半查找判定树

从图8-5可知，查找一个记录时要进行比较的关键字序列恰好位于从根结点到该记录结点的路径上。例如，在查找22时，进行比较的关键字序列是45,18,22，它们所在的位置分别是5,2,3，查找过程恰好是沿路径进行的，关键字间的运算次数恰好是该路径中所含的结点数，即目标结点在二叉树中的层数加1。因此，查找成功时所用的运算次数不超过树的高度。若查找不成功，则都是遇到二叉树中的空指针的地方。例如，在查找85时，经过的路径是结点5，8，9，然后应该查找结点9的左孩子结点，此时遇到空指针，表明查找不成功。查找不成功时的运算次数也约等于树的高度。

【例8-4】对有序表1,4,5,7,9,12,15,20,21进行折半查找，给出对应于查找过程的两棵判定树。

在折半查找时，因为求 mid 位置时可能存在上取整或下取整的情况，所以折半查找树不唯一。在对给定的有序表进行折半查找时，对应的两棵折半查找判定树如图 8-6 所示。



a) 折半查找判定树一　　b) 折半查找判定树二

图 8-6　两棵折半查找判定树

那么，折半查找的平均查找长度是多少呢？

为了讨论方便，设有序表的长度 $n=2^h-1$，h 是整数，则描述折半查找过程的二叉树为满二叉树，其高度 $h=\log_2(n+1)$。在这棵二叉树中，层次从 0 开始计数，层次为 0 的结点有 1 个，层次为 1 的结点有 2 个……层次为 $h-1$ 的结点有 2^{h-1} 个。现仍设表中每个记录的查找概率相等，即 $P_i=1/n$，则查找成功时折半查找的平均查找长度为

$$\mathrm{ASL}=\sum_{i=1}^{n}P_iC_i=\frac{1}{n}\sum_{j=1}^{h}j\times 2^{j-1}$$

上式的计算结果约为 $\log_2 n$。因此，当查找成功时，折半查找的平均查找长度对 n 来说是对数级的，即 $O(\log n)$。而当查找不成功时，折半查找算法必须从二叉树的根“走”到空指针，也就是“走”了树的高度的路径，平均查找长度自然也是对数级的。例如，当 $n=1000$ 时，折半查找的平均查找长度不超过 10，比顺序查找的速度要快得多，这是它的主要优点。当然，使用折半查找必须是在顺序存储方式下，且必须做到按记录的关键字值排序。正是因为这两个条件，换来了折半查找的较高效率。相对于折半查找，顺序查找虽然速度慢，但是对存储结构和记录存放没有更多的限制。

折半查找时的一个关键位置是查找区间的中间位置。修改这种方法，预先存储记录的关键字值的预期分布情况，查找时一般不从查找区间的中间位置开始，而是根据关键字值，预测出它的可能位置，然后再在该位置附近的小区间内进行寻找。这种方法类似于我们在英文字典中查找单词，所以称为字典检索。在字典中查找单词时，基于的是我们对单词在字典中所处位置的预判断，当判断较准确时，能很快找到目标，否则可能会花费较多的时间。一般来讲，当预期的关键字值分布符合实际分布时，字典检索比折半查找法更好。但当两个分布不符合实际分布时，字典检索的效率可能很差。

三、索引顺序查找方法

索引顺序查找又称为分块查找，是结合了顺序查找和二分查找特点的一种查找方法，适用于数据较多的情况。

首先，将众多的数据分成若干块，将大的查找池分为若干小的查找池。每块中的值可以有序，也可以无序，但块与块之间必须有序，即第一块中的所有关键字都小于第二块中的所有关键字，第二块中的所有关键字都小于第三块中的所有关键字，依此类推。一般来讲，第

i 块中的所有关键字值都小于第 $i+1$ 块中的所有关键字值。块的这种特性通常称为整体有序。

例如，下面 3 个块是整体有序的，每个块内的数据可以是无序的。

块 1：22,12,13,8,9,20,6,16

块 2：33,42,44,38,24,48,26,37

块 3：60,58,74,49,86,53,76,91

在分块后，将每个块中的最大关键字抽取出来，按块的次序保存在一个一维数组中。每个数组元素中，除了最大关键字以外，还有指示该块实际存储位置的指针。这两项共同组成索引项。保存索引项的这个一维数组称为索引表。因为块是整体有序的，所以索引表是一个有序表。

对应于上面这三个块的索引表及对应的块如图 8-7 所示。

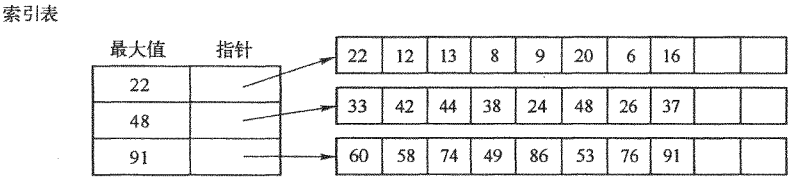


图 8-7　索引结构示意图

索引顺序查找的基本思想是：首先查找索引表，确定要查找的关键字可能在哪个块中，然后在确定的块内再进行进一步查找。因为索引表是有序表，所以查找索引表时可以使用折半查找，当然，如果索引表较小，也可以使用顺序查找。在每个块内，通常使用顺序查找。当然，如果块内是有序的，则也可以使用折半查找。

虽然索引表占据了额外的存储空间，索引表的查找也增加了一定的系统开销，但因为将查找池进行分块，使得在块内查找时，查找范围缩小，与顺序查找法相比，在一定程度上提高了效率。当数据量较多，不能全部放到内存中时，也可以使用这样的方法，将使用索引表查找到的对应块放到内存中。

【例 8-5】 索引顺序查找示例。

针对前面给出的三个块，分别查找关键字 38 和 50。

按每块中最大关键字值建立索引表：22,48,91。

查找 38 的过程：先查找索引表，找到第一个大于查找目标的关键字。采用顺序查找法，使用两次比较。然后，在 48 对应的块 2 内进行顺序查找，进行 4 次比较，查找成功。总的查找次数是 2+4=6。

查找 50 的过程：先查找索引表，找到第一个大于查找目标的关键字。采用顺序查找法，使用 3 次比较。然后，在 91 对应的块 3 内进行顺序查找，进行 8 次比较，查找不成功。总的查找次数是 3+8=11。

假设总的记录数为 n，分块时，每个块内的记录数是 s，块数为 b，则 $b=\lceil n/s \rceil$。当然，最后一个块中的记录数可能不足 s。

当查找索引表及在块内进行查找时，均采用顺序查找方法。当查找索引表时，查找成功的平均查找长度为 $(b+1)/2$，在查找目标所在的块内进行查找时，查找成功的平均查找长度为 $(s+1)/2$。因此，查找成功的平均查找长度 $ASL_{成功}=(b+1)/2+(s+1)/2$。当 $s=\sqrt{n}$ 时，$ASL_{成功}$ 能达到最小值，即这样分块时整体的查找效率最好。

例如，当记录数为256时，每个块内的记录数为16，块数为16，查找成功时的平均查找长度 $ASL_{成功}=17$。

第三节　树形结构的查找

第二节介绍的查找方法主要针对顺序存储结构，而有些结构，比如有序表，为了保证有序性，进行插入和删除操作很不方便。

本节介绍的三种查找方法利用树形结构来存储记录，这些方法不仅能达到较高的查找效率，也能较好地解决在查找表中插入和删除记录的问题。

一、二叉查找树

在分析折半查找算法的性能时，已经用到了二叉树。实际上，二叉树很好地体现了“一分为二”的思想。可以用二叉树的每个结点存储一个记录，并且按一定的规律加以组织，使之便于查找。下面给出二叉查找树的定义。

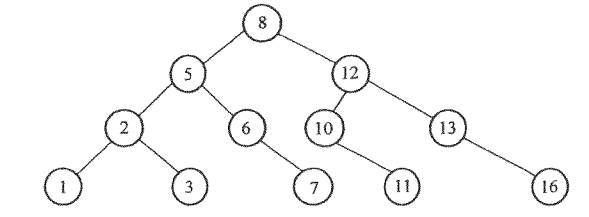
定义 8-2　二叉查找树（Binary Search Tree，BST）或者是一棵空树，或者是具有下列性质的二叉树：

（1）若它的左子树不空，则左子树所有结点保存的记录的关键字值均小于它的根结点保存的记录的关键字值。

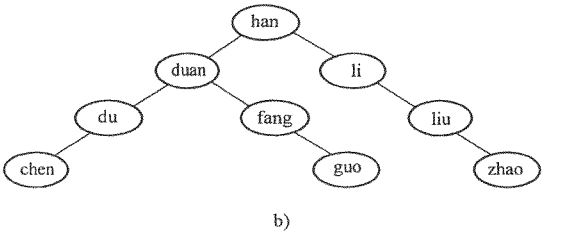
（2）若它的右子树不空，则右子树所有结点保存的记录的关键字值均大于它的根结点保存的记录的关键字值。

（3）它的左子树、右子树也都是二叉查找树。

图8-8所示的两棵二叉树都是二叉查找树，在它们的结点中列出了所存记录的关键字。在图8-8a中，关键字为整数，在图8-8b中，关键字为字符串（姓氏的汉语拼音），这些关键字都是可以比较大小的，可以验证，它们都满足二叉查找树的定义要求。二叉查找树也称为二叉搜索树。中序遍历二叉查找树可以得到一个有序序列。



a)



b)

图8-8　二叉查找树示例

二叉查找树的类型定义与二叉树是一样的，结点类及二叉查找树的定义如下。

```
typedef int ELEMType;
typedef struct BNode                    //二叉查找树结点
{   ELEMType data;                      //数据域
    struct BNode *left, *right;         //指向左孩子、右孩子结点的指针
}BstTNode;
typedef BstTNode *BstTree;              //二叉查找树
```

用到的常量定义如下。

```
#define TRUE 1
#define FALSE 0
#define ERROR -1
```

1. 二叉查找树的查找

根据二叉查找树的定义，在树中进行查找是比较容易的。设要查找的目标为 target，查找的基本思想是：从根结点开始，如果根结点的关键字等于查找目标 target，则查找成功，并返回指向根结点的指针。如果目标 target 小于根结点的关键字值，则在它的左子树继续查找；如果目标 target 大于根结点的关键字值，则在它的右子树继续查找；依此类推。在这个过程中，沿着从根结点到叶结点的一条路径向下查找，如果路径中某结点的关键字值与目标相等，则表示查找成功，返回 TRUE；若遇到空指针，则表示查找失败，返回 FALSE，表示二叉查找树中不存在查找目标 target。

【例 8-6】 在二叉查找树中进行查找示例。

根据二叉查找树的查找算法，在图 8-9 所示的二叉查找树中，分别查找关键字 82 和 37。

先查找 82。将关键字 82 与根结点关键字 60 进行比较，因为 60<82，所以应该在 60 的右子树中继续查找。仍与子树的根结点进行比较，此处是 65，发现 65 仍小于 82，所以还要在 65 的右子树中继续查找。此时，遇到的关键字为 97，它大于目标 82，进而转到 97 的左子树中继续查找，这棵子树的根结点中保存的值正好等于 82，查找成功。查找结束。

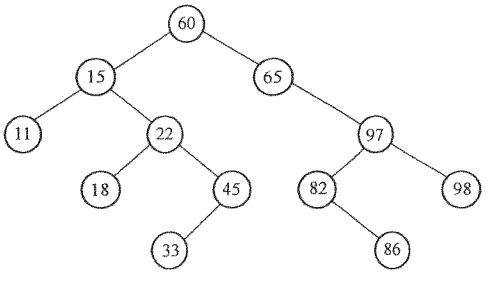


图 8-9　在二叉查找树中进行查找示例

查找 37 的过程与查找 82 的过程类似。从根结点 60 开始，依次将 37 与 60、15、22、45 及 33 相比较，当比较到 33 时，由于 37>33，故需要在 33 的右子树中继续查找，而 33 的右子树为空，表明二叉查找树中不存在关键字值等于 37 的结点。查找结束。

在查找过程中，与查找目标相比较的各关键字所在的结点刚好位于从根结点开始的一条路径上，这些关键字构成一个查找序列。由树的性质可知，查找序列的长度不大于树的高度。

在二叉查找树中进行查找的方法很容易用一个递归程序实现。

```
int BstSearch(ELEMType target,BstTree t)          //二叉查找树的查找，递归实现
{   //查找成功返回 TRUE，查找不成功返回 FALSE
```

```
    if (t==NULL) return FALSE;
    if (t->data==target) return TRUE;                    //查找成功返回 1
    if (t->data>target) return BstSearch(target,t->left);    //搜索左子树
    else return BstSearch(target,t->right);                  //搜索右子树
}
```

在算法 BstSearch 中，若查找成功，则返回 TRUE，否则返回 FALSE。

也可以使用非递归方式实现二叉查找树的查找过程，实现如下。

```
int BstSearch1(ELEMType target,BstTree t)          //二叉查找树的查找，非递归实现
{   //查找成功，返回 TRUE，若不成功，返回 FALSE
    BstTNode *temp=t;
    while (temp!=NULL){
        if (temp->data==target) return TRUE;       //查找成功返回 1
        if (temp->data>target) temp=temp->left;    //搜索左子树
        else temp=temp->right;                     //搜索右子树
    }
    return FALSE;
}
```

2. 二叉查找树的生成

实际上，二叉查找树的生成就是从空树开始依次将结点插入到树中的过程。在一棵二叉查找树中插入一个新结点的算法是：若二叉查找树为空，则新结点为二叉查找树的根结点；若二叉查找树非空，则比较新结点的关键字值和根结点的关键字值，若新结点的关键字值小于根结点的关键字值，则新结点插入到根结点的左子树中，否则插入到根结点的右子树中。这也是一个递归过程，而且和查找算法非常相似。实际上，新结点的插入位置即在二叉查找树中查找该值失败时的那个空指针的位置。

在二叉查找树中插入新结点的具体算法如下。

```
//二叉查找树的插入算法，递归实现
void BstInsert(ELEMType k,BstTree *t)        //插入结点的关键字为 k，树根为 t
{   if ((*t)==NULL){
        (*t)=(BstNode *)malloc(sizeof(BstNode));
        (*t)->data=k;
        (*t)->left=NULL;
        (*t)->right=NULL;
    }
    else{
        if (k<(*t)->data){
            BstInsert(k,&(*t)->left);
        }
        else{
            BstInsert(k,&(*t)->right);
        }
```

```
        }
    return;
}
```

按照这个插入算法，要插入的新结点一定会插入到某结点的空指针位置，成为它的左子结点或右子结点，并且成为二叉查找树的一个新叶结点。在插入之前，先申请新结点要占用的空间，新结点的关键字是参数 k，其左孩子指针、右孩子指针均为空。

【例 8-7】 二叉查找树中的插入示例。

按照二叉查找树的插入算法，在图 8-9 所示的树中依次插入关键字为 58 和 17 的两个新结点。

先看第一个值 58。因为 58 小于根结点的关键字 60，故应在左子树上插入，再看左子树的根结点，其关键字为 15，小于 58，所以要在其右子树上插入。依此类推，接下来，依次与关键字 22、45 进行比较，得知新值应插入在 45 的右子树中。45 的右孩子指针为空，所以新结点 58 成为 45 的右子结点。

再看第二个值 17。查找插入位置的过程与前面的过程类似，在分别与 60、15、22 和 18 进行比较后，找到新结点的插入位置，即 18 的左子树中。而 18 的左孩子指针为空，所以结点 17 成为 18 的左子结点。

两个关键字插入后得到的新二叉查找树如图 8-10 所示。

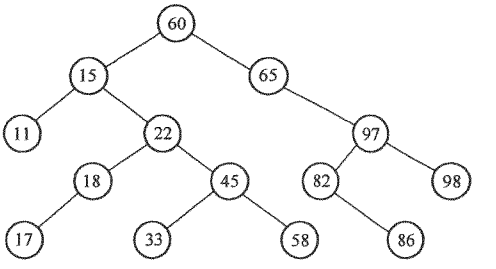


图 8-10　二叉查找树中的插入示例

【例 8-8】 从空树开始，依次插入关键字分别为 h、a、r、d 的结点，建立一棵二叉查找树，给出建树过程。

这棵树的建立过程如图 8-11 所示。

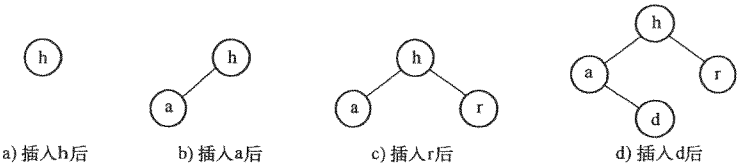


图 8-11　二叉查找树的建立过程

【例 8-9】 按选项中所给的次序，分别将 15 个整数依次插入到初始为空的二叉查找树中，能得到有最低高度的树的插入顺序是（　）。

A. 先依次插入奇数 1,3,…,15，再插入 8,14，最后依次插入其余偶数 2,4,…

B. 先依次插入偶数 2,4,…,14，再插入 9,5,11，最后依次插入其余奇数 1,3,…

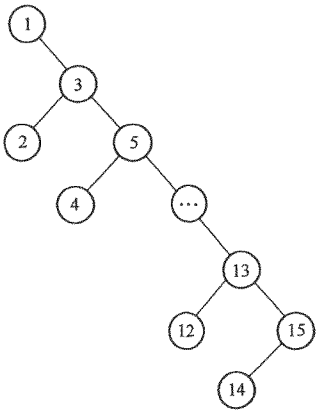
C. 先插入 4,8,12，再依次插入奇数 1,3,…,15，最后依次插入其余偶数 2,6,…

D. 先插入 9,5,11，再依次插入偶数 2,4,…,14，最后依次插入其余奇数 1,3,…

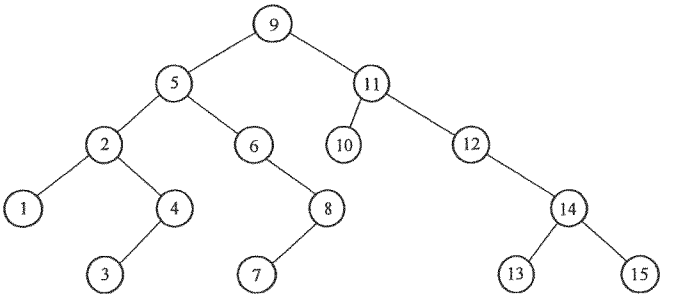
答案为 D。

可以将各选项生成的二叉查找树画出来。例如，选项 A 生成的二叉查找树如图 8-12a

所示。选项 D 生成的二叉查找树如图 8-12b 所示。其他的不再赘述。



a) 选项A生成的二叉查找树



b) 选项D生成的二叉查找树

图 8-12　例 8-9 选项生成的二叉查找树

3. 二叉查找树中的删除

现在讨论如何从二叉查找树中删除一个结点。删除一个结点比插入一个结点要麻烦些，因为要删除的可以是树中任一结点，删除后不但要保证剩余的结点仍构成一棵二叉树，而且要保持二叉查找树的有序特性。

假设在二叉查找树中要删除的结点为 x，它的父结点是 f，不失一般性，设 x 为 f 的左孩子结点（x 为 f 的右孩子结点时的删除算法与之类似）。这时，分以下三种情况考虑。

（1）x 的度为 0，即 x 为叶结点，此种情况最为简单，删除该结点后，令 f 的左孩子结点指针为空即可。这个过程如图 8-13a 所示。

（2）x 的度为 1，即 x 只有左子树 LTree 或右子树 RTree，删除 x 结点后，令 LTree 或 RTree 直接成为其父结点 f 的左子树即可。x 只有右子树 RTree 的情况如图 8-13b 所示，x 只有左子树 LTree 的情况与此类似。

（3）x 的度为 2，即 x 结点的左、右子树均不为空，这种情况比较复杂。因为删除 x 结点后，其左子树 LTree、右子树 RTree 不能全都连接在其父结点 f 的左孩子结点指针上，故破坏了原来树的结构。必须找到一个合适的方法，才能将 LTree 和 RTree 都安排妥当。为了保持树的结构，一般会在树中寻找另一个结点 w 来顶替被删除的结点 x，然后转而删除 w。



a) 删除二叉查找树中的叶结点　b) 删除二叉查找树中有一个孩子结点的分支结点　c) 删除二叉查找树中有两个孩子结点的分支结点

图 8-13　在二叉查找树中进行删除

那么 w 是哪个结点呢？要满足的条件是，以 w 顶替 x 后，要保证满足二叉查找树的有序性。

分析二叉查找树的有序性可知，w 可以选择树的中序遍历序列中 x 的直接前驱或直接后继。再分析二叉树的中序遍历特性可知，x 的直接前驱是 x 左子树中的最大值，即左子树的中序遍历序列的最后一个结点，它一定没有右孩子结点；x 的直接后继是 x 右子树中的最小值，即右子树的中序遍历序列的第一个结点，它一定没有左孩子结点。

从根结点开始，沿左孩子结点指针一直向下查找，能到达二叉查找树中最小值的结点；沿右孩子结点指针一直向下查找，能到达二叉查找树中最大值的结点。

无论 w 是选择 x 的直接前驱还是直接后继，它的度最多是 1，那么删除 w 的操作可以归结为前述（1）或（2）的情况。

设要删除的结点为 x，不失一般性，以它的直接前驱 w 来顶替它。根据刚才的分析，以 w 填充 x 的位置后，仍能保证二叉查找树的特性。这样删除 x 就变为删除 w。这个过程如图 8-13c 所示。

【例 8-10】 二叉查找树中结点删除示例。

仍以图 8-9 所示的二叉查找树为例，分别删除关键字 45 和 60 所在的结点。

先看第一个关键字 45。它只有一个孩子结点，删除时，让其父结点直接指向它唯一的左孩子结点就可以了，也就是让 22 的右孩子结点指针指向结点 33。删除结果如图 8-14a 所示。

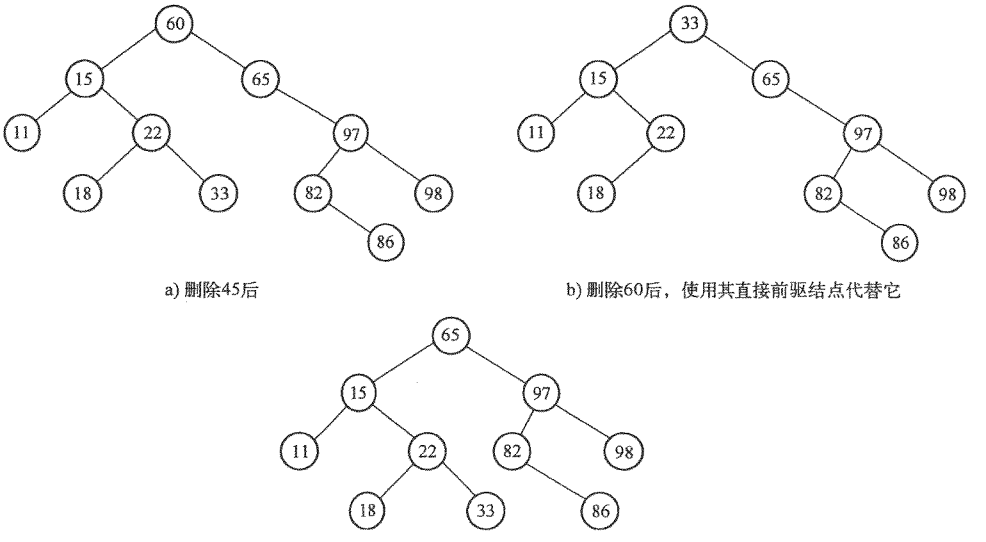
再看关键字 60。它有两个孩子结点，所以需要使用它的直接前驱或直接后继来顶替它。假设使用直接前驱顶替它。它的直接前驱是 33，让 33 成为新树的根，同时删除 33 所在的结点，因为这个结点是叶结点，直接删除就可以了。删除结果如图 8-14b 所示。

也可以使用 60 的直接后继结点来代替它。它的直接后继结点是 65。令 65 成为新的根结点，然后删除 65 所在的结点。删除结果如图 8-14c 所示。

在二叉查找树上进行查找的效率究竟如何？根据查找算法，查找一个关键字时，最大的比较次数为树的高度。当结点个数已知时，二叉查找树的高度是多少呢？这不能一概而论。根据建树的算法，当关键字个数（也就是结点个数）一定时，所生成的二叉查找树的高度不仅与关键字个数有关，而且与这些关键字插入的次序有关。

对于同一组关键字，如果插入次序改变了，则可能会生成不同的二叉查找树，如图 8-15

所示。关键字序列 a，g，e，b，f，d，c 会生成图 8-15a 所示的二叉查找树，关键字序列 a，b，g，f，c，e，d 会生成图 8-15b 所示的二叉查找树，关键字序列 a，b，c，d，e，f，g 会生成图 8-15c 所示的二叉查找树。这 7 个关键字形成的最“矮”的树如图 8-15d 所示。由此可见，同为 7 个结点的二叉查找树，树形和高度可能会有很大的不同。

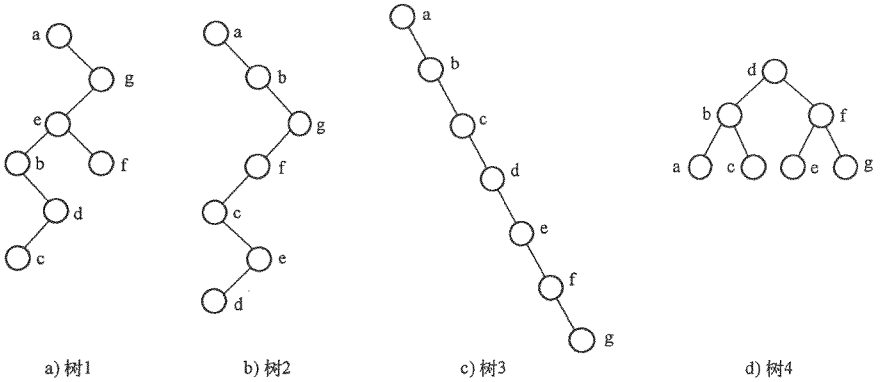


a) 删除45后

b) 删除60后，使用其直接前驱结点代替它

c) 删除60后，使用其直接后继结点代替它

图 8-14　二叉查找树中结点删除示例



a) 树1　　b) 树2　　c) 树3　　d) 树4

图 8-15　同一组关键字对应的不同二叉查找树

二叉查找树的高度不仅决定了查找时最大的比较次数，也影响了平均查找长度。现在假设对树中 7 个关键字的查找概率相等，都是 1/7，则图 8-15d 所示树的查找成功的平均查找长度为

$$ASL_{成功} = 1/7(1+2+2+3+3+3+3) = 17/7$$

而图 8-15a 所示树的查找成功的平均查找长度为

$$ASL_{成功} = 1/7(1+2+3+4+4+5+6) = 25/7$$

实际上，图 8-15b 和图 8-15c 所示的两棵树都是单链形式，它们的平均查找长度与对顺序表进行顺序查找时的平均查找长度相同。当关键字以递增或递减的次序插入树中时，就会生成如图 8-15c 所示的线性表或类似的树形。一般来说，对于 n 个记录，当它们的关键字随机出现时，所构成的二叉查找树还是比较均衡的。在这样的二叉查找树上进行查找，其查找成功的平均时间复杂度为 $O(\log n)$。这个时间复杂度与对线性表进行折半查找时的时间复杂度是同级的，但是它没有折半查找那样严格的限制条件，特别是插入和删除比较方便，因此，在二叉查找树中的查找是一种实用的、有效的查找方法。

二、AVL 树

图 8-15 表明，对于同一组数据，可以形成几棵不同形状的二叉查找树。在这些树中，有些二叉查找树的树高差异较大。因为很多操作的时间复杂度与树高相关，所以，生成的二叉查找树应尽可能“矮”。

两位数学家 G. M. Adel'son-Vel'skii 和 E. M. Landis 在 1962 年提出了一种新的树结构，改进了二叉查找树的操作效率。

在二叉查找树的每个结点中增加一个标记，称为平衡因子，定义为该结点左子树的高度减去右子树的高度。当每个结点的平衡因子的绝对值不大于 1 时，称二叉查找树为 AVL 树。AVL 取自提出该结构的两位数学家的名字。AVL 树是平衡的，有些教材也称其为平衡二叉树。

定义 8-3 AVL 树或者是一棵空树，或者是具有下列性质的二叉查找树：

（1）它的左子树和右子树都是 AVL 树。

（2）它的左子树和右子树的高度之差的绝对值不大于 1。

若二叉查找树中所有结点的平衡因子均介于-1 和 1 之间，则称树是平衡的，否则称为失平衡。例如，在图 8-15 所示的四棵树中，前三棵都是失平衡的，只有最后一棵是平衡的。

【例 8-11】 判断图 8-16 所示的二叉查找树是否是一棵 AVL 树。

将各结点的平衡因子标记出来，可以看出，平衡因子均介于-1 和 1 之间，所以它是一棵 AVL 树。

【例 8-12】 在高度为 4 的 AVL 树中，最少有多少个结点？

解： 最少有 7 个结点。

在所有等高的 AVL 树中，满足“除叶结点以外，每个分支结点的平衡因子是-1 或 1”条件的 AVL 树所含的结点个数最少。

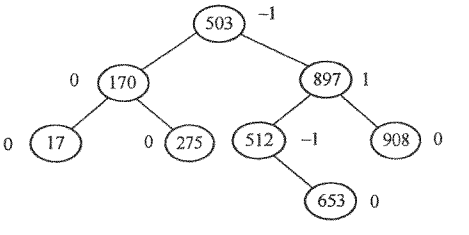


图 8-16 标记了平衡因子的一棵二叉查找树

高度为 0 的 AVL 树所含有结点个数为 0。高度为 1 的 AVL 树仅含有 1 个结点。高度为 2 的 AVL 树，一棵子树的高度为 1，另一棵子树的高度为 0，树中含有的结点个数=1+0+1=2。依此类推。在高度为 3 的 AVL 树中，两棵子树的高度分别是 2 和 1，所含结点的个数=2+1+1=4。高度为 4 的 AVL 树中所含结点个数=4+2+1=7。

AVL 树是满足平衡条件的二叉查找树。AVL 树中的基本操作与二叉查找树中的基本操作是一样的，也包括查找、插入及删除。在二叉查找树中进行的插入、删除关键字的操作会改变树形。如果操作后二叉查找树是平衡的，则它是 AVL 树。当失平衡时，需要重排结点以恢复树的平衡性。AVL 树通过旋转操作，将失平衡的树恢复为平衡的树。本书只介绍 AVL 树的查找和插入。

1. AVL 树的查找

AVL 树是满足平衡条件的特殊二叉查找树，AVL 树的查找过程与二叉查找树的查找过程是一样的。

例如，在图 8-16 所示的 AVL 树中查找关键字 512，查找过程中关键字的比较序列是 503、897、512，查找成功。

2. AVL 树的插入

从空树开始，向 AVL 树中逐个插入关键字，生成 AVL 树。每插入一个关键字，都要保证得到的树是平衡的。将关键字插入 AVL 树的过程分以下两个步骤：

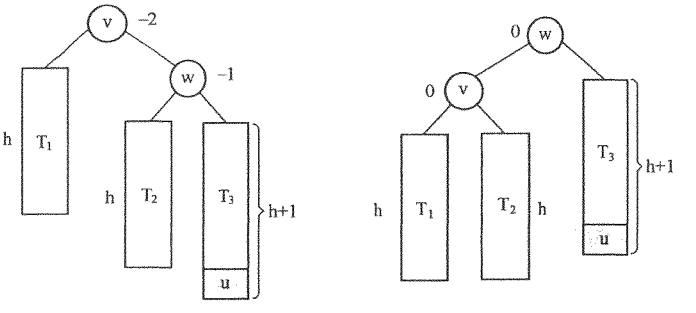
① 按照二叉查找树的插入规则，将关键字插入到 AVL 树中。

② 如果得到的新树是平衡的，则插入完成，否则进行相应的旋转以恢复平衡。

在将一个关键字插入到 AVL 树中时，按照二叉查找树的插入规则，新结点都是叶结点，其平衡因子为 0。插入之前树是平衡的，但新结点的插入会改变一些结点的平衡因子。如果新树仍是平衡的，则结点的插入结束。如果插入过程使得某些结点的平衡因子的绝对值大于 1，则导致树失平衡，需要通过旋转使得树恢复平衡。旋转共有四种，两种是单旋转，两种是双旋转。

因为新结点所在子树树高的变化最多是 1，所以失平衡结点的平衡因子的绝对值只能是 2。找到距新插入结点最近的失平衡结点，根据新插入结点 u 与失平衡结点 v 之间的相对位置关系，从四种旋转中选择一种来执行。

如果 u 插入在 v 右孩子结点的右子树中（称为 RR 型），如图 8-17a 所示，则进行向左的单旋转。



a) 插入结点后失平衡的树　　b) 左单旋转后恢复平衡的AVL树

图 8-17　失平衡的树经向左的单旋转后恢复平衡

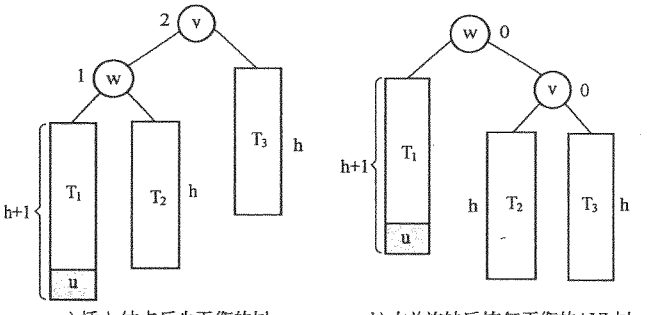
此时旋转的关键结点是结点 w，即失平衡的结点 v 的右孩子结点。让 w 升为 v 的父结点，v 成为 w 的左孩子结点。结点 w 原来的左子树 T_2 转移给 v，作为其右子树，其他的两棵

子树 T_1 和 T_3 维持不变。左单旋转后的树如图 8-17b 所示。

可以验证，这样旋转后得到的新树，既满足二叉查找树关键字的大小关系，也符合树中各结点的平衡因子介于-1 和 1 之间的要求，所以树恢复平衡。

与 RR 型旋转呈镜像的一种情况是 LL 型旋转。新插入的结点 u 插入在 v 左孩子结点的左子树中，如图 8-18a 所示，则进行向右的单旋转，右单旋转后的树如图 8-18b 所示。

当 u 插入在 v 右孩子结点的左子树中时称为 RL 型，不失一般性，u 插入在 t 的右子树中，如图 8-19 所示。



a) 插入结点后失平衡的树　　b) 右单旋转后恢复平衡的AVL树

图 8-18　失平衡的树经向右的单旋转后恢复平衡

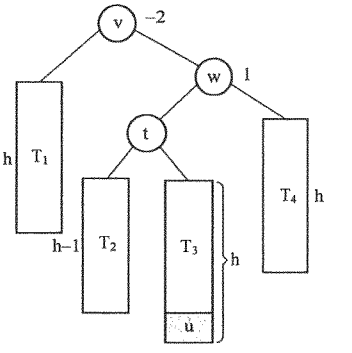
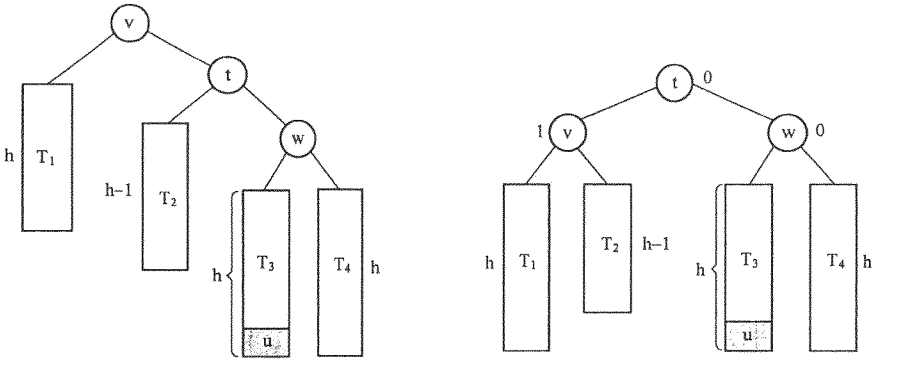


图 8-19　插入结点后失平衡的树一

此时一次单旋转不能解决问题，需要进行双旋转，即两次单旋转。先进行以结点 t 为关键结点的右单旋转，结果如图 8-20a 所示，再进行以 t 为关键结点的左单旋转，结果如图 8-20b 所示。



a) 右单旋转后的树　　b) 左单旋转后的树

图 8-20　进行 RL 型双旋转的情况

在进行 RL 型双旋转后，树又恢复了平衡。

与 RL 型双旋转呈镜像的一种情况是 LR 型双旋转。结点 u 插入在 v 左孩子结点的右子树中，不失一般性，u 插入在 t 的左子树中，如图 8-21 所示。

此时先进行以结点 t 为关键结点的左单旋转，再进行以 t 为关键结点的右单旋转，结果如图 8-22 所示。

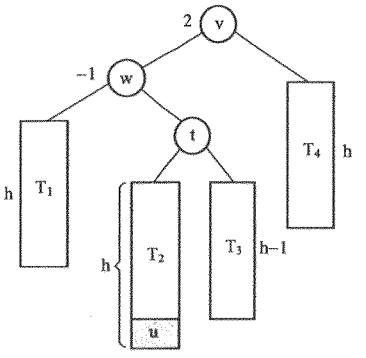


图 8-21　插入结点后失平衡的树二

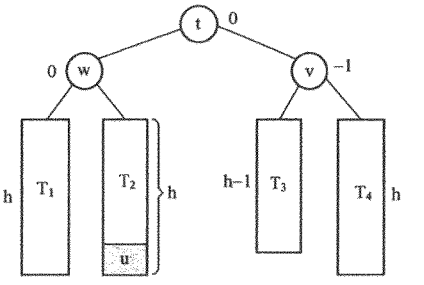


图 8-22　进行 LR 双旋转后恢复平衡的树

【例 8-13】 将下列关键字添加到初始为空的 AVL 树中，给出生成树的过程。

70,80,90,20,10,50,60,40,30

解：依次插入 70、80 后，树是平衡的，不需要再进行旋转。得到的 AVL 树如图 8-23 所示。

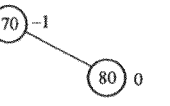


图 8-23　插入 70 和 80 后得到的 AVL 树

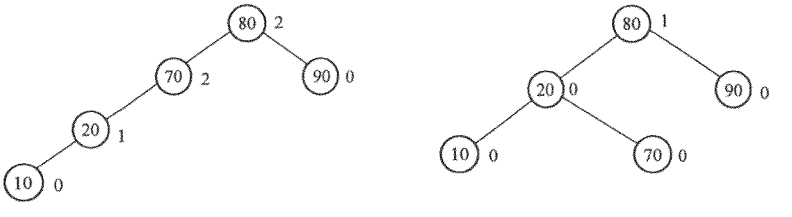
插入关键字 90，树失平衡，结果如图 8-24a 所示。失平衡的结点为 70，新结点 90 插入在 70 的右孩子结点的右子树中，所以进行左单旋转，恢复平衡，结果如图 8-24b 所示。



a) 插入90后失平衡的树　　b) 单左旋转后恢复平衡的AVL树

图 8-24　插入 90 后，由失平衡到恢复平衡的过程

接下来，依次插入 20、10 后，树再次失平衡，如图 8-25a 所示。此时需要进行单右旋转，恢复平衡，结果如图 8-25b 所示。

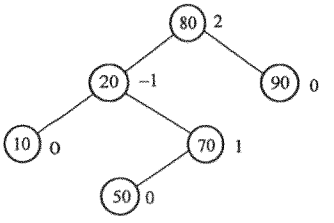


a) 插入20和10后失平衡的树　　b) 单右旋转后恢复平衡的AVL树

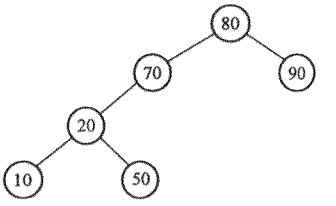
图 8-25　插入 20 和 10 后，由失平衡到恢复平衡的过程

接下来，插入 50，树再次失平衡，如图 8-26a 所示。通过分析新插入的结点与失平衡结点的相对位置可知，需要进行 LR 双旋转。先进行的是以 20 为根的这棵子树的向左的单

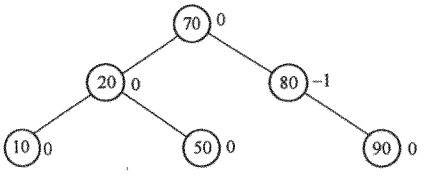
旋转，70 为子树的根，20 为 70 的左孩子结点，将 70 原来的左子树作为 20 的新右子树，结果如图 8-26b 所示。然后再进行以 80 为根的这棵树的向右的单旋转，80 作为 70 的右孩子结点，70 升为根，旋转后的结果如图 8-26c 所示。



a) 插入50后失平衡的树



b) 向左的单旋转（中间结果）



c) 向右的单旋转后恢复平衡的AVL树

图 8-26　插入 50 后，由失平衡到恢复平衡的过程

依次插入关键字 60 和 40 后，树仍是平衡的，如图 8-27 所示。

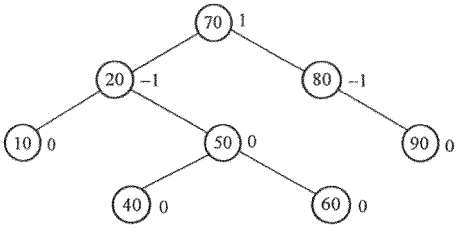


图 8-27　依次插入 60、40 后的 AVL 树

插入最后一个关键字 30，树再次失平衡了，如图 8-28 所示。

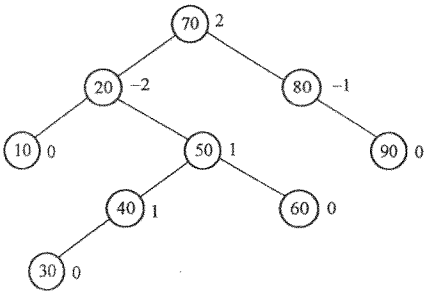


图 8-28　插入 30 后失平衡的树

最接近新插入结点 30 的失平衡结点是 20，30 位于 20 右孩子结点的左子树中，所以需要进行 RL 旋转。40 先调整为 20 的右孩子结点，然后调整为 70 的左孩子结点。调整后，20 和 50 分别是 40 的左孩子结点与右孩子结点。最终得到的 AVL 树如图 8-29 所示。

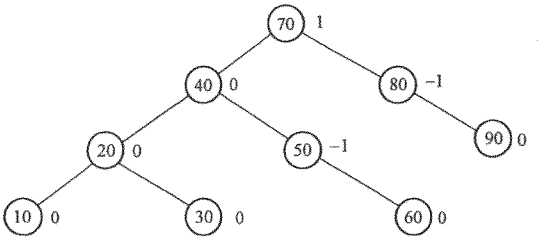


图 8-29　最终得到的 AVL 树

当树失平衡时，树中失平衡的结点个数可能多于 1 个，如图 8-28 中结点 20 和 70 都是失平衡的结点。调整时，仅调整以 20 为根的子树，既实现了这棵子树中所有结点的平衡，也使得其父结点 70 恢复了平衡。在插入操作中，一次插入最多需要一次调整。

三、B 树

扩展二叉查找树的概念，在树的每个结点中不只保存一个关键字，而是允许保存多个关键字。这样，树的每一层中可容纳的记录个数显著增大，树的高度有所降低。

1. B 树的概念和特性

1970 年，Bayer 等人提出一种多路平衡查找树，称为 B 树。

定义 8-4　一棵 m 阶 B 树或者为空，或者为满足下列性质的 m 叉树：

（1）树中每个结点至多有 m 棵子树。

（2）根结点至少有两棵子树。

（3）除根结点以外，每个结点至少有$\lceil m/2 \rceil$棵子树。

（4）所有叶结点都出现在同一层上。

（5）所有结点都包含如下形式的数据。

$$(n, A_0, K_1, A_1, K_2, A_2, \cdots, K_n, A_n)$$

其中，n 为关键字的个数，K_i（$i=1,2,\cdots,n$）为关键字，且满足 $K_1<K_2<\cdots<K_n$。A_i（$i=0,1,\cdots,n$）为指向子树根结点的指针，且对于 $i=1,2,\cdots,n-1$，A_i所指向子树中全部结点的关键字均大于 K_i而小于 K_{i+1}。A_0所指向子树中全部结点的关键字均小于 K_1，A_n所指向子树中全部结点的关键字均大于 K_n。对于叶结点，所有指针 A_i皆为空。对于具有 n 个关键字的非叶结点，将有 $n+1$ 棵子树。

当 $m=3$ 时，每个结点中最多含有 2 个关键字、3 个指针，最少时可以只含有 1 个关键字、2 个指针，所以 3 阶 B 树又称为 2-3 树。图 8-30 所示的是一棵 3 阶（$m=3$）B 树，每个结点中或者含有一个关键字，或者含有两个关键字。通常情况下，各结点中并不列出关键字的个数 n。

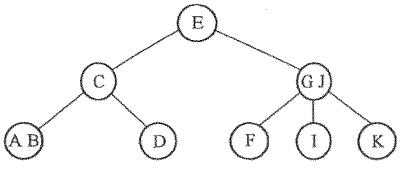


图 8-30　3 阶 B 树

B 树的一个结点中可以含有多个关键字，这与通常的树中每个结点只含有一个关键字是不同的。关键字的个数要满足一定的条件，既不会太多，也不会太少。

B 树的所有叶结点都在同一层中，这表明：对于树中的每个结点，它们的所有子树的高都是一样的。特别地，从根结点到叶结点的路径长度都相同。例如，在图 8-30 中，根结点

有 2 棵子树，每棵子树的高都是 2。含关键字（G J）的结点有 3 棵子树，每棵子树的高都是 1。树中共有 5 个叶结点，从根结点到任何叶结点的路径长度都是 2。这个特性称为树的平衡性。

B 树中，各关键字在结点中的位置满足大小规律，这可以看作树的有序性。结点中相邻的两个关键字中间的指针指向的子树内的所有关键字值介于两个关键字值之间。

2. B 树的查找

B 树的有序性保证在进行查找时，可以按照类似二叉查找树中的查找方法进行。

【例 8-14】 B 树中的查找示例。

在图 8-30 所示的 3 阶 B 树上分别查找关键字 I 和 M。

首先从根开始，根结点中只有一个关键字 E，I 排在它的后面，故转到 E 的最右子树中继续查找。这棵子树的根中含有两个关键字 G 和 J，而 I 位于这两个关键字的中间，所以应该沿 G 与 J 中间的指针进入到相应的子树中进行查找。子树的根只含有关键字 I，正是查找的目标，查找成功。

再看关键字 M 的查找过程。M 大于 E，进入 E 的最右子树中。M 大于这棵子树的根结点中的两个关键字，故再进入它的最右子树中进行查找。而这棵子树的根中只有一个关键字 K，它小于 M，故再进入 K 的最右子树中。而此时已经遇到空指针，即 K 的子树都是空树，表明树中不存在关键字 M，查找失败。

从查找过程可以看出，在 m 阶 B 树上进行查找时，其比较次数与两个因素有关；第一个因素是结点中关键字的数目（最多 $m-1$ 个，可以采用顺序查找。由于它们是有序的，当 m 较大时，也可以用折半查找方法）；第二个因素是树的高度。现在假设共有 N 个关键字，且 m 是已确定的，那么 m 阶 B 树的最大高度为对数级的。也就是说，如果不考虑在结点内部的查找代价，则在含有 N 个关键字的 m 阶 B 树上进行查找时，其平均查找长度对 N 来说也是对数级的。

3. B 树的建立

建立 B 树的过程，即从空树开始逐个插入关键字的过程，关键是要解决在 B 树中如何插入一个关键字的问题。

下面以最简单的 3 阶 B 树为例，说明在 B 树中插入一个关键字的过程。这个插入规则适用于任意阶的 B 树。

先考察最简单的情况，在空树中插入一个关键字。这个过程非常简单，使用待插入的关键字建立一个结点，该结点成为新 B 树的根结点，从另一个角度来看，它也是新 B 树的叶结点。插入过程结束。

现在假设，目前已经有了一棵非空的 3 阶 B 树，待插入的关键字是 X。插入过程分为两个阶段，第一个阶段是找到插入位置，第二个阶段是完成插入过程。

借助于前面介绍的 B 树的查找算法来查找插入位置。从根结点开始查找 X，查找的结果有两种可能，即查找成功和查找失败。如果树中有关键字 X，则不需要再插入 X 了。当查找失败时，一定是到达了叶结点，且遇到了空指针。这个叶结点就是插入位置。第一阶段完成。

按关键字的大小关系，将 X 插入相应的叶结点中。若插入后该叶结点的关键字数目是 2（2-3 树中每个结点最多可容纳两个关键字），则插入完成。如果是 3 个关键字，则将这个叶

结点分裂为两个结点，两个结点中分别含有 3 个关键字中的最小关键字和最大关键字，而将中间的关键字提升到该叶结点的父结点中，相当于在父结点中插入一个关键字。按照大小关系，将父结点中的关键字排好序。父结点中多了一个关键字，也同样地增加了一个指针，这个指针用来指向刚才叶结点分裂时多出来的新结点。

然后递归地处理父结点中的情况。在这个过程中，如果父结点中的关键字个数是 2，则插入过程完成。否则，继续进行结点分裂、中间关键字提升的过程，这个过程与前面的过程是类似的。由于树高是有限的，因此在某一步操作后提升过程一定会结束。

最坏的情况是结点分裂、中间关键字的提升要一直进行到根结点，即根结点中接收了一个新的关键字，并造成根结点的分裂。此时，中间关键字提升成为新的根结点，即新的根结点中只含有一个关键字。这种情况下，树高增加 1，这是树长高的唯一一种情况。

注意，在 B 树中插入新关键字时，都是在叶结点中插入关键字，而不是在树中增加一个新的结点，这与二叉查找树有很大的不同。正因为有这个规则，才能保证 B 树中的所有叶结点都在同一层上。

【例 8-15】 建立 B 树示例。

给定数据序列如下：18,15,41,10,45,30,25,3,71,60,50,72，现将各个元素依次插入到初始为空的 5 阶 B 树中，画出得到的结果。

根据定义，5 阶 B 树的每个结点中，关键字最多为 4 个，最少为 2 个，相应的指针最多为 5 个，最少为 3 个。当一个结点中所含的关键字少于 4 个时，就允许直接插入，当达到 5 个时，结点分裂为 2 个，各含有最小的 2 个关键字和最大的 2 个关键字，位于中间的关键字提升到父结点中。

所以，依次插入前 4 个关键字时，不会出现结点分裂现象。这 4 个关键字都在一个结点中，这个结点既是叶结点，也是根结点。得到的 B 树如图 8-31a 所示。

现在插入关键字 45。此时，叶结点中含有 5 个关键字，从小到大依次是 10、15、18、41、45。因为超出限制，所以结点分裂为 2 个结点。其中，10、15 在较小（左边）的结点中，41、45 在较大（右边）的结点中。中间的关键字 18 提升至父结点中。因为现在还没有父结点，所以创建一个新的结点作为分裂后两个结点的父结点，它其实是新的根结点。得到的 B 树如图 8-31b 所示。

关键字 30、25 插入在根的右孩子结点中，3 插入到根的左孩子结点中。它们均不会导致结点分裂。得到的 B 树如图 8-31c 所示。

现在插入关键字 71。这个关键字的插入会导致根结点的右孩子结点的分裂。提升到父结点中的关键字是 41，父结点中含有的两个关键字是 18 和 41，对应的孩子结点由 2 个变为 3 个。除原来的最左孩子结点以外，还有 25、30 所在的结点及 45、71 所在的结点。得到的 B 树如图 8-31d 所示。

然后插入关键字 60 和 50。它们插入在 45 和 71 所在的结点中，结点不分裂。得到的 B 树如图 8-31e 所示。

最后插入关键字 72。它的插入过程与关键字 71 的插入类似，也使得最初插入的结点分裂，中间关键字 60 提升至父结点中，最终得到的 B 树如图 8-31f 所示。

在 m 阶 B 树中进行插入且结点需要分裂时，如果 m 是偶数，则中间关键字有两个，提升这两个关键字中的哪一个都是可以的。

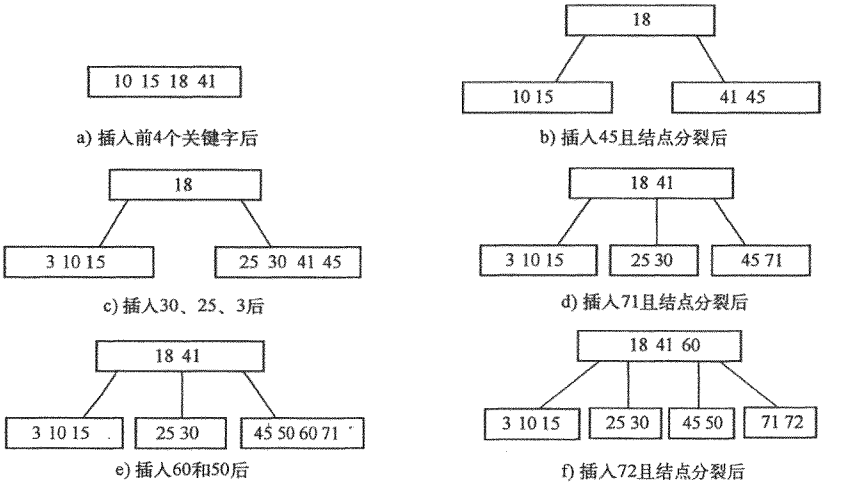


图 8-31　5 阶 B 树的生成过程

第四节　哈希表及其查找

哈希方法是一种非常实用的方法，它按照某种特殊的策略将数据放置在数组中，并按照同样的策略在数组中进行查找。所以，哈希方法不仅仅是查找方法，还是一种数据保存方法。

哈希方法尽可能地减少了关键字之间的比较，这是与前面介绍的顺序查找方法、折半查找方法、索引顺序查找方法及树形查找方法所不同的。本节介绍哈希方法及其实现过程。

一、哈希方法

在顺序存储方式中，如果数据随机存放，则只能使用顺序查找方法，查找效率很低。如果按照关键字的大小有序存放，虽然提高了查找效率，但数据的变动性较弱。也就是说，当数据进行插入或删除时，必须将很大的开销用于数据移动，查找效率的提高被数据移动的开销抵消掉了。

在实际应用中，可能会遇到既需要高的查找效率，又需要灵活的数据变动机制的情况。本节介绍的哈希方法可以用来处理这种情况。

哈希方法使用的存储结构是数组，它采用特殊的机制存放数据，目的是既方便插入数据，也方便查找数据。它的数据处理机制是找出关键字值与数据存放地址之间的关系。

哈希方法把关键字值映射到数组中的一个位置，这通过一个函数来实现，这个函数称为哈希函数，通常用 H 来表示。存放记录的数组称为哈希表，用 T 来表示。哈希表中的一个位置也称为一个槽。从数据存放到数据访问的整套机制称为哈希方法。

哈希一词来自英文单词 hash 的音译，根据这个方法的含义，哈希也称为散列。所以，哈希方法也称为散列方法，哈希表也称为散列表，哈希技术也称为散列技术。

【例 8-16】 哈希方法示例。

假设某高校参加夏令营的学生全部住在一栋宿舍大楼内，该楼的楼层足够高，楼中每层有足够多的房间，每个房间内只住一位学生。为了便于管理和查询，现以学生的姓名（假设不超过三个汉字）为关键字，以姓名的汉字笔画数作为关键字的值，来决定他（她）所住的房间号，其中姓的笔画数决定楼层号，名字的笔画数决定本层的房间号。

例如，表 8-1 就是一张学生姓名和所住房间号的对照表。

表 8-1　学生住宿表

学生姓名	房间号
丁一	201
于立	305
王方	404
田小华	536
刘力文	624
李中元	744
⋮	⋮

无论是安排学生住宿（建表）还是查询某个学生所住的房间号，这种方法都是十分方便的。但是，若有两位学生的姓名笔画数相同，就会出现问题。例如，已按表 8-1 为学生分配了住宿房间，现在又有一位名叫卞云的学生加入夏令营，她的姓名编码是 404，可是 404 号房间已经安排了学生王方，不能再住第二个人了。这就带来了问题，这种情况称为发生了“冲突”。

为学生分配宿舍，进而查询某位学生所住房间号的过程，与哈希方法所采用的思想是类似的。所谓的哈希方法就是根据记录的关键字值，使用特殊的计算策略来确定该记录在哈希表中的存储位置。查询目标时也使用同样的计算策略，针对目标进行计算。担负计算策略的就是哈希函数，它以关键字值为自变量，得到的值是数组的下标。

哈希方法需要解决的问题有两个：一是选择什么样的哈希函数；二是当有冲突发生时，如何解决。哈希方法不可避免地会发生冲突，所以解决冲突的方法也非常重要。在解决冲突时，既要决定发生冲突的记录新的存放位置，也要保证在后续的查询时能顺利地找到目标。

例如，在上述为学生分配住宿房间的问题中，如何处理冲突呢？当然，处理的方法有很多种，最简单的方法是看下一个房间号 405，若 405 号房间为空，则安排卞云住在该房间；若 405 号房间也有学生住宿，则看 406 号房间是否为空，依此类推，直到找到一个空房间为止。在这种安排下，若要询问卞云的住处，首先要到 404 号房间查看，若此房间住的学生不是卞云，则查下一个房间，依此类推。只要为卞云分配了住宿房间，则肯定能找到她。

在哈希方法中，需要为待存储的数据准备足够的存储空间，并设计一个哈希函数 H，该函数将关键字 k 转换成一个非负整数 L，即 $H(k)=L$，L 对应的是哈希表的地址，简称为哈希地址或哈希值。一方面，L 不能超过哈希表的大小；另一方面，哈希表的大小影响着哈希函数的设计。若有两个不同的关键字 k_1 和 k_2，即 $k_1 \neq k_2$，有 $H(k_1)=H(k_2)=L$，则称发生了“冲突”。没有“冲突”的哈希函数称为“完美”哈希函数。不幸的是，当不能预先知道全体关键字时，构造的哈希函数通常不是完美哈希函数。

二、哈希函数的构造方法

因为实际问题中的关键字是多种多样的，所以不可能构造通用的哈希函数，甚至不可能归纳为有限的几种方法。这里只提出构造哈希函数的基本原则，并介绍几种常用的哈希函数的构造方法。

构造哈希函数时通常考虑的因素有：

（1）计算哈希函数所需的时间。

（2）关键字的长度。

（3）哈希表的大小。

（4）关键字的分布情况。

（5）记录的查找频率。

构造哈希函数的基本原则是：

（1）算法简单，计算量小。

（2）均匀分布，减少冲突。

算法简单及冲突少都是为了提高哈希方法的效率。但在许多情况下，这两个原则是有矛盾的，不可能都达到最好。在构造具体的哈希函数时，要将两个原则折中考虑。下面介绍几种常用的构造哈希函数的方法。

1. 直接定址法

直接定址法是一种计算最简单且冲突最少的方法，在某些适合的情况下应尽量采用。这种哈希函数是关键字值的线性函数，即 $H(k)=k$ 或 $H(k)=a\times k+b$，其中 a、b 为常数。

【例 8-17】直接定址法示例。

假设要统计某地区 1949～2000 年每年的出生人数，统计结果可列在一张表中，此时，年份为关键字。因为共有 2000-1949+1=52 年，所以哈希表中有 52 个位置，位置值是 0～51，位置值与数组下标一致。如何将关键字值对应到表中位置呢？取 $H(k)=k-1949$ 即可，其中 k 为年份数。这样的哈希表示意如下：

	0	1	…	51
年份	1949	1950	…	2000
人数	…	…	…	…

在哈希表构造完成后，若要查询 1949 年至 2000 年间任意年份 M（$1949\leqslant M\leqslant 2000$）出生的人数，则在数据表的位置 M-1949 处即可找到。

2. 平方取中法

平方取中法是一种较常用的构造哈希函数的方法。若关键字值的位数超出了哈希表地址的范围，又不能简单地截取其中的几位作为哈希函数值，那么，为了让关键字值的每一位都能影响计算的结果，可以先将关键字值自乘，然后截取中间几位并将它们对应到哈希表地址。

【例 8-18】平方取中法示例。

设有一组关键字 ABC，BCD，CDE，DEF，…，其对应的机内代码分别为 010203，020304，030405，040506，…，假定地址空间大小为 10^3，编号为 0～999。

哈希表内共有 1000 个位置，如果直接将关键字的机内代码作为哈希地址，则远远超过哈希表的大小。如果直接选取机内代码的若干位，则不能全面反映机内代码的所有信息。在一个数自乘后，乘积的中间几位既能反映原数低位的情况，也能反映原数高位的情况。依据这个思想，将一个数自乘（平方），然后抽取中间的若干位（取中），结果作为哈希值。

在本例中，可取关键字机内代码平方后的中间三位作为存储位置，见表 8-2。

表 8-2　关键字及对应编码计算

关键字	机内代码	机内代码的平方数	哈希地址
ABC	010203	0104101209	410
BCD	020304	0412252416	225
CDE	030405	0924464025	446
⋮	⋮	⋮	⋮
XYZ	242526	58818860676	886
⋮	⋮	⋮	⋮

以 ABC 为例，看看哈希地址的计算过程。将 ABC 转换为数值 010203，取平方的结果是 0104101209，选择中间的 3 位数 410（舍掉最右边的 4 位数，然后选择 3 位），将它作为哈希地址。

中间的几位，既能反映关键字低位的信息，也能反映关键字高位的信息。至于选择哪几位，可以依实际情况而定。

3. 折叠法

折叠法是另一类处理多位关键字的方法。当关键字的位数较多，远超出哈希表地址的范围时，可将关键字分割为位数相等的几段，然后将各段叠加，叠加后将最高位的进位舍去，所得结果即哈希地址。

例如，关键字值 $k=123203241712$，哈希表长度为 1000，地址号为 0~999。可以将 k 按三位分段，共分成 4 段，分别是 123、203、241、712。将这 4 个值相加，去掉进位 1 之后得到哈希地址 279。这种方法也是尽可能让关键字的每一位都起作用，以达到使哈希函数值均匀分布的目的。同时，采取几位折叠方式，又避免了最后哈希地址太大，超出哈希表范围的问题。

在折叠时，也可以间隔地让分段后的数先反转再相加。例如，对于 123、203、241、712，间隔反转后得到 123、302、241、217，相加后得到 883。

4. 基数转换法

基数转换法将关键字值先看成另一种进制的数，然后转换成原来进制的数（如十进制），再选其中的几位作为哈希地址。

例如，现有十进制的关键字值 210485，先把它看成十三进制数，再转换成十进制数，其过程是：

$$210485_{13}=2\times13^5+1\times13^4+0\times13^3+4\times13^2+8\times13+5=771932_{10}$$

然后从中选几位作为哈希地址即可。通常要求两个基数互素，且新基数要比原基数大。

5. 除留余数法

除留余数法是一种常用的方法，它通过对关键字值进行取模运算得到哈希地址。设给出

的关键字值为 k，哈希表长度为 m，则设计哈希函数为：

$$H(k)=k \bmod p$$

其中，p 为小于等于 m 的某个正整数。理论分析和试验结果均证明，p 应取小于等于表长 m 的最大素数，这样才能达到使哈希函数值均匀分布的目的。例如，当 $m=1000$ 时，p 可以取 997 这个素数。对某些关键字计算哈希地址的结果见表 8-3。

表 8-3　关键字及哈希地址计算

关　键　字	机内代码（k）	$H(k)=k \bmod 997$
KEYA	11052501	756
EYB	052502	658
AKEY	01110525	864
BKEY	02110525	873
ABCD	01020304	373
DCBA	04030201	327

构造哈希函数的几种方法也可以混合使用，例如，可以先利用折叠法，得到一个中间结果，再利用除留余数法得到最终哈希值。

三、处理冲突的方法

给定一个哈希函数 H 和两个关键字 k_1、k_2，如果 $H(k_1)=b=H(k_2)$，其中 b 是哈希表中的一个位置，则称 k_1 和 k_2 在哈希函数 H 下对于 b 有冲突。

若对于关键字集合中的任一关键字，经哈希函数映射到地址集合中任何一个地址的概率是相等的，则称此类哈希函数是均匀的哈希函数。换句话说，就是使关键字经过哈希函数得到一个“随机地址”，目的是让一组关键字的哈希地址均匀分布在整个地址空间中，从而减少冲突。

冲突在构造实际的哈希函数时是很难避免的，只能要求产生的冲突尽可能少。因此，一旦发生冲突，如何解决冲突就成为哈希技术中十分重要的问题。本节介绍解决冲突的两类基本方法。其中一类称为开放地址法，也叫闭哈希法。当发生冲突时，用某种方法形成一个探测下一地址的序列，沿着这个序列一个个查找，直到找到一个空闲地址来保存发生冲突的记录为止。空闲地址也称为开放地址。另一类称为链地址法，也叫开哈希法。它将具有相同哈希函数值的关键字对应的记录组成一个单链表。发生冲突的记录都保存在该单链表中。发生冲突的关键字称为同义词。

1. 开放地址法

这一类解决冲突的方法的精髓是如何得到下一个空闲地址。常用的是如下两种确定探测序列的方式：

① 线性探测法。

② 二次探测法。

假设哈希表长度为 m，地址为 $0\sim m-1$，哈希函数为 $H(k)$。

在用线性探测法解决冲突时，求下一地址的公式是：

$$d_{i+1}=(d_i+1) \bmod m$$

其中，$d_1=H(k)$。这个方法的思想是从哈希地址基位置 $H(k)$ 出发，依次探查其后面第一个位置、后面第二个位置等，以寻找空闲地址。当到达哈希表位置 $m-1$ 时，再返回到哈希表表头以继续探查，即将哈希表看成循环的。

在这个探查过程中，如果哈希表不满，则一定能够找到一个空位置，将记录放入哈希表中。如果转了一圈后又回到基位置 d_1，仍未找到空闲位置，则表明哈希表已满。

在用二次探测法解决冲突时，求下一空闲地址的公式是：

$$d_{2i}=(d_1+i^2) \bmod m$$

$$d_{2i+1}=(d_1-i^2) \bmod m \quad (i=1,2,\cdots)$$

其中，$d_1=H(k)$。这个方法的基本思想是以哈希地址基位置 $H(k)$ 为中心，跳跃式地向两边搜索下一地址，即探查的地址序列为 $d_1+1^2, d_1-1^2, d_1+2^2, d_1-2^2, d_1+3^2, d_1-3^2, \cdots$。它仍将哈希表看成循环的，而且是双向循环的，即如果下标变大，越过表尾，则返回表头继续；如果下标变小，越过表头，则返回表尾继续。

二次探测法跳跃式地查找空闲地址，有可能无法遍历哈希表中的所有位置。所以，即使哈希表不满，也可能找不到空闲位置。有研究表明，当哈希表的表长为素数，且表中至少有一半的空闲位置时，使用二次探测法肯定能找到空闲地址。

还有其他方式，例如，可以将伪随机数序列当作相对于发生冲突位置 $H(k)$ 的偏移量。注意，存放和查找时的伪随机数序列要一致。

【例 8-19】 开放地址法示例。

哈希表中使用哈希函数 H(key)= key mod 11，哈希表地址为 0~10，将关键字序列 22,41,53,46,30,13,5,19 填入表中，分别采用线性探测法和二次探测法处理冲突，并分别给出两种方法得到的哈希表。

根据题意，哈希表地址为 0~10，表长为 11。哈希函数为 H(key)= key mod 11，则按照哈希函数计算的各关键字对应的哈希值列在表 8-4 中。

表 8-4　关键字对应的哈希值（地址）

	22	41	53	46	30	13	5	19
H	0	8	9	2	8	2	5	8

根据表 8-4 中各关键字对应的哈希值，将各关键字放入哈希表中，先按线性探测法处理冲突。例如，关键字 22 直接放入下标为 0 的位置，关键字 41 直接放入下标为 8 的位置。接下来，关键字 53 和 46 分别直接放入下标为 9 与 2 的位置。再接下来，关键字 30 应该放在下标为 8 的位置，但这个位置已经存入了关键字 41，所以出现了冲突。

寻找下标 8 后面第一个位置，找到下标为 9 的位置，这个位置已放置了 53，故再找下一个位置。下标 10 是空闲的，可以将 30 放入位置 10。关键字 13 与关键字 46 在位置 2 处发生冲突，可以将 13 放入下标为 3 的空闲位置。关键字 5 放在下标为 5 的位置。关键字 19 的基位置是 8，与 41 发生冲突，寻找空闲位置时，依次查看位置 9 和位置 10，均不空闲。此时再转回表头，位置 0 已有数据，位置 1 是空闲的，所以 19 放置在位置 1 处。使用线性探测法处理冲突后得到的哈希表见表 8-5。

表 8-5　线性探测法得到的哈希表

0	1	2	3	4	5	6	7	8	9	10
22	19	46	13		5			41	53	30

再看按二次探测法处理冲突的情况。

因为放置关键字 22、41、53 和 46 时均不发生冲突，所以都是直接放置到位的。关键字 30 与 41 在位置 8 处发生冲突，按二次探测法解决冲突，先探测位置 8+1=9，若仍冲突，则再探测位置 8-1=7，此位置空闲，所以 30 放置在位置 7 处。关键字 13 与 46 在位置 2 处冲突，探测位置 2+1=3，此位置空闲，所以 13 放置在位置 3 处。关键字 5 放置在位置 5 处，没有发生冲突。最后，关键字 19 与 41 发生冲突，探测位置序列是 8+1=9、8-1=7 和 $(8+2^2) \bmod 11=1$，找到空闲位置。使用二次探测法处理冲突后得到的哈希表见表 8-6。

表 8-6　二次探测法得到的哈希表

0	1	2	3	4	5	6	7	8	9	10
22	19	46	13		5		30	41	53	

可以改变哈希表的结构，得到桶式哈希法。桶式哈希把哈希表分成多个桶，每个桶类似于原来的每个位置。每个桶又划分为多个槽。所有桶的最后还有一个溢出桶。当放置记录时，将记录放到相应桶（位置）的第一个空槽中。如果相应位置的各个槽都满了，则将记录放到溢出桶中。溢出桶采用顺序存储方式。

2. 链地址法

链地址法是一种经常使用而且很有效的解决冲突的方法。在哈希表的每个位置放置一个指针，每个指针都指向由所有哈希到该位置的关键字形成的单链表。这个单链表称为同义词表。

【例 8-20】 链地址法处理冲突示例。

使用链地址法处理例 8-19 中出现的冲突。

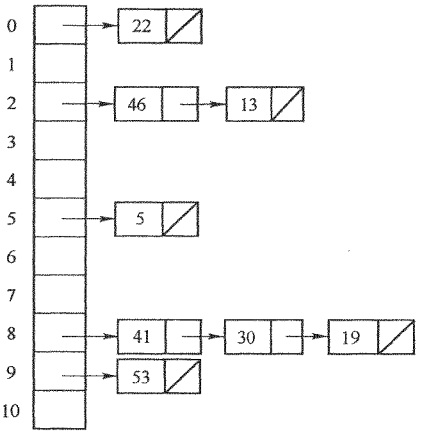


图 8-32　采用链地址法解决冲突

与例 8-19 一样，先计算各关键字对应的存储位置，在位置 8 发生冲突的关键字是 41、30 和 19，在位置 2 发生冲突的关键字是 46 和 13，其他关键字不发生冲突。采用链地址法解决冲突后得到的结果如图 8-32 所示。

四、哈希表的查找

在哈希表中查找关键字类似于将关键字放入哈希表的过程。在开放地址法中，放置关键字时的探查位置序列就是查找该关键字时要探查的位置序列。在桶式哈希法中，如果在桶中查找到关键字，则查找成功；如果查找到桶中最后一个存放位置，且还没有查找到，则需要分两种情况考虑。若此时桶是满的，则说明可能还有记录存放在溢出桶中，还需要顺序查找溢出桶；如果桶中还有空位置，则查找失败。在链地址法中，需要在对应的同义词表中采用顺序查找法进行查找。

以例 8-19 得到的表 8-5 所示的哈希表为例，分别查找关键字 46、30 和 2。先看 46 的查找过程。通过哈希函数，计算 46 的哈希地址为 2，查找哈希表的位置 2，查找到 46，查找成功。查找过程中进行了一次关键字间的比较。关键字 30 的哈希地址是 8，访问哈希表的位置 8，不等于 30，此时需要探查下一个位置。因为解决冲突的策略是线性探测法，所以探查位置 8+1=9，仍没有找到，继续探查位置 8+2=10，找到 30，查找成功，查找过程中分别与关键字 41、53 和 30 进行了比较，共 3 次。最后查找 2。2 的哈希地址是 2，在位置 2 没有找到，继续查找位置 3 和位置 4，在位置 4 遇到了空闲位置，表明查找失败，哈希表中没有关键字 2。其比较次数也是 3。

哈希表的查找效率使用平均查找长度来度量，分为查找成功的平均查找长度 $ASL_{成功}$ 和查找失败的平均查找长度 $ASL_{失败}$。

仍以表 8-5 为例，计算其平均查找长度。

假设各关键字的查找概率相等。对于不发生冲突的关键字，查找成功时只需要比较 1 次，即查找长度是 1。这样的关键字共有 5 个。对于发生冲突的关键字，要和放置该关键字时探查位置序列中的各关键字均比较 1 次。比如，在查找关键字 30 时，首次找到的位置是 8，再查找的位置是 9，最后放置的位置是 10。所以，为查找 30 进行比较的次数是 3。同样，查找关键字 13 时需要进行两次比较。实际上，查找时关键字的比较次数与放置该关键字时的探查次数是相等的。对应于表 8-5 所示哈希表的各关键字的探查次数见表 8-7。

表 8-7　哈希表及各关键字探查次数

0	1	2	3	4	5	6	7	8	9	10
22	19	46	13		5			41	53	30
1	5	1	2		1			1	1	3

查找成功的平均查找长度 $ASL_{成功}=(1\times5+5+2+3)/8=15/8=1.875$。

查找不在哈希表中的所有关键字均导致查找失败。假设这些关键字的查找概率相等。不在哈希表中的关键字个数是无穷多，均匀哈希函数将它们映射到哈希表中每个位置的概率相等，所以按等概率来计算每个位置查找失败的平均查找长度。

仍以表 8-5 为例。若要查找的某关键字 K 的哈希地址为 0，则 K 要依次与 22、19、46 和 13 进行比较，遇到位置 4 时才可判定查找失败，共进行 5 次比较。若 K 的哈希地址为 1，则 K 要进行 4 次比较。若 K 的哈希地址为 4，因为这是空闲位置，所以比较次数为 1。对于哈希地址为 8 的关键字 K，若要判定其查找失败，则需要将 K 依次与从位置 8 到位置 10，再到位置 4 中的各个值进行比较，共 8 次比较。各位置查找失败时的比较次数见表 8-8。

表 8-8　查找失败的比较次数

0	1	2	3	4	5	6	7	8	9	10
5	4	3	2	1	2	1	1	8	7	6

查找失败的平均查找长度 $ASL_{失败}=(5+4+3+2+1+2+1+1+8+7+6)/8=40/8=5$。

在哈希表中进行查找时，空闲位置对查找失败的判定非常重要。哈希表中连续排列的关键字序列称为簇。簇的长度直接影响查找失败的平均查找长度。

对于线性探测法，每解决一次冲突，都会使原来的簇更长。比如，关键字 46 和 13 在位

置 2 处发生冲突，放置 13 时，将其放置在 46 的后面，使得原来长度为 1 的簇变为长度为 2 的簇。这种现象称为聚集。由线性探测法解决冲突时形成的聚集称为基本聚集。线性探测法不只使得同义词形成的簇变长，还可能使得不是同义词形成的簇连接起来。例如，关键字 41 和 30 本是同义词，它们和 53 在同一个簇中。

在二次探测法中，从基位置开始，向左或向右跳跃式地远离基位置，以探查空闲位置，所以较不易形成基本聚集。但探查位置序列是一样的，当两个关键字在某个位置发生冲突时，仍要访问相同的探查序列，也会形成聚集，这称为二级聚集。二级聚集也会增大查找失败时的平均查找长度。

在哈希表中，通常不进行删除操作。从哈希表中直接删除一个关键字后，这个位置变为空闲位置，会影响查找后续关键字的正确性。仍以表 8-5 为例，删除关键字 30，导致位置 10 变为空闲。当查找关键字 19 时，导致查找失败。为了解决这个问题，通常将删除关键字后的空闲位置标记为特殊位置，它不同于空闲位置，也不同于有数据的位置。当插入关键字时，特殊位置被当作空闲位置使用，即可以放置关键字。当查找关键字时，特殊位置被当作非空闲位置，继续探查下一个位置。

哈希表是通过哈希函数进行计算后直接求出存储地址的，当哈希函数能得到均匀的地址分布，且没有发生冲突时，不需要任何比较运算就可以直接找到所要的记录。但实际上不可能完全避免冲突，因此查找时还需要进行若干次探查比较，才能找到所要的记录。

如果哈希表较满，则会增大发生冲突的可能性。使用装填因子描述哈希表的充满程度。设 α 是哈希表的装填因子，其计算公式为：

$$\alpha=\frac{\text{哈希表中已有的记录数}}{\text{哈希表的长度}}$$

在开放地址法中，$0\leqslant\alpha\leqslant1$。直观地看，当 α 很小时，表中已有的记录稀少，发生冲突的可能性就小；反之，当 α 很大时，说明表中已有的记录稠密，发生冲突的可能性就很大。在采用链地址法时，α 的值可能会大于 1。如果各同义词表的长度均匀，则查找效率较高。

有研究表明，当哈希表中的数据个数很大时，平均查找长度仅与装填因子有关。将 3 种解决冲突方法的查找成功的平均查找长度和查找失败的平均查找长度列在表 8-9 中。

表 8-9　3 种解决冲突方法的平均查找长度

	$ASL_{成功}$	$ASL_{失败}$
线性探测法	$\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$	$\frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right)$
二次探测法	$-\frac{1}{\alpha}\ln(1-\alpha)$	$\frac{1}{1-\alpha}$
链地址法	$1+\frac{\alpha}{2}$	$\alpha+e^{-\alpha}$

若采用开放地址法解决冲突，那么，当对哈希表进行查找时，平均查找长度不依赖于表的长度 n，而只依赖于表的装填因子 α。α 越接近于 1，平均查找长度越大。当哈希表的充满程度在 80%（$\alpha=0.8$）时，线性探测法查找成功的平均查找长度也不过等于 3。

哈希方法不适合于范围查找，且删除数据时不方便。

【例 8-21】 下列选项中，哈希技术最适合的操作是（　）。

A. 删除给定关键字的元素

B. 在哈希表中查找给定关键字

C. 输出关键字升序排列时位于第 k 位的元素

D. 按关键字升序排列并输出哈希表中的所有元素

答案为 B。

在哈希表中查找某个关键字是最方便的（选项 B 正确）。但查找的关键字根据哈希函数计算存储地址，并不知道关键字的排序情况，所以输出排在第 k 位（无论是升序还是降序）的元素（选项 C）或按关键字升序排列并输出各元素（选项 D）都是困难的。删除给定关键字的元素是可行的，但删除是在哈希表中标记了特殊标记，与查找给定关键字相比，删除并不占优势。

内 容 小 结

本章介绍了查找的概念，介绍了三类查找方法，分别是基于顺序表的查找方法、基于树形结构的查找方法和哈希方法，实现了相关的查找算法并进行了复杂度分析，给出了各算法的适用条件。

对于顺序表的查找方法，介绍了适用于任何情况的顺序查找方法、适用于关键字有序的有序表的折半查找方法，以及结合顺序查找和折半查找特点的索引顺序查找方法。对于基于树形结构的查找方法，介绍了二叉查找树、AVL 树和 B 树的概念及相关操作。本章最后介绍了非常重要的哈希方法，讨论了哈希函数的构造策略及解决冲突的方法。

习　　题

一、单项选择题

1. 在对含 n 个元素的查找表进行顺序查找时，若查找每个元素的概率相同，则查找成功的平均查找长度为_____。

A. $(n+1)/2$　　B. $n/2$　　C. n　　D. $((1+n)\times n)/2$

2. 下列关于折半查找的叙述中，正确的是_____。

A. 表必须有序，表可以顺序方式存储，也可以链表方式存储

B. 表必须有序且表中数据必须是整型、实型或字符型

C. 表必须有序，而且只能从小到大排列

D. 表必须有序，且表只能以顺序方式存储

3. 与二叉查找树的查找效率有关的是二叉树的_____。

A. 高度　　B. 结点个数　　C. 树形　　D. 结点位置

4. 下列二叉查找树的情形中，查找效率最差的是_____。

A. 结点太多　　B. 完全二叉树

C. 没有度为 2 的结点　　D. 结点太复杂

5. 以下列序列分别构造二叉查找树，得到的树形与其他三个不同的是_____。

A. 100,80,90,60,120,110,130

B. 100,120,110,130,80,60,90

C. 100,60,80,90,120,110,130

D. 100,80,60,90,120,130,110

6. 结点数为4的AVL树可能的不同树形个数是_____。

A. 4　　B. 5　　C. 6　　D. 7

7. 若将关键字1、2、3、4、5、6和7依次插入到初始为空的AVL树T中，则T中平衡因子为0的分支结点的个数是_____。

A. 0　　B. 1　　C. 2　　D. 3

8. 下列关于m阶B树的说法中，错误的是_____。

A. 根结点至多有m棵子树

B. 所有叶结点都在同一层次上

C. 非叶结点至少有$m/2$（m为偶数）或$m/2+1$（m为奇数）棵子树

D. 根结点中的数据是有序的

9. 下列关于m阶B树的说法中，正确的是_____。

Ⅰ. 每个结点至少有两棵非空子树

Ⅱ. 树中每个结点至多有$m-1$个关键字

Ⅲ. 所有叶结点在同一层次上

Ⅳ. 当插入一个数据项，引起B树结点分裂后，树长高一层

A. Ⅲ　　B. Ⅱ、Ⅲ　　C. Ⅰ、Ⅱ、Ⅲ　　D. Ⅱ、Ⅲ、Ⅳ

10. 下列关于哈希查找的说法中，正确的是_____。

A. 哈希函数的构造越复杂越好，因为随机性好，冲突小

B. 除留余数法是所有哈希函数中最好的

C. 不存在特别好与坏的哈希函数，要视具体情况而定

D. 若需要在哈希表中删除一个元素，那么，无论用何种方法解决冲突，只需要简单地将该元素删去

11. 设哈希表表长为14，哈希函数是H(key)=key mod 11，表中已有数据的关键字分别为15、38、61、84，共4个，现要将关键字为49的结点加入到表中，若用二次探测法解决冲突，则放入的位置是_____。

A. 8　　B. 3　　C. 5　　D. 9

12. 哈希函数有一个共同的性质，即函数值应当以_____取其值域的每个值。

A. 最大概率　　B. 最小概率　　C. 随机概率　　D. 相等概率

13. 哈希表的地址区间为0~17，哈希函数为H(K)=K mod 17。采用线性探测法处理冲突，并将关键字序列26,25,72,38,8,18,59依序存储到哈希表中。

（1）元素59存放在哈希表中的位置是_____。

A. 8　　B. 9　　C. 10　　D. 11

（2）存放元素59时需要搜索的次数是_____。

A. 2　　B. 3　　C. 4　　D. 5

二、解答题

1. 在顺序存储的条件下，当各记录满足什么条件时，可以分别采用什么策略进行查找？

2. 在4阶B树中，每个结点所含子树个数的上下限分别是多少？

3. 在5阶B树中，每个结点所含关键字个数最多是多少？最少是多少？

4. 由6个结点构造的二叉查找树的最大高度和最小高度分别是多少？

5. 二叉查找树具有什么特点？

6. 二叉查找树中最大的关键字和最小的关键字分别位于什么位置？

7. 设有一个包含15个关键字的有序表，其中关键字的次序为：1,2,3,4,5,6,7,8,9,10,11,12,13,14,15，当用折半查找法查找关键字2、10、7时，其比较次数分别是多少？

8. 设由空树开始，依次插入关键字D、E、G、B、C、J、A、I，构成二叉查找树。画出这棵树的生成过程。

9. 设有依以下次序出现的关键字序列22,41,53,46,30,13,1,67，构造一棵二叉查找树，画出最后得到的树形。

10. 设有依以下次序出现的关键字序列22,41,53,46,30,13,1,67，构造一棵AVL树，画出最后得到的树形。

11. 设有依以下次序出现的关键字序列35,16,18,20,5,50,22,60,3,17,45,7，构造一棵3阶B树。要求从空树开始，每插入一个关键字，画出一个树形。

12. 构造哈希函数的基本原则是什么？列出三种常用的构造方法。

13. 设有一组关键字，出现次序为：105,97,28,52,37,22,16,90,45,79,59,76，要求用哈希方法将它们存入长度为15的表中。

（1）采用除留余数法构造哈希函数。

（2）用二次探测法解决冲突。

给出最后得到的哈希表。

14. 已知哈希表的地址空间为A[0...11]，哈希函数为H(k)=k mod 11，采用线性探测法处理冲突。请将下列数据25,16,38,47,79,82,51,39,89,151,231依次插入哈希表中，并计算在等概率情况下查找成功时的平均查找长度。

15. 设哈希表的地址范围为0~17，哈希函数为H(K)=K mod 16，K为关键字，用线性探测法处理冲突，输入关键字序列：10,24,32,17,31,30,46,47,40,63,49，构造哈希表，试回答下列问题。

（1）给出最终的哈希表。

（2）若查找关键字63，需要依次与哪些关键字比较？

（3）若查找关键字65，需要依次与哪些关键字比较？

（4）假定每个关键字的查找概率相等，求查找成功时的平均查找长度。

三、算法设计题

1. 设二叉查找树中保存的是整数。编写算法，返回一棵二叉查找树上最大关键字与最小关键字的值。

2. 设二叉查找树中保存的是整数。编写算法，返回一棵二叉查找树上全部关键字的平均值。

3. 设二叉查找树中保存的是整数。编写算法，输出二叉查找树中满足$k_1 \leqslant x \leqslant k_2$的所有关键字x。

4. 设二叉查找树中保存的是整数。编写算法，判别给定二叉树是否为二叉查找树。

参 考 文 献

[1] 苏仕华．数据结构［M］．北京：外语教学与研究出版社，2012.
[2] 殷人昆．数据结构：C 语言版［M］．2 版．北京：清华大学出版社，2017.
[3] 严蔚敏，吴伟民．数据结构：C 语言描述［M］．北京：清华大学出版社，2012.
[4] CLIFFORD A S. 数据结构与算法分析［M］．张铭，刘晓丹，译．北京：电子工业出版社，2010.
[5] 殷人昆．数据结构算法解析［M］．北京：清华大学出版社，2021.
[6] 殷人昆．数据结构精讲与习题详解［M］．北京：清华大学出版社，2012.
[7] 邓俊辉．数据结构习题解析［M］．3 版．北京：清华大学出版社，2013.
[8] 李春葆．新编数据结构习题与解析［M］．2 版．北京：清华大学出版社，2019.
[9] 陈锐．数据结构：C 语言实现［M］．北京：机械工业出版社，2020.
[10] 王争．数据结构与算法之美［M］．北京：人民邮电出版社，2021.
[11] 王曙燕．数据结构与算法［M］．北京：高等教育出版社，2019.
[12] 徐孝凯．数据结构：C 语言描述［M］.2 版．北京：清华大学出版社，2018.
[13] 熊岳山．数据结构与算法［M］.2 版．北京：清华大学出版社，2016.
[14] 张岩．数据结构与算法［M］.5 版．北京：高等教育出版社，2020.

后　记

经全国高等教育自学考试指导委员会同意，由电子、电工与信息类专业委员会负责高等教育自学考试《数据结构与算法》教材的审定工作。

本教材由南开大学辛运帏教授、北京理工大学陈朔鹰副教授负责编写。国防科技大学熊岳山教授、天津大学喻梅教授进行评审，并提出修改意见，谨向他们表示诚挚的谢意。

电子、电工与信息类专业委员会最后审定通过了本教材。

全国高等教育自学考试指导委员会
电子、电工与信息类专业委员会
2023 年 12 月

封面设计　万卷九州

本书数字资源

本书配有电子课件、习题解答等教辅资源，需要的读者可登录 www.cmpedu.com 免费注册，审核通过后下载，或扫描关注机械工业出版社计算机分社官方微信订阅号——身边的信息学，回复 76103 即可获取本书配套资源链接。

机工教育微信服务号

身边的信息学

ISBN 978-7-111-76103-7

定价：75.00 元